Designing, Installing & Operating a Production Kubernetes Cluster

A Cluster Architect's Textbook — built around TicketHub, a real-world event-ticketing microservices platform

Author's role: You are the **Kubernetes Cluster Architect** for TicketHub. This book walks the entire journey — from **bare-metal servers** in a data center, through **VM slicing**, **cluster installation**, **platform services**, **containerizing 9 microservices**, and every workload, scaling, scheduling, security, and observability concern a production cluster needs.

Assumed knowledge: none beyond basic Linux and "what a container is". Every concept is introduced from first principles, then applied to TicketHub with **real diagrams, Dockerfiles, and YAML** you can run from the companion `repo/`. If any term feels assumed, start with the **Chapter 0 primer** and the **Glossary** — they define the recurring vocabulary (the object model, kubectl, Helm, CIDR, Kafka/Redis) so no later chapter has to stop and explain it.

Platform decisions (fixed for the whole book):

Layer	Choice
Virtualization	KVM / libvirt (Proxmox) on bare metal
Kubernetes install	`kubeadm` (vanilla upstream), HA control plane
CNI	**Cilium** (eBPF) + **Hubble** observability
Load balancer	**MetalLB** (bare-metal `LoadBalancer` services)
Edge routing	**Kubernetes Gateway API** (Cilium implementation)
Storage	**Rook-Ceph** (block, file, S3 object)
Policy / security	Pod Security Admission, **Kyverno**, **Falco**, RBAC, NetworkPolicy
Autoscaling	Metrics Server, **HPA**, **VPA**, Cluster Autoscaler, **KEDA**
Delivery	**Argo CD** (GitOps)
DB Replication	Custom CRDs (`PostgresReplicationCluster`, `ReplicationSlot`)

Table of Contents

Part VI — Observability & Operations

Appendices

How this book is organized

Each chapter follows the same rhythm: **concept from basics → a color-coded diagram → the TicketHub application → real YAML/commands → an architect's checklist**. The six parts build strictly on one another — from bare-metal physical design, through cluster installation and service deployment, to reliability, security, and day-2 operations. Every Dockerfile and manifest referenced lives in the companion `repo/`, organized by bootstrap order.

The TicketHub platform at a glance

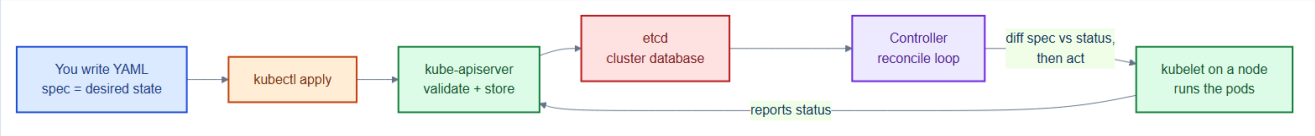
9 microservices — **Frontend UI, API Gateway, Users/Auth, Catalog, Inventory, Orders, Payments, Notifications, Search** — backed by **PostgreSQL, Redis, Kafka, and a Ceph object store**, running on a **12-node** cluster (3 control plane + 9 workers across general / data / infra pools).

0. Prerequisites & a Five-Minute Primer

This book assumes only **basic Linux** and **"what a container is."** Everything else is built up from first principles. But five ideas recur on almost every page — the **declarative object model**, **kubectl/kubeconfig**, **Helm**, **CIDR notation**, and the **stateful backends** (Postgres, Redis, Kafka). Rather than interrupt later chapters to define them, we front-load them here. Skim this once; refer back whenever a term feels assumed. A full **Glossary** sits at the back of the book (Appendix A).

0.1 Everything is a declarative object

You almost never *tell* Kubernetes to "do" something imperatively. Instead you **declare a desired state** as a YAML object, hand it to the cluster, and a **controller** continuously works to make reality match. This one idea underpins Deployments, Services, autoscaling, GitOps — the whole book.



Every Kubernetes object has the **same four-part shape**. Once you can read one, you can read all of them:

```
# See: repo/manifests/ for the full manifest
apiVersion: apps/v1        # which API group + version defines this kind
kind: Deployment           # the type of object
metadata:                  # name, namespace, labels — how it's identified
 name: catalog
 namespace: tickethub
spec:                      # DESIRED state — what YOU want (the only part you write)
 replicas: 3
status:                    # ACTUAL state — what IS (filled in by Kubernetes, read-only)
 readyReplicas: 3
```

Field	Who writes it	Meaning
`apiVersion`	You	Which API group/version (e.g. `apps/v1`, `v1`, `networking.k8s.io/v1`)
`kind`	You	The object type (`Pod`, `Service`, `Deployment`, …)
`metadata`	You	Name, namespace, **labels** (key/value tags used everywhere)
`spec`	You	**Desired** state — your request

`status`	Kubernetes	**Actual** state — the controller's report back

Mental model — a thermostat, not a light switch

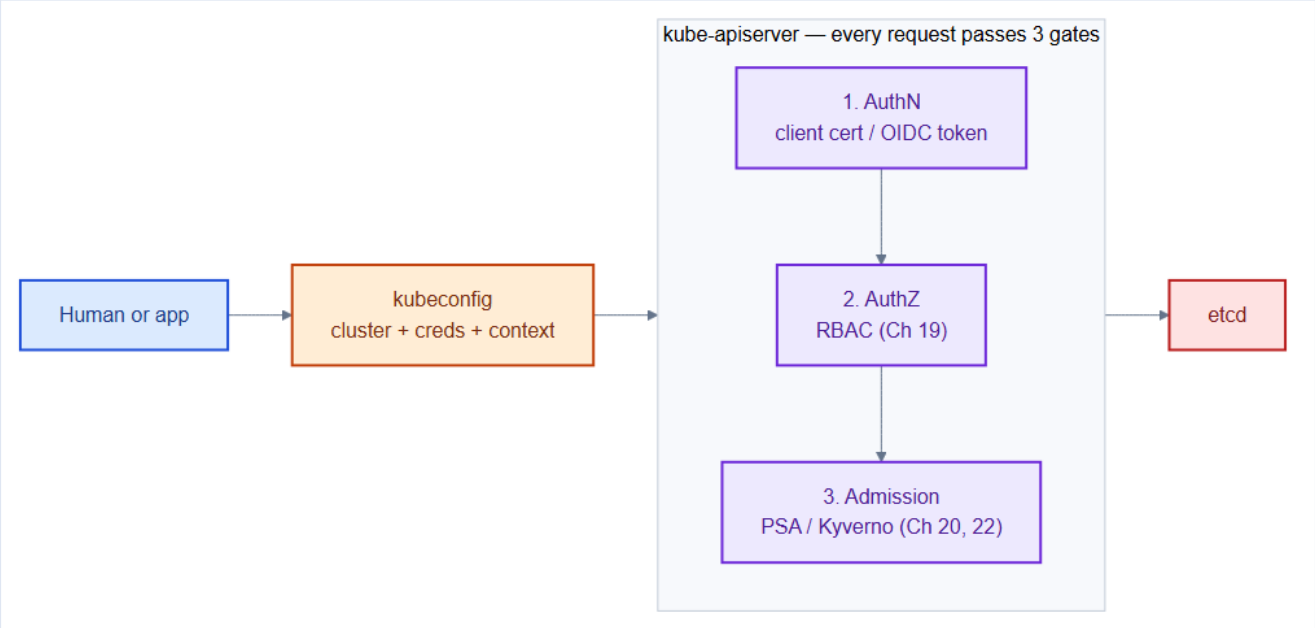
A light switch is **imperative**: you flip it, the light obeys once. A thermostat is **declarative**: you set 21°C (the `spec`) and it *continuously* works to reach and hold it, compensating when a window opens. Kubernetes is a giant rack of thermostats. You declare `replicas: 3`; if a pod dies, a controller notices the gap between `spec` and `status` and starts a replacement — no command from you.

Labels are the glue of the whole system

A **label** is a `key: value` tag in `metadata.labels` (e.g. `app: orders`). Services find pods by label, Deployments own pods by label, NetworkPolicies select pods by label, monitoring scrapes pods by label. When something "can't find" something else in later chapters, a **label/selector mismatch** is the first thing to check.

0.2 Talking to the cluster — kubectl, kubeconfig & contexts

`kubectl` is the command-line client. It never talks to nodes directly — it sends every request to the **kube-apiserver**, the single front door. How does `kubectl` know *which* cluster and *who you are*? A file called the **kubeconfig** (default `~/.kube/config`).



A kubeconfig bundles three things into named **contexts** so you can switch clusters instantly:

Kubeconfig part	Holds
cluster	The API server URL (our VIP `https://10.10.0.10:6443`) + its CA cert

user	*Your* credentials — a client certificate or an OIDC token
context	A named pairing of (cluster + user + default namespace)

```
kubectl config get-contexts          # list clusters you can reach
kubectl config use-context tickethub-prod
kubectl config set-context --current --namespace=tickethub   # stop typing -n
kubectl get pods                     # now scoped to tickethub
```

Kubernetes has no 'User' object — identity comes from outside

This surprises everyone. You cannot `kubectl create user alice`. A **human identity** is proven to the API server one of two ways: a **client certificate** signed by the cluster CA, or an **OIDC token** from an external identity provider (Okta, Entra ID, Google). The API server only **authenticates** (verifies who you are) and then **authorizes** (RBAC, Chapter 19) — it never *stores* users. That's why Chapter 19's "subjects" are `User`/`Group` strings that come "from the auth layer": that layer is your certs or OIDC provider, not Kubernetes.

Every request the API server accepts passes three gates, in order — worth memorizing because Chapters 19–22 each live at one of them:

1. **Authentication (authN)** — *who are you?* (client cert or OIDC token)
2. **Authorization (authZ)** — *are you allowed?* (**RBAC**, Chapter 19)
3. **Admission** — *is the object itself acceptable?* (**PSA**/**Kyverno**, Chapters 20, 22)

0.3 Helm in five minutes

Many platform components in this book (Cilium, the Cilium Gateway API, Falco, the Prometheus stack) are installed with **Helm** — the Kubernetes package manager. You'll see `helm install …` repeatedly, so here's the whole model:

Helm term	Analogy	What it is
Chart	A software installer package	A bundle of templated YAML manifests
Values	The installer's settings screen	Your overrides (`--set key=value` or `values.yaml`)
Release	An installed program	One deployment of a chart into your cluster
Repo	An app store	A server hosting charts (`helm repo add …`)

```
helm repo add cilium https://helm.cilium.io/        # register a chart source
helm install cilium cilium/cilium \                 # install chart -> named release "cilium"
 --namespace kube-system \
 --set kubeProxyReplacement=true                    # override a default value
helm upgrade cilium cilium/cilium --set hubble.ui.enabled=true   # change the release
helm list -A                                         # what's installed
```

Helm renders YAML, then applies it — nothing magic

A chart is just **templates + your values → plain Kubernetes YAML**, which Helm then applies like any manifest. Anything Helm installs, you can inspect with normal `kubectl get`. When a `--set` flag in this book looks mysterious, it's simply filling a blank in the chart's templates. `helm template …` prints the final YAML without installing — great for seeing exactly what you're about to get.

0.4 Reading IP addresses & CIDR notation

Chapter 4 plans four separate networks using notation like `10.244.0.0/16`. If the `/16` is unfamiliar, here's all you need.

An IPv4 address is four bytes: `10.244.0.5`. The **`/N` suffix (the "prefix length")** says how many leading bits are *fixed* as the network; the rest are free for hosts.

CIDR	Fixed prefix	Usable-ish size	Reads as
`10.10.0.0/24`	first 24 bits	256 addresses	"the `10.10.0.x` block"
`10.244.0.0/16`	first 16 bits	65,536 addresses	"all `10.244.x.x`"
`10.96.0.0/12`	first 12 bits	~1,048,576	"`10.96.x.x`–`10.111.x.x`"

Smaller /N = bigger network

The number is *fixed bits*, so a **smaller** number means **fewer fixed bits** and a **larger** address range. `/16` (65k addresses) is far bigger than `/24` (256). The Pod CIDR is a `/16` because a cluster has *lots* of pods; a single VLAN is a `/24`. The one rule that matters for the whole book: these ranges must **never overlap** (Chapter 4).

0.5 The stateful backends — Postgres, Redis & Kafka

TicketHub's services are stateless, but they lean on three stateful backends the book deploys as StatefulSets (Chapters 11, 14). You don't need to *operate* them expertly, but you should know what each *is*.

Backend	Category	One-line role	Why TicketHub uses it

PostgreSQL	Relational database (SQL)	Durable, transactional records	Users, catalog, orders, payments — the source of truth
Redis	In-memory key/value store	Very fast, short-lived data	Seat **holds** (expire in minutes), dedupe caches
Kafka	Distributed event log / message broker	Durable stream of **events** between services	Async fan-out: emails, search indexing, analytics

Kafka deserves a few extra terms because Chapters 1, 16 and 26 use them:

Kafka term	Meaning
Topic	A named stream of events (e.g. `ticket-events`)
Broker	One Kafka server node; a cluster has several for redundancy
Producer / Consumer	A service that *writes* / *reads* events
Consumer group	A set of consumer replicas sharing the work of one topic
Lag	How many events a consumer is **behind** — the backlog. KEDA scales on this (Chapter 16)

> **Mental model — Kafka is a shared, replayable mailbox**

> A **synchronous** call (REST/gRPC) is a **phone call** — both parties must be present. Kafka is a **mailbox**: Orders drops a "ticket purchased" letter (**event**) into a **topic** and moves on. Notifications reads the mailbox whenever it's ready. If Notifications is down, letters wait; when it recovers it catches up on the **lag**. That decoupling is why a slow email provider can never fail a ticket sale.

0.6 A few kernel terms you'll meet at install time

Chapters 2 and 5 prepare Linux nodes and touch low-level terms. Quick definitions so the install commands aren't black boxes (all also in the Glossary):

Term	Plain meaning

cgroup	A Linux kernel feature that limits/accounts a process's CPU & memory. Requests/limits (Chapter 15) are enforced via cgroups
cgroup driver	*Who* manages cgroups — `systemd` or `cgroupfs`. containerd and kubelet must pick the **same** one or pods fail to start (Chapter 5)
static pod	A pod the kubelet runs directly from a file on the node (not from the API server). The control-plane components run this way (Chapter 5)
network namespace	A private network stack (its own interfaces/routes) that isolates a pod's networking
veth pair	A virtual "cable" — one end in the pod's namespace, one on the node — that the CNI creates to connect a pod to the network (Chapter 6)

0.9 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- The **kubeconfig context** is not the same as a user account — it is a named combination of cluster, user credentials, and namespace. You can have many contexts pointing at the same cluster with different credentials; `kubectl config use-context` just changes which combination is active.

- `kubectl` communicates only with the **kube-apiserver** — never with kubelets directly. Every operation (even `kubectl exec`) is proxied through the API server.

- Helm charts are just **templates** that render to Kubernetes YAML. The actual objects live in the cluster; Helm tracks release state in a Secret in the same namespace. Deleting that Secret orphans the objects — `helm list` shows nothing but the resources still run.

Gotchas — traps that catch experienced engineers

- Confusing `kubectl apply` (declarative, idempotent, tracks last-applied-configuration annotation) with `kubectl create` (imperative, fails if the object already exists). Always prefer `apply` in automation.

- Using `kubectl delete pod X` to "restart" a pod managed by a Deployment just causes the Deployment to create a replacement — the correct restart idiom is `kubectl rollout restart deployment/X`.

- Assuming all objects are namespaced — `Node`, `PersistentVolume`, `ClusterRole`, `StorageClass`, and `CustomResourceDefinition` are **cluster-scoped**. Passing `-n my-ns` does nothing for them.

Architect Considerations

1. **Single kubeconfig vs per-cluster** — should operators use a shared admin kubeconfig (simple but over-privileged) or per-person OIDC tokens (auditable, revokable)? Choose OIDC + `kubectl oidc-login` for any team larger than 2.

2. **kubectl version skew** — the client must be within ±1 minor version of the server. Enforce this via a cluster-local `kubectl` wrapper script that pins the correct version.

3. **Helm vs raw manifests** — Helm adds release lifecycle management but introduces template complexity. Use Helm for third-party software you consume; use raw manifests (or Kustomize) for your own services where you control the YAML.

4. **etcd as the source of truth** — everything in `kubectl get` is a live read from etcd. There is no separate "config database" to sync; the cluster state IS the database.

5. **Preview environments** — namespaces make cheap preview environments only if your storage (PVCs, secrets) can also be namespace-scoped and cheaply provisioned. Plan PVC provisioning speed before committing to PR-per-namespace patterns.

Chapter 0 checklist — you're ready if you can say…

- Every object is `apiVersion` + `kind` + `metadata` + **`spec` (desired)** + `status` (actual).
- `kubectl` talks to the **apiserver**; **kubeconfig** holds *which cluster* + *who you are*.
- Kubernetes **doesn't store users** — identity is a client cert or OIDC token; RBAC authorizes.
- **Helm** = charts (templated YAML) + values → a named **release**.
- `/16` is **bigger** than `/24`; cluster CIDRs must never overlap.
- **Postgres** = SQL truth, **Redis** = fast ephemeral, **Kafka** = durable event log (topics, lag).

With the vocabulary in place, Part I begins the real work: designing TicketHub and the data center it will run on.

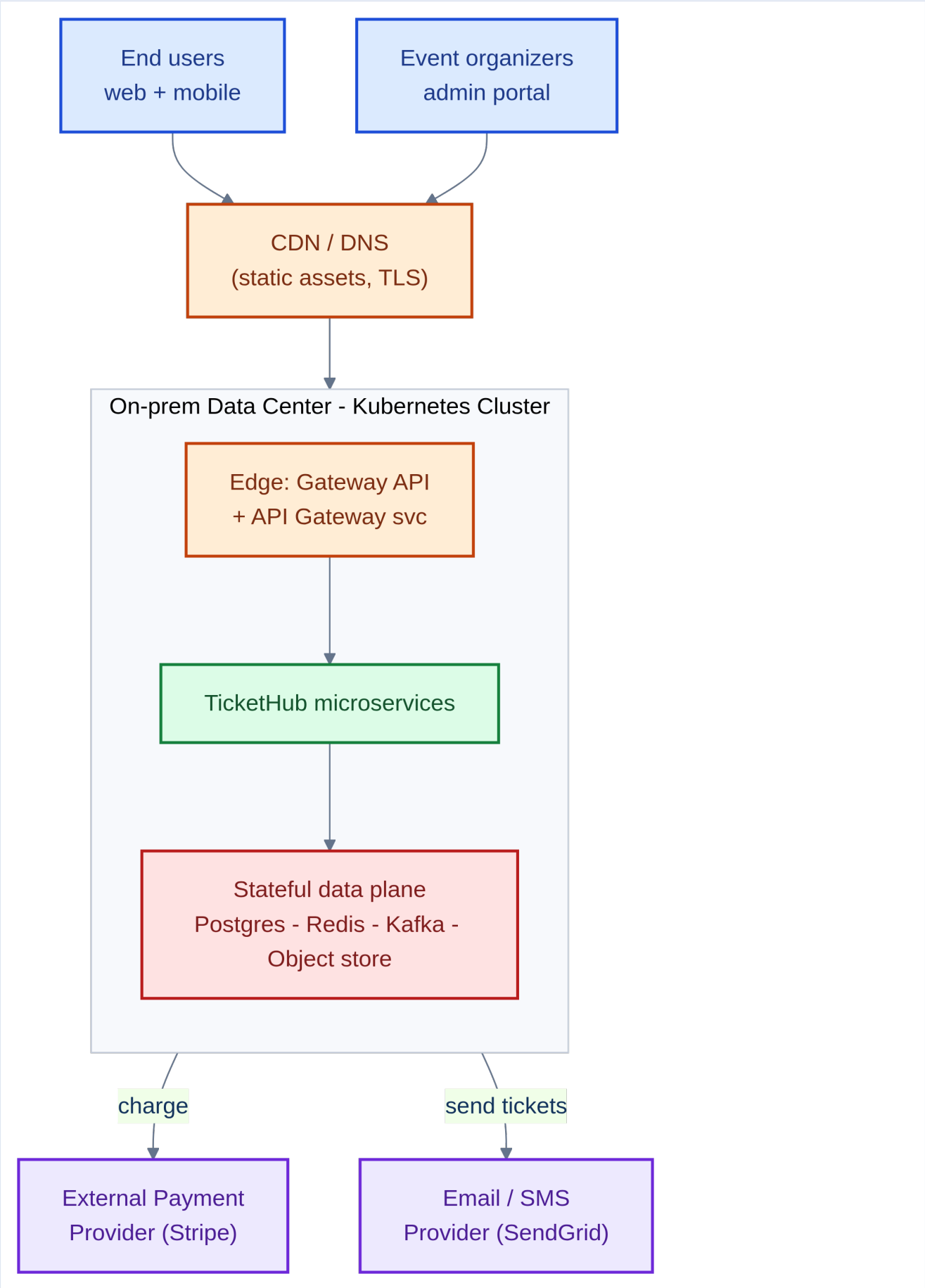
---

1. The TicketHub Scenario — Services, Interfaces & Interactions

Before we rack a single server, an architect must understand **what we are building and why**. Infrastructure exists to serve an application, so we design the application's shape first, then let it drive every cluster decision that follows.

1.1 The business scenario

TicketHub is an online platform where **event organizers** publish events (concerts, sports, theatre) and **end users** browse, hold seats, pay, and receive tickets. It must handle spiky traffic — when a popular event goes on sale, thousands of users hit "buy" in the same minute — which makes it a perfect case study for **autoscaling, resilience, and stateful data** on Kubernetes.


End users
web + mobile
Event organizers
admin portal
CDN / DNS
(static assets, TLS)
On-prem Data Center - Kubernetes Cluster
Edge: Gateway API
+ API Gateway svc
TicketHub microservices
Stateful data plane
Postgres - Redis - Kafka -
Object store
charge
send tickets
External Payment
Provider (Stripe)
Email / SMS
Provider (SendGrid)

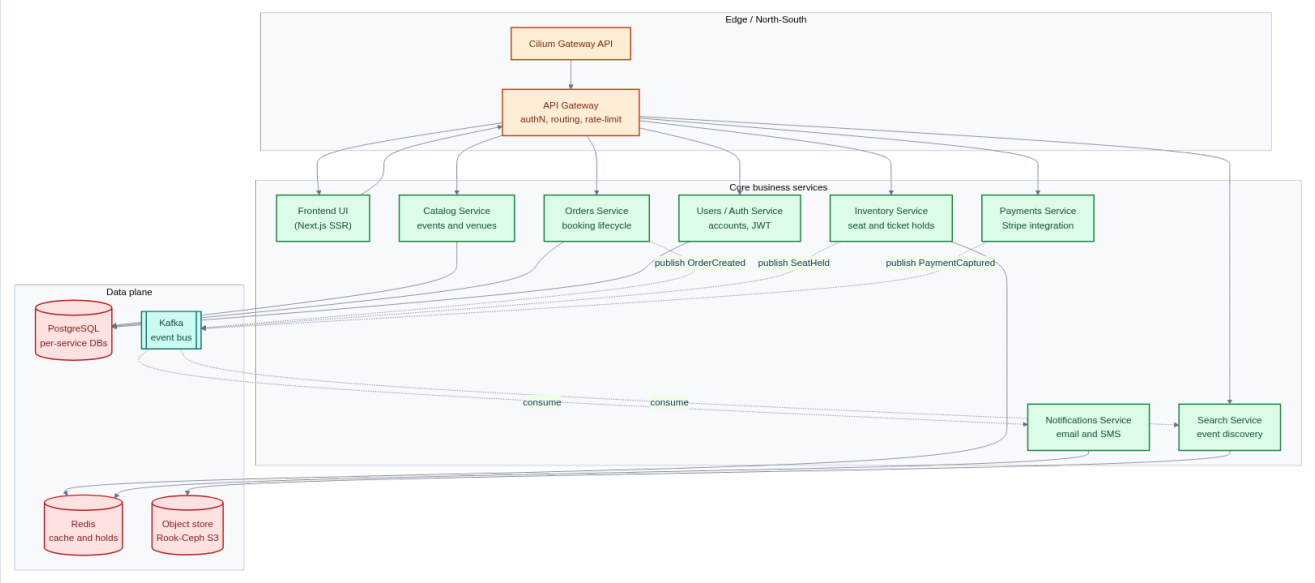
The platform lives in an **on-prem data center** (our Kubernetes cluster) and talks to two **external providers**: a payment gateway (Stripe) and an email/SMS provider (SendGrid).

> **Mental model — the cluster is a city, services are shops**

> Think of the cluster as a **city**. The **API Gateway** is the city gate (everyone enters through it). Each **microservice** is a specialized shop with its own staff and its own storeroom (**database-per-service**). Some interactions are a face-to-face conversation (**synchronous** REST/gRPC), others are dropping a letter in the mail that gets processed later (**asynchronous** events via Kafka).

1.2 The service catalog

TicketHub is decomposed into **9 microservices**, each with a single responsibility, its own API, and its own data. This is the heart of the architecture:



#	Service	Responsibility	Stateless?	Backing store	Scaling driver
1	**Frontend UI**	Server-rendered web app (Next.js)	Yes	—	Web traffic (HPA on CPU)
2	**API Gateway**	Single entry point: authN, routing, rate-limiting	Yes	—	Request rate
3	**Users / Auth**	Accounts, login, JWT issuance	Yes	PostgreSQL `users_db`	Login rate

4	**Catalog**	Events, venues, seat maps (read-heavy)	Yes	PostgreSQL `catalog_db`	Read QPS
5	**Inventory**	Seat availability & short-lived **holds**	Yes*	Redis (holds) + PostgreSQL	Hold rate (spiky)
6	**Orders**	Booking lifecycle & saga orchestration	Yes	PostgreSQL `orders_db`	Order rate
7	**Payments**	Stripe integration, capture/refund	Yes	PostgreSQL `payments_db`	Order rate
8	**Notifications**	Email/SMS on events (async consumer)	Yes	Redis (dedupe)	Kafka lag (**KEDA**)
9	**Search**	Event discovery / indexing	Yes	Object store + index	Query rate

Plus the **stateful data plane** (runs *inside* the cluster as StatefulSets in Part III): **PostgreSQL**, **Redis**, **Kafka**, and **Rook-Ceph** object storage.

> **Why 'stateless' services still have databases**

> A service is **stateless** when the *pod* holds no durable data — any replica can serve any request, and losing a pod loses nothing. The durable state lives in the **database**, not the pod. That's what lets us scale Orders from 3 to 30 replicas freely. Truly **stateful** components (Postgres, Kafka) get special treatment (StatefulSets, stable identity, persistent volumes) in Chapter 11 and 14.

1.3 Interfaces each service exposes

An architect defines **contracts**, not implementations. Each service exposes a narrow, versioned interface:

Service	Protocol	Key endpoints / interface	Consumed by
API Gateway	HTTPS (REST)	`/*` → routes to services; verifies JWT	Internet (via the Gateway API)

Users/Auth	REST + gRPC	`POST /login`, `POST /register`, `GET /verify`	Gateway, all services (token verify)
Catalog	REST + gRPC	`GET /events`, `GET /events/{id}`, `GET /venues/{id}`	Gateway, Search
Inventory	gRPC	`HoldSeats()`, `ReleaseSeats()`, `ConfirmSeats()`	Orders
Orders	REST	`POST /orders`, `GET /orders/{id}`	Gateway
Payments	gRPC	`Authorize()`, `Capture()`, `Refund()`	Orders
Notifications	Kafka consumer	subscribes `orders.*`, `payments.*`	(event-driven)
Search	REST	`GET /search?q=`	Gateway

REST, gRPC and JWT in one breath

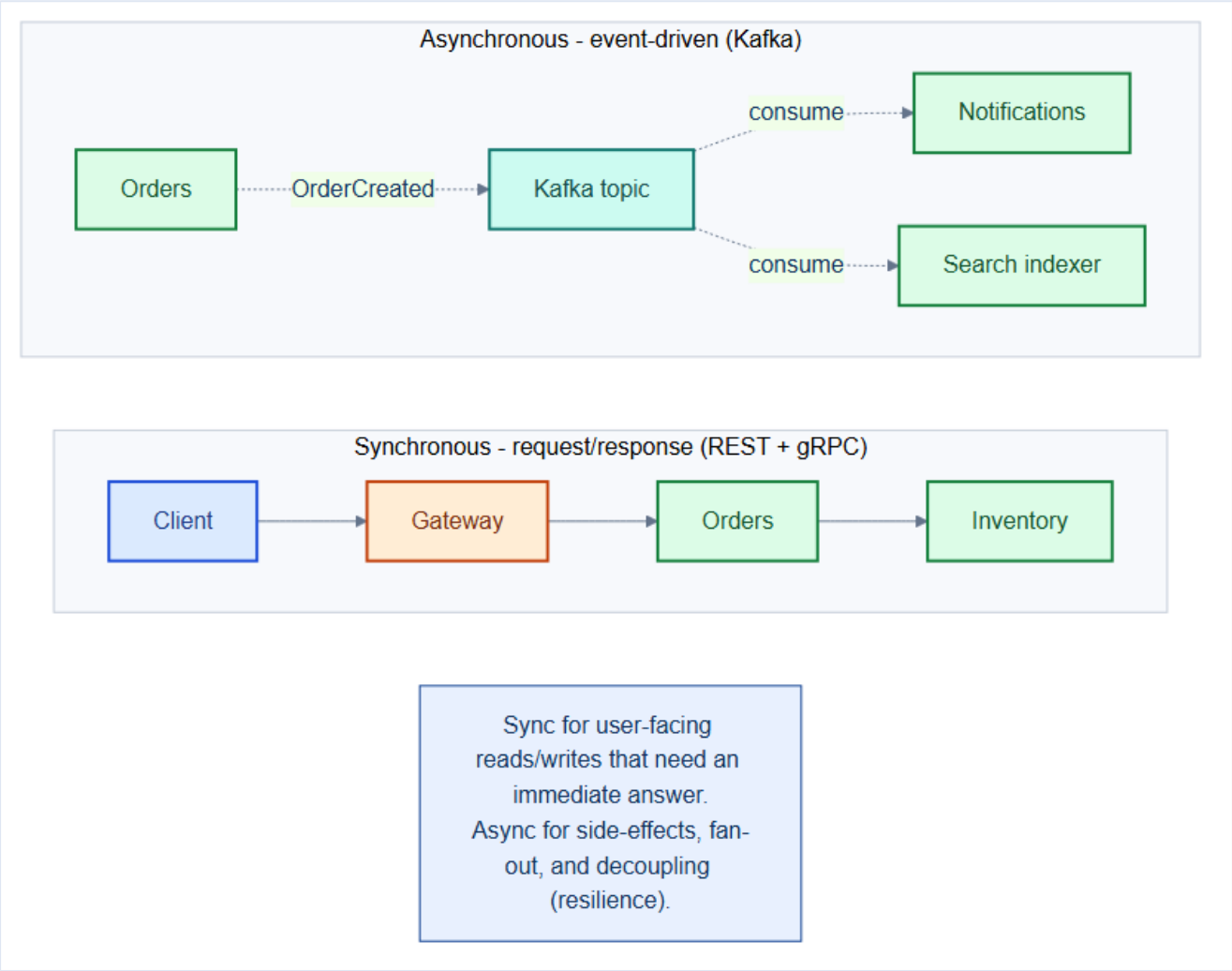
REST is resource-oriented HTTP/JSON — simple and browser-friendly, so TicketHub speaks it at the **edge**. **gRPC** is a fast, strongly-typed binary protocol over HTTP/2, better for high-volume **internal** service-to-service calls. A **JWT** (JSON Web Token) is a signed token the Users/Auth service issues at login; the gateway verifies it on every request to prove who the caller is. All three are in the table above — see the Glossary (Appendix A) for one-line refreshers.

gRPC internally, REST at the edge

A common architect pattern: expose **REST/JSON at the edge** (browser-friendly, via the Gateway) but use **gRPC between internal services** (faster, strongly typed, streaming). Kubernetes `ClusterIP` services and Cilium handle both equally — we'll wire this up in Chapter 12.

1.4 Synchronous vs asynchronous interactions

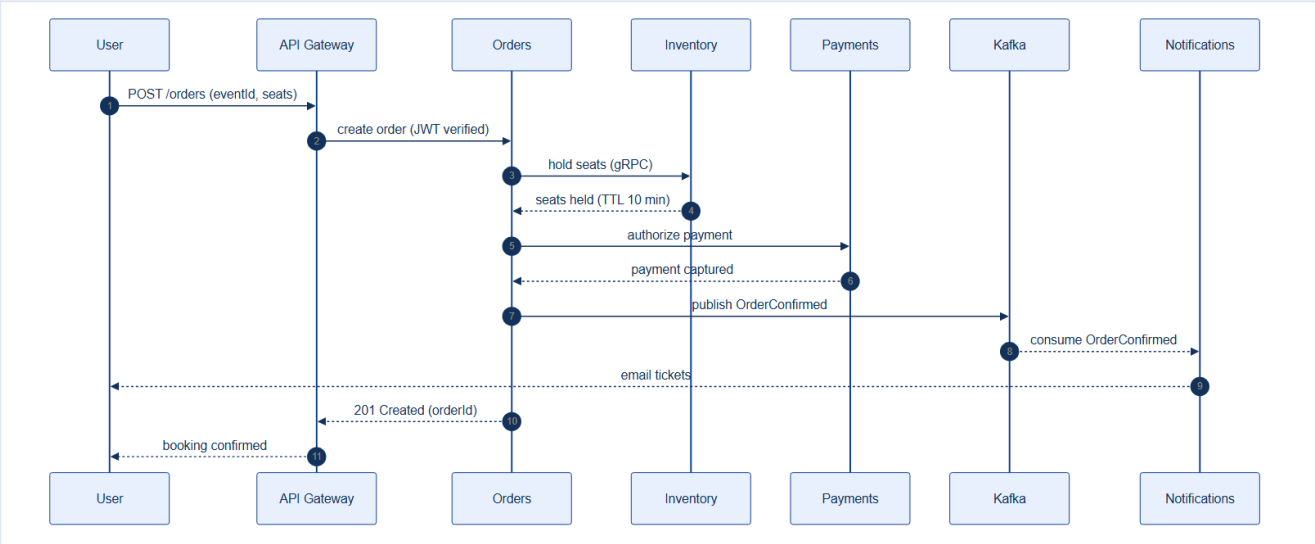
Not every interaction should be a blocking call. Choosing sync vs async per interaction is one of the most important design decisions:



- **Synchronous (REST/gRPC)** — the caller waits for an answer. Use when the user needs an **immediate result**: "is this seat available?", "did my payment succeed?".
- **Asynchronous (Kafka events)** — the caller emits an event and moves on. Use for **side-effects and fan-out**: sending a confirmation email, updating the search index, analytics. If Notifications is briefly down, orders still succeed and emails flush when it recovers — **decoupling for resilience**.

1.5 A booking, end to end

Here's how the services collaborate for the core "buy a ticket" flow — a mix of synchronous calls (for the transaction) and an asynchronous event (for the receipt):



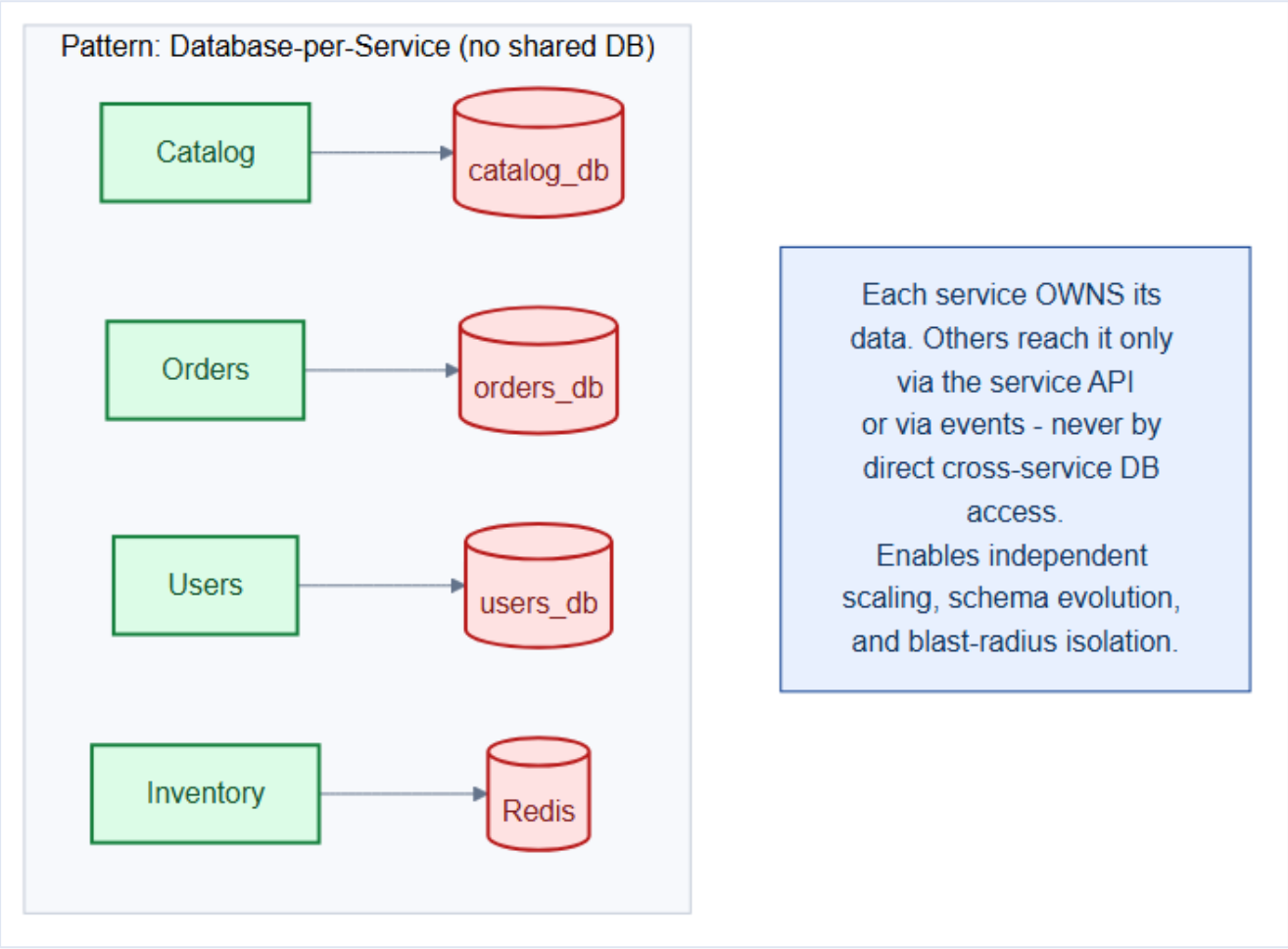
This flow also illustrates the **Saga pattern**: Orders orchestrates a multi-step transaction across Inventory and Payments. If payment fails, Orders issues a compensating `ReleaseSeats()` — because there are no distributed ACID transactions across microservices.

> **What the Saga pattern is (and why we need it)**

> A single database gives **ACID** guarantees — a multi-step change either fully commits or fully rolls back. Across *separate* service databases (Inventory, Payments, Orders) there is **no** shared transaction, so a crash mid-way could leave seats held but unpaid. A **Saga** replaces the one big transaction with a *sequence of local commits*, each paired with a **compensating** undo. Orders runs the saga — hold seats, charge, confirm — and if the charge fails it fires `ReleaseSeats()` to compensate. You trade a distributed lock for explicit undo steps and eventual consistency.

1.6 Database-per-service

Notice each service owns its **own** database and never reaches into another's:



Architect's rule — own your data

A service's database is **private**. No other service connects to it directly. Others get that data only through the owning service's **API** or through **events**. This is what allows each service to scale, evolve its schema, and fail **independently** — the entire promise of microservices. Violating it (a shared database) recreates a distributed monolith, the worst of both worlds.

1.7 What this means for the cluster (foreshadowing)

Every application decision above creates an infrastructure requirement we'll fulfil in later chapters:

Application need	Cluster capability	Chapter
Spiky on-sale traffic	HPA + KEDA + Cluster Autoscaler	16
Stateful Postgres/Kafka	StatefulSets + Rook-Ceph PVs	11, 14
Service-to-service calls	ClusterIP + Cilium + NetworkPolicy	12, 21
Isolation between teams/services	Namespaces + RBAC + ResourceQuota	9, 15, 19

Secure payments	Secrets, PSA, Falco, image signing	20, 23, 24
Zero-downtime releases	Rolling updates, PDB, Argo CD	18, 28

1.8 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- A "stateless" service pod can still hold **in-flight request state** (goroutine/thread memory) — pod loss during a request causes a 5xx to the client. This is unavoidable without client-side retry logic or a service mesh retry policy.

- The Kafka `OrderConfirmed` event is **at-least-once** delivered. Notifications must check a Redis dedup key before sending an email to prevent duplicate receipts — a subtle invariant that often gets dropped when the Notifications service is rewritten.

- The 10-minute seat-hold TTL must be enforced in three places consistently: Redis key TTL, application-level expiry check on the Orders write path, AND the UI countdown timer. Any mismatch causes ghost holds (seats held but expired) or double-booking (hold released while user still on checkout page).

Gotchas — traps that catch experienced engineers

- **Shared database anti-pattern**: connecting Orders directly to `users_db` to avoid an RPC call seems harmless but creates a hidden schema coupling. Resist it — it is the most common path back to a distributed monolith.

- **Synchronous saga orchestration** means Orders holds a DB transaction open while waiting for Inventory and Payments RPCs. Slow external calls (Stripe latency spikes) translate directly to Postgres connection exhaustion. Timeout every external RPC and compensate explicitly.

- **Missing idempotency keys on payment capture**: if the Orders pod restarts mid-saga after `Authorize()` but before writing the result, it may call `Authorize()` again on retry — resulting in a double-charge. Every payment RPC must carry a stable idempotency key derived from the `orderId`.

Architect Considerations

1. **Thundering-herd on sale open**: 50,000 users hit the Inventory service in the same second. Is Redis `SETNX` for holds safe under this load, or do you need a distributed queue (Redis Streams, Kafka) to serialize the seat-hold requests?

2. **Stripe outage strategy**: should failed payment attempts be queued in Kafka and retried asynchronously (better UX for users), or returned as 402 immediately (simpler, but worse conversion)?

3. **Eventual consistency visibility**: when a seat is held by User A, how quickly does the event page for User B show it as unavailable? A 10-second lag is acceptable for concerts; a 1-second lag is acceptable for limited edition sneakers. Define the SLO before building.

4. **Decomposition boundary review**: is a separate Payments service justified for TicketHub, or should Orders own payment capture? The boundary matters because it determines who handles Stripe webhook callbacks.

5. **Event schema versioning**: `OrderConfirmed` v1 carries `{ orderId, userId, seats[] }`. When you add `promoCode` in v2, Notifications (consuming v1) must not break. Plan Avro/Protobuf schema registry or envelope versioning from the start.

6. **Capacity model**: a single sold-out stadium event generates ~60,000 concurrent users over 5 minutes. Work backwards to per-service RPS, then to pod count, then to node count — this is the exercise that determines your HPA max replicas in Chapter 16.

Chapter 1 checklist — the architect's design outputs

- A **service catalog** with one clear responsibility per service.
- A defined **interface/contract** for each (protocol + endpoints).
- A **sync vs async** decision for every interaction.
- **Database-per-service** ownership boundaries.
- An **event flow** for the critical path (booking saga).

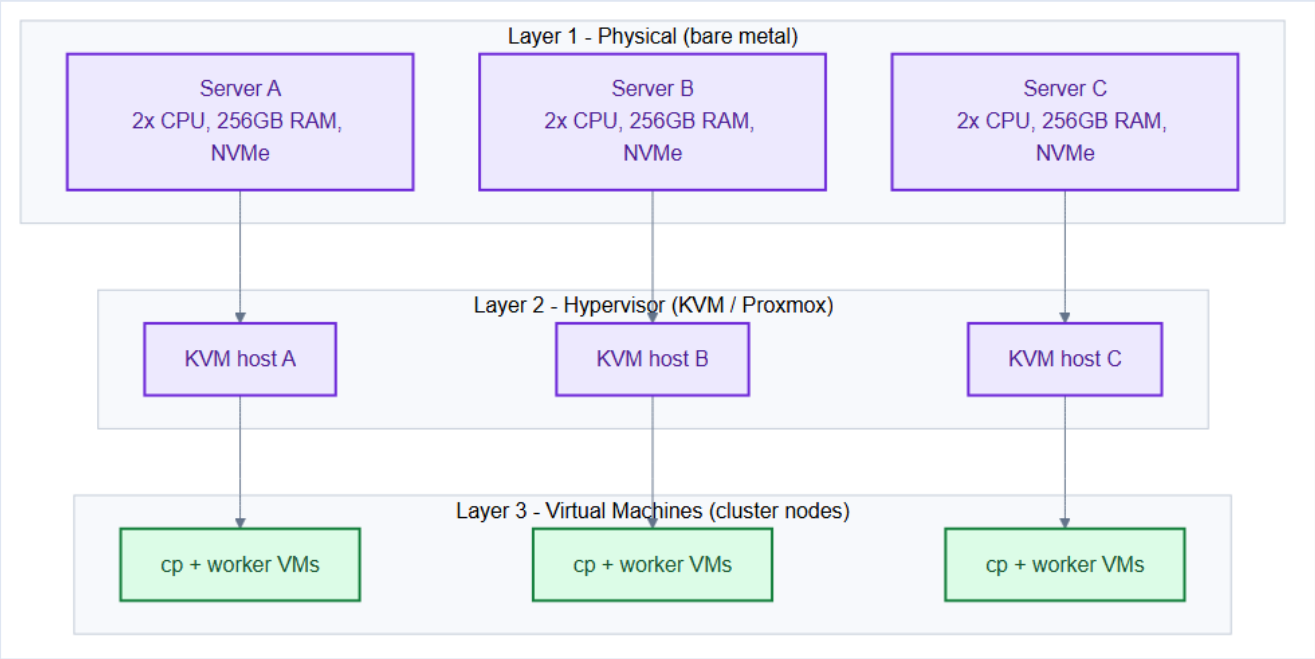
With the *what* nailed down, Part I now turns to the *where* — the physical servers and VMs the cluster will run on.

---

2. From Bare Metal to Virtual Machines

TicketHub runs in our own data center — there is no cloud "give me a node" button. As the architect, you start with **physical servers** and must turn them into the fleet of **VMs** that will become Kubernetes nodes. This chapter builds that foundation layer by layer.

2.1 The three layers



Layer	What it is	Our choice
1. Physical (bare metal)	Real servers: CPU sockets, RAM, NVMe disks, NICs	3 × dual-socket servers, 256 GB RAM, NVMe
2. Hypervisor	Software that slices one physical host into many VMs	**KVM** via **Proxmox VE**
3. Virtual machines	The isolated "computers" that become k8s nodes	12 VMs across the 3 hosts

Mental model — an apartment building

A **bare-metal server** is a plot of land. The **hypervisor** is the construction company that builds an apartment building on it. Each **VM** is an apartment — isolated, with its own allotted space (vCPU, RAM, disk), sharing the underlying plumbing (physical hardware). Kubernetes then moves its "tenants" (pods) into those apartments.

2.2 Why virtualize at all? Why not run Kubernetes on bare metal directly?

You *can* run Kubernetes directly on physical machines, but virtualization buys an architect crucial flexibility:

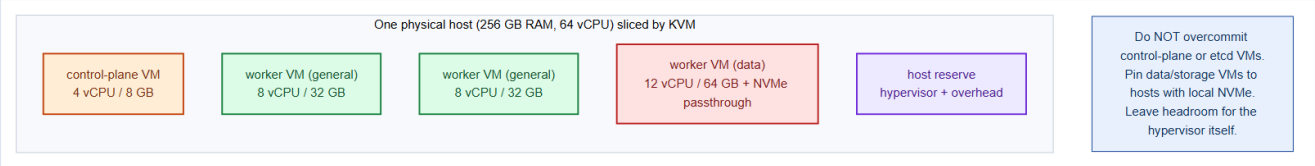
Benefit	Why it matters for TicketHub
Right-sizing	Carve a 256 GB host into differently-sized nodes (small control-plane, large data node)
Isolation	A misbehaving node VM can't take down the whole physical host
Fast rebuild	Re-provision a node VM from a template in minutes (immutable infra)
Live migration	Move a VM to another host for hardware maintenance with no downtime
Density	Run 3–4 nodes per physical host instead of wasting a whole server on one node

> **The trade-off: a small performance tax**

> Virtualization adds ~2–5% overhead and a layer to debug. For latency-critical or storage-heavy nodes you can mitigate with **CPU pinning**, **huge pages**, and **NVMe passthrough** (giving a VM direct access to a physical disk) — which we do for the **data** node pool.

2.3 Slicing one host into nodes

Here's how a single 256 GB / 64 vCPU physical host is divided into cluster-node VMs:



Sizing guidance an architect applies:

VM role	vCPU	RAM	Disk	Overcommit?
Control-plane	4	8 GB	50 GB SSD	**No** — etcd is latency-sensitive
Worker (general)	8	32 GB	100 GB	Light CPU overcommit OK

Worker (data)	12	64 GB	NVMe passthrough	**No** — storage & DB
Host reserve	—	~16 GB	—	Never allocate 100%

> **Never starve etcd, never allocate 100%**

> • **etcd** (on control-plane VMs) is extremely sensitive to disk and CPU latency. Give control-plane VMs **dedicated** (non-overcommitted) resources and fast SSD.

> • Always leave **host headroom** (RAM + CPU) for the hypervisor itself. Allocating every last GB to VMs causes swapping and cascading instability.

2.4 Spreading VMs across physical hosts (failure domains)

Which physical host each VM lands on is a critical availability decision. If all 3 control-plane VMs sat on one physical host, that host failing would **destroy etcd quorum** and take down the cluster's brain.

Failure-domain rule

Spread the **3 control-plane VMs across 3 different physical hosts**. Do the same for stateful replicas (Postgres primary/replica, Kafka brokers, Ceph OSDs). One physical host failure should cost you **at most one** member of any quorum or replica set. We'll enforce the pod-level half of this with **anti-affinity** and **topology spread** in Chapter 17.

2.5 Provisioning VMs as immutable templates

An architect doesn't hand-build 12 VMs. You build **one golden image** and clone it:

```
# On each Proxmox host: create a cloud-init Ubuntu template once...
qm create 9000 --name ubuntu-2204-template --memory 2048 --cores 2 \
 --net0 virtio,bridge=vmbr0
qm importdisk 9000 jammy-server-cloudimg-amd64.img local-nvme
qm set 9000 --scsihw virtio-scsi-pci --scsi0 local-nvme:vm-9000-disk-0
qm set 9000 --ide2 local-nvme:cloudinit --boot c --bootdisk scsi0 --serial0 socket
qm template 9000

# ...then clone a right-sized worker from it in seconds
qm clone 9000 210 --name worker-gen-1 --full
qm set 210 --cores 8 --memory 32768
qm resize 210 scsi0 100G
qm start 210
```

Immutable infrastructure

Treat node VMs as **cattle, not pets**. Never hand-patch a node in place — if it misbehaves, `kubectl drain` it, delete the VM, and clone a fresh one from the template. Everything reproducible lives in the template + `kubeadm` config + GitOps (Chapter 28). This is the foundation of a maintainable cluster.

2.6 Base OS preparation (every node)

Whatever tool provisions the VMs, every node needs the same kernel-level prep before `kubeadm` (Chapter 5). Foreshadowing the install:

```
# Kubernetes requires swap OFF
swapoff -a && sed -i '/ swap / s/^/#/' /etc/fstab

# Kernel modules + sysctls for container networking (Cilium)
modprobe overlay && modprobe br_netfilter
cat <<EOF | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.ipv4.ip_forward                 = 1
EOF
sysctl --system
```

2.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- KVM **CPU pinning** eliminates NUMA cross-traffic for memory-intensive VMs (Postgres data nodes) but reduces the hypervisor's scheduling freedom — pin only where latency matters, not cluster-wide.

- Proxmox default disk format is **qcow2** (copy-on-write, flexible but ~15% slower than raw). Switch VM disks to **raw** format on Ceph RBD-backed storage for production database VMs to eliminate the double CoW overhead.

- VM **balloon driver** (virtio-balloon) can dynamically return unused guest RAM to the hypervisor — useful for dev/staging but dangerous for Postgres nodes that use a large shared_buffers. Disable it for data-pool VMs.

Gotchas — traps that catch experienced engineers

- **Forgetting to disable swap in the VM** after provisioning: `swapoff -a` survives the session but `/etc/fstab` re-enables it on reboot, causing kubelet to refuse to start with `failed to run Kubelet: running with swap on is not supported`.

- **CPU passthrough breaks live migration**: `cpu: host` gives best performance but means you cannot live-migrate VMs to a host with a different CPU generation. Use the lowest common denominator microarch (e.g., `cpu: Cascadelake-Server`) for any VM you may need to migrate.

- **NTP drift between VMs**: the hypervisor clock is the authoritative source; each VM must sync from the KVM host, not from an external NTP server. etcd requires < 500ms clock skew between members — larger drift causes leader election instability.

Architect Considerations

1. **VM density vs performance**: a rule of thumb is allocate no more than **1.5× physical cores** in total vCPUs across all VMs on a host (avoid CPU steal for latency-sensitive workloads). What is the actual peak CPU utilization per VM in your load model?

2. **Dedicated CP hosts vs shared**: separating control-plane VMs onto dedicated physical hosts prevents a noisy application workload from starving the etcd leader, but wastes hardware. Justified for clusters > 50 nodes or SLA-critical platforms.

3. **VM count vs bare-metal workers**: every VM layer adds latency (network virtio, disk virtio). For very I/O-intensive workloads (Kafka, Ceph OSD), evaluate whether the VMs are introducing unacceptable p99 latency — consider dedicated bare-metal workers in the data pool.

4. **Recovery time objective for a lost hypervisor**: if a KVM host fails, how long does it take to bring up replacement VMs (manual re-provision vs Terraform + cloud-init)? This directly sets your node-level RTO.

5. **Ceph OSD placement**: Ceph OSDs must run on nodes where the raw block devices reside. If Ceph runs on VMs, the VMs must have RBD-backed disks that are NOT themselves backed by the same Ceph cluster (avoid circular dependency).

Chapter 2 checklist

- Physical hosts chosen and sized (3 × 256 GB/64 vCPU).

- Hypervisor (KVM/Proxmox) installed on each.
- A **golden VM template** built; nodes cloned and right-sized per role.
- VMs **spread across hosts** to protect quorums (failure domains).
- Base OS prep (swap off, modules, sysctls) baked into the template.

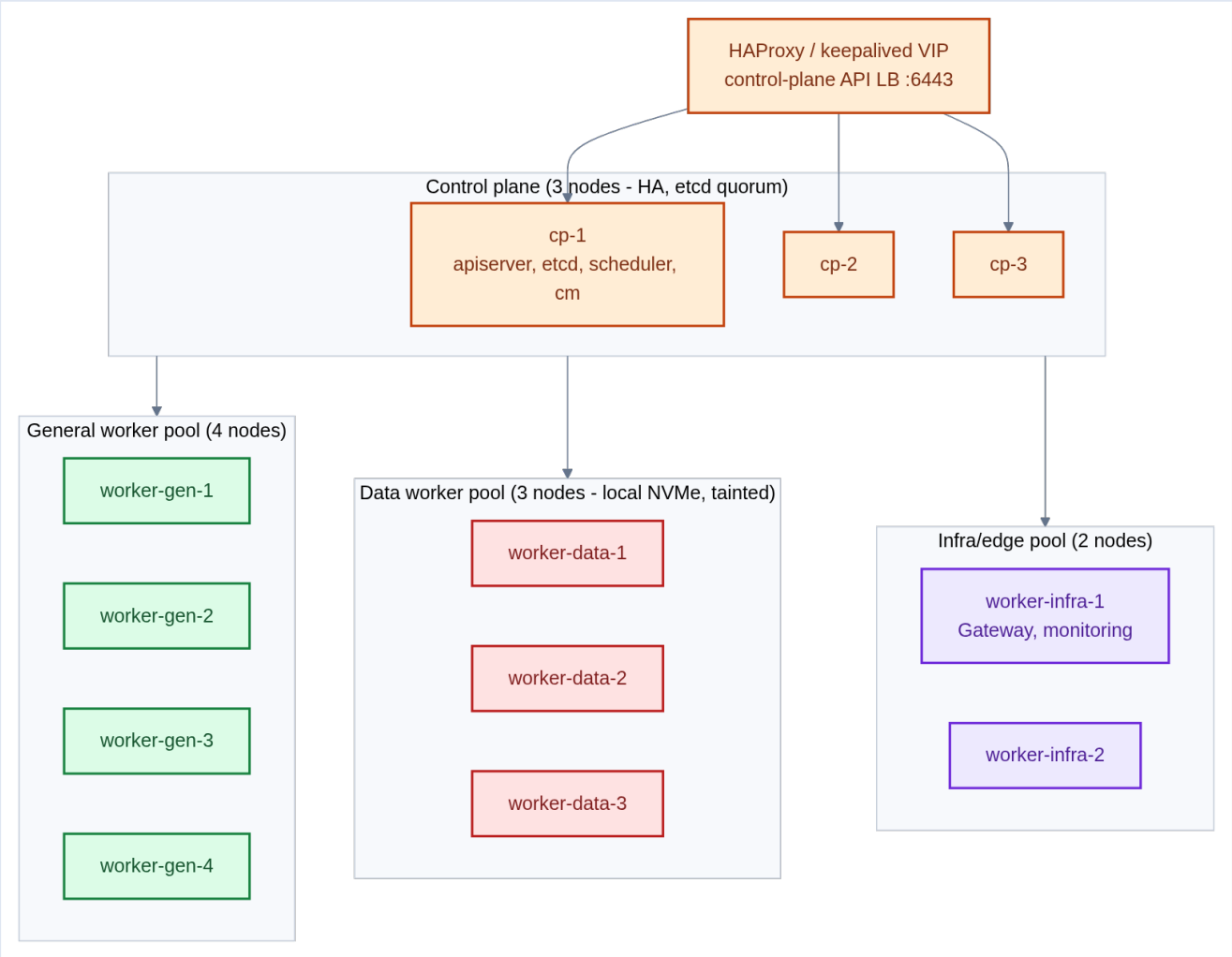
Next: how these 12 VMs are organized into a **cluster topology** of control-plane and worker pools.

---

3. Cluster Topology — Control Plane & Worker Node Design

We have 12 VMs. Now we decide **which VM does what**. The topology of a cluster — how many control-plane nodes, how workers are grouped — determines its availability, its blast radius, and how cleanly workloads are isolated.

3.1 The full TicketHub topology



12 nodes total, organized as:

Group	Count	Role	Node labels / taints
Control plane	3	apiserver, etcd, scheduler, controller-manager	control-plane role — auto-tainted `NoSchedule`
General workers	4	Stateless services (UI, gateway, orders, ...)	`pool=general`

Data workers	3	Postgres, Kafka, Redis, Ceph OSDs	`pool=data`, taint `data=true:NoSchedule`
Infra/edge workers	2	Gateway API, Prometheus, Grafana, Loki	`pool=infra`, taint `infra=true:NoSchedule`

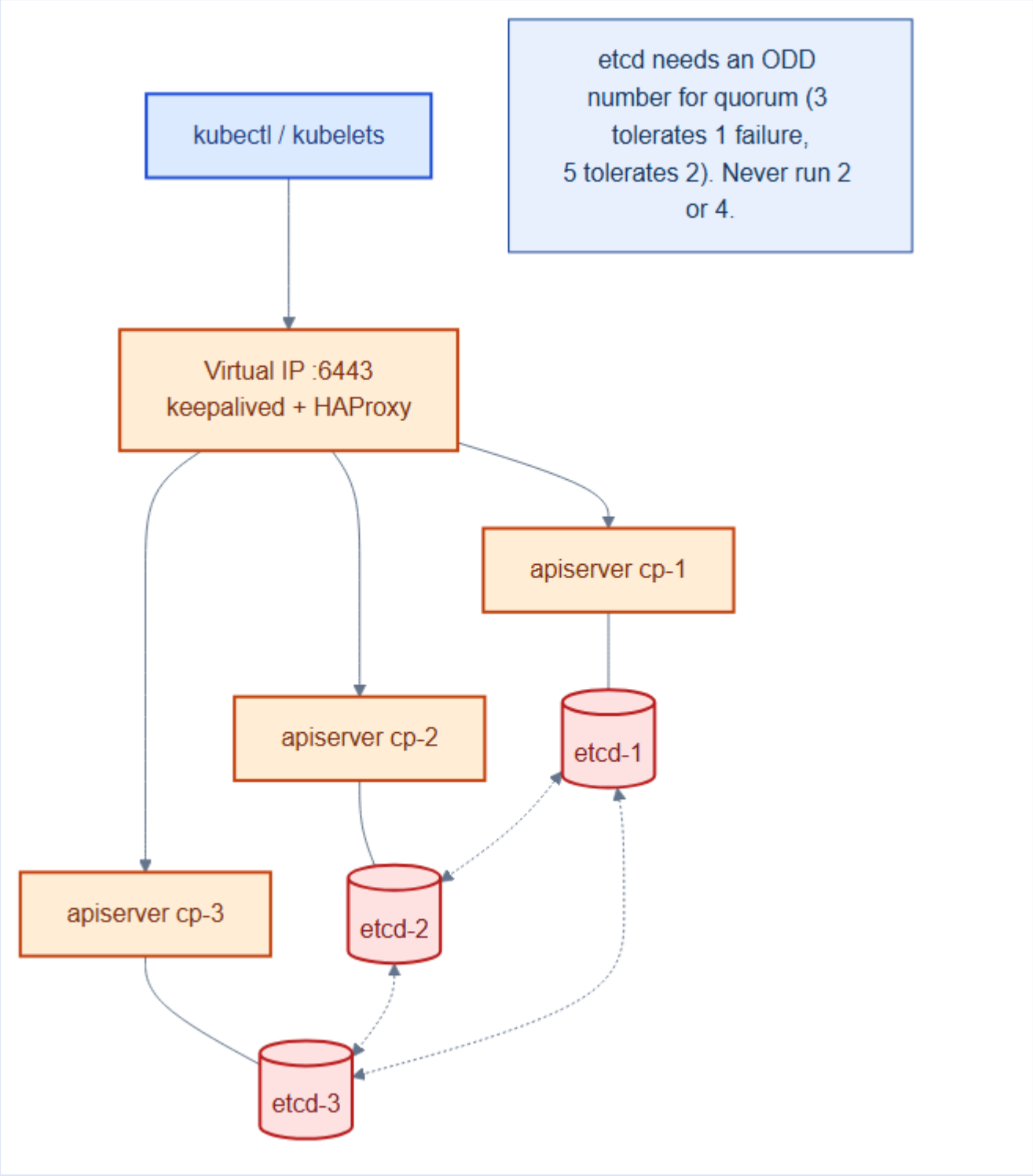
Mental model — brain, hands, vaults, doors

- **Control plane = the brain.** It makes decisions but runs no app workloads.
- **General workers = the hands.** They do the everyday stateless work.
- **Data workers = the vaults.** Guarded (tainted), they hold precious state on fast local disks.
- **Infra workers = the doors & cameras.** The Gateway (the doors) and monitoring (the cameras) live here, isolated from noisy app pods.

3.2 The control plane — what runs on it

Each control-plane node runs the components that *are* Kubernetes:

Component	Role	Notes
kube-apiserver	The single front door; all reads/writes go through it	Stateless → run 3, load-balanced
etcd	The cluster's database (all state)	**Quorum-based**, needs odd count
kube-scheduler	Assigns pods to nodes	One active (leader-elected)
kube-controller-manager	Runs control loops (Deployments, etc.)	One active (leader-elected)



> **Why 3 control-plane nodes, and why ODD**

> etcd uses the **Raft** consensus protocol, which needs a **majority (quorum)** to commit writes. With **3** members, quorum is 2 — the cluster survives **1** node failure. With **5**, it survives 2. An **even** number is pointless: 4 members still only tolerate 1 failure (quorum 3) while costing more coordination. **Rule: always 3 or 5, never 2 or 4.**

An external **virtual IP** (keepalived) fronts the three API servers via **HAProxy**, so `kubectl` and kubelets talk to one stable `:6443` endpoint regardless of which control-plane node is healthy.

```
# /etc/haproxy/haproxy.cfg (on the load-balancer VIP nodes)
frontend k8s-api
   bind *:6443
   mode tcp
   default_backend k8s-cp
backend k8s-cp
   mode tcp
   balance roundrobin
   option tcp-check
   server cp-1 10.10.0.11:6443 check
   server cp-2 10.10.0.12:6443 check
   server cp-3 10.10.0.13:6443 check
```

VIP, keepalived and Raft — the three HA words here

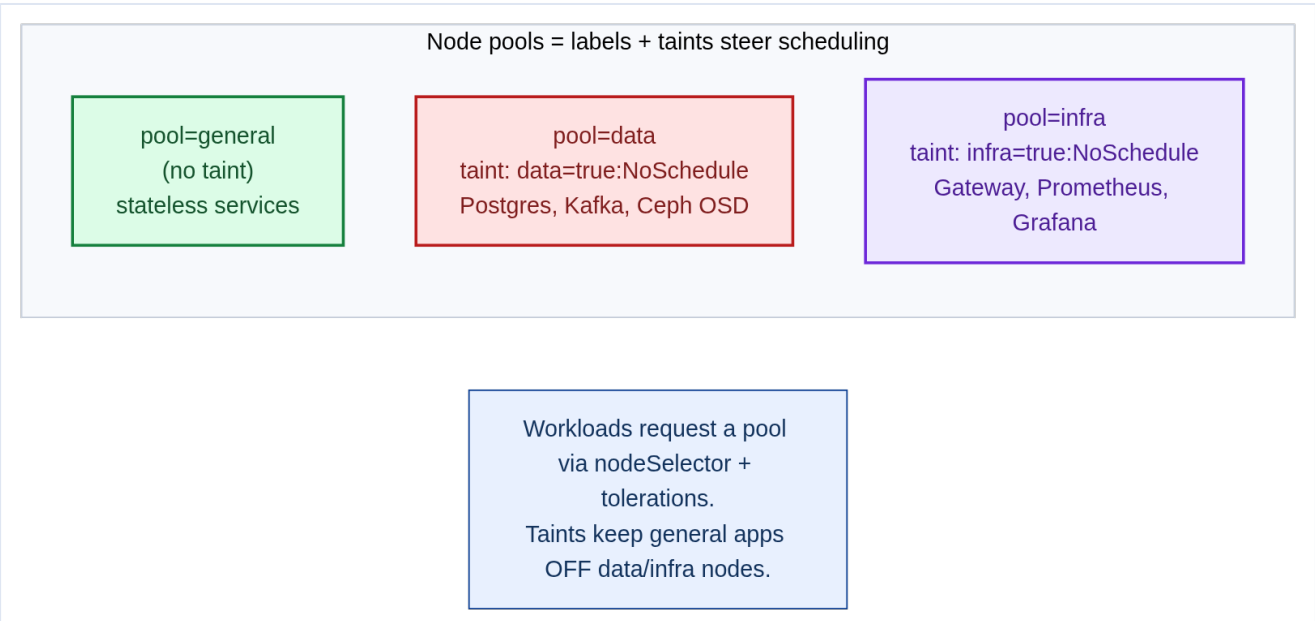
A **VIP (virtual IP)** is a single floating address clients always use. **keepalived** (via the VRRP protocol) moves it to a healthy load-balancer node if one dies, so the `:6443` endpoint never disappears. Behind it, etcd keeps its three copies in agreement with the **Raft** consensus protocol — an elected leader plus a majority **quorum** — which is exactly why the member count must be odd (see Glossary, Appendix A).

Control-plane nodes are tainted for a reason

kubeadm automatically taints control-plane nodes with `node-role.kubernetes.io/control-plane:NoSchedule` so **app pods never land on them**. Keep it that way in production — a runaway app pod must never compete with etcd for CPU or disk. (On a tiny dev cluster you might remove the taint; never in prod.)

3.3 Worker node pools — labels and taints

We don't want a batch Kafka rebalance stealing CPU from the edge Gateway, or a stateless UI pod landing on a precious NVMe data node. **Node pools** solve this using two primitives:



- **Labels** — attract pods (`nodeSelector`/affinity says "put me on `pool=data`").
- **Taints** — repel pods (`data=true:NoSchedule` says "only pods that *tolerate* this may land here").

```
# Label and taint the data pool
kubectl label node worker-data-1 worker-data-2 worker-data-3 pool=data
kubectl taint node worker-data-1 worker-data-2 worker-data-3 data=true:NoSchedule

# Label and taint the infra pool
kubectl label node worker-infra-1 worker-infra-2 pool=infra
kubectl taint node worker-infra-1 worker-infra-2 infra=true:NoSchedule
```

A pod that wants a data node must **both** select the label **and** tolerate the taint (full example in Chapter 17):

```
    nodeSelector:
      pool: data
  tolerations:
    - key: data
      operator: Equal
      value: "true"
      effect: NoSchedule
```

> **Labels attract, taints repel — you usually need both**

> A label alone doesn't stop *other* pods from landing on the node. A taint alone doesn't guide *your* pod to it. The combination — **label + nodeSelector** to attract the right pods and **taint + toleration** to exclude everyone else — gives clean, dedicated pools.

Failure domains — zone labels on bare metal. `pool` labels say *what kind* of node; **zone** labels say *which independent failure domain* it lives in — a rack, a power feed, a top-of-rack switch. On a cloud, the provider sets `topology.kubernetes.io/zone` for you. **On bare metal nobody does — you must label the nodes yourself**, or the spread constraints in Chapter 17 have nothing to spread across. Map each node to its physical rack:

```
# Standard well-known key; value = your real failure domain (rack / power feed).
kubectl label node worker-gen-1 worker-data-1 worker-infra-1 topology.kubernetes.io/zone=rack-a
kubectl label node worker-gen-2 worker-data-2 worker-infra-2 topology.kubernetes.io/zone=rack-b
kubectl label node worker-gen-3 worker-gen-4  worker-data-3  topology.kubernetes.io/zone=rack-c
```

Now a stateful set spread `topology.kubernetes.io/zone` puts one replica per rack, so losing a rack (power or switch) never takes out a quorum. Keep the mapping honest: two "zones" sharing one PDU are not independent failure domains.

3.4 Sizing the control plane vs workers

Concern	Control plane	Workers

Scales with	Cluster size (nodes, objects, API QPS)	Workload demand
Bottleneck	etcd disk latency, apiserver CPU	Pod CPU/memory
HA strategy	3–5 members, spread across hosts	Many nodes, autoscale
Grows by	Vertical (bigger CP nodes)	Horizontal (more workers)

Rule of thumb for control-plane sizing

Up to ~50 nodes / a few thousand pods, 3 control-plane nodes at 4 vCPU / 8–16 GB are plenty. Past that, scale etcd to faster disks and more RAM before adding a 4th/5th member. TicketHub at 12 nodes is comfortably in the small-cluster range.

3.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- etcd quorum requires `(n/2)+1` members available. With 3 control-plane nodes you can lose exactly **one** and still write. Losing two makes the API server read-only — no new pod scheduling, no ConfigMap updates, no Secret creation. Plan your maintenance window accordingly.

- **Zone labels on bare metal are advisory only** — nothing in Kubernetes enforces that pods assigned to `zone=rack-a` actually run on physical rack A. The labels are only honoured by topology spread constraints and affinity rules you configure. If you mislabel, you silently lose the HA guarantee.

- The `infra` pool taint means DaemonSets for platform components (Falco, node-exporter, Cilium agent) must carry the matching toleration or they will not run on infra nodes — leaving them unmonitored. Always audit DaemonSet tolerations when adding a new taint.

Gotchas — traps that catch experienced engineers

- **Adding zone labels after workloads are running** does not re-balance existing pods. You must delete and recreate (or drain-and-uncordon) the pods to trigger rescheduling with topology spread constraints applied.

- **Forgetting to taint data nodes** before deploying application workloads: without the taint, a Deployment's pods may be scheduled onto data nodes, competing for CPU/RAM with Postgres and Kafka.

- **etcd on shared disk**: etcd is extremely sensitive to disk I/O latency. Running etcd on the same NVMe as application workloads causes sporadic leader elections and `etcdserver: request timed out` errors. Always dedicate a disk or partition exclusively to `/var/lib/etcd`.

Architect Considerations

1. **Scale ceiling**: the data pool has 3 nodes. Postgres (3 pods) + Kafka (3 pods) already fills the pool with minimal headroom. At what point does adding a Redis cluster, a second Postgres instance for a new service, or Elasticsearch require a pool expansion plan?

2. **Control plane upgrade strategy**: with 3 control-plane nodes you must drain one at a time, leaving the etcd cluster at 2/3 quorum during upgrade. Plan the maintenance window so you never start upgrading the second node while the first is still upgrading.

3. **Worker pool segmentation trade-offs**: strict `NoSchedule` taints on data/infra pools mean a traffic spike on the general pool cannot borrow capacity from infra nodes. Is this acceptable, or should infra nodes have `PreferNoSchedule` to allow emergency overflow?

4. **Node labels for feature detection**: beyond `pool=` labels, consider labelling nodes with hardware features (`nvidia.com/gpu`, `storage.ssd=nvme`) so workloads can request specific hardware via `nodeSelector` without hard-coding hostnames.

5. **Multi-rack failure domains**: if racks share a single top-of-rack switch, a switch failure is a zone failure. Verify physical network redundancy matches your logical zone model — otherwise `topology.kubernetes.io/zone` labels overstate HA.

Chapter 3 checklist

- **3 control-plane** nodes (odd, HA), spread across physical hosts, **tainted**.
- An external **VIP + HAProxy** fronting the API servers.
- Worker **pools** (general / data / infra) defined with **labels + taints**.
- Nodes labeled with **`topology.kubernetes.io/zone`** by physical rack (bare metal has no auto-zoning).
- Sizing plan: control plane scales **up**, workers scale **out**.

Next: the **network** that stitches these nodes — and their pods — together.

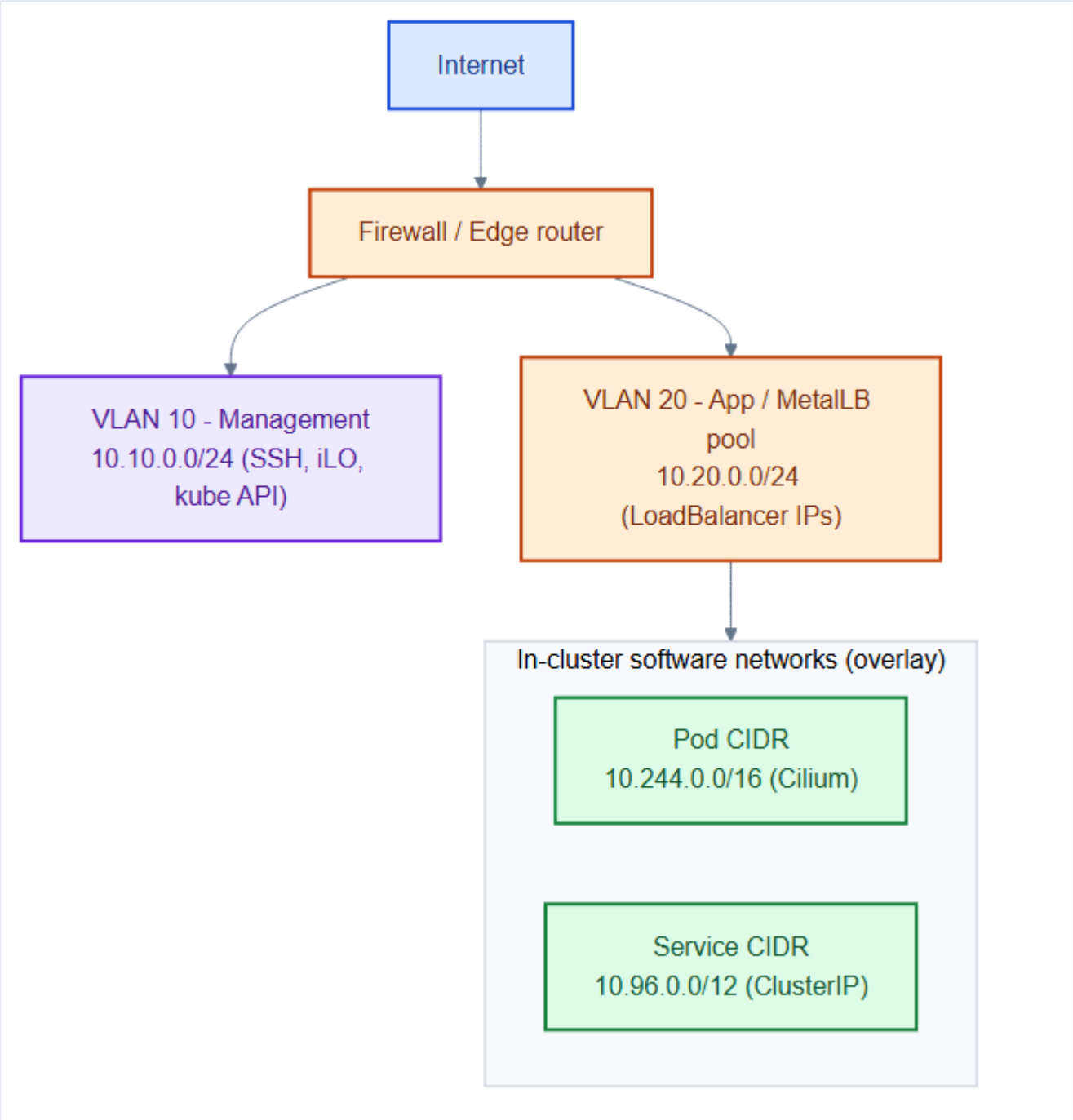
---

4. Network Design — Subnets, CIDRs, North-South & East-West

Networking is where most bare-metal Kubernetes projects stumble. Unlike a cloud, nobody hands you load balancers, routable pod networks, or DNS. As the architect you must **plan every IP range** and understand the two directions traffic flows: **North-South** (in/out of the cluster) and **East-West** (pod to pod).

4.1 The four networks in play

A Kubernetes cluster juggles **four distinct address spaces**. Confusing them is the #1 source of "my pods can't talk" incidents:



Network	Example CIDR	Who lives here	Routable outside cluster?
Node / management	`10.10.0.0/24`	VM NICs, SSH, API `:6443`	Yes (physical VLAN)
MetalLB pool	`10.20.0.0/24`	External IPs for `LoadBalancer` services	Yes (physical VLAN)
Pod CIDR	`10.244.0.0/16`	Every pod gets an IP here	No (Cilium overlay)

Service CIDR	`10.96.0.0/12`	Virtual `ClusterIP`s	No (virtual, kube-proxy/eBPF)

The golden rule of cluster CIDRs

The **Pod CIDR** and **Service CIDR** must **not overlap** with each other or with your **physical/VLAN** ranges. An overlap causes silent, maddening routing failures. Write the IP plan down **before** installing, and pick private ranges that are clearly distinct from your data-center subnets.

4.2 The IP plan (write this before installing)

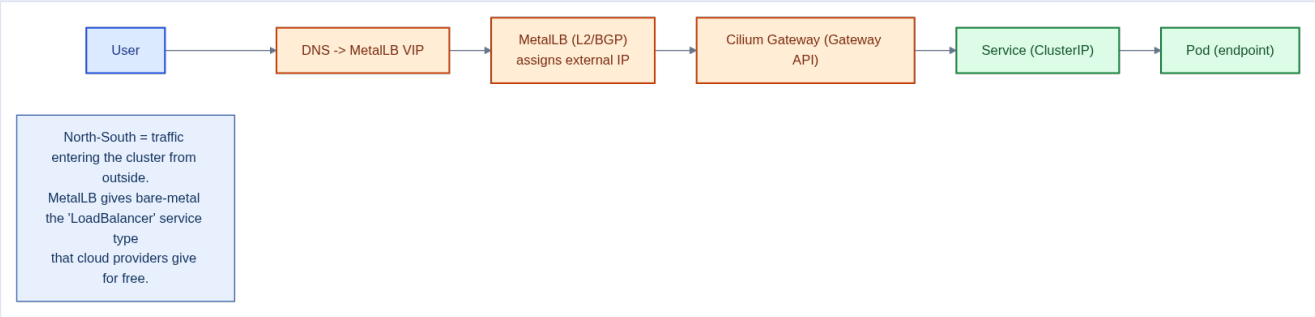
Purpose	Range	Notes
Control-plane VMs	`10.10.0.11–13`	cp-1..3
General workers	`10.10.0.21–24`	worker-gen-1..4
Data workers	`10.10.0.31–33`	worker-data-1..3
Infra workers	`10.10.0.41–42`	worker-infra-1..2
API VIP (keepalived)	`10.10.0.10`	HAProxy front
MetalLB address pool	`10.20.0.100–200`	Gateway + any LB services
Pod CIDR	`10.244.0.0/16`	`kubeadm --pod-network-cidr`
Service CIDR	`10.96.0.0/12`	`kubeadm --service-cidr` (default)

VLAN separation

Put **management** traffic (VLAN 10) and **application/LB** traffic (VLAN 20) on separate VLANs. You don't want user traffic hitting the LoadBalancer pool to share a broadcast domain with etcd/SSH management. This is basic data-center hygiene that also limits blast radius.

4.3 North-South traffic — getting users *into* the cluster

North-South is traffic crossing the cluster boundary — a user's browser reaching TicketHub. On bare metal this is the part the cloud normally does for you, so we assemble it from **MetalLB + the Gateway API**:



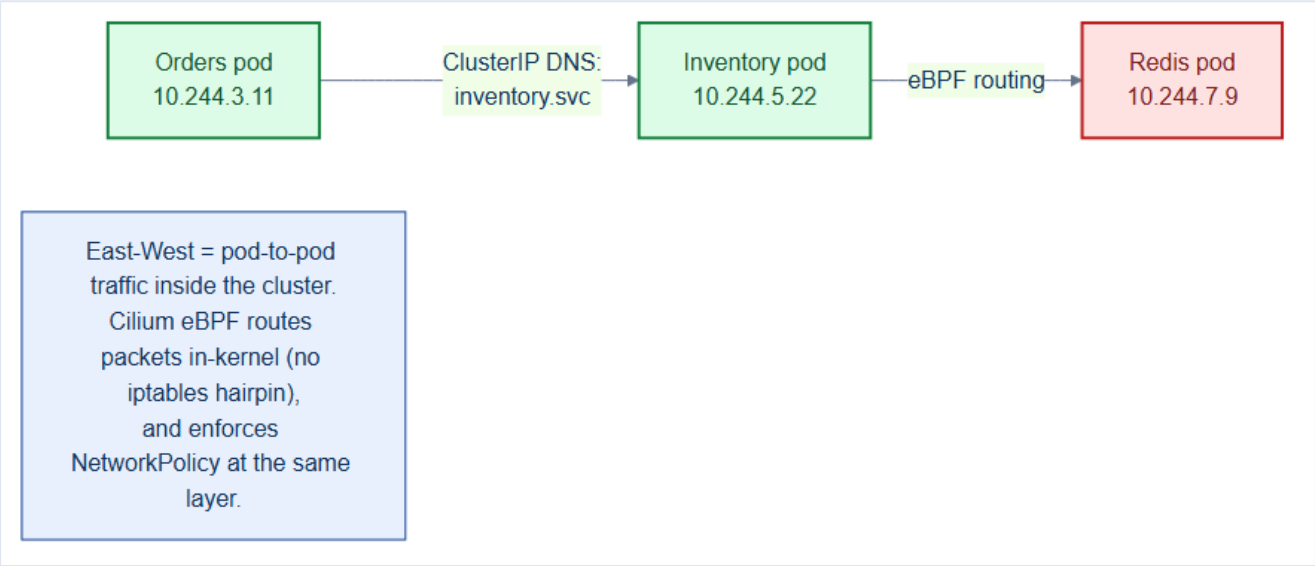
1. **DNS** points `tickethub.com` at a MetalLB external IP (from the `10.20.0.0/24` pool).
2. **MetalLB** makes `Service type=LoadBalancer` actually work on bare metal by announcing that IP via **L2 (ARP)** or **BGP** to your router.
3. The **Gateway** (Cilium's Gateway API implementation) receives the traffic and does host/path routing (`/api → gateway`, `/ → frontend`), TLS termination, etc.
4. It forwards to the target **Service (ClusterIP)**, which lands on a healthy **Pod**.

> **L2/ARP vs BGP, briefly**
>
> To make an external IP reachable, MetalLB must *advertise* it to the physical network. **L2 mode** answers **ARP** (the LAN's "who has this IP?" broadcast) from a single elected node — simple, but all traffic funnels through that one node. **BGP mode** peers with your router using the **BGP** routing protocol so several nodes serve the IP at once (true load-sharing). L2 for simplicity, BGP for scale.

> **Mental model — airport arrivals**
>
> North-South is the **arrivals hall** of an airport. **MetalLB** is the runway that lets planes land at all (a public gate/IP). The **Gateway** is passport control and the signage that routes each traveler to the right terminal (service). Without MetalLB, planes have nowhere to land; without the Gateway, travelers wander the tarmac.

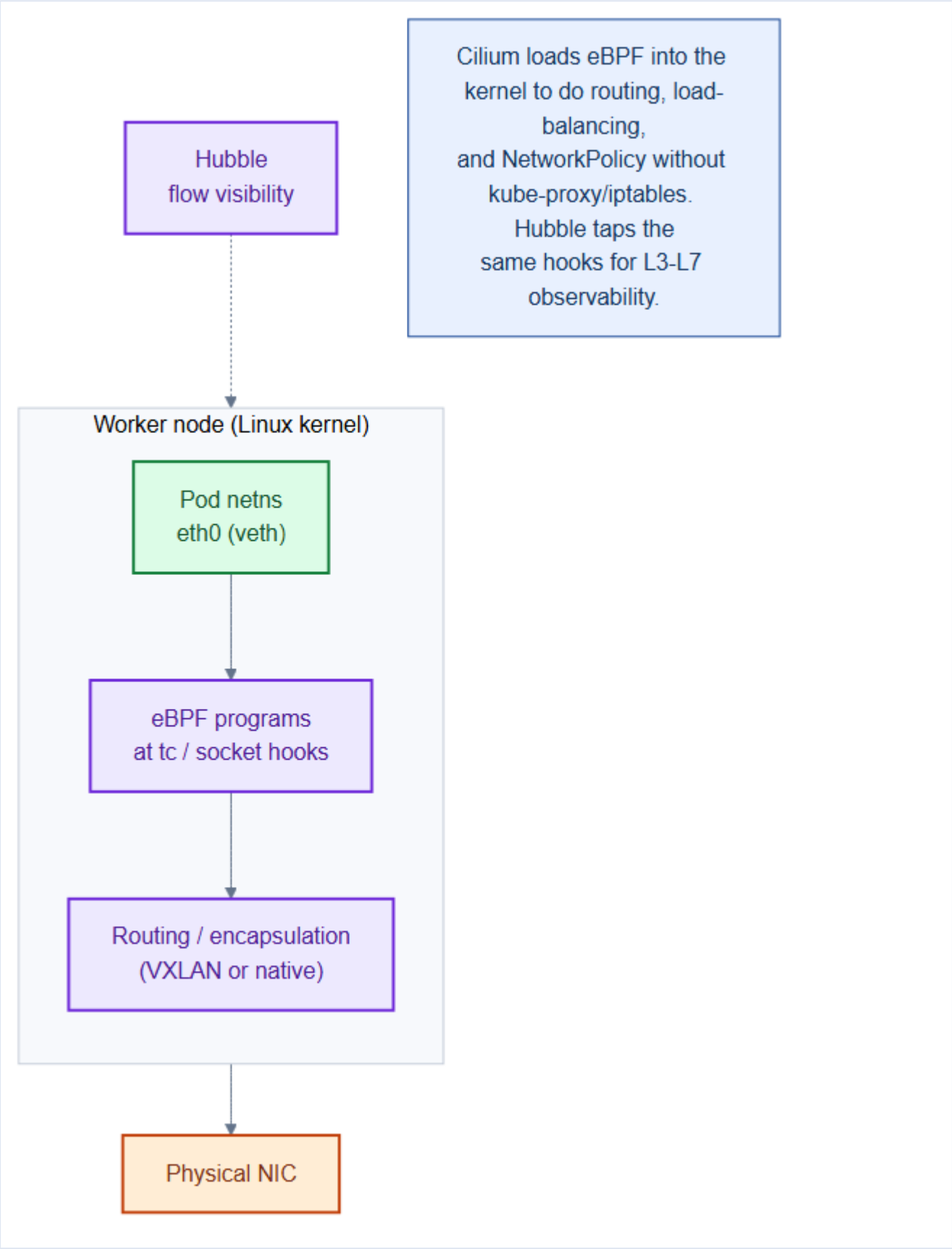
4.4 East-West traffic — pods talking to each other

East-West is the far larger volume: Orders calling Inventory, everything hitting Redis. Every pod gets a **routable-within-the-cluster IP** from the Pod CIDR, and reaches others via **Service DNS**:



- A pod calls `inventory.tickethub.svc.cluster.local` — CoreDNS resolves it to the Inventory **ClusterIP**.
- **Cilium** (our CNI) programs the kernel (via **eBPF**) to route the packet straight to a backend pod, load-balancing across replicas — **without** the traditional `iptables` hairpin that kube-proxy uses.

4.5 How Cilium moves the packets (the data path)



Cilium attaches **eBPF programs** to kernel hooks so that routing, service load-balancing, and **NetworkPolicy enforcement** all happen in-kernel at the same layer. Benefits for TicketHub:

Capability	Why it matters
eBPF routing	Faster than iptables at scale; no giant rule chains
kube-proxy replacement	Cilium can *be* the service proxy — fewer moving parts
NetworkPolicy (L3–L7)	Zero-trust between services (Chapter 21)
Hubble	Live flow maps — *see* every Orders→Inventory call

```
# Foreshadowing Chapter 6 — Cilium is installed with these CIDRs
cilium install \
 --set ipam.mode=cluster-pool \
 --set ipam.operator.clusterPoolIPv4PodCIDRList=10.244.0.0/16 \
 --set kubeProxyReplacement=true \
 --set hubble.relay.enabled=true --set hubble.ui.enabled=true
```

4.6 Cluster DNS

Inside the cluster, **CoreDNS** resolves service names. Every Service gets a stable DNS name:

```
<service>.<namespace>.svc.cluster.local
 inventory.tickethub.svc.cluster.local  -> Inventory ClusterIP
 postgres-primary.data.svc.cluster.local -> Postgres StatefulSet pod
```

Bare-metal gotchas to plan for now

- **MetalLB L2 mode** funnels all traffic for an IP through **one** node at a time (failover, not load-share). Use **BGP mode** with a capable router for true multi-node load distribution.
- `hostNetwork` **pods** bypass the Pod CIDR and grab node ports directly — use sparingly (e.g., the edge Gateway data plane) and track those ports.
- Keep the **MetalLB pool** comfortably larger than your expected number of `LoadBalancer` services so you never run out of external IPs.

4.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- Pod CIDRs (`10.244.0.0/16`) and Service CIDRs (`10.96.0.0/12`) must **never overlap** with each other or with the node network (`10.10.0.0/16`). Cilium allocates a `/24` per node from the pod CIDR — with `/16` you can have up to 256 nodes before you need a larger CIDR (plan for growth from day one).

- **DNS round-robin for Services is not load balancing** — it is address discovery. Cilium's eBPF does the actual per-connection load balancing at the kernel level, not at the DNS layer. This means long-lived gRPC streams to a Service IP may stay on a single backend pod until the connection is closed.

- Node-to-node traffic uses the **node network CIDR** (`10.10.0.0/16`), not the pod CIDR. Firewall rules between nodes must allow the full pod CIDR range (for pod-to-pod across nodes) AND the Service CIDR (for return traffic through ClusterIP virtual IPs).

Gotchas — traps that catch experienced engineers

- **Picking a pod CIDR that overlaps with a future on-prem subnet**: once a cluster is bootstrapped you cannot change the pod or service CIDRs without rebuilding. Reserve a block of RFC 1918 address space (e.g., `100.64.0.0/10`, CGNAT range) that will never appear in your corporate network.

- **kube-dns / CoreDNS hardcoded to `10.96.0.10`**: if you choose a non-standard service CIDR, the CoreDNS ClusterIP will be different — update all references, including the kubelet `--cluster-dns` flag in the kubeadm config, or DNS resolution fails cluster-wide.

- **MetalLB pool overlapping with node IPs**: MetalLB hands out IPs from `10.10.0.200-250` as LoadBalancer Service IPs. If a new server is assigned an IP in that range, ARP conflicts will cause intermittent routing failures. Document the split in your IP address management (IPAM) system and enforce it.

Architect Considerations

1. **Address space future-proofing**: `10.244.0.0/16` gives 65,536 pod IPs. With 256 nodes × 110 pods each = 28,160 pods max. A `/15` gives twice the room; a `/14` four times. Choose based on your 3-year node growth forecast, not your current node count.

2. **East-West encryption**: should all pod-to-pod traffic be encrypted (WireGuard overlay in Cilium) or only traffic crossing a trust boundary? Encryption adds ~5% CPU overhead. For TicketHub's on-prem cluster where physical network access is controlled, selective encryption (gateway ↔ payments) may suffice.

3. **Egress NAT design**: pods use the node IP as the SNAT address for outbound traffic. If Payments calls Stripe from any of 9 worker IPs, Stripe must whitelist all 9. Consider a dedicated egress IP (Cilium EgressGateway) for external API calls from specific namespaces.

4. **IPv6 dual-stack readiness**: Cilium supports dual-stack. If your data center is moving toward IPv6, plan the pod and service CIDRs to include `fd00::/112` ranges from the start — retrofitting IPv6 post-launch is expensive.

5. **BGP vs ARP for MetalLB**: `L2 ARP` mode is simpler but has a single-node failure window (the node holding the ARP entry). `BGP` mode distributes the announcement but requires a BGP router in your rack. Choose based on your network team's capabilities.

Chapter 4 checklist — the network blueprint

- A written **IP plan**: node, MetalLB, Pod, and Service ranges that **don't overlap**.
- **VLAN separation** of management vs application traffic.
- **North-South** path designed: DNS → MetalLB → Cilium Gateway (Gateway API) → Service → Pod.
- **East-West** path understood: Pod IP + Service DNS, routed by **Cilium eBPF**.
- **CoreDNS** naming convention known by every service.

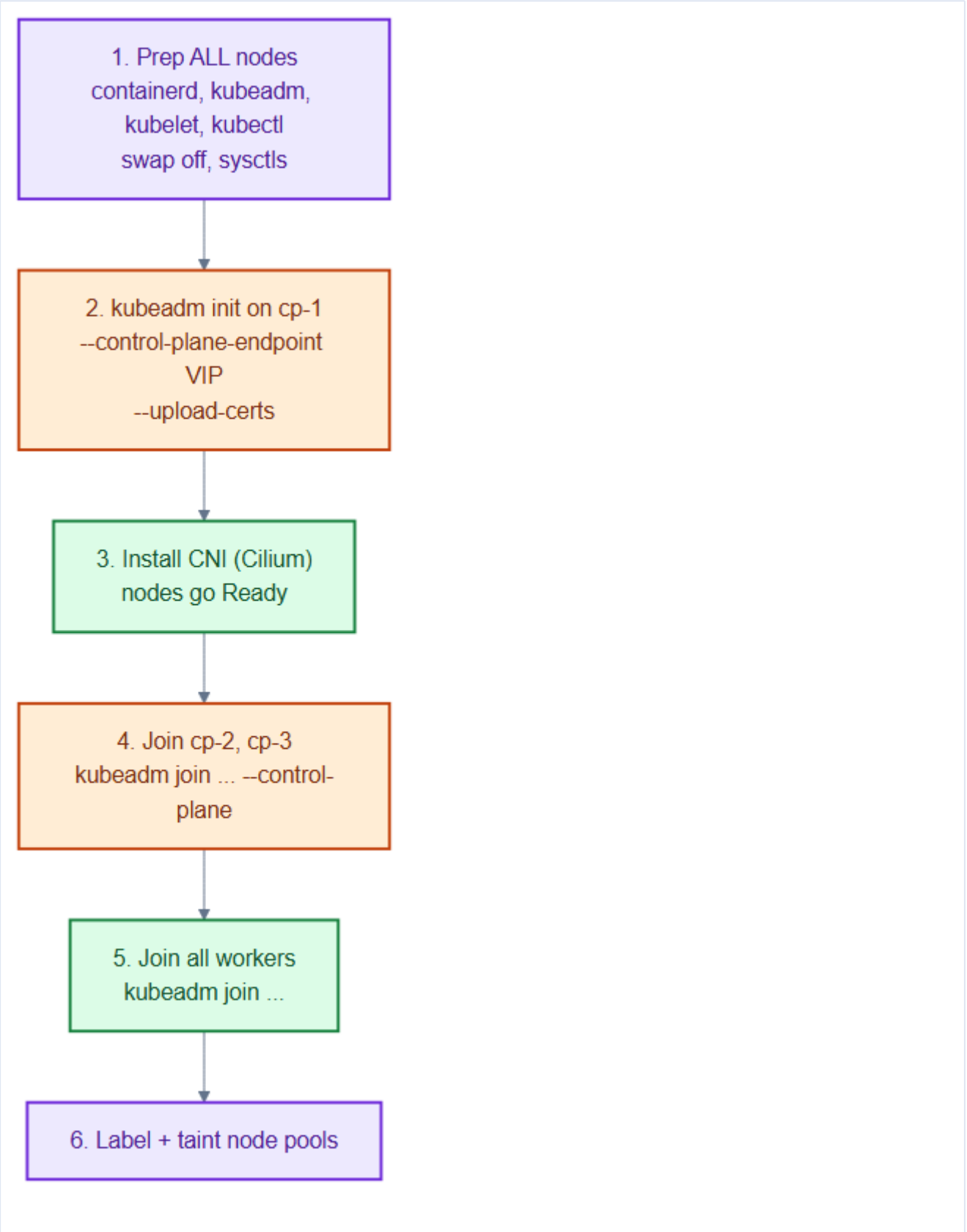
Part I is complete — we know **what** we're building (Ch 1) and **where** it runs (Ch 2–4). Part II installs Kubernetes onto this foundation.

---

5. Installing Kubernetes with kubeadm (HA Control Plane)

Part I gave us 12 prepared VMs and a network plan. Now we turn them into an actual Kubernetes cluster using **kubeadm** — the official, vendor-neutral bootstrapping tool. We build a **highly available** control plane from the start, because retrofitting HA later is painful.

5.1 The install, end to end



The sequence never changes: prep every node → `init` the first control-plane → install a CNI → join the other control-plane nodes → join workers → label/taint pools.

> **Mental model — founding a company**

> `kubeadm init` on cp-1 is **incorporating the company**: it creates the official seal (the cluster **CA**), the headquarters (etcd + apiserver), and issues a **join token** — like an employee badge template. Every other node "joins" by presenting that token and getting badges (certs) signed by the same CA.

5.2 Step 0 — container runtime on every node

Kubernetes doesn't run containers itself; it delegates to a **CRI runtime**. We use **containerd**:

```
# On EVERY node
apt-get update && apt-get install -y containerd
containerd config default | tee /etc/containerd/config.toml
# Use the systemd cgroup driver (must match kubelet)
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
systemctl restart containerd && systemctl enable containerd

# Install the kube tooling (pin the version!)
apt-get install -y kubelet=1.30.* kubeadm=1.30.* kubectl=1.30.*
apt-mark hold kubelet kubeadm kubectl
```

> **The cgroup driver must match**

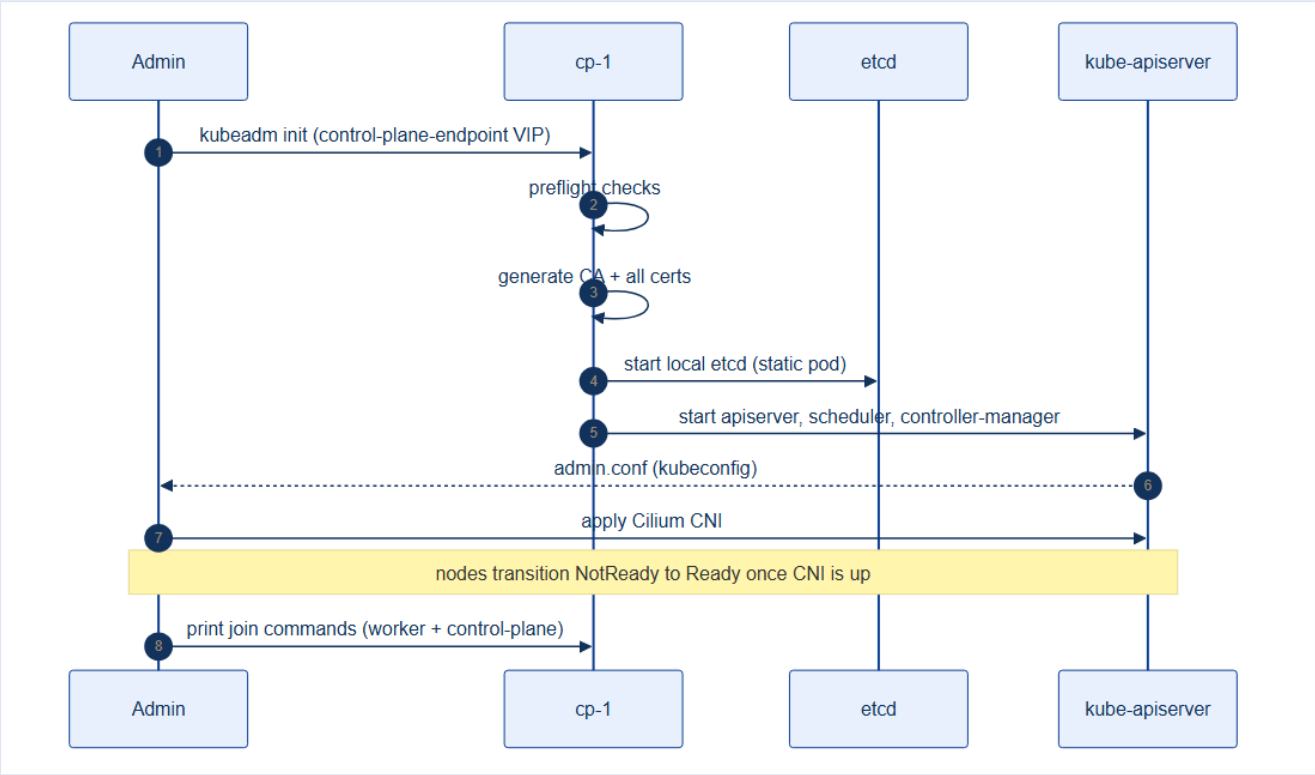
> `containerd` and `kubelet` must use the **same** cgroup driver (`systemd`). A mismatch causes kubelet to fail with cryptic errors and pods stuck in `CreateContainerError`. This is one of the most common kubeadm install failures.

5.3 Step 1 — initialize the first control-plane node

The critical flag is `--control-plane-endpoint`, pointing at the **VIP** (from Chapter 3), not cp-1's own IP. This is what makes HA possible.

```
# On cp-1
kubeadm init \
 --control-plane-endpoint "10.10.0.10:6443" \  # the HAProxy VIP
 --upload-certs \                                # share CA certs for other CP joins
 --pod-network-cidr "10.244.0.0/16" \            # matches Cilium (Ch 4)
 --service-cidr "10.96.0.0/12" \
 --skip-phases=addon/kube-proxy                  # Cilium replaces kube-proxy
```

What happens under the hood:



kubeadm runs preflight checks, generates the **cluster CA** and all component certificates (the cluster is its own **PKI** — a private certificate authority that signs an identity for every component and node; see the Chapter 0 primer, the Glossary, and **Chapter 7A** for the full certificate inventory, HA SANs, and renewal), starts etcd and the control-plane components as **static pods** (pods the kubelet runs directly from a file, not via the API server — Chapter 0), and prints the **join commands**.

```
# Set up kubectl access
mkdir -p $HOME/.kube && cp /etc/kubernetes/admin.conf $HOME/.kube/config
kubectl get nodes    # cp-1 shows NotReady — no CNI yet, that's expected
```

5.4 Step 2 — install the CNI (nodes go Ready)

Nodes stay `NotReady` until a network plugin is installed. We install **Cilium** (full detail in Chapter 6):

```
cilium install --version 1.16.1 \
 --set kubeProxyReplacement=true \
 --set ipam.mode=cluster-pool \
 --set ipam.operator.clusterPoolIPv4PodCIDRList=10.244.0.0/16
cilium status --wait
kubectl get nodes    # cp-1 now Ready
```

5.5 Step 3 — join the other control-plane nodes

```
# On cp-2 and cp-3 — note the --control-plane flag and certificate-key
kubeadm join 10.10.0.10:6443 \
 --token <token> \
 --discovery-token-ca-cert-hash sha256:<hash> \
 --control-plane \
 --certificate-key <cert-key>
```

Now all three API servers sit behind the VIP; etcd forms a 3-member quorum.

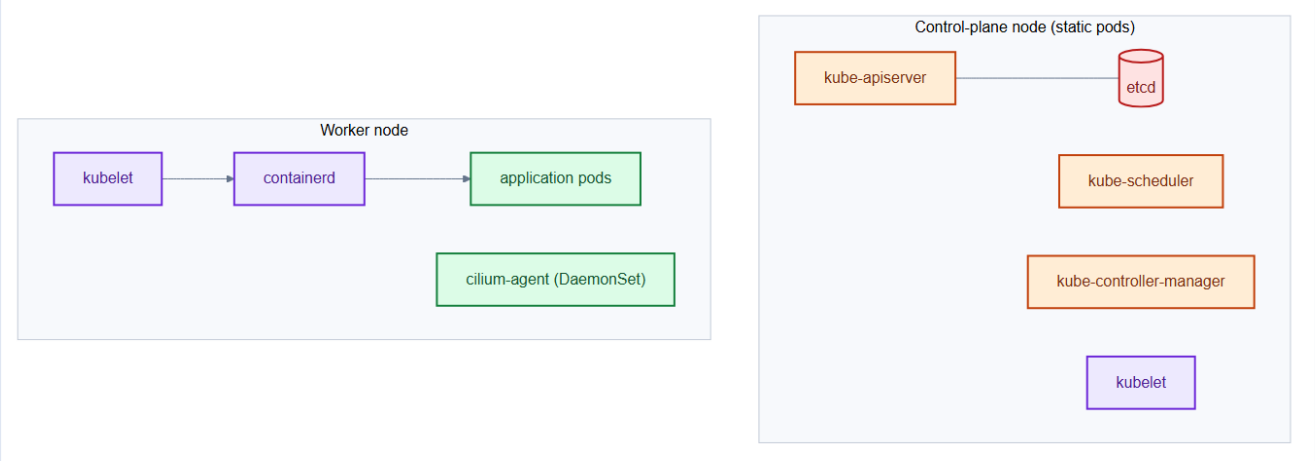
5.6 Step 4 — join the workers, then label & taint

```
# On each worker
kubeadm join 10.10.0.10:6443 \
 --token <token> \
 --discovery-token-ca-cert-hash sha256:<hash>
```

```
# From cp-1 — organize the pools (Chapter 3)
kubectl label node worker-gen-{1..4} pool=general
kubectl label node worker-data-{1..3} pool=data
kubectl taint node worker-data-{1..3} data=true:NoSchedule
kubectl label node worker-infra-{1..2} pool=infra
kubectl taint node worker-infra-{1..2} infra=true:NoSchedule

# Zone = physical failure domain (rack). No cloud provider sets this on bare
# metal — do it by hand so topology spreads work (Chapter 3, 17).
kubectl label node worker-gen-1 worker-data-1 worker-infra-1 topology.kubernetes.io/zone=rack-a
kubectl label node worker-gen-2 worker-data-2 worker-infra-2 topology.kubernetes.io/zone=rack-b
kubectl label node worker-gen-3 worker-gen-4  worker-data-3  topology.kubernetes.io/zone=rack-c
```

After install, here's what runs where:



Join tokens expire — regenerate on demand

The bootstrap token expires after 24h. To add a node later:

```
    kubeadm token create --print-join-command
```

For a new **control-plane** node you also re-upload certs:

```
    kubeadm init phase upload-certs --upload-certs
```

5.7 Verifying a healthy cluster

```
kubectl get nodes -o wide                 # all 12 Ready
kubectl get pods -n kube-system           # control-plane static pods + cilium
kubectl get --raw='/readyz?verbose'       # apiserver health
kubectl -n kube-system exec etcd-cp-1 -- etcdctl endpoint health --cluster
```

Architect's HA install principles

- Always `init` with `--control-plane-endpoint` = **VIP**, even for a single CP you plan to grow. You cannot add it cleanly afterwards.
- **Pin** kubelet/kubeadm/kubectl versions and `apt-mark hold` them — accidental upgrades break clusters.
- Match the **cgroup driver** across containerd and kubelet.
- Keep the kubeadm config and join process **scripted/GitOps'd** so any node is reproducible (immutable infra from Chapter 2).

5.7 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- `kubeadm init --upload-certs` stores the CA private key in a Secret in `kube-system` encrypted with a per-run key and **automatically expires after 2 hours**. If the second CP node join happens after 2 hours, the `--certificate-key` will no longer work — regenerate with `kubeadm init phase upload-certs --upload-certs`.
- **Static pods bypass the scheduler**: etcd, kube-apiserver, kube-controller-manager, and kube-scheduler run as static pods managed by kubelet directly from `/etc/kubernetes/manifests/`. They cannot be managed with `kubectl delete pod` — deleting the static pod manifest file is the only way to stop them.
- The `--skip-phases=addon/kube-proxy` flag during `init` leaves the cluster without ANY service routing until Cilium is installed. This means the init job completes successfully but `kubectl get nodes` may show `NotReady` even for the first CP node — that is expected.

Gotchas — traps that catch experienced engineers

- **Pinning kubeadm/kubelet versions**: `apt-mark hold` is essential. An unintended `apt upgrade` that bumps kubelet to a newer minor version than the kube-apiserver violates the version skew policy and can break the node.
- **Forgetting `--control-plane-endpoint` at init time**: you cannot add this flag post-installation. If you init with `--apiserver-advertise-address` (single IP) instead of a VIP, joining additional CP

nodes later will require a kubeadm upgrade + cert regeneration — painful.

- **Certificate SANs**: `kubeadm init` auto-includes the CP node IP and hostname in the apiserver cert SANs, but NOT the VIP if you add the load balancer later. Always pass `--apiserver-cert-extra-sans=<VIP>` at init time, or regenerate the apiserver cert afterward with `kubeadm init phase certs apiserver`.

Architect Considerations

1. **Bootstrap token security**: the join token printed by `kubeadm init` is valid for 24 hours and grants unauthenticated join capability. Rotate it (`kubeadm token create`) immediately after all nodes have joined, and restrict token creation permissions in RBAC.

2. **etcd topology — stacked vs external**: kubeadm defaults to stacked etcd (etcd co-located on CP nodes). External etcd (separate VMs) gives stronger isolation and allows etcd to be upgraded independently, but adds 3+ VMs to manage. For a 12-node cluster, stacked is adequate; for 50+ nodes, consider external.

3. **kubeadm config file vs flags**: all the `--flags` above should be committed to a `ClusterConfiguration` YAML (`repo/cluster/kubeadm-config.yaml`) and checked into git. Never run kubeadm with flags from memory — the config file IS your cluster's source of truth.

4. **Certificate rotation policy**: by default, kubelet rotates its client certificates automatically. The kube-apiserver serving cert must be manually renewed annually (`kubeadm certs renew`). Add a PrometheusRule alert for cert expiry < 30 days (Chapter 27 covers this).

5. **Disaster recovery with etcd snapshots**: the cluster is recoverable from etcd only if you have a recent snapshot AND the CA key. Test your etcd restore procedure against a clone cluster before the first production incident.

Chapter 5 checklist

- containerd + pinned kube tools on all nodes; swap off; sysctls set.
- `kubeadm init` on cp-1 with the **VIP** endpoint and correct CIDRs.
- CNI installed → nodes **Ready**.
- cp-2/cp-3 and all workers **joined**; pools **labeled + tainted**.
- Cluster health verified (nodes, etcd quorum, apiserver readyz).

---

6. The CNI — Cilium (eBPF) + Hubble

A freshly `init`ed cluster has no pod networking until you install a **CNI (Container Network Interface)** plugin. The CNI is arguably the most consequential platform choice an architect makes — it determines how pods get IPs, how Services are load-balanced, how NetworkPolicy is enforced, and what observability you get. We chose **Cilium**.

6.1 What a CNI actually does

When the scheduler places a pod on a node, the kubelet calls the CNI plugin to:

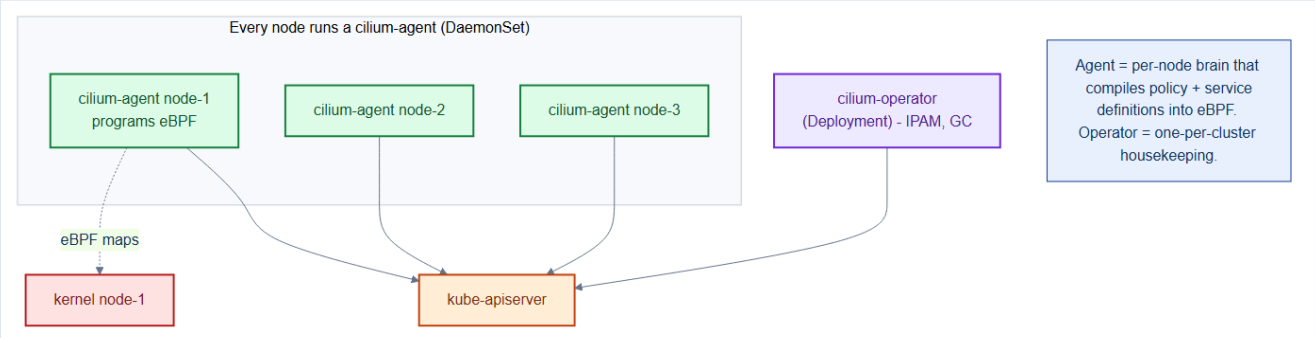
1. Create the pod's network namespace and a **veth** pair.
2. Assign the pod an **IP** from the node's slice of the Pod CIDR (**IPAM**).
3. Program the **routes** so the pod can reach other pods and Services.
4. Enforce **NetworkPolicy** for traffic in/out of the pod.

Traditional CNIs (e.g., Flannel + kube-proxy) do steps 3–4 with **iptables**, which becomes a giant, slow rule-chain at scale. **Cilium uses eBPF** instead — small programs loaded into the Linux kernel.

> **Mental model — programmable kernel vs paper rulebook**

> iptables is a **paper rulebook** the kernel reads top-to-bottom for every packet — fine for 50 rules, painful for 50,000. **eBPF** is like **installing a custom chip** into the kernel's networking path that already *knows* the routing and policy decisions as fast hash-map lookups. Same decisions, dramatically faster, plus deep visibility.

6.2 Cilium's components



Component	Kind	Role
cilium-agent	DaemonSet (one per node)	Compiles Services + NetworkPolicy into **eBPF** on that node

cilium-operator	Deployment (one per cluster)	Cluster-wide housekeeping: IPAM, garbage collection
eBPF programs	In-kernel	The actual datapath: routing, LB, policy

```
# Install with the settings that match our design (Ch 4)
cilium install --version 1.16.1 \
 --set kubeProxyReplacement=true \                # Cilium IS the service proxy
 --set ipam.mode=cluster-pool \
 --set ipam.operator.clusterPoolIPv4PodCIDRList=10.244.0.0/16 \
 --set hubble.relay.enabled=true \
 --set hubble.ui.enabled=true \
 --set loadBalancer.mode=dsr                       # direct server return, lower latency

cilium status --wait
cilium connectivity test                           # end-to-end self-test
```

kube-proxy replacement is a real architectural win

By setting `kubeProxyReplacement=true`, Cilium removes kube-proxy entirely and implements `ClusterIP`/`NodePort`/`LoadBalancer` service routing in eBPF. Fewer components, no bloated iptables/IPVS rules, and lower latency for TicketHub's heavy East-West traffic (Orders↔Inventory↔Redis).

What kube-proxy actually did

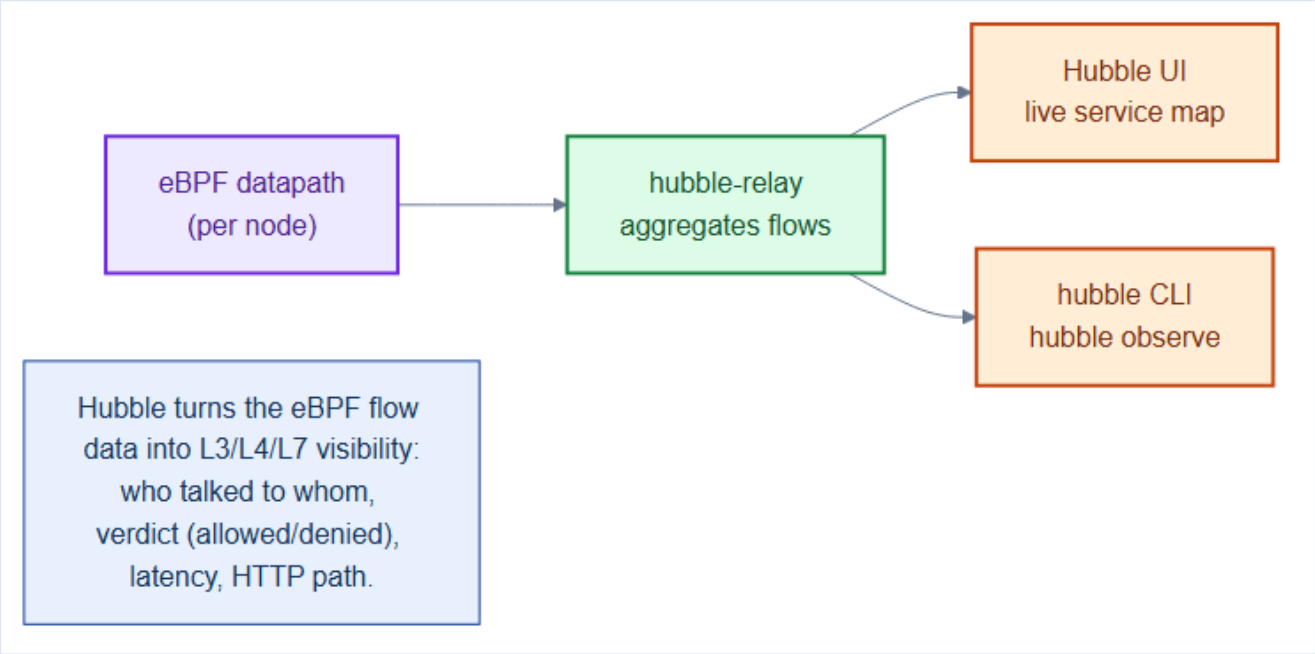
Classic Kubernetes ran **kube-proxy** on every node to turn a Service's virtual IP into real pod IPs by programming large `iptables`/IPVS rule tables — which grow slow as services multiply. Cilium performs the same routing in **eBPF** (fast in-kernel hash lookups), so `kubeProxyReplacement=true` lets us drop kube-proxy entirely: one fewer moving part, and lower East-West latency.

6.3 Why Cilium for TicketHub specifically

Need (from Part I)	Cilium capability
Zero-trust between 9 services	**L3–L7 NetworkPolicy** (even HTTP-path-aware) — Chapter 21
High East-West throughput	eBPF datapath, DSR load balancing
"Why can't Orders reach Payments?"	**Hubble** flow visibility with allow/deny verdicts
Fewer moving parts	kube-proxy replacement
Future multi-cluster	Cluster Mesh (out of scope, but available)

6.4 Hubble — seeing the network

Cilium's sister project **Hubble** taps the same eBPF hooks to give you a live map of **every flow** in the cluster: source, destination, port, L7 protocol, and whether policy **allowed or denied** it.




```
# Observe live flows to the Payments service, denied only
hubble observe --to-pod tickethub/payments --verdict DROPPED

# Open the graphical service map
cilium hubble ui        # port-forwards the Hubble UI
```

Hubble makes NetworkPolicy debuggable

The usual pain with NetworkPolicy is "I applied a policy and now something's broken, but *what*?" Hubble answers directly: it shows the **dropped** flow with the exact source, destination, and port, so you know precisely which rule to add. We'll lean on this heavily in Chapter 21.

6.5 Verifying pod networking

```
kubectl run test --image=nicolaka/netshoot -it --rm -- bash
# inside: curl inventory.tickethub.svc.cluster.local:8080/healthz
# inside: dig frontend.tickethub.svc.cluster.local
cilium status                     # agents healthy, eBPF programs loaded
cilium endpoint list              # every pod endpoint + policy state
```

CNI is hard to change later — choose deliberately

Swapping a CNI on a running production cluster is a disruptive, drain-and-migrate exercise. Decide up front. Cilium fits TicketHub's needs (policy depth, performance, observability); Flannel would be simpler but has **no NetworkPolicy**, which is a non-starter for a payment platform.

6.6 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- Cilium's **BPF maps** are kernel data structures with a fixed maximum size. The default `bpf-map-dynamic-size-ratio` of 0.25 sizes maps based on total system RAM. On a node with very low RAM (< 4GB), the map sizes may be too small for clusters with many services, causing `map full` errors. Tune `--bpf-policy-map-max` and `--bpf-lb-map-max` proactively.

- **L7 policy (HTTP/gRPC path-aware)** requires Cilium to proxy the connection through an Envoy sidecar on the node — this adds ~0.5ms latency per hop. Use L7 policy only where HTTP-path granularity is genuinely needed; use L3/L4 for everything else.

- `kubeProxyReplacement=true` takes full ownership of `ClusterIP` routing. If you later add a component that tries to install its own iptables rules for service routing (e.g., an older Istio version), you will get a conflict. Verify all installed components are compatible with kube-proxy-free mode.

Gotchas — traps that catch experienced engineers

- **CNI is hard to replace post-install**: migrating from Cilium to another CNI (or vice versa) requires draining all nodes, removing the CNI, reinstalling, and reprogram all NetworkPolicy — effectively a cluster rebuild. Choose deliberately and commit.

- **Hubble relay not enabled by default**: `hubble observe` requires `hubble.relay.enabled=true` at install time. If you forget it, you need to `helm upgrade` Cilium later. Not a disaster, but it means you're flying blind on network flows during the most vulnerable early phase.

- **`cilium connectivity test` requires unrestricted egress**: the test creates pods in `cilium-test` namespace that make external HTTP requests. If a default-deny NetworkPolicy is in place before the test, it will fail with misleading errors. Run the connectivity test before applying NetworkPolicy.

Architect Considerations

1. **Hubble data retention**: Hubble stores flow records in a ring buffer in kernel memory — it has no persistent store. For compliance or forensics, you need to configure a Hubble export to an external log store (Loki, Elasticsearch) via the Hubble Kafka/S3 exporter.

2. **eBPF kernel version requirements**: Cilium 1.16 requires kernel ≥ 5.10 for all features. Verify your VM kernel version before cluster bootstrap — older RHEL/Ubuntu LTS kernels may not support all Cilium features (notably WireGuard encryption requires ≥ 5.6, BPF-based masquerading requires ≥ 5.10).

3. **DSR (Direct Server Return) compatibility**: DSR load balancing (`loadBalancer.mode=dsr`) bypasses kube-proxy completely and returns traffic directly from the backend pod to the client. It

requires the backends to see the original client IP — check that your HAProxy VIP configuration is compatible before enabling this.

4. **Network policy migration strategy**: migrating from broad `allow all` to zero-trust NetworkPolicy (Chapter 21) is the highest-risk operational change on a live cluster. Use Hubble in audit mode first, generate policy from observed flows, then enforce incrementally per namespace.

5. **Cluster Mesh for multi-cluster**: if TicketHub grows to span multiple data centers, Cilium Cluster Mesh provides a single policy domain across clusters. This is a significant architectural commitment — design the initial address space and naming conventions with multi-cluster in mind from day one.

Chapter 6 checklist

- CNI installed → all nodes **Ready**, `cilium connectivity test` passes.
- **kube-proxy replaced** by Cilium eBPF.
- **Hubble** enabled for flow visibility (UI + CLI).
- Pod-to-pod and Service DNS verified from a debug pod.

---

7. Load Balancing & Gateway API — MetalLB + Cilium

On a cloud, `Service type=LoadBalancer` magically provisions a real load balancer with a public IP. On **bare metal, nobody provides that** — the Service would sit forever in `<pending>`. **MetalLB** fills the gap, and the **Kubernetes Gateway API** (implemented here by **Cilium**) sits on top to do smart HTTP routing for TicketHub's many services.

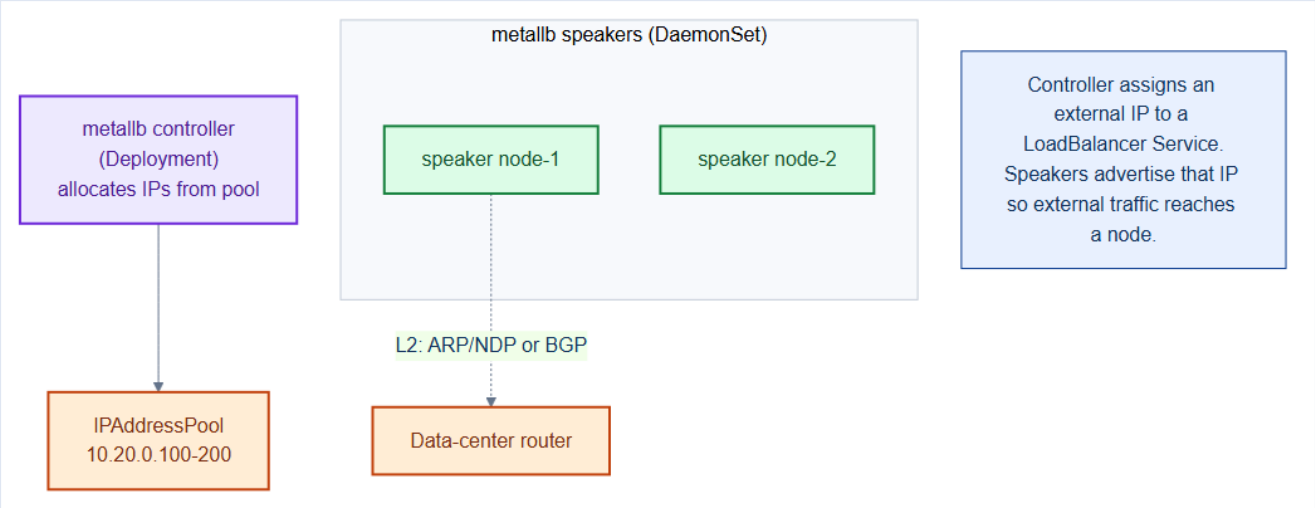
> **What changed in this edition — Ingress → Gateway API**

> Earlier versions of this chapter used a classic **NGINX Ingress**. TicketHub has since **migrated to the Kubernetes Gateway API**, the official successor to Ingress. Ingress is now feature-frozen upstream — no new capabilities are being added to it — and the whole ecosystem is moving to Gateway API. Section **7.6** explains the *why*, the *cons of Ingress*, and *how Gateway API resolves them* in plain terms. If you have only ever seen Ingress, read 7.6 first, then come back.

7.1 The problem MetalLB solves

```
kubectl get svc -n platform cilium-gateway-tickethub
# NAME                         TYPE           EXTERNAL-IP   ...
# cilium-gateway-tickethub     LoadBalancer   <pending>     <-- stuck forever on bare metal
```

MetalLB watches for `LoadBalancer` Services, **allocates an external IP** from a pool you define, and **advertises** it to your physical network so packets actually arrive.



Component	Kind	Role
controller	Deployment	Allocates IPs from the pool to Services
speaker	DaemonSet	Advertises the IP (L2 ARP/NDP, or BGP)

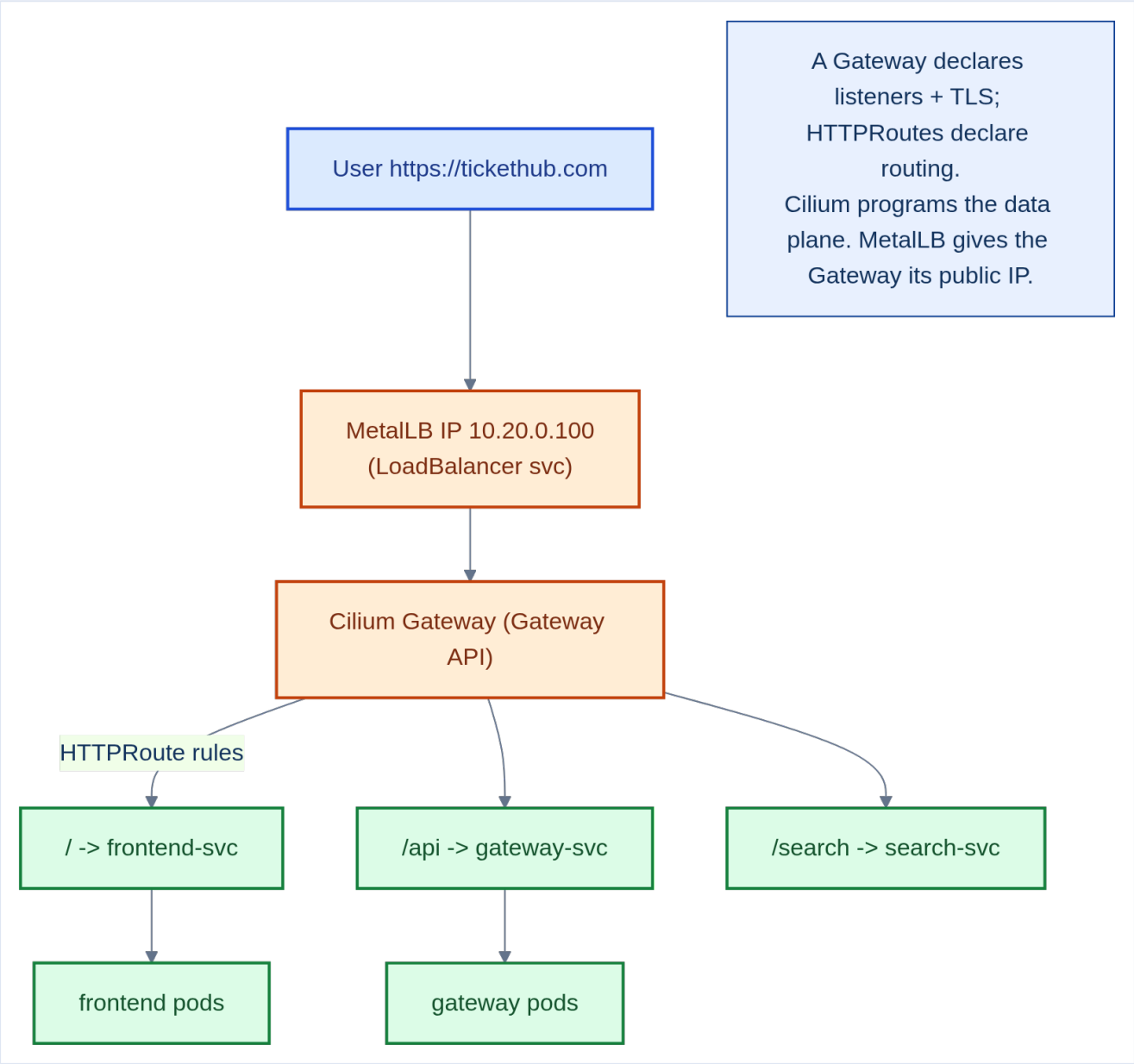
```
# repo/manifests/10-platform/metallb-pool.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
 name: tickethub-pool
 namespace: metallb-system
spec:
 addresses:
   - 10.20.0.100-10.20.0.200      # the VLAN 20 range from Chapter 4
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement            # simplest mode; use BGPAdvertisement with a capable router
metadata:
 name: tickethub-l2
 namespace: metallb-system
spec:
 ipAddressPools: [tickethub-pool]
```

L2 mode is failover, not load-sharing

In **L2 mode**, one elected node answers ARP for a given IP — all traffic for that IP flows through that single node (fast failover if it dies, but no load distribution). For true multi-node balancing, use **BGP mode** and peer MetalLB speakers with your data-center router. Plan this with your network team early.

7.2 Why you still need a routing layer on top

MetalLB gives you an **IP**; it knows nothing about HTTP. Without a routing layer, every service needing external access would burn its own external IP and have no TLS or path routing. The **Gateway** consumes **one** MetalLB IP and routes by **host/path** to many services.



Mental model — building receptionist

MetalLB is the **street address** of the building (a public IP). The **Gateway** is the **receptionist** inside: one desk, but it reads where each visitor wants to go (`/api`, `/search`, `/`) and directs them to the right office (Service) — while also checking their credentials (TLS). You need both: an address to be found, and a receptionist to route.

7.3 Installing the Cilium Gateway API (pinned to infra nodes)

We already run **Cilium** as the CNI (Chapter 6), and Cilium ships a built-in Gateway API implementation — so there is **no separate ingress controller to install**. We just enable the feature and let Cilium provision the data plane:

```
helm upgrade cilium cilium/cilium -n kube-system --reuse-values \
 --set gatewayAPI.enabled=true \                # turn on the Gateway API controller
```

```
 --set gatewayAPI.hostNetwork.enabled=false \    # use a LoadBalancer Service (MetalLB)
 --set nodeSelector.pool=infra                    # keep edge components on the infra pool (Ch 3)
```

The Gateway API **CRDs** must exist first (they are *not* built into Kubernetes like Ingress is):

```
kubectl apply -f \
 https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-install.yaml
kubectl get gatewayclass          # cilium should show ACCEPTED=True
```

Gateway API is CRDs, not core — install them or nothing works

`Ingress` ships inside the Kubernetes API server. `Gateway`, `HTTPRoute`, and friends are **CRDs** you must install (and keep version-matched to your controller). A `Gateway` applied before the CRDs exist fails with `no matches for kind "Gateway"`.

7.4 The TicketHub Gateway + HTTPRoute objects

Where Ingress crammed *listeners, TLS, and routing* into one object, the Gateway API splits them by **owner**. The platform team owns the **Gateway** (the ports, the IP, the TLS cert); each app team owns its **HTTPRoute** (the paths and backends). Both live in the repo:

```
# repo/manifests/10-platform/gateway-tickethub.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
 name: tickethub
 namespace: tickethub
 annotations:
   cert-manager.io/cluster-issuer: letsencrypt-prod   # auto TLS (see 7.5)
spec:
 gatewayClassName: cilium                             # Cilium realises this as a LB Service
 listeners:
   - name: http                                        # :80 — ACME challenge + redirect
     protocol: HTTP
     port: 80
     hostname: tickethub.example.com
   - name: https                                       # :443 — TLS terminated here
     protocol: HTTPS
     port: 443
     hostname: tickethub.example.com
     tls:
       mode: Terminate
       certificateRefs: [{ kind: Secret, name: tickethub-tls }]
---
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
 name: tickethub
 namespace: tickethub
spec:
 parentRefs: [{ name: tickethub, sectionName: https }]
 hostnames: [tickethub.example.com]
 rules:
   - matches: [{ path: { type: PathPrefix, value: /api } }]   # API Gateway service
```

```
    backendRefs: [{ name: gateway, port: 8080 }]
  - matches: [{ path: { type: PathPrefix, value: / } }]      # everything else -> frontend
    backendRefs: [{ name: frontend, port: 80 }]
```

```
kubectl get gateway,httproute -n tickethub
# NAME                                  CLASS    ADDRESS        PROGRAMMED
# gateway.../tickethub                  cilium   10.20.0.100    True
# NAME                                  HOSTNAMES
# httproute.../tickethub                ["tickethub.example.com"]
```

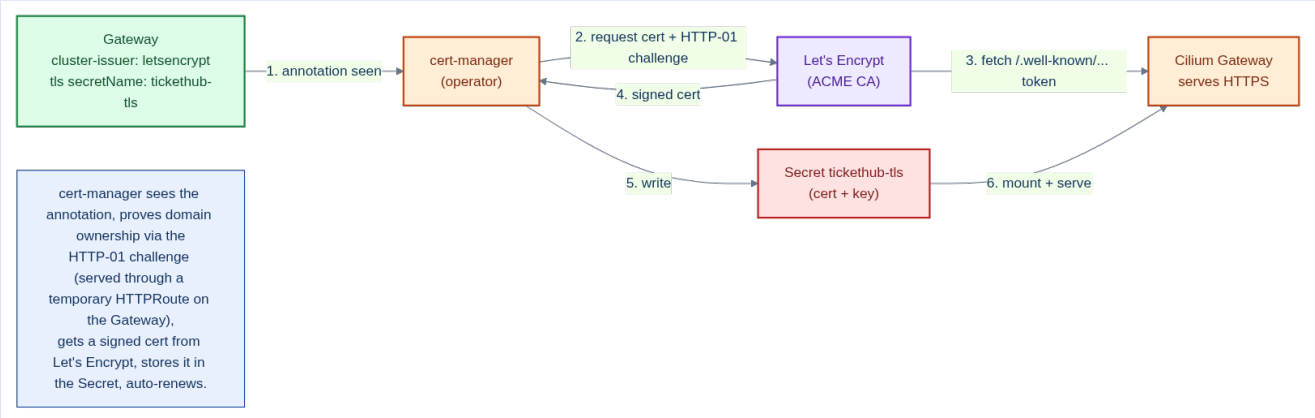
> **The old `ingressClassName: nginx` is now `gatewayClassName: cilium`**

> The routing engine is selected on the **Gateway**, not scattered across each route. Swapping implementations (Cilium → Envoy Gateway → NGINX Gateway Fabric) is a one-line change on the Gateway; the `HTTPRoute` objects are portable and untouched.

7.5 Automatic TLS with cert-manager

That one annotation — `cert-manager.io/cluster-issuer: letsencrypt` — is doing a lot of quiet work. On its own, the Gateway declares it *wants* HTTPS on `tickethub.com` and expects the certificate in a Secret named `tickethub-tls`. But **nothing creates that Secret** unless something obtains a real, browser-trusted certificate. That something is **cert-manager**.

cert-manager is an operator (Chapter 25) that automates the whole certificate lifecycle: request, prove ownership, store, and **renew before expiry** — so no human ever copies a `.pem` file onto a server at 2am.



Install it once, cluster-wide (it lives in the `platform` namespace, Chapter 9):

```
helm install cert-manager jetstack/cert-manager \
 -n platform --create-namespace \
 --set crds.enabled=true          # installs the Certificate/Issuer CRDs
```

Then define **where** certificates come from. A **ClusterIssuer** is a cluster-scoped recipe for talking to a Certificate Authority — here, **Let's Encrypt** over the **ACME** protocol:

```
# repo/manifests/10-platform/cert-manager-issuers.yaml
apiVersion: cert-manager.io/v1
```

```
kind: ClusterIssuer
metadata:
 name: letsencrypt                  # matches the Gateway annotation
spec:
 acme:
   server: https://acme-v02.api.letsencrypt.org/directory   # production CA
   email: platform@tickethub.com                            # expiry warnings go here
   privateKeySecretRef: { name: letsencrypt-account-key }   # your ACME account key
   solvers:
     - http01:
         gatewayHTTPRoute:                                  # prove ownership via HTTP-01
           parentRefs:
             - { name: tickethub, namespace: tickethub, kind: Gateway, group: gateway.networking.k8
```

How ownership is proven (the ACME challenge). Let's Encrypt won't sign a cert for `tickethub.com` unless you prove you control it. The two common challenges:

Challenge	How you prove control	When to use
HTTP-01	cert-manager serves a token at `http://tickethub.com/.well-known/...`; the CA fetches it	Public HTTP reachable (our case)
DNS-01	cert-manager writes a `TXT` DNS record the CA checks	Wildcards (`*.tickethub.com`) or no inbound HTTP

The end-to-end loop, all automatic:

1. You apply the Gateway with the `cluster-issuer` annotation.
2. cert-manager notices and creates a **Certificate** object for the TLS hosts.
3. It requests the cert from Let's Encrypt and solves the **HTTP-01** challenge (temporarily attaching an HTTPRoute for `/.well-known/acme-challenge/...` to the Gateway's `:80` listener).
4. Let's Encrypt signs; cert-manager writes the cert + key into the **`tickethub-tls`** Secret.
5. The Gateway picks up the Secret and serves **HTTPS**. cert-manager renews ~30 days before the 90-day expiry — untouched by humans.

Watching a certificate actually get generated. "Automatic" doesn't mean "invisible" — cert-manager builds a chain of objects you can watch and, when something's wrong, debug. Each `Certificate` spawns a `CertificateRequest`, which spawns an ACME `Order`, which spawns a `Challenge`:

```
Gateway -> Certificate -> CertificateRequest -> Order -> Challenge -> Secret (tickethub-tls)
```

Apply the Gateway, then follow the chain to completion:

```
# 1. The Certificate starts NOT ready while issuance runs.
kubectl get certificate -n tickethub
# NAME            READY   SECRET          AGE
# tickethub-tls   False   tickethub-tls   5s
```

```
# 2. Watch the request/order/challenge objects cert-manager created.
kubectl get certificaterequest,order,challenge -n tickethub

# 3. Follow the HTTP-01 challenge until the CA authorizes the domain.
kubectl describe challenge -n tickethub
# Status:  valid
# Reason:  Successfully authorized domain "tickethub.com"

# 4. READY flips to True; the Secret now holds the signed cert + key.
kubectl get certificate -n tickethub
# tickethub-tls   True    tickethub-tls   90s
kubectl get secret tickethub-tls -n tickethub -o jsonpath='{.type}'   # kubernetes.io/tls
```

If a cert is stuck, walk the chain downward — the failing object holds the reason

A `Certificate` stuck `READY: False` for minutes means a step below it failed. Run `kubectl describe` down the chain — **Certificate -> CertificateRequest -> Order -> Challenge** — and read the **Events/Status** of the *lowest* object; that's where the real error is. The usual culprits, all on the **HTTP-01** step: the DNS `A` record doesn't point at the MetalLB Gateway IP yet, **port 80 is blocked** (the CA fetches the token over plain HTTP before HTTPS exists), or you hit a Let's Encrypt **rate limit** (use staging). If you install the `cmctl` plugin, `cmctl status certificate tickethub-tls -n tickethub` prints the whole chain and its errors in one view.

Use the staging issuer first — Let's Encrypt rate-limits hard

Production Let's Encrypt allows only a handful of certs per domain per week. A misconfigured issuer can **burn your weekly quota** in minutes and lock you out. Always validate against the **staging** CA (`https://acme-staging-v02.api.letsencrypt.org/directory`) first — it issues **untrusted** certs (browser warning) but has generous limits — then flip the annotation to the production issuer once the flow works end-to-end.

Mental model — a robotic notary on a subscription

Manually managing TLS is a chore: buy a cert, prove you own the domain, install it, and diary a reminder to redo it before it expires. cert-manager is a **robotic notary**: it does the ownership proof, files the paperwork, installs the certificate where the Gateway expects it, and renews the subscription automatically — forever.

This is only the public edge — the cluster has a much larger PKI

The Let's Encrypt cert here secures the *front door*. Behind it, every control-plane component, node, and (optionally) internal service also runs on certificates. For the complete picture — the cluster PKI inventory, HA certificate SANs for a 3-node control plane, renewal/rotation, and an **internal CA with cert-manager for service mTLS** — see **Chapter 7A, Certificate & PKI Management**.

7.6 Why we migrated from Ingress to the Gateway API

If you have only ever used **Ingress**, this section is for you. It explains — in plain terms — what Ingress is, where it hurts, and how the **Gateway API** fixes each pain point. This is the reasoning behind the migration you see in this chapter.

What Ingress was. An `Ingress` is a single Kubernetes object that says "expose these HTTP paths on this hostname with this TLS cert." It was the standard way to get traffic into a cluster for years. It works — but it was designed early, kept deliberately minimal, and is now **feature-frozen**: the Kubernetes project has stopped adding capabilities to it and points everyone at the Gateway API instead. "Deprecated in spirit" is a fair summary — Ingress still runs, but it is a dead-end road.

The cons of Ingress (why it hurts in practice)

Problem with Ingress	What it means for a fresh grad	Consequence
One object, mixed owners	Listeners, TLS, *and* routing are all crammed into one `Ingress`. The platform team (who owns TLS/IP) and the app team (who owns paths) must edit the **same file**.	Change collisions, unclear ownership, risky reviews.
Annotation soup	Anything beyond basic path routing — rewrites, timeouts, rate limits, header matching, canary splits — lives in `nginx.ingress.kubernetes.io/...` **annotations**: free-form strings, not validated fields.	A typo in an annotation fails silently. Your YAML is **locked to one controller** (NGINX annotations don't work on Traefik/HAProxy).
No portability	Because behaviour hides in vendor annotations, moving from NGINX to another controller means rewriting them all.	Vendor lock-in; migrations are painful.
Weak expressiveness	Ingress natively understands host + path only. Header/method-based routing and traffic splitting (blue-green, canary) are not first-class.	You bolt on service meshes or CRDs to do routine things.
Shared-fate reloads	Classic controllers render **one big config** for all Ingresses. One bad Ingress can break the config reload for everyone.	A single team's mistake can take down unrelated services.
Frozen	No new features are being added upstream.	You are investing in a technology with no future roadmap.

How the Gateway API resolves each one

Gateway API answer	How it fixes the Ingress con
Role-oriented objects — `GatewayClass` (infra provider), `Gateway` (platform team: ports, IP, TLS), `HTTPRoute` (app team: paths, backends).	Ownership is split cleanly across separate objects and RBAC. Teams stop editing each other's files.
Typed spec fields, not annotations — header matches, method matches, redirects, traffic splitting, timeouts are **real, validated fields** in the CRD.	The API server rejects mistakes; behaviour is portable across implementations.
Portable & implementation-agnostic — selecting the engine is one line (`gatewayClassName`). Cilium, Envoy Gateway, Istio, NGINX Gateway Fabric all read the *same* `HTTPRoute`.	Swapping controllers no longer means rewriting routes.
Rich routing built in — weighted `backendRefs` (canary), header/method matches, request mirroring, redirects.	Common patterns need no annotations or extra CRDs.
Per-route objects — each `HTTPRoute` is independent; a broken one affects only itself.	One team's mistake no longer risks the whole edge.
The active standard — Gateway API is where all new investment goes; it also unifies mesh (east-west) routing under GAMMA.	You build on the technology with a future.

Mental model — Ingress is a Swiss-army knife, Gateway API is a toolbox

Ingress is one folding tool: every blade crammed into a single handle that one person holds. Handy, but everyone fights over the handle and you can't add new blades. **Gateway API** is a labelled toolbox: the platform team owns the *box and the power outlet* (the `Gateway`), each app team grabs the *tool they need* (their `HTTPRoute`), and you can swap the whole brand of tools without changing how anyone works.

Migration gotchas — what actually bites during the switch

- **CRDs are not core.** `Ingress` lives inside the API server; `Gateway`/`HTTPRoute` are **CRDs** you must install *first* and keep version-matched to the controller.
- **cert-manager solver changes.** The ACME HTTP-01 solver moves from `http01.ingress` to `http01.gatewayHTTPRoute`, and needs cert-manager's `ExperimentalGatewayAPISupport` (GA in recent releases) enabled at install.
- **Annotations don't carry over.** Any `nginx.ingress.kubernetes.io/*` behaviour must be re-expressed as typed fields or filters — there is no automatic translation.

- **Run both during cutover.** Keep the old Ingress and the new Gateway live on *different* hostnames/IPs, validate, shift DNS, then delete the Ingress. Don't flip in place.

- **`Gateway` is namespaced but cross-namespace routing needs `ReferenceGrant`.** If an `HTTPRoute` in namespace A targets a Service in namespace B, you must grant it explicitly — a deliberate safety boundary Ingress never had.

Layered edge: MetalLB → Gateway → Service → Pod

- **MetalLB** = L2/L3, gives bare metal a real external IP.
- **Gateway (Cilium)** = L7, host/path routing + TLS termination, one IP for many services.
- **Service** = stable virtual IP + DNS for a set of pods.
- **Pod** = the actual workload.

Each layer has one job; together they replace what a cloud LB + ALB gives you.

7.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- MetalLB in L2 mode announces the LoadBalancer IP from a **single node** at a time using ARP/NDP. Traffic arrives at that node, which then routes internally. This means a single node carries all north-south traffic — the load balancing happens AFTER the packet enters the cluster, not at the IP level. The announcing node becomes a bottleneck for very high-throughput services.
- With **Cilium's Gateway API**, TLS is terminated inside the Cilium-managed Envoy proxy on the Gateway node — backend pods receive plain HTTP (or mTLS if you configure it separately). There is **no separate NGINX pod** to size; capacity is governed by the Cilium agent/Envoy resource limits on the infra nodes. Ensure they can absorb your peak TLS handshake rate.
- `cert-manager` uses **ACME DNS-01 or HTTP-01 challenges** for Let's Encrypt. The HTTP-01 solver now attaches an **HTTPRoute** to the Gateway's `:80` listener rather than an Ingress. On a private on-prem cluster without internet access, HTTP-01 is impossible — use DNS-01 with your DNS provider's API, or an internal CA (Chapter 7A) for cluster-internal certificates.

Gotchas — traps that catch experienced engineers

- **MetalLB pool overlapping with DHCP range**: if your data center DHCP server allocates IPs in the `10.10.0.200-250` range, ARP conflicts will cause intermittent LoadBalancer IP unreachability.

Co-ordinate with the network team and document the reserved range.

- **Gateway API CRDs missing or version-skewed**: unlike Ingress, `Gateway`/`HTTPRoute` are CRDs. Applying a `Gateway` before the CRDs exist fails with `no matches for kind "Gateway"`; a CRD version newer than the Cilium controller understands leaves the Gateway `PROGRAMMED: False` with no traffic. Install the CRDs first and match versions.

- **Forgetting `parentRefs`/`sectionName`**: an `HTTPRoute` that doesn't reference the Gateway (or references the wrong listener section) is silently ignored — the modern equivalent of the old `ingressClassName` mismatch. Check `kubectl describe httproute` for an `Accepted: True` condition.

- **cert-manager CRD version drift**: `Certificate`, `Issuer`, and `ClusterIssuer` CRDs are versioned. Upgrading cert-manager without reading the migration guide can cause CRD schema validation failures that block certificate renewals silently.

Architect Considerations

1. **Single vs multi-Gateway**: running one `Gateway` for all traffic makes it a shared-fate component. Because `HTTPRoute`s are independent per-route objects, a bad route no longer breaks the whole edge — but the listeners/IP are still shared. Consider per-team or per-tier `Gateway`s (each its own IP) for strong isolation.

2. **MetalLB BGP upgrade path**: L2 mode is simpler but has a single-node bottleneck. If north-south throughput requirements grow (>10 Gbps), plan the migration to BGP mode with ECMP — this requires network team involvement and a BGP router change.

3. **WAF placement**: with Cilium's Gateway you can attach L7 policy/filters at the edge, or run a dedicated WAF. For a payments platform, should WAF rules live at the Gateway (easier), at the API Gateway service (more context-aware), or both? Layer 7 inspection adds latency — measure before committing.

4. **Gateway API maturity by feature**: core `HTTPRoute` is GA, but some pieces (TLSRoute, GRPCRoute, mesh/GAMMA) are at different stability levels across implementations. Pin to the feature set your controller (Cilium) supports as GA, and track the conformance matrix before relying on experimental fields.

5. **Certificate wildcard vs per-service**: a `*.tickethub.io` wildcard cert simplifies management but must be rotated for every domain — including unrelated ones. Per-service certs (automated by cert-manager) are noisier but limit blast radius if a cert is compromised.

Chapter 7 checklist

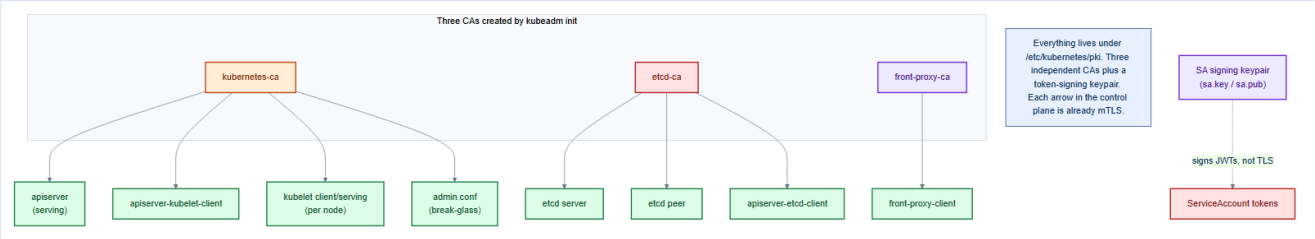
- **MetalLB** installed with an **IPAddressPool** from the VLAN-20 range.
- **Gateway API CRDs** installed and version-matched to the Cilium controller.
- **Cilium Gateway API** enabled, pinned to the **infra** pool; the Gateway's LoadBalancer Service got a MetalLB IP.
- A **Gateway** + **HTTPRoute** route `/api` and `/` with TLS terminated at the `:443` listener.
- **cert-manager** installed with a **ClusterIssuer** using the `gatewayHTTPRoute` solver; TLS auto-issued and auto-renewed.
- Validated against the **staging** ACME CA before flipping to production.
- Decided **L2 vs BGP** mode with the network team.

7A. Certificate & PKI Management

Chapter 7 gave TicketHub a browser-trusted certificate at the edge. But that public cert is the *tip* of a much larger iceberg: **a Kubernetes cluster is, from the first second of `kubeadm init`, a full Public Key Infrastructure.** Every control-plane component, every node, and every internal service proves who it is with an X.509 certificate. If those certificates are wrong, missing a name, or expired, the cluster doesn't "degrade" — it **stops**. This chapter is the complete picture: every certificate in the cluster, how HA multiplies them, how to inspect and renew them, and how to extend the same discipline to application-to-application traffic with an internal CA and mTLS.

7A.1 The cluster is a PKI — the certificate inventory

`kubeadm init` doesn't just start pods; it stands up **three independent Certificate Authorities** plus a token-signing keypair, then issues a leaf certificate to every identity that needs one. Everything lives under `/etc/kubernetes/pki` on each control-plane node.



CA (root of trust)	Signs certificates for	Why it's separate
kubernetes-ca (`ca.crt`)	apiserver serving, apiserver→kubelet client, kubelet client/serving, admin, controller-manager, scheduler	The cluster's main identity domain
etcd-ca (`etcd/ca.crt`)	etcd server, etcd peer, etcd healthcheck-client, apiserver→etcd client	etcd is the crown jewels; its own CA limits blast radius
front-proxy-ca	front-proxy-client	Isolates the API aggregation layer (extension API servers)
SA signing keypair (`sa.key`/`sa.pub`)	*Not X.509* — signs ServiceAccount **JWTs**	Token signing, not TLS identity (Chapter 19)

The individual leaf certificates every cluster has:

Certificate	Identity it proves	Key SAN / subject

`apiserver.crt`	The API server **to clients**	VIP, every CP host, `kubernetes.default`, service IP
`apiserver-kubelet-client.crt`	API server **to each kubelet**	CN in `system:masters`
`apiserver-etcd-client.crt`	API server **to etcd**	client auth
`etcd/server.crt`	An etcd member **to its clients**	that member's IP + `localhost`
`etcd/peer.crt`	An etcd member **to other members**	that member's IP + host
`kubelet.crt` (per node)	A kubelet **to the API server**	`system:node:<host>`, group `system:nodes`
`front-proxy-client.crt`	The aggregation layer	client auth
`admin.conf` cert	Your `kubectl` **break-glass** identity	`system:masters`

Two ways to prove identity, one PKI

Every arrow between components in Kubernetes is **mutual TLS already** — the client presents a cert, the server presents a cert, and each validates the other against the right CA. The API server is the hub: it is a **TLS server** to kubectl and kubelets, and a **TLS client** to etcd and the kubelets. Get the CA trust and the SANs right and the whole mesh "just works"; get them wrong and nothing connects.

7A.2 HA multiplies the certificates — SANs, the VIP & the etcd mesh

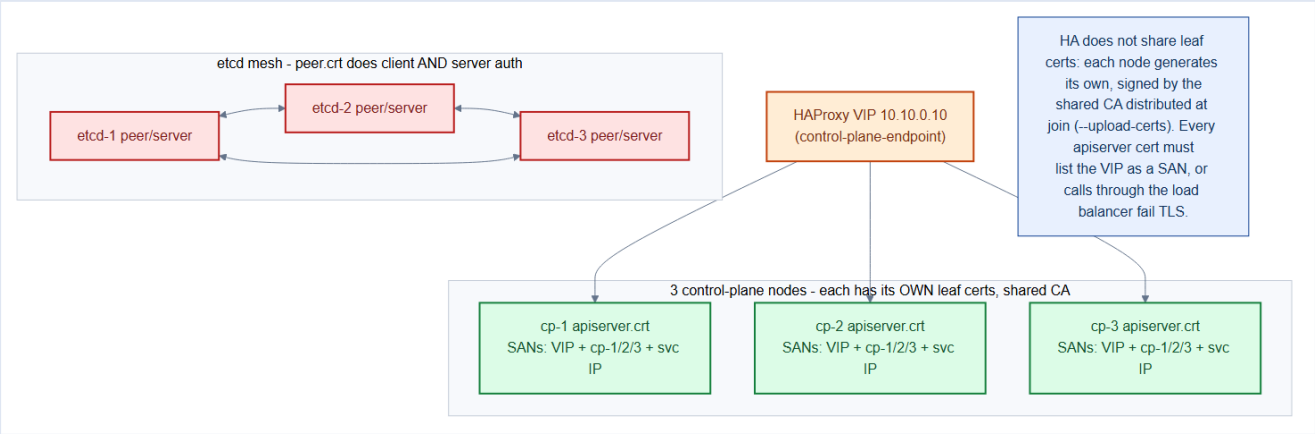
A single-node control plane has one of each cert. **TicketHub's HA control plane has three** (Chapter 3: `cp-1/2/3` behind the HAProxy **VIP**). HA doesn't share leaf certs — each node generates **its own** apiserver/etcd certs, all signed by the **shared CAs** that `--upload-certs` distributes at join time. Two things must be correct for HA to work:

1. The apiserver serving cert must carry every name a client might use. A client can hit any CP node *or* the VIP, so the `apiserver.crt` **Subject Alternative Names** must include all of them. Miss the VIP and every `kubectl` through the load balancer fails TLS verification:

```
# repo/cluster/kubeadm-config.yaml  — apiServer.certSANs feed apiserver.crt
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
controlPlaneEndpoint: "10.10.0.10:6443"     # the HAProxy VIP (Chapter 3)
apiServer:
 certSANs:
   - "10.10.0.10"          # VIP  <-- the one everyone forgets
   - "api.tickethub.internal"   # stable DNS for the VIP
```

```
    - "cp-1"
    - "cp-2"
    - "cp-3"
    - "10.10.0.11"          # cp-1 IP
    - "10.10.0.12"          # cp-2 IP
    - "10.10.0.13"          # cp-3 IP
etcd:
 local:
   serverCertSANs: ["10.10.0.11", "10.10.0.12", "10.10.0.13"]
   peerCertSANs:   ["10.10.0.11", "10.10.0.12", "10.10.0.13"]
```

2. etcd is a 3-member mesh, and every member authenticates every other. Each etcd `peer.crt` is used for **both** client and server auth (a member is both when replicating), and its SANs must list that member's address. The three peers form a fully-authenticated quorum; lose the peer certs and the Raft cluster (Chapter 3) can't form.



The single most common HA cluster outage: a missing VIP SAN

If you add the load-balancer VIP or its DNS name *after* `kubeadm init`, the existing `apiserver.crt` won't include it and every call through the LB fails with `x509: certificate is valid for <hosts>, not <vip>`. Fixing it means regenerating the apiserver cert on **all three** nodes (`kubeadm certs renew apiserver` after adding the SAN to the config) and restarting each apiserver. Put every name in `certSANs` **before** you init.

7A.3 Inspecting & verifying what you have

You cannot manage what you cannot see. Two commands answer "what certs exist, and are they valid?"

```
# The whole inventory, expiry dates, and which CA signed each — run on every CP node.
kubeadm certs check-expiration
# CERTIFICATE                EXPIRES                  RESIDUAL TIME   EXTERNALLY MANAGED
# apiserver                  Aug 11, 2027 09:00 UTC   364d            no
# apiserver-etcd-client      Aug 11, 2027 09:00 UTC   364d            no
# etcd-peer                  Aug 11, 2027 09:00 UTC   364d            no
# ...
# CERTIFICATE AUTHORITY      EXPIRES                  RESIDUAL TIME
# ca                         Aug 09, 2035 09:00 UTC   9y

# Read one cert's SANs directly — confirm the VIP is really in there.
```

```
openssl x509 -in /etc/kubernetes/pki/apiserver.crt -noout -text \
 | grep -A1 "Subject Alternative Name"
# DNS:kubernetes, DNS:api.tickethub.internal, ..., IP Address:10.10.0.10
```

Leaf certs are short-lived (1 year); CAs are long-lived (10 years)

`kubeadm` issues component leaf certs with a **1-year** lifetime and the CAs with a **10-year** lifetime. That asymmetry is deliberate: renewing a leaf is cheap and routine; rotating a CA is rare and disruptive (every kubeconfig and kubelet must learn the new trust anchor). Plan for leaf renewal as a **calendar event**, not an emergency.

7A.4 Renewal & rotation — the day-2 job that prevents outages

Leaf renewal (routine). Every `kubeadm upgrade apply` **auto-renews all control-plane leaf certs** — so a cluster upgraded at least yearly rarely expires. To renew out-of-band, do it **one control-plane node at a time** (HA lets the other two serve while you restart one):

```
# On each CP node in turn:
kubeadm certs renew all              # re-signs every leaf from the existing CA
# restart the control-plane static pods so they load the new certs:
kubectl -n kube-system delete pod \
 kube-apiserver-$(hostname) kube-controller-manager-$(hostname) \
 kube-scheduler-$(hostname) etcd-$(hostname)
kubeadm certs check-expiration       # confirm fresh 1-year dates
```

Kubelet rotation (automatic). Kubelets rotate their **client** cert automatically via the CSR API (`rotateCertificates: true`, auto-approved by the controller-manager). Turn on **serving**-cert rotation too (`serverTLSBootstrap: true`) so kubelet serving certs are CSR-issued and rotated instead of self-signed — then approve or auto-approve those CSRs.

Expiry monitoring (non-negotiable). Tie certificates into Chapter 26: run the **x509-certificate-exporter** to surface every cert's expiry as a Prometheus metric and **alert weeks ahead** — on all three CP nodes.

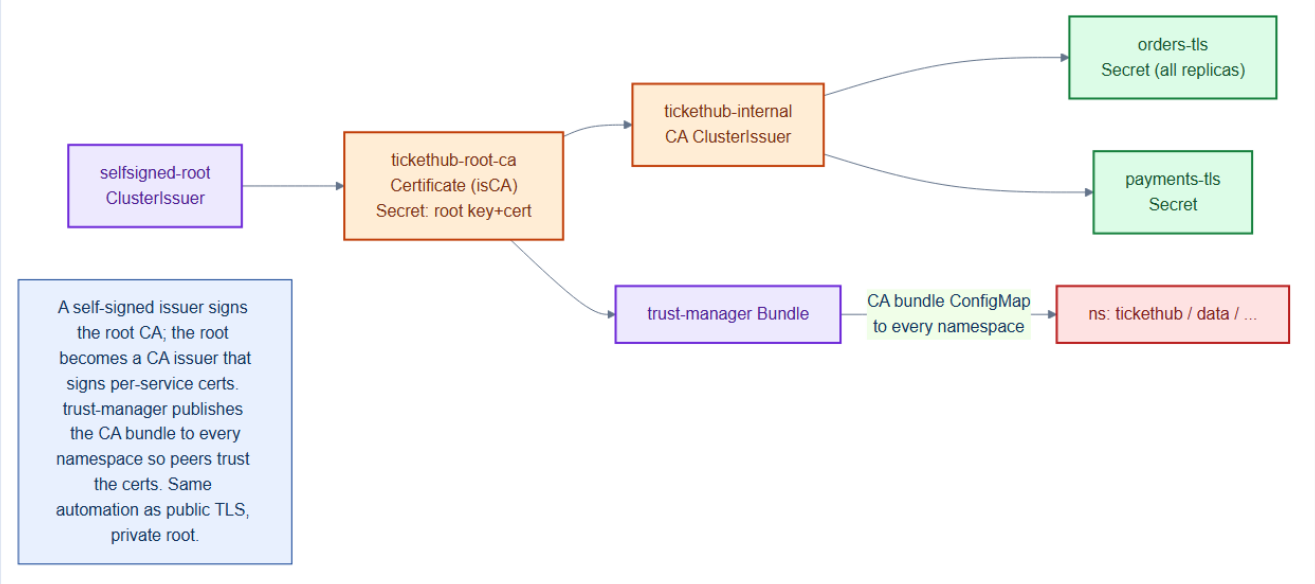
```
# repo/manifests/70-observability/cert-expiry-rule.yaml
- alert: ControlPlaneCertExpiringSoon
 expr: (x509_cert_not_after - time()) / 86400 < 21   # < 3 weeks left
 for: 1h
 labels: { severity: page }
 annotations: { summary: "A control-plane certificate expires in under 21 days" }
```

Expired control-plane certs are a full outage, not a warning

If `apiserver.crt` or the etcd certs expire, the API server and etcd **refuse to start** — you cannot `kubectl` your way out, because kubectl itself needs a valid cert. Recovery is manual, on-console, per node. This is the one PKI failure that takes the whole cluster down at once, so the exporter alert above is mandatory, and CA-expiry (the 10-year clock) belongs in your team's long-term calendar.

7A.5 Application PKI — cert-manager as an internal CA

Chapter 7 used cert-manager with **Let's Encrypt** for the *public* edge. Internal service-to-service traffic is different: you don't want (or can't get) public certs for `orders.tickethub.svc`, and you don't want the internet in your trust path. The answer is to run **your own CA inside the cluster** with cert-manager — same automation, private trust.



Bootstrap a root CA once (a self-signed issuer signs the root, which becomes a CA issuer that signs everything else):

```
# repo/manifests/15-pki/internal-ca.yaml
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: selfsigned-root }
spec: { selfSigned: {} }
---
apiVersion: cert-manager.io/v1
kind: Certificate
metadata: { name: tickethub-root-ca, namespace: cert-manager }
spec:
 isCA: true
 commonName: tickethub-internal-ca
 secretName: tickethub-root-ca        # the CA key + cert land here
 duration: 87600h                     # 10 years, like the cluster CA
 privateKey: { algorithm: ECDSA, size: 256 }
 issuerRef: { name: selfsigned-root, kind: ClusterIssuer }
---
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: tickethub-internal }  # every service cert is issued by this
spec:
 ca: { secretName: tickethub-root-ca }
```

Now issue a certificate per service. **HA is the interesting part:** all replicas of a Deployment sit behind one **Service DNS name**, so they **share one certificate** whose SAN is that Service name — every pod mounts the same Secret and that is correct, because clients dial the Service, not a pod. For a **StatefulSet**, where clients address *individual* pods (`orders-0.orders-headless...`), the cert must also carry **per-pod DNS SANs** (or a wildcard):

```
# repo/manifests/15-pki/orders-internal-cert.yaml
apiVersion: cert-manager.io/v1
kind: Certificate
metadata: { name: orders-tls, namespace: tickethub }
spec:
 secretName: orders-tls              # mounted by ALL orders replicas
 duration: 2160h                     # 90 days; cert-manager auto-renews
 dnsNames:
   - orders.tickethub.svc.cluster.local          # the Service (covers every replica)
   - "*.orders-headless.tickethub.svc.cluster.local"  # per-pod names (StatefulSet HA)
 issuerRef: { name: tickethub-internal, kind: ClusterIssuer }
```

Trust distribution — the other half. A certificate is useless if peers don't trust the CA that signed it. Manually copying the CA into every namespace is exactly the toil cert-manager exists to kill, so use its companion **trust-manager** to publish the CA bundle as a ConfigMap into **every** namespace automatically:

```
# repo/manifests/15-pki/trust-bundle.yaml
apiVersion: trust.cert-manager.io/v1alpha1
kind: Bundle
metadata: { name: tickethub-ca }
spec:
 sources:
   - secret: { name: "tickethub-root-ca", key: "tls.crt" }
 target:
   configMap: { key: "ca.crt" }       # appears as ConfigMap tickethub-ca in all namespaces
```

Deployment replicas share a cert; StatefulSet pods often need their own SANs

The rule that trips people up: a **Service name** covers *all* replicas behind it, so a stateless Deployment's replicas correctly share **one** Secret. Only when clients connect to a **specific pod** — StatefulSet members like a Postgres primary or a Kafka broker (Chapter 14) — do you need **per-pod DNS SANs** (or a headless-service wildcard) so each pod can present a name that matches how it's addressed.

7A.6 mTLS between services — app-managed vs mesh-managed

With per-service certs and a distributed CA bundle, you can turn on **mutual TLS** east-west. Two paths, and the right choice depends on scale:

Aspect	App-managed mTLS	Mesh-managed mTLS (Cilium / Istio)
How	Each app mounts its cert Secret + CA bundle and enforces client certs in code	A sidecar/eBPF layer wraps every connection transparently
Identity	The cert-manager Certificate per service	**SPIFFE** identity per workload, issued & rotated automatically

Rotation	cert-manager renews the Secret; app must reload	Fully automatic, seconds-scale, no app change
Best for	A handful of sensitive links	Cluster-wide zero-trust across many services
HA behavior	All replicas share the Service cert	Every pod gets its own rotating identity, automatically

7A.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- cert-manager **does not renew certificates that are manually edited**. If you patch the `spec.duration` or `spec.renewBefore` of a `Certificate` object, cert-manager detects the discrepancy and re-issues immediately — useful for forcing a renewal, but unexpected if you're just inspecting it.

- The cluster CA generated by kubeadm is a **10-year self-signed cert** by default. It is NOT automatically rotated by kubeadm — you must rotate it manually, which is one of the most disruptive cluster operations (it requires restarting every component and re-issuing every signed cert). Plan for this before year 10.

- **Trust bundle distribution via `trust-manager`** is one-way: trust-manager copies CA bundles INTO ConfigMaps, but pods that mount the ConfigMap as a volume do NOT see updates until the pod restarts. Add an application-level cert reload mechanism (SIGHUP handler, inotify watcher) for services that use the bundle.

Gotchas — traps that catch experienced engineers

- **`NotReady` after cert rotation**: when you rotate the kubelet serving cert, the API server briefly loses connectivity to the kubelet, causing pods to show `Unknown` status. This is transient — wait 60 seconds before debugging.

- **`Certificate` object `spec.secretName` collision**: if two `Certificate` objects in the same namespace target the same `secretName`, cert-manager will fight over the Secret content, causing rapid re-issuance. Always use unique Secret names per Certificate.

- **Let's Encrypt rate limits**: the production Let's Encrypt CA has a hard rate limit of 50 certificates per registered domain per week. In a staging environment with many ephemeral namespaces, each generating a Certificate, you can exhaust this limit in hours. Always use the Let's Encrypt staging issuer in non-production environments.

Architect Considerations

1. **Internal CA lifetime and rotation plan**: a 10-year cluster CA means you have 10 years before a forced rebuild — but also 10 years of accumulated risk from the CA key being on disk. Consider a 5-year lifetime and document the rotation procedure before you need it.

2. **mTLS scope**: cert-manager + trust-manager + istio/linkerd can give you mTLS between every pod pair. For TicketHub, mTLS is most critical for the `orders → payments → postgres` path. Define explicitly which service pairs require mTLS vs which are covered by NetworkPolicy alone.

3. **PKI hierarchy depth**: the current model is a 2-level hierarchy (cluster CA → service certs). A 3-level hierarchy (root CA → intermediate CA → service certs) allows you to rotate intermediate CAs without touching the root — better for long-lived installations.

4. **Certificate observability**: cert expiry is a leading indicator of outage, not a trailing one. The `x509-certificate-exporter` DaemonSet (Chapter 27) plus a Prometheus alert firing 30 days before expiry should be considered non-optional for any production cluster.

5. **Vault integration for production PKI**: cert-manager's Vault issuer allows Vault to be the root of trust, keeping private keys off-cluster in a secrets manager with HSM backing. For a payments platform, this is the correct long-term architecture — the current cluster CA is a stepping stone.

Chapter 7A checklist

- **Cluster PKI inventory** understood; `kubeadm certs check-expiration` clean on all CP nodes.
- `apiServer.certSANs` includes the **VIP + DNS + every CP host/IP**; etcd server/peer SANs per member.
- Leaf renewal **automated via upgrades**; out-of-band renewal done **one CP node at a time**.
- Cert **expiry exported to Prometheus** and alerted **weeks ahead** (Chapter 26); CA expiry on the long-term calendar.
- **kubelet** client (and serving) cert rotation enabled.
- Internal **CA ClusterIssuer** via cert-manager; **trust-manager** distributes the CA bundle to all namespaces.
- Per-service certs cover the **Service DNS**; StatefulSets add **per-pod SANs**; all replicas share/trust correctly.
- East-west **mTLS** via a mesh (SPIFFE, auto-rotating) or app-managed for sensitive paths — no plaintext for regulated traffic.

---

8. Storage — Rook-Ceph, StorageClasses & Dynamic PV/PVC

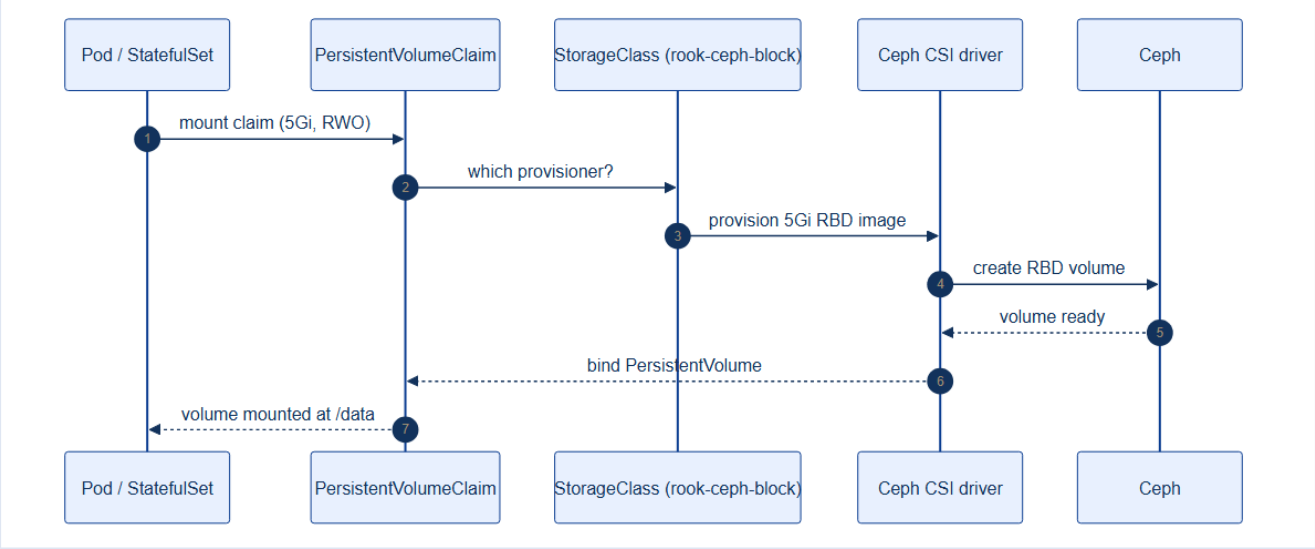
TicketHub's databases, Kafka logs, uploaded assets, and backups all need **durable storage that survives pod rescheduling**. On bare metal there's no cloud disk service, so we run our own distributed storage: **Ceph**, operated by **Rook**. This chapter also introduces the core Kubernetes storage abstractions — **PV, PVC, StorageClass** — that every stateful workload depends on.

8.1 The storage abstractions (from basics)

Kubernetes deliberately separates **what a workload wants** from **how storage is provided**:

Object	Analogy	Who creates it
PersistentVolumeClaim (PVC)	A **request**: "I need 5Gi, read-write-once"	App developer
StorageClass (SC)	A **catalog entry**: "gold = Ceph RBD, SSD"	Cluster architect
PersistentVolume (PV)	The **actual volume** provisioned	Auto-created by the provisioner

With **dynamic provisioning**, the developer only writes a PVC referencing a StorageClass; the PV is created automatically. No manual disk wrangling.

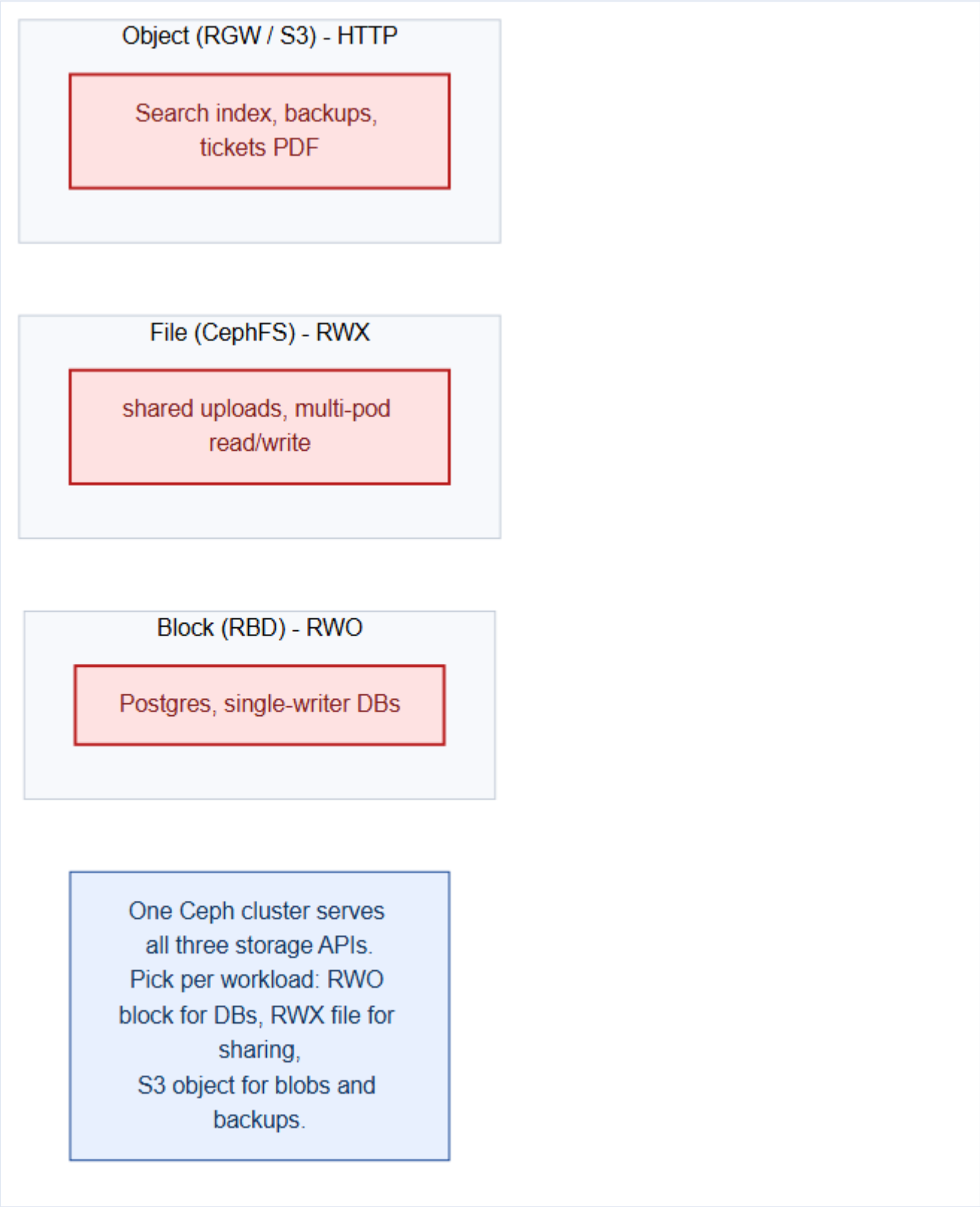


> **Mental model — ordering at a restaurant**

> A **PVC** is your **order** ("a 5Gi steak, well done"). The **StorageClass** is the **menu item** describing how the kitchen makes it. The **PV** is the **plated dish** delivered to your table. You (the app) never enter the kitchen (Ceph) — you just order from the menu and receive food.

8.2 Why Rook-Ceph on bare metal

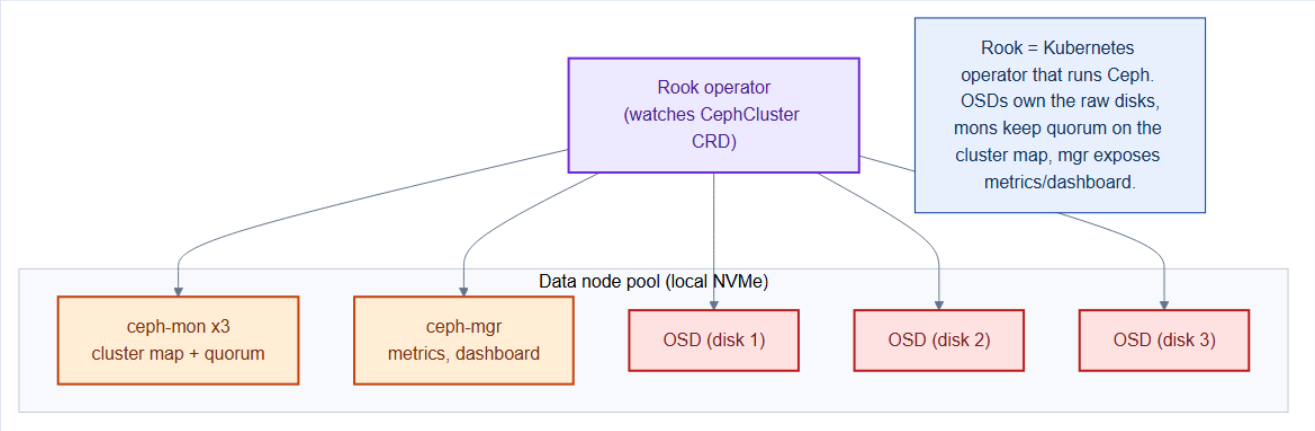
TicketHub needs three *kinds* of storage, and Ceph provides all three from one cluster:



Type	Ceph API	Access mode	TicketHub use

Block	RBD	`ReadWriteOnce`	Postgres, Kafka, Redis data dirs
File	CephFS	`ReadWriteMany`	Shared uploads read by many pods
Object	RGW (S3)	HTTP	Search index snapshots, backups, ticket PDFs

Rook is the Kubernetes **operator** that runs and manages Ceph for you — turning a notoriously complex storage system into declarative CRDs.



Ceph daemon	Role
mon (x3)	Keep the cluster map + quorum (odd count, like etcd)
mgr	Metrics, dashboard, orchestration
OSD (one per disk)	Store the actual data on raw NVMe

8.3 Deploying Rook-Ceph on the data pool

Ceph runs on the **data** node pool (tainted, local NVMe from Chapter 3):

```
# repo/manifests/10-platform/ceph-cluster.yaml (excerpt)
apiVersion: ceph.rook.io/v1
kind: CephCluster
metadata:
 name: rook-ceph
 namespace: rook-ceph
spec:
 mon: { count: 3, allowMultiplePerNode: false }   # quorum, spread across hosts
 storage:
   useAllNodes: false
   nodes:
     - name: worker-data-1
       devices: [{ name: "nvme1n1" }]
```

```
    - name: worker-data-2
      devices: [{ name: "nvme1n1" }]
    - name: worker-data-3
      devices: [{ name: "nvme1n1" }]
 placement:
   all:
     tolerations:
       - key: data
         operator: Exists
         effect: NoSchedule
     nodeAffinity:
       requiredDuringSchedulingIgnoredDuringExecution:
         nodeSelectorTerms:
           - matchExpressions:
               - { key: pool, operator: In, values: [data] }
```

8.4 Defining StorageClasses (the architect's catalog)

```
# repo/manifests/10-platform/storageclasses.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
 name: rook-ceph-block                 # the "gold" tier for databases
provisioner: rook-ceph.rbd.csi.ceph.com
parameters:
 pool: replicapool
 imageFormat: "2"
 csi.storage.k8s.io/fstype: ext4
reclaimPolicy: Retain                   # keep data if PVC deleted (safety for DBs)
allowVolumeExpansion: true              # grow volumes online
volumeBindingMode: WaitForFirstConsumer # bind only once a pod is scheduled
```

Two StorageClass settings every architect must set deliberately

- **`reclaimPolicy`**: `Retain` for anything precious (databases) so deleting a PVC doesn't wipe data; `Delete` for scratch/ephemeral.

- **`volumeBindingMode: WaitForFirstConsumer`**: delays PV creation until the pod is scheduled, so the volume lands in the **same failure domain/zone** as the pod. Critical for topology-aware placement.

CSI — the storage plugin standard

The `provisioner` value (`rook-ceph.rbd.csi.ceph.com`) is a **CSI (Container Storage Interface)** driver. CSI is the vendor-neutral plugin API that lets Kubernetes provision, attach, snapshot, and expand volumes on *any* backend — Ceph here, a cloud disk elsewhere — without Kubernetes knowing the storage internals. Rook ships the Ceph CSI driver; the StorageClass just names it.

8.5 A developer consuming storage

A stateful workload just asks — the platform delivers:

```
# See: repo/manifests/ for the full manifest
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: postgres-data
 namespace: data
spec:
 accessModes: [ReadWriteOnce]
 storageClassName: rook-ceph-block
 resources:
   requests:
     storage: 20Gi
```

```
kubectl get pvc,pv -n data
# PVC Bound to an auto-created PV, backed by a Ceph RBD image.
```

For StatefulSets (Postgres, Kafka), we don't even write PVCs by hand — `volumeClaimTemplates` mints one **per replica** with stable identity (Chapter 14).

8.6 Access modes (know the difference)

Mode	Meaning	Backed by
RWO ReadWriteOnce	One node mounts read-write	Ceph RBD (block)
RWX ReadWriteMany	Many nodes mount read-write	CephFS (file)
ROX ReadOnlyMany	Many nodes mount read-only	CephFS

Don't ask a database for RWX

Databases want **RWO block** storage — a single writer with block semantics. Trying to run Postgres on an **RWX** shared filesystem invites corruption. Reserve **RWX** (CephFS) for genuinely shared, concurrency-safe use (static assets, uploads).

8.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- Ceph's **CRUSH map** determines which OSDs receive each placement group's replicas. Without explicit CRUSH rules, all three replicas of a PG can land on OSDs of the same host — providing no host-level redundancy even with `replicasPerFailureDomain: 1`. Verify the CRUSH topology with `ceph osd crush tree` after cluster bootstrap.

- **`reclaimPolicy: Retain`** means deleting a PVC does NOT delete the backing Ceph RBD image. The image persists in Ceph consuming space indefinitely. You must manually `kubectl delete pv`

`<name>` AND run `rbd rm` or the Rook cleanup policy to actually reclaim the storage.

- The StorageClass `volumeBindingMode: WaitForFirstConsumer` delays PV provisioning until the pod is scheduled. This is intentional for topology awareness — but it means `kubectl get pvc` will show `Pending` until the first pod is scheduled, which can look like a bug during initial cluster setup.

Gotchas — traps that catch experienced engineers

- **OSD placement on VMs backed by the same Ceph cluster**: if your worker VMs' root disks are Ceph RBD images and you then run Ceph OSDs on those VMs, a Ceph outage prevents the VMs from booting, which prevents Ceph from recovering. Never run Ceph OSDs on VMs whose disks depend on Ceph.

- **Ceph health `HEALTH_WARN` on OSDs_down during node drain**: when you drain a data node for maintenance, its OSDs go down. Ceph begins backfilling immediately. If the node comes back within `osd_down_out_interval` (default 600s), no data movement occurs. If it takes longer, Ceph redistributes all PGs — generating significant I/O. Increase `osd_down_out_interval` during planned maintenance windows.

- **Block device selection for OSDs**: Rook requires dedicated block devices (no partition table, no filesystem). A freshly provisioned VM disk that was previously used as a Ceph OSD may have leftover LVM signatures — clean it with `wipefs -a /dev/sdX` and `dd if=/dev/zero of=/dev/sdX bs=1M count=10` before Rook can claim it.

Architect Considerations

1. **OSD count and raw capacity planning**: Ceph with 3× replication means usable capacity = raw capacity / 3. For 9 OSDs × 1TB each = 3TB raw, you get ~3TB usable. This needs to cover all PVCs plus the internal Ceph metadata overhead (~5%). Model capacity growth against your planned 3-year PVC growth rate.

2. **Separate OSD disks for block vs object**: Ceph block (RBD) and object (RGW) workloads have very different I/O patterns (random IOPS vs sequential throughput). Separate OSDs using all-flash for block and HDD for object storage if your hardware budget allows it.

3. **Rook operator upgrade isolation**: Rook manages CephCluster upgrades — but a Rook operator upgrade and a Ceph upgrade are separate events. Always upgrade Rook first, then trigger the Ceph image upgrade via the CephCluster CR. Never skip a Ceph minor version.

4. **Multi-site replication for DR**: Ceph RGW supports multi-site replication of object storage between clusters. If your DR strategy requires off-site data copies (Chapter 27), the Ceph multi-site design must be in place before the cluster goes live.

5. **Performance testing before production**: run `fio` benchmarks against both RBD and CephFS before declaring the storage layer production-ready. Key targets: Postgres needs > 5,000 random IOPS at 4K block size; Kafka needs > 200 MB/s sequential write throughput per broker.

Chapter 8 checklist

- **Rook operator + CephCluster** running on the **data** pool (3 mons, OSDs on NVMe).
- **StorageClasses** defined: block (DBs), file (shared), object (S3/backups).
- `reclaimPolicy` and `volumeBindingMode` set deliberately per tier.
- Dynamic provisioning verified: a PVC **Binds** to an auto-created PV.
- Access modes matched to workloads (RWO for DBs, RWX for shared).

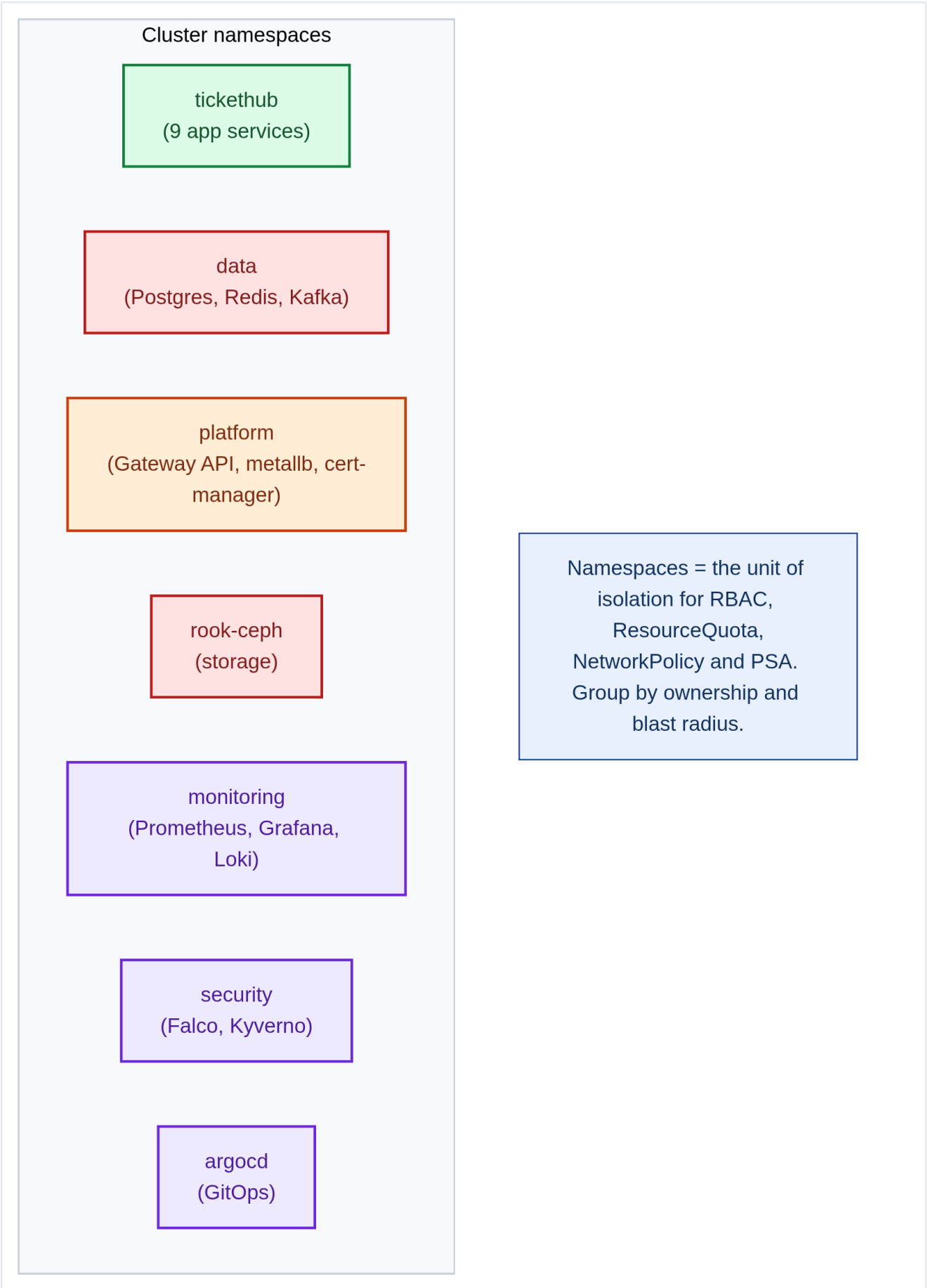
---

9. Namespaces, the Resource Model & Bootstrap Ordering

The cluster now has networking and storage. Before we deploy a single application pod, an architect defines the **organizational structure** — namespaces — and the **order** in which everything must come up. Get this wrong and you get tangled dependencies, noisy-neighbor problems, and security gaps.

9.1 Namespaces — the unit of isolation

A **namespace** is a virtual cluster inside the cluster. It's the boundary for **RBAC**, **ResourceQuota**, **NetworkPolicy**, and **Pod Security Admission** — nearly every governance control is applied per namespace.


Cluster namespaces
tickethub
(9 app services)
data
(Postgres, Redis, Kafka)
platform
(Gateway API, metallb, cert-manager)
rook-ceph
(storage)
monitoring
(Prometheus, Grafana, Loki)
security
(Falco, Kyverno)
argocd
(GitOps)
Namespaces = the unit of isolation for RBAC, ResourceQuota, NetworkPolicy and PSA. Group by ownership and blast radius.

Namespace	Contents	Why separate
`tickethub`	The 9 application services	The product; app-team RBAC
`data`	Postgres, Redis, Kafka	Stateful; stricter policy, data-team RBAC
`platform`	Gateway API, MetalLB, cert-manager	Shared infra; platform-team only
`rook-ceph`	Storage operator + Ceph	Isolated blast radius
`monitoring`	Prometheus, Grafana, Loki	Cross-cutting; read access clusterwide
`security`	Falco, Kyverno	Enforcement plane, tightly locked
`argocd`	GitOps controller	Deploys everything else

Mental model — departments in a company

Namespaces are **departments**. Each has its own budget (**ResourceQuota**), its own staff access (**RBAC**), its own security rules (**PSA/NetworkPolicy**), and its own door policy (who may talk to whom). You wouldn't give the summer intern keys to the finance vault — likewise the app team doesn't get write access to `rook-ceph`.

```
# repo/manifests/00-namespaces/namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
 name: tickethub
 labels:
   team: app
   pod-security.kubernetes.io/enforce: restricted   # PSA from day one (Ch 20)
   pod-security.kubernetes.io/warn: restricted
---
apiVersion: v1
kind: Namespace
metadata:
 name: data
 labels:
   team: data
   pod-security.kubernetes.io/enforce: baseline      # DBs may need slightly more
```

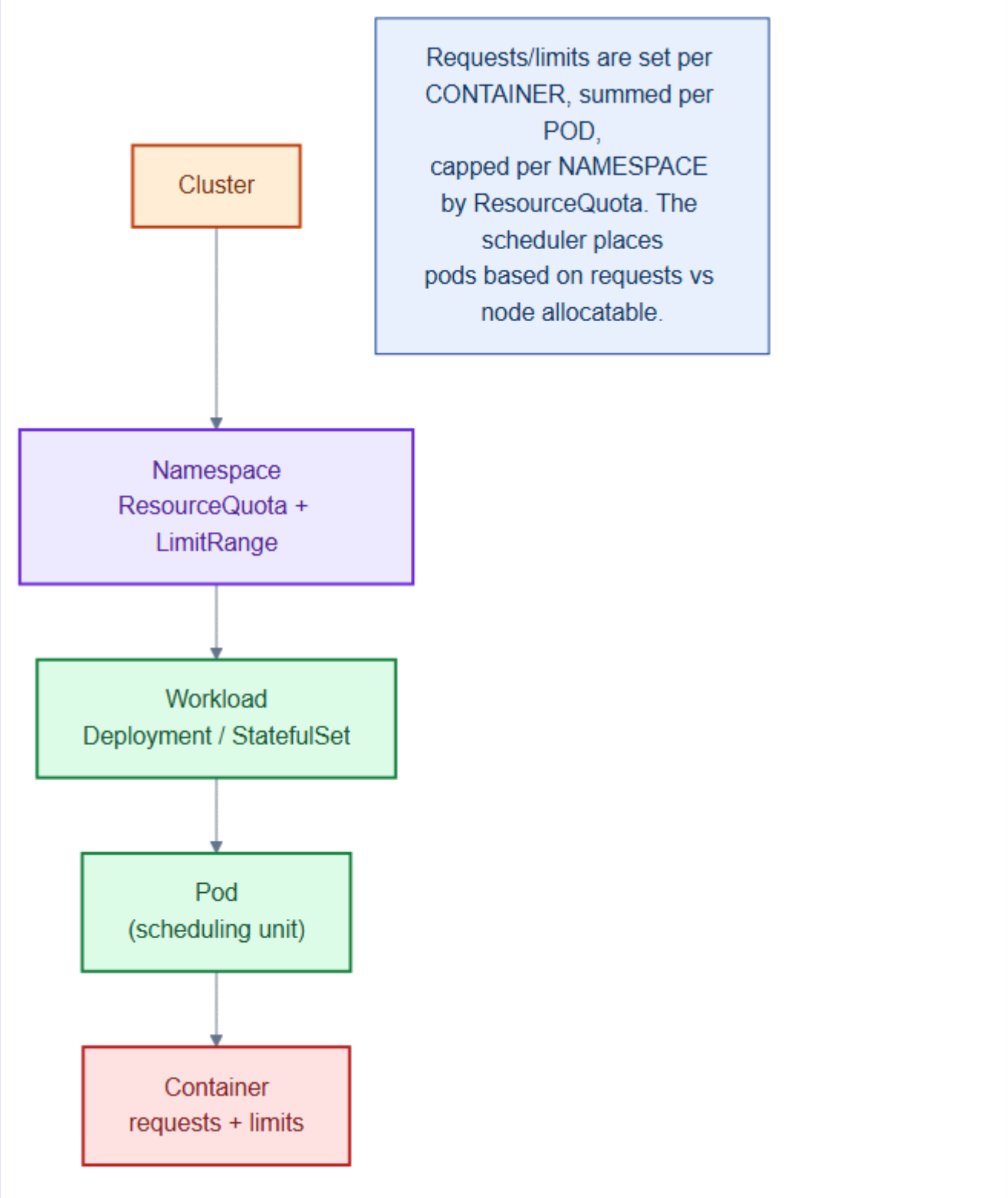
Apply Pod Security labels at namespace creation

Bake the `pod-security.kubernetes.io/enforce` label into the namespace definition from the start (Chapter 20). Adding it later to a namespace full of running, non-compliant workloads is a painful

> retrofit.

9.2 The resource model — how requests/limits nest

Understanding the **nesting** of resources is essential before deploying workloads (deep dive in Chapter 15):



- **Container** declares `requests` (guaranteed) and `limits` (ceiling) for CPU/memory.
- **Pod** resources = the sum of its containers'.
- **Namespace** caps the total via **ResourceQuota**, and sets per-container defaults via **LimitRange**.
- The **scheduler** places pods by comparing **requests** to each node's **allocatable** capacity.

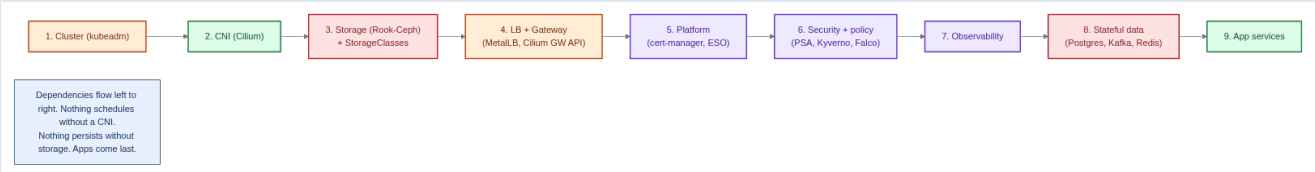
```
# repo/manifests/00-namespaces/quota-limits.yaml
apiVersion: v1
kind: ResourceQuota
metadata: { name: tickethub-quota, namespace: tickethub }
spec:
 hard:
   requests.cpu: "40"
   requests.memory: 80Gi
   limits.cpu: "80"
   limits.memory: 160Gi
   pods: "200"
   persistentvolumeclaims: "50"
---
apiVersion: v1
kind: LimitRange           # default requests/limits so pods can't be "unbounded"
metadata: { name: tickethub-defaults, namespace: tickethub }
spec:
 limits:
   - type: Container
     default:        { cpu: "500m", memory: 512Mi }
     defaultRequest: { cpu: "100m", memory: 128Mi }
```

A namespace without a ResourceQuota is a runaway risk

Without a quota, one buggy Deployment scaling to hundreds of pods can starve the whole cluster. A LimitRange also prevents pods that specify **no** requests (which the scheduler treats as near-zero, packing nodes dangerously). Set both on every app namespace.

9.3 Bootstrap ordering — dependencies flow one way

Platform components have a strict dependency order. Installing them out of order causes pods stuck `Pending` (no CNI), PVCs stuck unbound (no storage), or Services with no external IP (no MetalLB).



#	Layer	Depends on	Why
1	Cluster (kubeadm)	—	The foundation
2	**CNI (Cilium)**	1	**Nothing schedules** without pod networking

3	Storage (Rook-Ceph) + SCs	2	Stateful things need PVs
4	LB + Gateway (MetalLB, Cilium GW API)	2	External access
5	Platform (cert-manager, ESO)	3,4	TLS, secrets
6	Security/policy (PSA, Kyverno, Falco)	1	Enforce **before** apps land
7	Observability	3	Persist metrics/logs
8	Stateful data (Postgres, Kafka, Redis)	3	Needs storage
9	**App services**	all	Come **last**

Two ordering rules that prevent most bootstrap pain

1. **CNI before anything** — until a CNI is up, every pod is `Pending`.

2. **Policy before apps** — install PSA/Kyverno/Falco *before* workloads, so the first app pod is already governed. Retrofitting policy onto running apps means discovering violations in production.

9.4 Making bootstrap reproducible

An architect never clicks through this by hand. The ordering is encoded as **Argo CD sync waves** (Chapter 28) or a Helmfile, so the entire platform reconstructs itself deterministically:

```
# Argo CD sync-wave annotation encodes the order declaratively
metadata:
 annotations:
   argocd.argoproj.io/sync-wave: "2"    # Cilium in wave 2, apps in wave 9
```

9.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Namespaces are not a security boundary** — a compromised pod in `tickethub` namespace can still reach pods in `data` namespace via ClusterIP Services. Namespaces provide isolation of RBAC, ResourceQuota, and LimitRange; NetworkPolicy provides the actual traffic boundary.

- **ResourceQuota does not enforce existing objects**: creating a ResourceQuota on a namespace that already has running pods doesn't evict them even if they exceed the quota. The quota only applies to NEW objects. Audit existing resource usage before applying quota to a live namespace.

- **`LimitRange` defaults apply at admission time**, not retroactively. Pods created before the LimitRange was applied keep their original (possibly unlimited) resource settings. This creates a mixed-state namespace that is hard to reason about.

Gotchas — traps that catch experienced engineers

- **Namespace deletion hangs**: `kubectl delete namespace tickethub` hangs if any object in the namespace has a finalizer that hasn't been cleared. Common culprits: Velero backup objects, cert-manager Certificates with `deleteOnTermination`, and CRDs with cross-namespace references. Debug with `kubectl get namespace tickethub -o json | jq '.spec.finalizers'`.

- **Cross-namespace Service access requires full DNS name**: a pod in `tickethub` that calls `postgres` (short hostname) resolves to `postgres.tickethub.svc.cluster.local` — which doesn't exist. The full name `postgres.data.svc.cluster.local` is required. Enforce this via ConfigMap (`DATABASE_URL`) values, not code.

- **ResourceQuota on `count/pods` without `count/deployments`**: setting a pod quota without a Deployment quota means a misbehaving Deployment can create thousands of pods quickly (e.g., a crash-loop flood) until the quota kicks in — but by then the node is already under pressure. Always pair pod quotas with replica-level limits.

Architect Considerations

1. **Namespace proliferation governance**: namespaces are cheap to create but expensive to manage (each needs quota, RBAC, NetworkPolicy, possibly LimitRange). Define a namespace creation policy: who can create namespaces, what template gets applied, how are they decommissioned?

2. **Soft vs hard multi-tenancy**: namespaces provide soft tenancy (API isolation + RBAC). Hard tenancy (cryptographic workload isolation) requires separate clusters or vCluster. For a single TicketHub cluster where all tenants are internal teams, namespace-based soft tenancy is sufficient.

3. **Bootstrap ordering and GitOps**: the bootstrap order (namespaces first, quotas second, platform CRDs third, workloads last) must be encoded in Argo CD Application sync waves (Chapter 28) so that a full cluster restore doesn't fail on ordering dependencies.

4. **LimitRange and VPA interaction**: VPA (Chapter 16) overwrites pod resource requests at admission time — but VPA recommendations must fit within the LimitRange min/max. If they don't, VPA silently skips the pod. Verify LimitRange maximums are generous enough for VPA to work.

5. **Quota for ephemeral storage**: `ephemeral-storage` requests/limits are rarely set but can cause node eviction under log-spamming containers. Add `ephemeral-storage` to your LimitRange defaults.

Chapter 9 checklist — Part II complete

- **Namespaces** created with team labels + **PSA** labels from day one.
- **ResourceQuota + LimitRange** on every app namespace.
- The **bootstrap order** understood and encoded (CNI → storage/LB → policy → data → apps).
- Platform install made **reproducible** (GitOps sync waves).

The platform is now ready. **Part III** containerizes the 9 TicketHub services and deploys them with the right workload controllers.

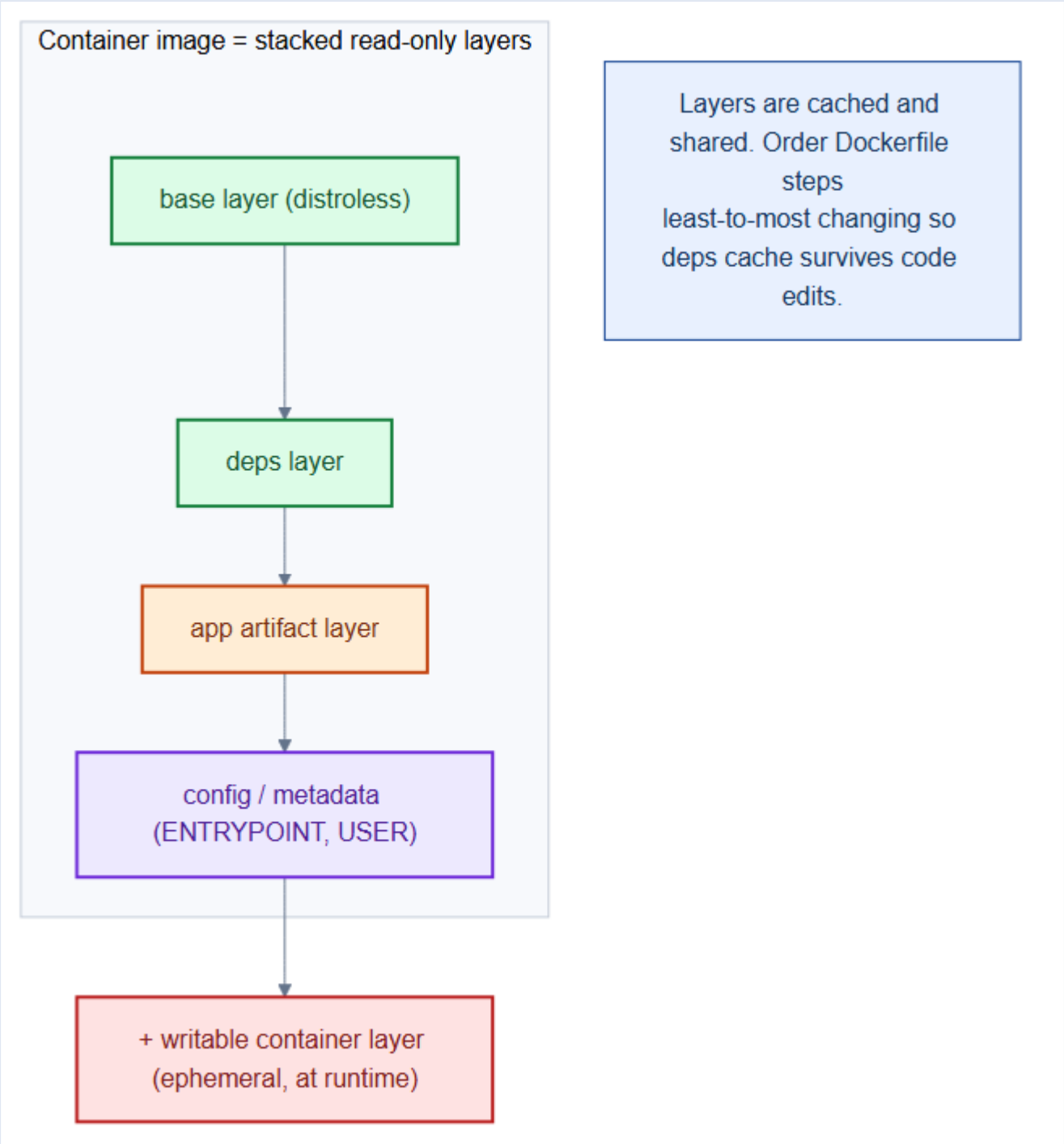
---

10. Containerizing the Services — Dockerfiles Done Right

The platform is running; now we ship code onto it. Every one of TicketHub's 9 services becomes a **container image**. How you build that image decides your **security posture, image size, cold-start speed, and supply-chain trust**. This chapter builds production-grade Dockerfiles from first principles.

10.1 What a container image actually is

An image is not a VM — it's a **stack of read-only layers** plus metadata (the `ENTRYPOINT`, the `USER`, environment). At runtime the container adds one thin **writable layer** on top.



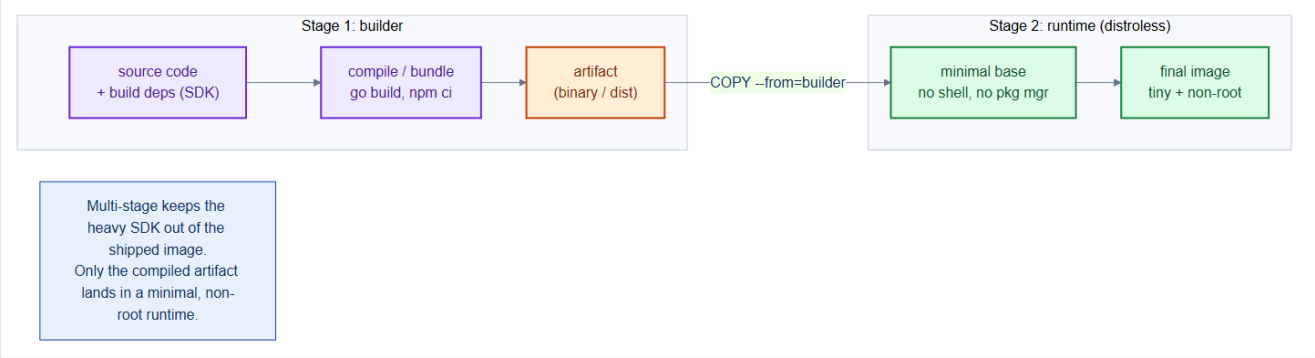
Property	Consequence for the architect
Layers are **cached**	Order Dockerfile steps least-changing → most-changing
Layers are **shared**	A common base image is downloaded once per node
Writable layer is **ephemeral**	Anything not on a volume is lost on restart

> **Mental model — a printed book vs. scratch paper**

> The image layers are the **printed pages** of a book — fixed, shared by every reader. The writable container layer is a **sheet of scratch paper** clipped on top: you can scribble on it, but it's thrown away when the container dies. Durable data goes to a **volume** (Chapter 14), never the scratch paper.

10.2 Multi-stage builds — the single most important technique

A naive Dockerfile ships the entire build toolchain (compilers, `node_modules` with dev deps, package managers) inside the running image — bloated and full of attack surface. **Multi-stage builds** compile in one stage and copy only the finished artifact into a tiny runtime stage.



Here is TicketHub's **Orders** service (Go) as a multi-stage build:

```
# repo/services/orders/Dockerfile
# ---- Stage 1: build ----
FROM golang:1.25 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download                # cached unless deps change
COPY . .
RUN CGO_ENABLED=0 go build -o /out/orders ./cmd/orders

# ---- Stage 2: runtime (distroless, non-root) ----
FROM gcr.io/distroless/static:nonroot
COPY --from=builder /out/orders /orders
USER 65532:65532                   # 'nonroot' uid, never root
EXPOSE 8080
ENTRYPOINT ["/orders"]
```

The **API Gateway** (Go) follows the same distroless pattern — it's a pure stdlib reverse proxy so the final image has zero external dependencies:

```
# repo/services/gateway/Dockerfile
FROM golang:1.22 AS builder
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -o /out/gateway ./cmd/gateway

FROM gcr.io/distroless/static:nonroot
COPY --from=builder /out/gateway /gateway
USER 65532:65532
EXPOSE 8080
ENTRYPOINT ["/gateway"]
```

And the **Frontend** (Node/React) service, build → unprivileged NGINX runtime:

```
# repo/services/frontend/Dockerfile
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install                    # installs deps from package.json
COPY . .
RUN npm run build                  # Vite produces dist/

FROM nginxinc/nginx-unprivileged:1.27-alpine   # runs as non-root by design
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 8080
```

Four Dockerfile rules every TicketHub image obeys

1. **Multi-stage** — the SDK never ships to production.
2. **Minimal base** — `distroless` or `-alpine`; no shell, no package manager = tiny attack surface.
3. **Non-root `USER`** — a compromised process isn't uid 0 (pairs with `runAsNonRoot`, Chapter 20).
4. **Pin versions + use lockfiles** — `go.sum` / `package-lock.json` for reproducible, auditable builds.

10.3 Distroless vs. Alpine vs. full OS

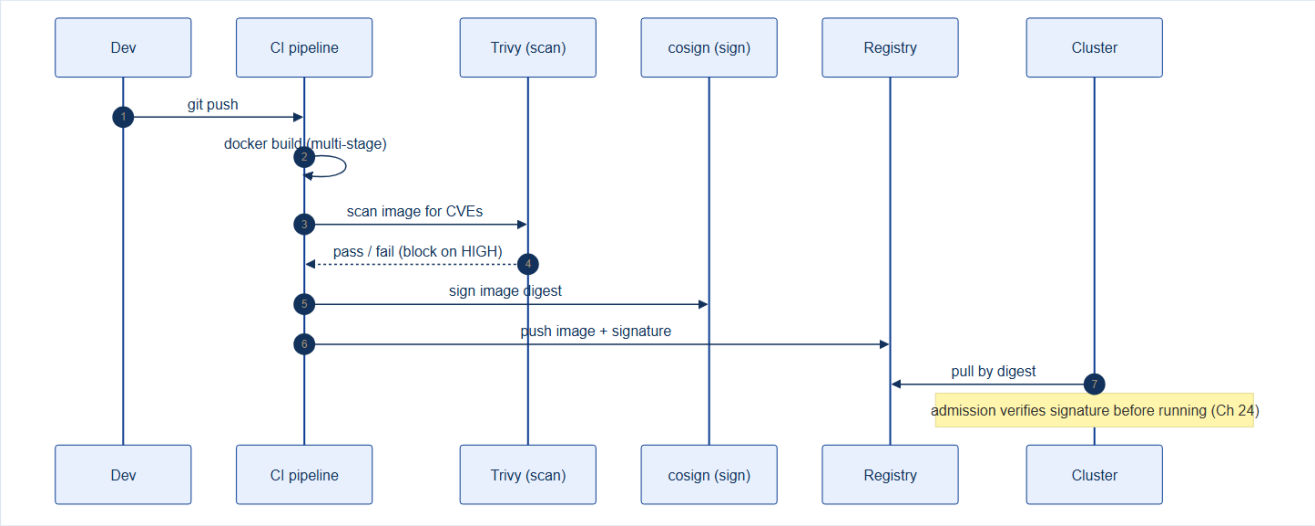
Base	Size	Shell?	When to use
`distroless/static`	~2 MB	none	Static Go/Rust binaries (best)
`alpine`	~7 MB	`/bin/sh`	Needs libc/tools, still small

`debian-slim`	~75 MB	full	Glibc/native deps required
full `ubuntu`	~180 MB	full	Avoid for services

Smaller isn't just disk — it's **fewer CVEs to patch** and **faster pulls** during a scale-up or node failure.

10.4 The build → trust → run supply chain

An image isn't trusted just because it built. In production every image is **scanned** for CVEs and **signed**, and the cluster verifies the signature before running it (enforced in Chapter 24).



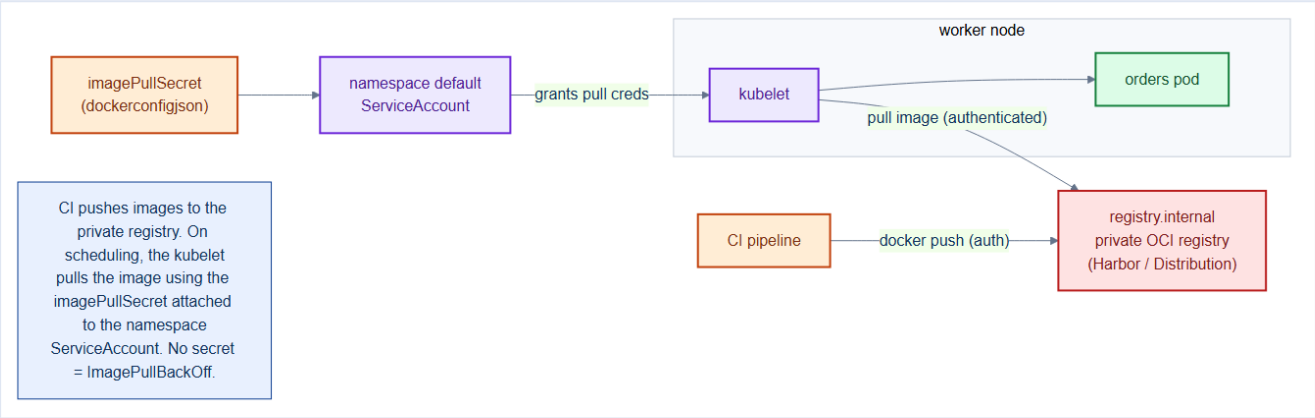

```
# In CI, per service:
docker build -t registry.internal/tickethub/orders:$(git rev-parse --short HEAD) .
trivy image --exit-code 1 --severity HIGH,CRITICAL registry.internal/tickethub/orders:$SHA
cosign sign --key cosign.key registry.internal/tickethub/orders:$SHA
docker push registry.internal/tickethub/orders:$SHA
```

> **Never deploy the `latest` tag to production**

> `latest` is a moving target — two nodes can pull *different* images under the same name, and rollbacks become guesswork. Deploy an **immutable tag** (git SHA) or, best, the **image digest** (`@sha256:...`). Reproducibility and rollback depend on it.

10.5 The private registry — where images live and how the cluster pulls them

Every image so far has been named `registry.internal/tickethub/...`. That prefix is not decoration — it's the hostname of **TicketHub's own container registry**, a service the platform team runs. On bare metal there's no cloud registry handed to you; you host one (Harbor, or the CNCF **Distribution** `registry:2`) so images never depend on Docker Hub being up or rate-limiting your node pulls, and so every image can be **scanned and signed** under your control.



Pushing is what CI does (previous section). **Pulling** is the part beginners miss: when the scheduler places a pod, the **kubelet** on that node pulls the image — and a private registry demands credentials. Anonymous pulls get `ErrImagePull` / `ImagePullBackOff`. You give the kubelet those credentials with an **imagePullSecret**: a Secret of type `kubernetes.io/dockerconfigjson`.

```
# Create the registry credential Secret in the namespace that needs it.
kubectl create secret docker-registry registry-internal \
 --namespace tickethub \
 --docker-server=registry.internal \
 --docker-username=ci-puller \
 --docker-password="$REGISTRY_TOKEN"
```

You can reference it two ways:

```
# (a) Per pod — explicit, but repeated in every workload:
spec:
 imagePullSecrets:
   - name: registry-internal
 containers:
   - name: orders
     image: registry.internal/tickethub/orders:v1
```

```
# (b) Better — attach it once to the namespace's default ServiceAccount, so
#     EVERY pod in the namespace inherits it with no per-workload change:
#     repo/manifests/10-platform/registry-pull-secret.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: default
 namespace: tickethub
imagePullSecrets:
 - name: registry-internal
```

Attach pull secrets to the ServiceAccount, not every Deployment

Option (b) is the pattern to reach for: one edit per namespace and every current and future pod can pull, with nothing to forget in individual manifests. Reserve per-pod `imagePullSecrets` for the odd workload that needs a *different* registry.

Don't commit the registry password to Git

> A `dockerconfigjson` Secret holds a real credential. Create it with `kubectl` (above) or, better, manage it with **External Secrets / Sealed Secrets** (Chapters 13 & 24) so Git never sees the plaintext token. The manifest in the repo is only the **ServiceAccount attachment** — the Secret itself is created out-of-band.

10.6 Running services locally

All three runnable services can be started on a development machine without Docker or a cluster. Each reads its upstream addresses from environment variables and falls back to `localhost` defaults.

Orders (Go — with Prometheus metrics + OpenTelemetry tracing):

```
cd repo/services/orders
go mod tidy          # resolve indirect deps on first run
go run ./cmd/orders  # listens :8080 (API) and :9090 (metrics)
```

Test: `curl http://localhost:8080/healthz` → `ok`
Test: `curl http://localhost:8080/api/orders` → JSON stub response

Gateway (Go — pure stdlib reverse proxy):

```
cd repo/services/gateway
# Point at wherever orders/catalog are running:
export ORDERS_URL=http://localhost:8080
export CATALOG_URL=http://localhost:8082   # stub if catalog isn't running
go run ./cmd/gateway                        # listens :8080 (gateway port)
```

Test: `curl http://localhost:8080/healthz` → `ok`
Test: `curl http://localhost:8080/api/orders` → proxied to orders service

Frontend (React + Vite):

```
cd repo/services/frontend
npm install          # first time only
npm run dev          # Vite dev server on :3000, proxies /api → gateway :8080
```

Open `http://localhost:3000` — the UI has two tabs: **Events** (calls `/api/catalog/events`) and **Orders** (calls `/api/orders`). If the backend isn't running, each tab shows a clear "could not reach service" error rather than a blank page.

> **Full local stack**

> Run orders on `:8081`, gateway on `:8080` pointing `ORDERS_URL=http://localhost:8081`, then `npm run dev` for the frontend. The Vite proxy handles CORS automatically in dev — no browser changes needed.

10.5 Nuances, Gotchas & Architect Considerations

> **Nuances — subtle behaviours to internalise**

- **Multi-stage build cache invalidation**: Docker's layer cache is invalidated from the first changed layer downward. Copying `go.mod`/`go.sum` BEFORE `COPY . .` means `go mod download` is only re-run when dependencies change, not on every code change. This is the single biggest build-time optimization for Go services.

- **Distroless images contain no shell** — you cannot `kubectl exec -it -- /bin/sh` into them for debugging. Instead, use ephemeral debug containers: `kubectl debug -it pod/X --image=busybox --target=go-service`. Never re-add a shell to a distroless prod image just for convenience.

- **Non-root UID must be consistent across image layers**: if the `COPY --from=builder` copies files owned by `root` and then the `USER 1000` directive switches to non-root, the app may not be able to read its own files at runtime. Always `COPY --chown=1000:1000` in the final stage, or set ownership in the builder stage.

Gotchas — traps that catch experienced engineers

- **`latest` tag in production**: `image: myapp:latest` combined with `imagePullPolicy: Always` means every pod restart pulls a new image — including breaking changes deployed after the pod was last scheduled. Always use immutable digest tags (`sha256:...`) or semver tags in production manifests.

- **Secret injection via build ARGs**: `ARG DB_PASSWORD` makes the secret visible in `docker history` and Docker layer cache. Secrets must be injected at **runtime** via environment variables or mounted files, never baked into the image layer.

- **Multi-arch build assumption**: building on an ARM Mac and pushing to a registry used by x86 nodes will cause `exec format error` on pod startup. Always build for `linux/amd64` explicitly in CI, or use `docker buildx` multi-platform manifests.

Architect Considerations

1. **Image registry strategy**: a private registry (registry.internal.tickethub.io) is required for images that contain proprietary business logic. Decide: run Ceph/Harbor on-cluster (adds operational burden) or use an external private registry (adds a network dependency and egress cost)?

2. **Base image governance**: who owns the base images (`golang:1.25-alpine`, `gcr.io/distroless/base`)? A team that pulls base images without verification is vulnerable to supply-chain attacks (Chapter 24). Define a process for: base image approval, vulnerability scanning, and scheduled rebuilds when base image CVEs are published.

3. **Build reproducibility**: a Dockerfile without pinned base image digests is not reproducible — two builds of the same commit can produce different images if the base image has been updated. Pin base images by digest in Dockerfiles for production services.

4. **Layer size vs layer count trade-off**: fewer, larger layers are generally faster to push/pull (fewer HTTP requests) but harder to cache incrementally. For a microservice with 50 MB of dependencies and 5 MB of code, separate layers make sense; for a 500 MB monolith, reconsider the decomposition.

5. **SBOM and CVE scanning integration**: generate a Software Bill of Materials (`syft`) and scan it with `grype` in the CI pipeline. Block merges when HIGH/CRITICAL CVEs are introduced. This is the first line of supply-chain defence (Chapter 24).

Chapter 10 checklist

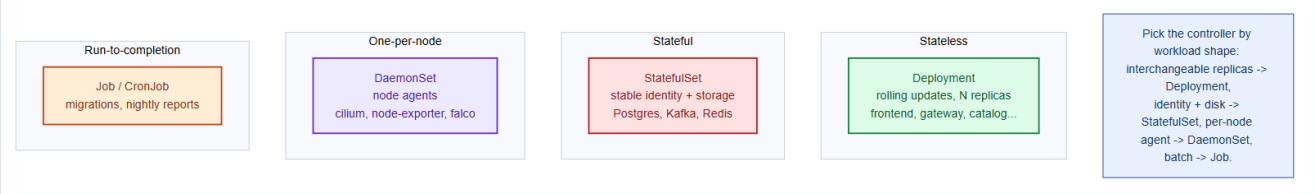
- Every service uses a **multi-stage** Dockerfile; SDK excluded from runtime.
- Runtime base is **distroless/alpine**, running as a **non-root** `USER`.
- Dependencies **pinned** with lockfiles; deps layer ordered for cache reuse.
- CI **scans (Trivy)** and **signs (cosign)** every image.
- Deploys reference an **immutable tag/digest**, never `latest`.
- Images live in the **private registry**; nodes pull with an **imagePullSecret** attached to the namespace **ServiceAccount**.
- **Gateway** and **Frontend** are fully runnable locally (`go run` / `npm run dev`).

---

11. Workload Controllers — Deployments, ReplicaSets, StatefulSets & DaemonSets

A pod is the smallest unit Kubernetes runs — but you almost **never** create a bare pod. A bare pod that dies stays dead. Instead you declare a **controller** that owns pods and continuously reconciles reality toward your desired state. Choosing the *right* controller for each workload shape is a core architect decision.

11.1 The controller family



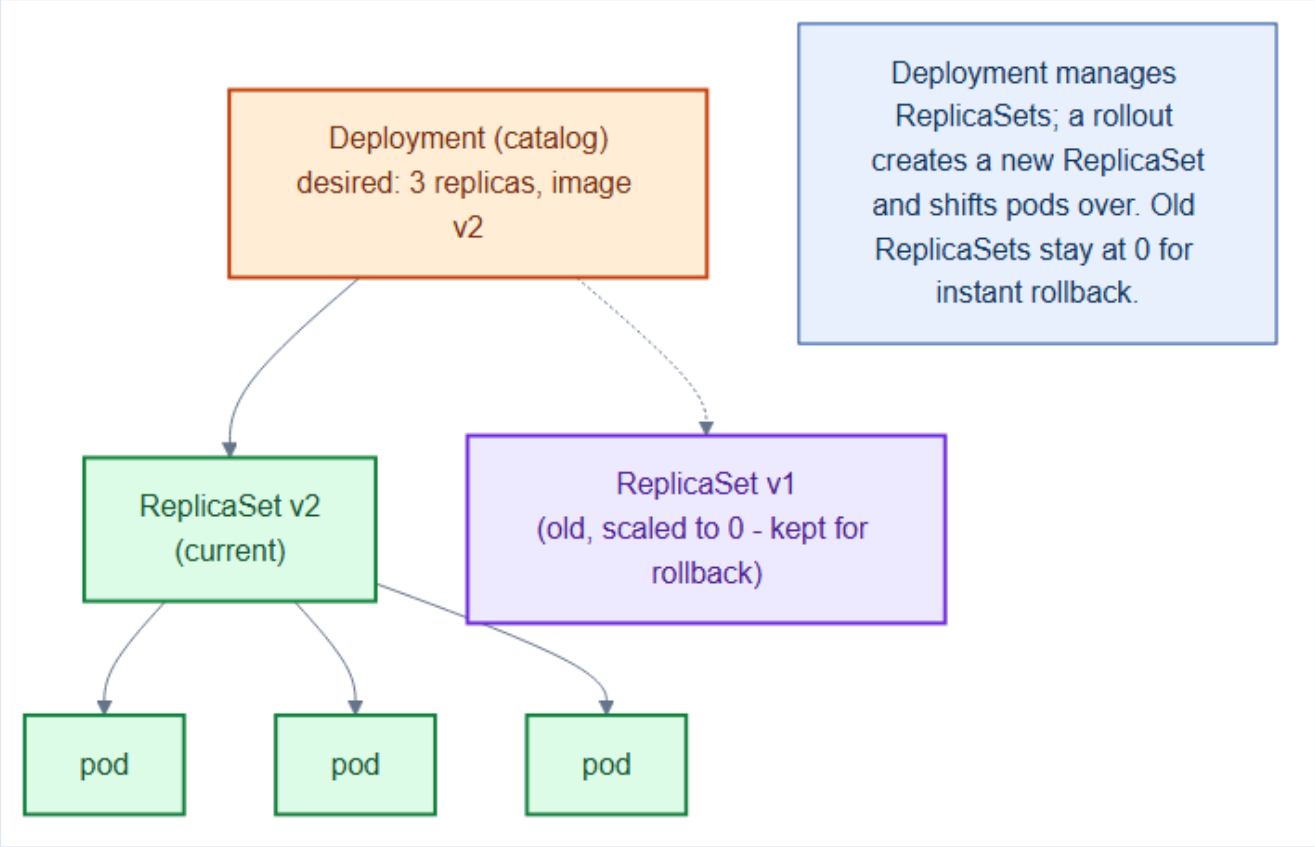
Controller	Guarantees	TicketHub workloads
Deployment	N interchangeable replicas, rolling updates, instant rollback	frontend, gateway, users, catalog, orders, payments, notifications, search
StatefulSet	Stable identity + stable per-pod storage, ordered ops	Postgres, Kafka, Redis (Ch 14)
DaemonSet	Exactly one pod per (matching) node	cilium, node-exporter, Falco, log shipper
Job / CronJob	Run to completion, once or on schedule	DB migrations, nightly reports

> **Mental model — cattle, pets, and sentries**

> **Deployment** pods are **cattle**: identical, numbered by chance, replaced without ceremony. **StatefulSet** pods are **pets**: each has a name and its own belongings (disk) that follow it. **DaemonSet** pods are **sentries**: one posted on every node, watching that node.

11.2 Deployments and the ReplicaSet underneath

A **Deployment** doesn't manage pods directly — it manages **ReplicaSets**. Each rollout creates a *new* ReplicaSet and gradually shifts pods to it, keeping the old one at zero for instant rollback.




```
# repo/manifests/30-workloads/catalog-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: catalog
 namespace: tickethub
spec:
 replicas: 3
 selector:
   matchLabels: { app: catalog }
 strategy:
   type: RollingUpdate
   rollingUpdate:
     maxUnavailable: 0        # never drop below desired during a rollout
     maxSurge: 1              # add one extra pod at a time
 template:
   metadata:
     labels: { app: catalog }
   spec:
     containers:
       - name: catalog
         image: registry.internal/tickethub/catalog@sha256:...  # immutable digest
         ports: [{ containerPort: 8080 }]
         resources:
           requests: { cpu: "100m", memory: 128Mi }
           limits:   { cpu: "500m", memory: 512Mi }
```

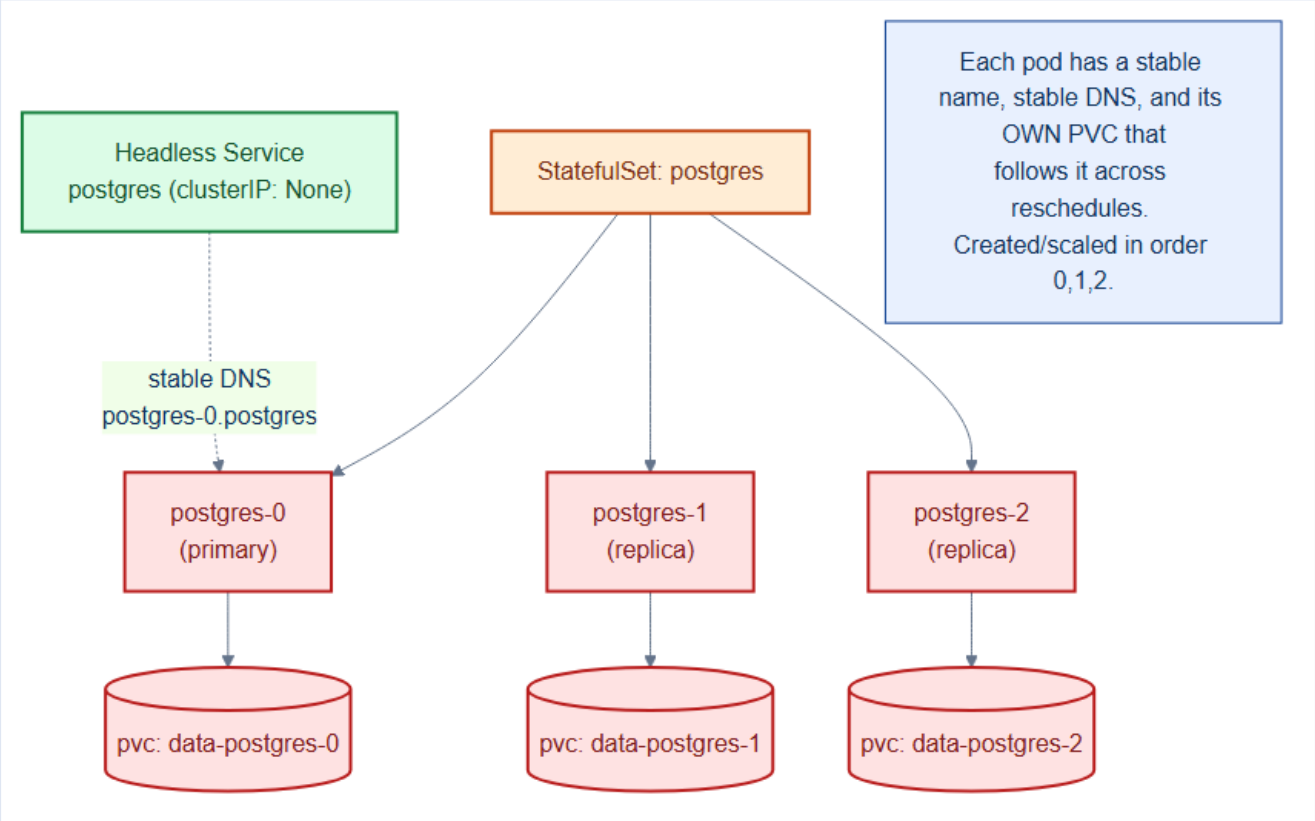
```
kubectl -n tickethub set image deploy/catalog catalog=...:v2   # triggers new ReplicaSet
kubectl -n tickethub rollout status deploy/catalog             # watch it converge
kubectl -n tickethub rollout undo deploy/catalog               # instant rollback
```

maxUnavailable: 0 for user-facing services

Setting `maxUnavailable: 0` with `maxSurge: 1` means Kubernetes brings up a new pod *before* removing an old one — zero capacity dip during deploys. Combine with **readiness probes** (Chapter 18) so traffic only shifts to pods that are actually up.

11.3 StatefulSets — when identity and storage matter

Stateless replicas are interchangeable; a database replica is **not**. `postgres-0` is the primary, has its *own* data, and must keep its name and disk across reschedules. That's a **StatefulSet**.



Deployment	StatefulSet
Random pod names (`catalog-7d9f-xk2`)	Ordinal names (`postgres-0`, `-1`, `-2`)
Shared or no storage	One PVC **per pod**, sticks to it
Any order, parallel	Ordered create/scale/delete (0→1→2)
ClusterIP Service	Usually a **headless** Service for stable DNS

We build these fully in Chapter 14; the point here is **controller selection**: identity + per-pod disk ⇒ StatefulSet.

11.4 DaemonSets — one pod per node

Some software must run on **every** node: the CNI agent, a metrics exporter, a security sensor. A **DaemonSet** guarantees exactly that, and automatically places a pod on any node that joins later.

```
# repo/manifests/70-observability/node-exporter-daemonset.yaml (excerpt)
apiVersion: apps/v1
kind: DaemonSet
metadata:
 name: node-exporter
 namespace: monitoring
spec:
 selector:
   matchLabels: { app: node-exporter }
 template:
   metadata:
     labels: { app: node-exporter }
   spec:
     tolerations:                # so it also lands on tainted data/infra nodes
       - operator: Exists
     containers:
       - name: node-exporter
         image: quay.io/prometheus/node-exporter:v1.8.0
         ports: [{ containerPort: 9100, hostPort: 9100 }]
```

Add broad tolerations to node-wide agents

Because data and infra nodes are **tainted** (Chapter 3), a DaemonSet that must cover *all* nodes needs `tolerations: [{ operator: Exists }]`. Otherwise your metrics or security agent silently skips exactly the nodes you most need to watch.

11.5 Jobs & CronJobs — batch work

Schema migrations and scheduled reports aren't long-running services — they **finish**.

```
# repo/manifests/30-workloads/db-migrate-job.yaml
apiVersion: batch/v1
kind: Job
metadata: { name: orders-db-migrate, namespace: tickethub }
spec:
 backoffLimit: 3
 template:
   spec:
     restartPolicy: Never
     containers:
       - name: migrate
         image: registry.internal/tickethub/orders-migrate@sha256:...
         command: ["/migrate", "up"]
```

Run migrations as a **Job** (often an Argo CD *PreSync* hook, Chapter 28) so schema changes land before the new app version rolls out.

11.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- A **ReplicaSet** is the actual controller that maintains the desired pod count — a Deployment manages ReplicaSets, not pods directly. When you do a rolling update, the Deployment creates a NEW ReplicaSet and scales it up while scaling down the old one. The old ReplicaSet (with 0 replicas) is kept as rollback history — `kubectl rollout history` lists them.

- **DaemonSets bypass the scheduler's bin-packing** — they place exactly one pod per matching node regardless of available resources. If a DaemonSet pod's resource requests cannot fit on a node, the pod stays `Pending` on that node forever (no eviction of other pods to make room). Size DaemonSet pods conservatively.

- **StatefulSet** `podManagementPolicy: Parallel` allows all pods to start simultaneously (faster), but the default `OrderedReady` is safer for databases that require pod-0 to be the primary before pod-1 starts replication. Never switch a Postgres StatefulSet to Parallel without understanding the startup coordination consequences.

Gotchas — traps that catch experienced engineers

- `kubectl delete rs` **on a Deployment-owned ReplicaSet**: the Deployment controller immediately re-creates the ReplicaSet. This is not a rollback — it creates a fresh RS with the same pod template. To rollback, use `kubectl rollout undo deployment/X`.

- **StatefulSet rolling update leaves a failed pod blocking the rollout**: if pod-0 fails its readiness probe after the update, the rollout pauses — pods 1 and 2 are never updated. The rollout does not time out or auto-rollback. You must manually fix pod-0 OR run `kubectl rollout undo` to unblock.

- **DaemonSet on tainted nodes**: a new `NoSchedule` taint on a node does not evict existing DaemonSet pods. The taint only affects newly scheduled pods. If you retaint a node, DaemonSet pods on it are unaffected until the next pod restart.

Architect Considerations

1. **Deployment vs StatefulSet boundary**: the line is "does pod identity matter?" If `orders-pod-7f4d` can replace `orders-pod-a1b2` without any state transfer, use a Deployment. If each pod has a named role (Postgres primary vs standby), use a StatefulSet. The gray area is services that use external session stores (Redis) — they are truly stateless and should use Deployments.

2. **DaemonSet for security vs performance**: Falco, node-exporter, and Cilium agents are natural DaemonSets. But a heavy DaemonSet (e.g., a 512Mi baseline logging agent) on every node burns constant cluster-wide RAM. Always benchmark DaemonSet overhead per node type before deploying.

3. **Job completion vs Deployment for one-time tasks**: `db-migrate-job.yaml` (repo/manifests/30-workloads/) runs schema migrations as a Job. Migrations run in a Deployment would run in a loop forever. The key Job parameters: `backoffLimit: 3` (max retries), `restartPolicy: Never` (don't restart the pod on failure, create a new one instead).

4. **CronJob concurrency policy**: `concurrencyPolicy: Forbid` means if the previous CronJob run is still running when the next trigger fires, the new run is skipped. For backup or batch jobs where overlapping runs would corrupt output, `Forbid` is the right choice; for idempotent jobs, `Allow` gives better throughput.

5. **Pod disruption budget interaction with Deployments**: a PDB with `minAvailable: 2` on a 3-replica Deployment means a `kubectl rollout restart` will drain only one pod at a time — the rollout serializes. This is the correct behavior for zero-downtime restarts but multiplies the rollout duration by the replica count.

Chapter 11 checklist

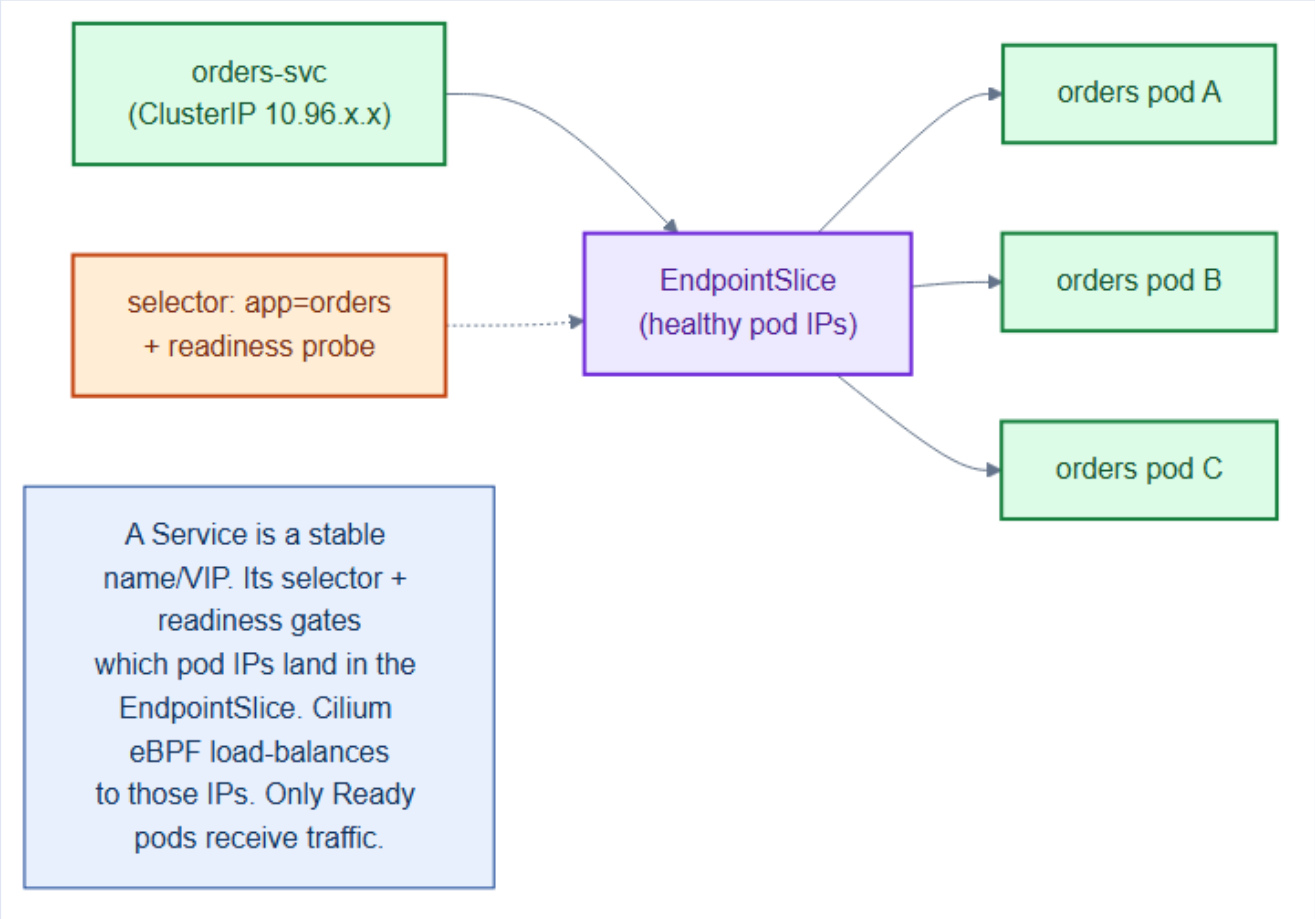
- Stateless services run as **Deployments** with `maxUnavailable: 0`, immutable image digests.
- Databases/brokers run as **StatefulSets** (identity + per-pod storage).
- Node-wide agents run as **DaemonSets** with `Exists` tolerations.
- Migrations/scheduled work run as **Jobs/CronJobs**, not long-lived pods.
- No bare pods anywhere — every pod is owned by a controller.

---

12. Services & Traffic — ClusterIP, Headless & Gateway Routing

Pods are **ephemeral** — they get new IPs on every restart. If the Orders service called Payments by pod IP, it would break constantly. A **Service** solves this: a stable name and virtual IP in front of a changing set of pods. This chapter covers how traffic actually flows through TicketHub, east-west and north-south.

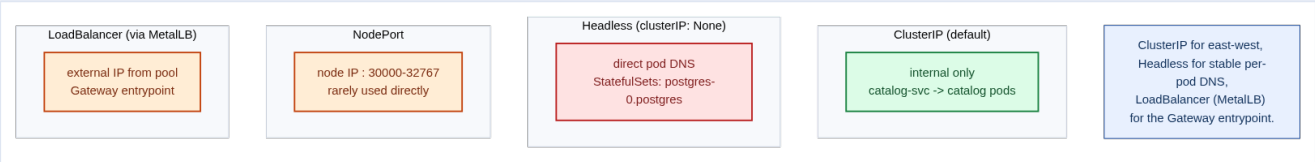
12.1 Why Services exist



A Service has a **selector** (`app: orders`). Kubernetes continuously maintains an **EndpointSlice** — the list of *Ready* pod IPs matching that selector. Cilium's eBPF datapath load-balances traffic across those IPs. When pods come, go, or fail readiness, the EndpointSlice updates automatically.

> **Mental model — a restaurant phone number**
>
> A Service is the restaurant's **published phone number**. Individual staff (pods) come and go, shifts change, but the number never does. You call the number; whoever is on duty and ready (passed the readiness probe) picks up. You never memorize an individual employee's cell (pod IP).

12.2 The Service types



Type	Reachable from	TicketHub use
ClusterIP (default)	Inside cluster only	All 9 services talk to each other
Headless (`clusterIP: None`)	Direct per-pod DNS	StatefulSets (Postgres, Kafka)
NodePort	`nodeIP:30000–32767`	Rarely direct; building block
LoadBalancer	External IP (MetalLB)	The Gateway's Service only

The standard TicketHub Service is a plain **ClusterIP**:

```
# repo/manifests/30-workloads/orders-deployment.yaml (Service section)
apiVersion: v1
kind: Service
metadata:
 name: orders
 namespace: tickethub
spec:
 selector: { app: orders }        # matches the Deployment's pod labels
 ports:
   - port: 80                      # stable virtual port
     targetPort: 8080              # container port
```

Now any pod resolves it by DNS: `http://orders.tickethub.svc.cluster.local` (or just `orders` within the same namespace).

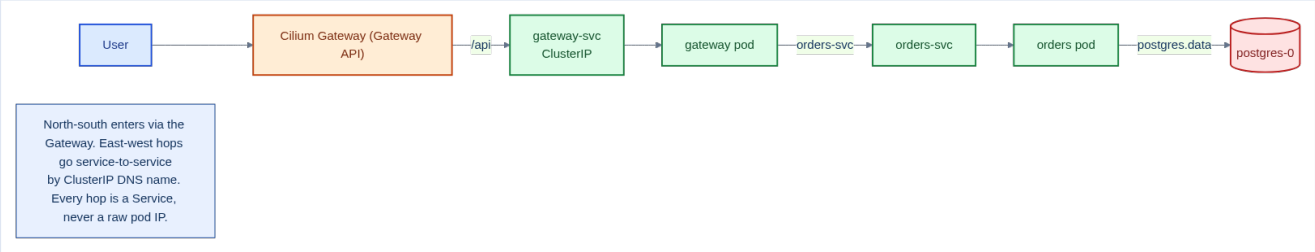
12.3 Headless Services for StatefulSets

A normal ClusterIP hides individual pods behind one VIP — but a Kafka client or a Postgres replica needs to reach **a specific pod**. A **headless** Service (`clusterIP: None`) returns the pod IPs directly and gives each StatefulSet pod stable DNS: `postgres-0.postgres.data.svc.cluster.local`.

```
# repo/manifests/20-data/postgres-statefulset.yaml (headless Service section)
apiVersion: v1
kind: Service
metadata:
 name: postgres
 namespace: data
spec:
 clusterIP: None                  # headless
 selector: { app: postgres }
 ports: [{ port: 5432 }]
```

12.4 The full request path

Put it together: north-south enters through the Gateway; east-west hops are service-to-service by DNS name — **never** a raw pod IP.



1. User hits `https://tickethub.example.com/api/orders` → MetalLB IP → **Gateway**.
2. The Gateway's HTTPRoute matches `/api` → **`gateway`** ClusterIP → a gateway pod.
3. Gateway calls **`orders`** ClusterIP → an orders pod.
4. Orders queries **`postgres-0.postgres.data`** (headless) → the primary.

Each arrow is a Service. That indirection is what lets pods scale, move, and fail without breaking callers.

The **Gateway** service (`repo/services/gateway`) implements this routing as a Go reverse proxy. Upstream addresses are injected via environment variables — no hard-coded DNS names inside the binary:

```
// repo/services/gateway/cmd/gateway/main.go (excerpt)
ordersURL  := getEnv("ORDERS_URL",  "http://localhost:8081")
catalogURL := getEnv("CATALOG_URL", "http://localhost:8082")

mux.Handle("/api/orders/",  newProxy(ordersURL,  ""))
mux.Handle("/api/catalog/", newProxy(catalogURL, ""))
```

In the cluster, the Deployment injects the real ClusterIP DNS names:

```
env:
 - name: ORDERS_URL
   value: http://orders.tickethub.svc.cluster.local
 - name: CATALOG_URL
   value: http://catalog.tickethub.svc.cluster.local
```

Locally, you override them to point at whatever port your services are running on — no cluster required. This pattern (env-var upstreams, localhost fallback) is what makes the service runnable both `go run` on a laptop and inside Kubernetes without any code changes.

> **DNS-based service discovery is the contract**

> Services always address each other by **DNS name**, resolved by CoreDNS (Chapter 4). Hard-coding pod IPs, or even ClusterIPs, is an anti-pattern — names are stable across restarts, rescheduling, and cluster rebuilds; IPs are not.

12.5 Gateway API for north-south routing

North-south HTTP (host/path routing + TLS) enters through the **Gateway API** (`Gateway` + `HTTPRoute`, Chapter 7), which TicketHub migrated to from the legacy **Ingress** object. The Gateway (owned by the platform team) declares the listeners, IP, and TLS; each **HTTPRoute** (owned by an app team) declares its paths and backends — a cleaner separation of infra vs app concerns, with richer traffic splitting and header routing than Ingress offered. East-west service-to-service calls never touch it; they use ClusterIP DNS directly (§12.1–12.4).

Readiness probes gate the EndpointSlice

A pod is only added to a Service's EndpointSlice once its **readiness probe** passes (Chapter 18). Forget the probe and traffic hits pods that are still booting → connection errors during every rollout and scale-up. Readiness is what makes zero-downtime deploys real.

12.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **kube-dns (CoreDNS) search domains** mean `postgres` inside a pod resolves to `postgres.<current-namespace>.svc.cluster.local`. If a pod in `tickethub` ns calls `postgres.data` (intending `postgres.data.svc.cluster.local`), it first tries `postgres.data.tickethub.svc.cluster.local` — which fails — before trying the correct form. Always use fully qualified names for cross-namespace DNS to avoid ndots resolution latency.

- **Session affinity (`sessionAffinity: ClientIP`) is hash-based, not sticky-session aware**: all connections from the same client IP hit the same pod, but a pod restart breaks affinity. If you need application-level stickiness (shopping cart, websocket), express it as a cookie-based `HTTPRoute` filter in your Gateway implementation (or a service mesh policy), not the Service affinity.

- **`ExternalTrafficPolicy: Local`** on a LoadBalancer Service preserves the original client IP (no SNAT) but means only nodes with a backend pod accept traffic — nodes without a pod will drop the connection. With 3 pods spread across 9 nodes, 6 out of 9 nodes will silently drop inbound traffic for that Service.

Gotchas — traps that catch experienced engineers

- **`ClusterIP: None` makes a Service headless** — it returns A records for individual pod IPs, not a virtual IP. Calling `postgres.data.svc.cluster.local` from the `orders` service returns all 3 pod IPs via DNS. If orders uses a naive HTTP client that doesn't re-resolve DNS on each connection, it may always route to the same pod. Headless Services require the client to implement its own load balancing.

- **Service port name must match Istio/Cilium L7 protocol detection**: naming a Service port `http` vs `tcp` changes how a service mesh or L7 NetworkPolicy processes it. Cilium uses the port name to decide whether to apply HTTP-aware policy. Always name ports with the correct protocol prefix.

- **Endpoint not ready after pod crash**: Kubernetes removes the pod's IP from the Service's EndpointSlice only after the readiness probe fails AND the pod is removed. During the gap (typically < 5s), the Service may route to a pod that is no longer serving. Ensure client retries are configured for this transient window.

Architect Considerations

1. **Headless Service for StatefulSets vs ClusterIP for Deployments**: this isn't a choice — StatefulSets that need stable per-pod DNS (Kafka brokers identifying themselves as `kafka-0.kafka.data`) MUST use headless. Deployments use ClusterIP for load-balanced access. Mixing them up is a common cause of mysterious connection failures.

2. **Service topology aware routing**: Kubernetes EndpointSlice topology hints route traffic preferentially to pods on the same node or zone. For TicketHub, routing Orders → Postgres within the same zone reduces cross-rack latency. Enable topology hints on Services where cross-AZ latency matters.

3. **East-West load balancing algorithm**: Cilium's eBPF uses maglev consistent hashing for Service load balancing by default — which gives better connection distribution than simple round-robin, especially for long-lived gRPC connections. Verify your connection pool sizes account for this distribution.

4. **NodePort port range**: the default NodePort range is `30000-32767`. Using NodePorts for production services is not recommended (port memorization burden, firewall complexity), but if needed for legacy integrations, document the port assignments explicitly to prevent conflicts.

5. **Service vs Gateway for internal services**: internal services (Orders calling Payments) should use ClusterIP Services directly — they don't need the Gateway. Only traffic entering from outside the cluster goes through the Gateway. Routing internal traffic through the edge Gateway adds unnecessary latency and a single point of failure.

Chapter 12 checklist

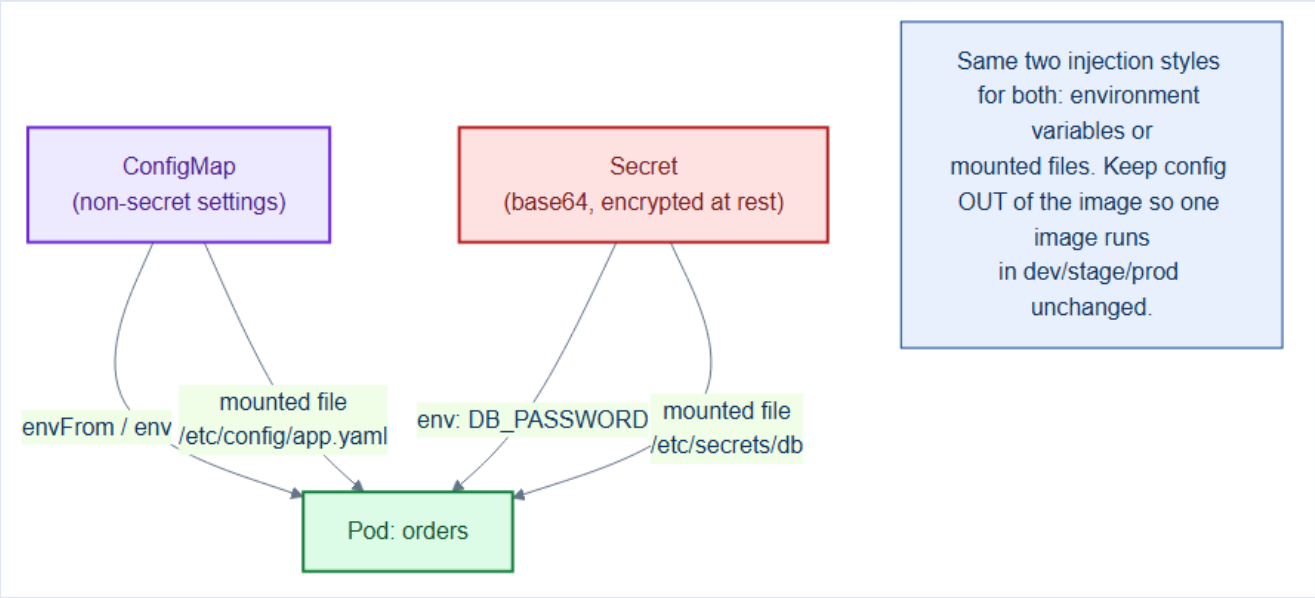
- Every service exposed as a **ClusterIP**; services call each other by **DNS name**.
- StatefulSets front a **headless** Service for stable per-pod DNS.
- Only the **Gateway**'s Service is a `LoadBalancer` (via MetalLB).
- **Readiness probes** defined so only Ready pods receive traffic.
- No raw pod IPs or ClusterIPs hard-coded anywhere.

---

13. Configuration & Secrets — ConfigMaps, Secrets & External Secrets

The same **image** must run unchanged in dev, staging, and production — only its **configuration** differs. Baking config or passwords into the image is a security and operability disaster. Kubernetes separates config from code with **ConfigMaps** and **Secrets**, and for real production, an **External Secrets** operator keeps credentials out of Git entirely.

13.1 The twelve-factor rule: config lives outside the image



Both ConfigMaps and Secrets inject into a pod the same two ways:

- as **environment variables**, or
- as **mounted files** in a volume.

```
# repo/manifests/40-config/configmaps.yaml
apiVersion: v1
kind: ConfigMap
metadata:
 name: orders-config
 namespace: tickethub
data:
 LOG_LEVEL: "info"
 PAYMENTS_URL: "http://payments.tickethub.svc.cluster.local"
 KAFKA_BROKERS: "kafka-0.kafka.data:9092,kafka-1.kafka.data:9092"
```

Mental model — one appliance, different wall sockets

The image is a **power tool**. The ConfigMap/Secret is the **plug adapter** for whichever country (environment) you're in. You don't manufacture a new drill for every country — you change the adapter. One image, many environments.

13.2 Secrets — same shape, more care

A **Secret** looks like a ConfigMap but is meant for sensitive values (DB passwords, API keys). Values are base64-encoded (**not** encryption on its own) and can be **encrypted at rest** in etcd (Chapter 24).

```
# See: repo/manifests/ for the full manifest
# Illustrative only — NOT committed to Git. Real values are synced by the
# External Secrets Operator (see external-secrets.yaml below).
apiVersion: v1
kind: Secret
metadata:
 name: orders-db
 namespace: tickethub
type: Opaque
stringData:                          # stringData: plaintext in, base64 stored
 DB_PASSWORD: "REPLACED_BY_EXTERNAL_SECRETS"
```

Consuming both in the Deployment:

```
# in the container spec
envFrom:
 - configMapRef: { name: orders-config }   # all keys as env vars
env:
 - name: DB_PASSWORD
   valueFrom:
     secretKeyRef: { name: orders-db, key: DB_PASSWORD }
volumeMounts:
 - name: appcfg
   mountPath: /etc/orders           # config as files
   readOnly: true
volumes:
 - name: appcfg
   configMap: { name: orders-config }
```

base64 is encoding, not encryption

`kubectl get secret -o yaml` shows base64 that anyone can decode in one command. Secrets protect data via **RBAC** (who can read them, Chapter 19) and **encryption at rest** (Chapter 24) — *not* by the base64 itself. Never commit real Secret values to Git.

13.3 Env vars vs. mounted files — which to use

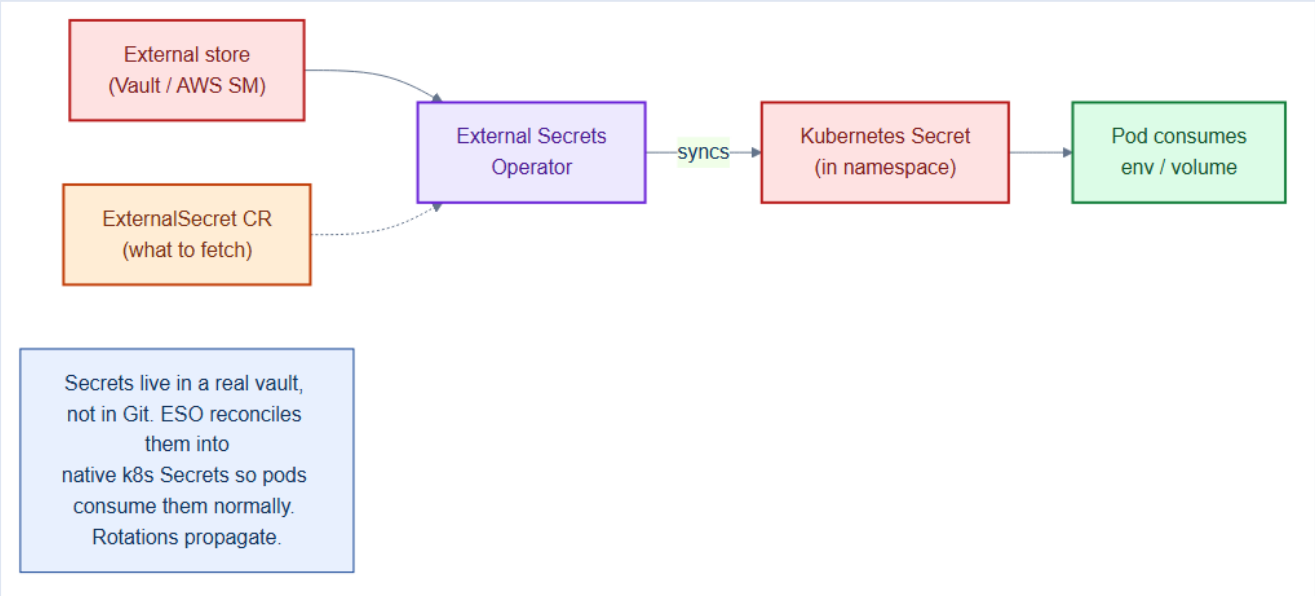
	Environment variables	Mounted files
Ease	Simplest	Slightly more setup
Live updates	No — needs pod restart	**Yes** — volume updates propagate
Leak risk	Visible in `/proc`, crash dumps, `env`	Lower; file perms apply

Best for	Small flags, URLs	Large configs, certs, rotating secrets

For rotating credentials and TLS certs, prefer **mounted files** so rotation doesn't require a restart.

13.4 External Secrets — keep credentials out of Git

Storing Secret YAML in Git (even encrypted) is fragile. The **External Secrets Operator (ESO)** keeps the source of truth in a real vault (HashiCorp Vault, AWS Secrets Manager) and **syncs** it into native Kubernetes Secrets. Git holds only a *reference* — the `ExternalSecret` — never the value.




```
# repo/manifests/40-config/external-secrets.yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
 name: orders-db
 namespace: tickethub
spec:
 refreshInterval: 1h              # re-sync + rotate
 secretStoreRef:
   name: vault-backend
   kind: ClusterSecretStore
 target:
   name: orders-db                # creates/updates this k8s Secret
 data:
   - secretKey: DB_PASSWORD
     remoteRef:
       key: tickethub/orders
       property: db_password
```

GitOps + secrets: reference, never the value

In a GitOps world (Chapter 28) the whole cluster is defined in Git — but secrets must **not** be. ESO squares the circle: Git stores the *ExternalSecret pointer*, the vault stores the *value*, ESO reconciles

them into a real Secret. Rotations in the vault propagate automatically on the next refresh.

Alternative: Sealed Secrets

If you have no external vault, **Sealed Secrets** lets you commit an *encrypted* SealedSecret to Git that only the in-cluster controller can decrypt. Good for smaller setups; ESO scales better when a real secrets manager already exists.

13.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Mounted ConfigMap volumes update automatically** (eventually consistent, with a kubelet sync delay of `syncPeriod`, default 60s) — but **environment variables from ConfigMaps do NOT update** without a pod restart. This asymmetry catches teams off-guard: changing a feature flag in a ConfigMap takes up to 60s to propagate to volume-mounted configs but requires a rollout for env-var configs.

- **Kubernetes Secrets are base64-encoded, not encrypted**, by default in etcd. The base64 encoding is not a security measure — anyone with etcd access sees plaintext values. Encryption at rest (`EncryptionConfiguration` with AES-GCM or Vault KMS provider) is a separate cluster-level config (Chapter 24).

- **ExternalSecret reconciliation interval**: the `ExternalSecret` CR has a `refreshInterval` (default 1 hour). If Vault revokes and reissues a secret (e.g., after a rotation event), the new value won't reach the Kubernetes Secret for up to 1 hour unless you `kubectl annotate externalsecret X force-sync=$(date +%s)` to trigger immediate reconciliation.

Gotchas — traps that catch experienced engineers

- **`envFrom: configMapRef` loads ALL keys as env vars**: if a ConfigMap has a key named `JAVA_TOOL_OPTIONS` it will override the JVM settings for every container that mounts it — silently. Prefer `env: valueFrom: configMapKeyRef` for explicit key selection in production.

- **Secret data keys with unsupported characters**: Kubernetes Secret keys must match `[-._a-zA-Z0-9]`. A key named `db.password` (with a dot) is valid, but many application frameworks that read env vars convert dots to underscores or vice versa. Use consistent naming conventions.

- **ExternalSecret fails silently if the Vault path doesn't exist**: the ExternalSecret controller sets a `Ready=False` condition on the CR, but the application pod starts anyway with the PREVIOUS secret value if the Kubernetes Secret already exists from a prior sync. Monitoring ExternalSecret conditions is non-optional.

Architect Considerations

1. **ConfigMap vs environment variable vs Vault secret decision tree**: use ConfigMaps for non-sensitive tunable configuration (log levels, feature flags, URLs). Use Secrets (backed by Vault via ExternalSecret) for credentials, tokens, and API keys. Never use ConfigMaps for secrets, even temporarily.

2. **Secret rotation zero-downtime strategy**: when a database password is rotated in Vault, ExternalSecret updates the Kubernetes Secret, but running pods don't see the update until they restart. Design connection pool reconnect logic (Postgres `target_session_attrs`, JDBC retry) to handle mid-session password changes, or coordinate pod rolling restarts with the rotation event.

3. **Vault namespace isolation**: does each team's ExternalSecret pull from a separate Vault namespace/path with its own policy? Sharing a single Vault policy that allows reading ALL secrets gives each service blast radius equal to the entire secret store — violates least privilege.

4. **Configuration drift detection**: with ConfigMaps managed by GitOps (Argo CD), a manual `kubectl edit configmap` creates drift that Argo CD will revert. Ensure your runbooks instruct operators to make configuration changes through git, not kubectl, to avoid surprise rollbacks.

5. **Sealed Secrets vs External Secrets**: Sealed Secrets encrypt secret values in git (useful for small teams without a Vault instance). External Secrets pull from an external store at runtime (better for enterprise governance). Choose based on your secret management maturity — Vault + External Secrets is the correct long-term architecture.

Chapter 13 checklist

- **One image per service** across all environments; config injected, never baked in.
- Non-secret settings in **ConfigMaps**; sensitive values in **Secrets**.
- Secrets protected by **RBAC** + **encryption at rest**, never committed as plaintext.
- Rotating creds/certs mounted as **files** for restart-free updates.
- Real credentials sourced via **External Secrets** (or Sealed Secrets); Git holds references only.

---

14. Stateful Storage — PV/PVC & StatefulSet volumeClaimTemplates

Chapter 8 built the storage *platform* (Rook-Ceph, StorageClasses). Chapter 11 introduced StatefulSets. This chapter joins them: how TicketHub's **databases and brokers** get durable, identity-bound storage that survives pod death, rescheduling, and node failure — without an operator manually creating a single volume.

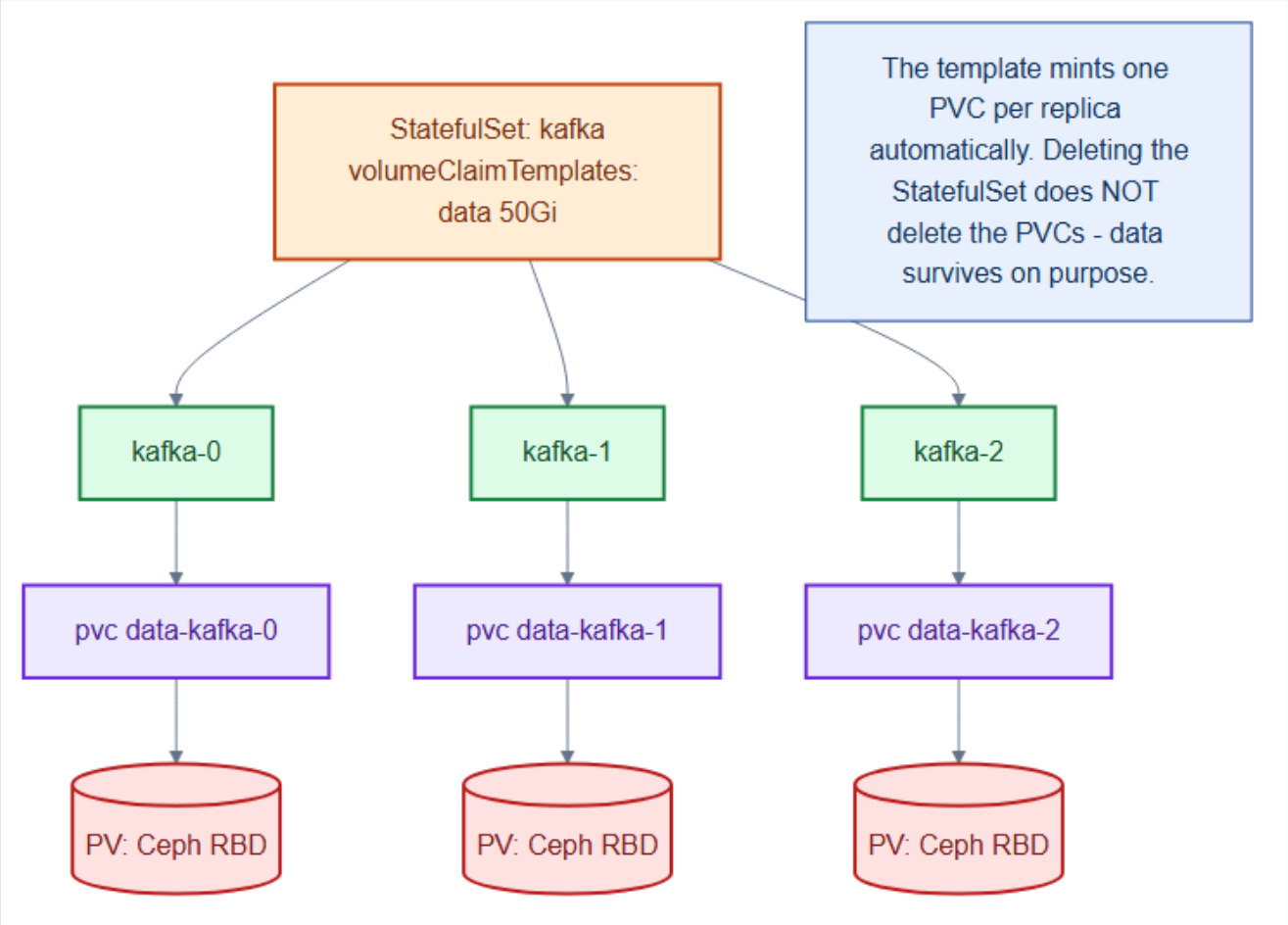
14.1 Recap — the storage abstractions in motion

A stateless pod can lose its writable layer and nobody cares. Postgres cannot. It needs a **PersistentVolume** that outlives the pod. The workload asks with a **PVC**; the **StorageClass** provisions a **PV** dynamically (Chapter 8). For StatefulSets, we don't write the PVCs by hand at all.

> **Mental model — assigned lockers at a gym**
>
> A Deployment pod is a **day-pass locker** — grab any free one, empty it when you leave. A StatefulSet pod has an **assigned annual locker**: `postgres-0` always returns to locker #0 with its own contents intact, even after it goes home and comes back (reschedules). The locker (PVC) is bound to the member (pod ordinal), not the visit.

14.2 volumeClaimTemplates — one PVC per replica, automatically

The magic of a StatefulSet is `volumeClaimTemplates`: for each replica it **mints a dedicated PVC** (`data-kafka-0`, `data-kafka-1`, …), each bound to its own PV, each following its pod forever.




```
# repo/manifests/20-data/kafka-statefulset.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
 name: kafka
 namespace: data
spec:
 serviceName: kafka               # the headless Service (stable DNS)
 replicas: 3
 selector:
   matchLabels: { app: kafka }
 template:
   metadata:
     labels: { app: kafka }
   spec:
     terminationGracePeriodSeconds: 60
     containers:
       - name: kafka
         image: bitnami/kafka:3.7
         ports: [{ containerPort: 9092 }]
         volumeMounts:
           - name: data
             mountPath: /bitnami/kafka
         resources:
           requests: { cpu: "1", memory: 2Gi }
           limits:   { cpu: "2", memory: 4Gi }
 volumeClaimTemplates:            # <-- one PVC created PER replica
   - metadata:
       name: data
```

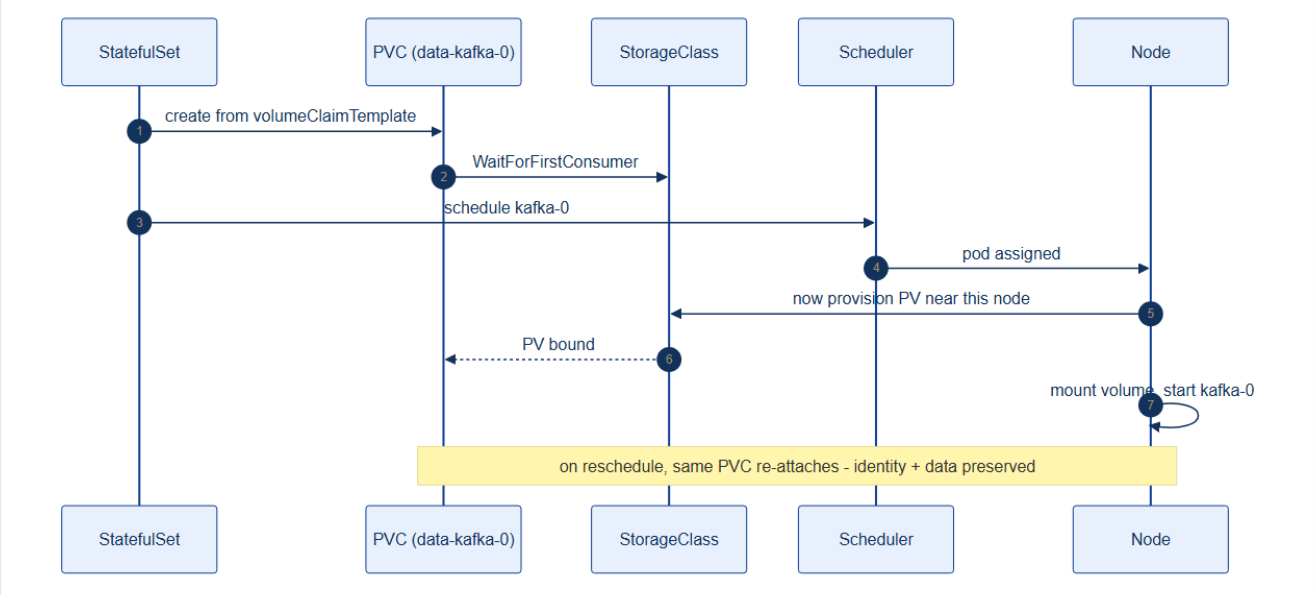
```
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: rook-ceph-block
      resources:
        requests:
          storage: 50Gi
```

The result:

```
kubectl -n data get pvc
# data-kafka-0   Bound   ...   50Gi   rook-ceph-block
# data-kafka-1   Bound   ...   50Gi   rook-ceph-block
# data-kafka-2   Bound   ...   50Gi   rook-ceph-block
```

14.3 How binding actually happens

Because the StorageClass uses `volumeBindingMode: WaitForFirstConsumer` (Chapter 8), the PV isn't provisioned until the pod is **scheduled** — so the volume lands in the right failure domain, next to its pod.



The crucial property: on **reschedule** (node failure, rollout), Kubernetes re-attaches the *same* PVC to the pod with the *same* ordinal. `kafka-1` always comes back with `data-kafka-1` — identity **and** data preserved.

> **Deleting a StatefulSet does NOT delete its PVCs**

> This is deliberate data-safety behavior. `kubectl delete statefulset kafka` removes the pods but **leaves `data-kafka-*` PVCs (and their PVs) intact**. Recreate the StatefulSet and it re-adopts the existing volumes. To actually reclaim storage you must delete the PVCs explicitly — a guardrail against accidental data loss.

14.4 Postgres with a primary + replicas

Postgres uses the same pattern, fronted by the headless Service from Chapter 12 so replicas find the primary at a stable address:

```
# repo/manifests/20-data/postgres-statefulset.yaml (excerpt)
spec:
 serviceName: postgres
 replicas: 3
 template:
   spec:
     containers:
       - name: postgres
         image: postgres:16
         volumeMounts:
           - name: data
             mountPath: /var/lib/postgresql/data
         env:
           - name: POSTGRES_PASSWORD
             valueFrom:
               secretKeyRef: { name: postgres-db, key: password }   # Ch 13
 volumeClaimTemplates:
   - metadata: { name: data }
     spec:
       accessModes: [ReadWriteOnce]
       storageClassName: rook-ceph-block   # reclaimPolicy: Retain (Ch 8)
       resources: { requests: { storage: 20Gi } }
```

14.5 Choosing access mode and reclaim policy per workload

Workload	Access mode	StorageClass reclaim	Why
Postgres / Kafka / Redis	**RWO** (block)	**Retain**	Single writer; never auto-wipe a DB
Shared uploads	**RWX** (CephFS)	Delete	Many pods read/write; ephemeral-ish
Backups / index snapshots	Object (S3)	n/a	Blob storage, lifecycle-managed

Match the access mode to the engine

Databases demand **RWO block** — a single writer with block semantics. Putting Postgres on an **RWX** shared filesystem invites corruption (Chapter 8). Reserve RWX for genuinely concurrency-safe shared data.

14.6 DB Replication with CRDs — moving beyond raw StatefulSets

A raw StatefulSet (sections 14.2–14.4) gives Postgres stable identity and durable storage. It does **not** give it:

- automatic **streaming replication** between primary and standbys,

- a **failover controller** that promotes a standby when the primary dies,
- a **connection pooler** to prevent connection-count exhaustion,
- **WAL archiving** for point-in-time recovery (PITR), or
- a **replication slot** lifecycle manager that prevents disk bloat.

Managing all of that by hand requires custom init-container scripts, sidecars, and maintenance procedures — work that is better encoded as an **operator** driven by CRDs (Chapter 25). TicketHub's `tickethub-db-operator` consumes two new API types in the `db.tickethub.io` group.

14.6.1 The two new CRDs

```
# CRD registration (abbreviated) — full definition in
# repo/manifests/20-data/postgres-replication-crd.yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
 name: postgresreplicationclusters.db.tickethub.io
spec:
 group: db.tickethub.io
 names: { kind: PostgresReplicationCluster, plural: postgresreplicationclusters, shortNames: [pgrc]
 scope: Namespaced
 versions:
   - name: v1alpha1
     served: true
     storage: true
     schema:
       openAPIV3Schema:
         type: object
         properties:
           spec:
             type: object
             properties:
               instances:          { type: integer, minimum: 1, maximum: 9 }
               postgresVersion:    { type: integer, enum: [14,15,16,17], default: 16 }
               replication:
                 type: object
                 properties:
                   synchronousMode: { type: string, enum: [none, first, quorum] }
                   hotStandby:      { type: boolean, default: true }
                   walLevel:        { type: string,  enum: [replica, logical] }
               walArchive:
                 type: object
                 properties:
                   enabled:     { type: boolean }
                   destination: { type: string }
               failover:
                 type: object
                 properties:
                   automaticFailover: { type: boolean, default: true }
                   promotionTimeout:  { type: integer, default: 60 }
               connectionPooler:
                 type: object
                 properties:
                   enabled:    { type: boolean }
                   poolMode:   { type: string, enum: [session, transaction, statement] }
                   maxClientConn: { type: integer }
               storage:
```

```
                type: object
                required: [size, storageClass]
                properties:
                  size:         { type: string }
                  storageClass: { type: string }
```

The second CRD, `ReplicationSlot`, models an individual physical or logical replication slot with a retention policy:

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
 name: replicationslots.db.tickethub.io
spec:
 group: db.tickethub.io
 names: { kind: ReplicationSlot, plural: replicationslots, shortNames: [rslot] }
 scope: Namespaced
 versions:
   - name: v1alpha1
     served: true
     storage: true
     schema:
       openAPIV3Schema:
         type: object
         properties:
           spec:
             type: object
             required: [clusterRef, type]
             properties:
               clusterRef: { type: string }
               type:       { type: string, enum: [physical, logical] }
               plugin:     { type: string }        # pgoutput | wal2json
               retentionPolicy:
                 type: object
                 properties:
                   maxWalSize:      { type: string, default: 2Gi }
                   inactiveTimeout: { type: string, default: 24h }
```

Why manage replication slots as CRs?

An inactive logical slot that accumulates WAL without a consumer will silently fill the disk and crash the primary. Declaring slots as `ReplicationSlot` CRs gives the operator **ownership** — it can monitor `pg_replication_slots`, fire a Kubernetes `Event`, and drop the slot before it becomes critical.

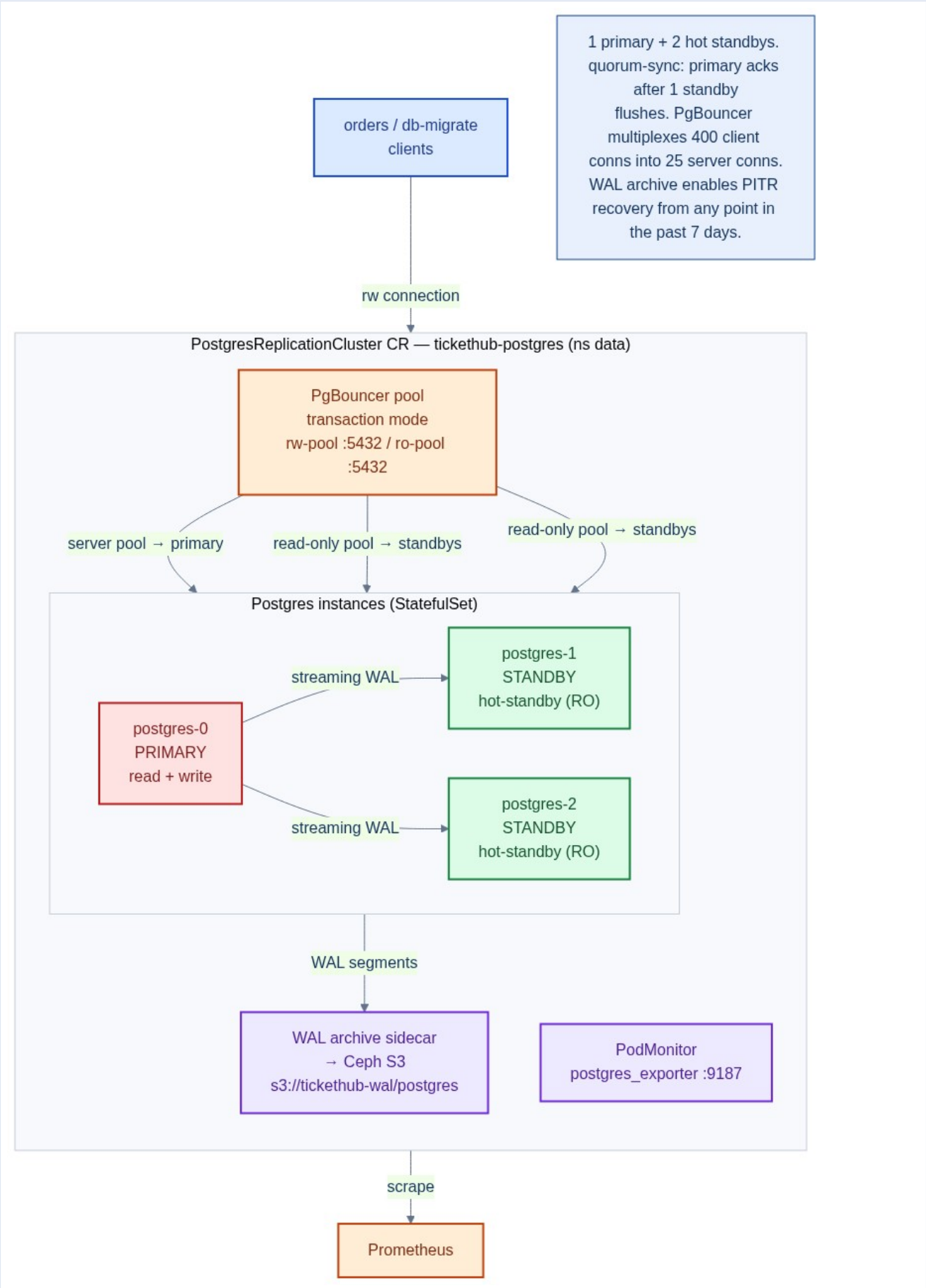
14.6.2 The TicketHub PostgresReplicationCluster CR

```
# repo/manifests/20-data/postgres-replication-crd.yaml (CR excerpt)
apiVersion: db.tickethub.io/v1alpha1
kind: PostgresReplicationCluster
metadata:
 name: tickethub-postgres
 namespace: data
spec:
 instances: 3                  # postgres-0 (primary) + 2 hot standbys
 postgresVersion: 16
```

```
  replication:
    synchronousMode: quorum     # ack after 1 standby flushes → RPO = 0
    synchronousStandbys: 1
    walLevel: replica
    hotStandby: true            # standbys serve read-only queries
  walArchive:
    enabled: true
    destination: s3://tickethub-wal/postgres
    credentialsSecret: ceph-s3-creds
    retentionDays: 7
  failover:
    automaticFailover: true
    promotionTimeout: 60
    primaryUpdateStrategy: RollingUpdate
  connectionPooler:
    enabled: true
    poolMode: transaction       # connection returned after each txn
    maxClientConn: 400
    defaultPoolSize: 25
  storage:
    size: 20Gi
    storageClass: rook-ceph-block
    walStorage:
      size: 10Gi                # separate disk prevents data-PVC WAL bloat
      storageClass: rook-ceph-block
  monitoring:
    enablePodMonitor: true
  bootstrap:
    method: initdb
    secretName: postgres-db
```

After `kubectl apply`, the operator creates a StatefulSet, two Services (read-write → primary, read-only → standbys), a PgBouncer Deployment, and a PodMonitor. The raw `postgres-statefulset.yaml` remains the conceptual teaching scaffold; the `PostgresReplicationCluster` CR is what production runs.

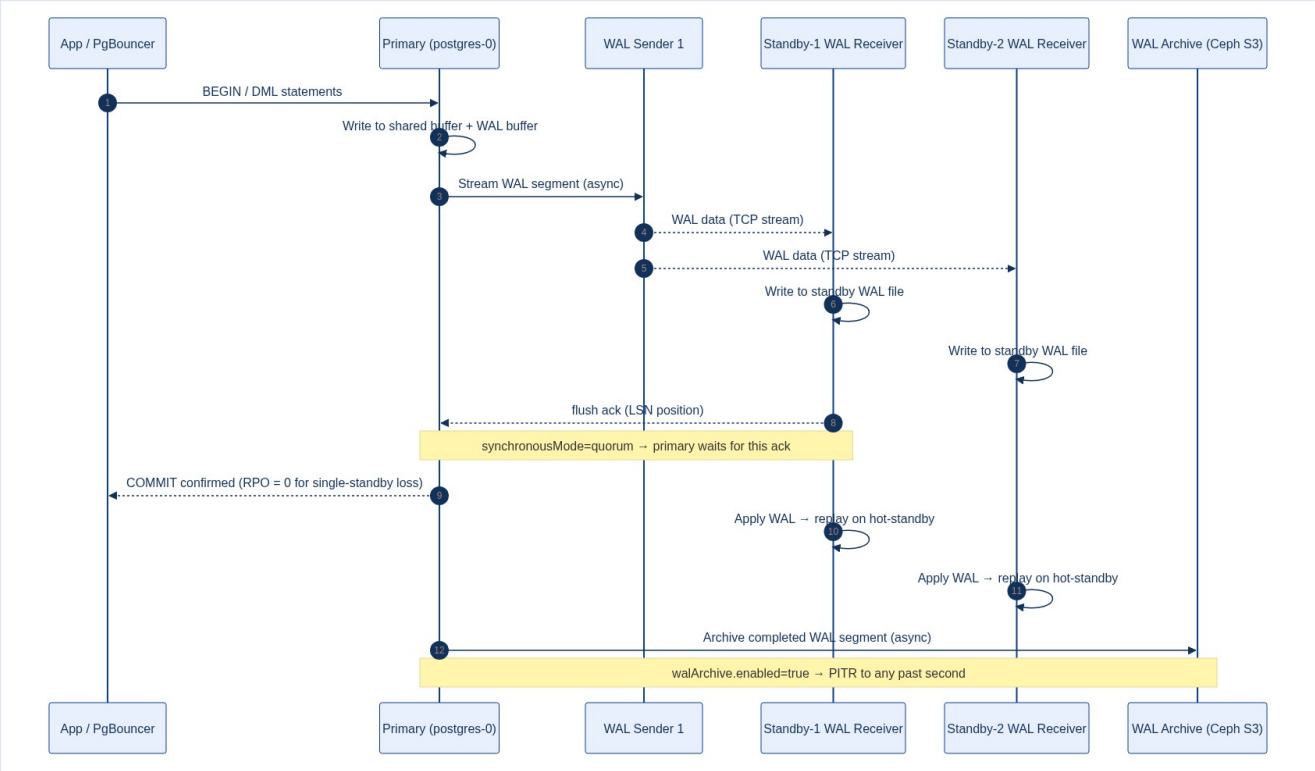
14.6.3 Streaming replication architecture



The three pods run the same `postgres:16` image. The operator:

1. Labels `postgres-0` as `role=primary` on bootstrap.
2. Configures `postgres-1` and `postgres-2` with `primary_conninfo` pointing at the primary's stable headless DNS name (`postgres-0.postgres.data`).
3. Creates **physical replication slots** on the primary for each standby, ensuring the primary keeps enough WAL for lagging standbys.
4. Starts the `pg_basebackup` in each standby init-container to clone data.

14.6.4 WAL streaming — how data flows



Every write on the primary appends to the **Write-Ahead Log (WAL)**. A **WAL sender** process streams these bytes to each standby's **WAL receiver**, which writes them to its own WAL file. A **startup process** on the standby replays the WAL continuously, keeping the standby within milliseconds of the primary.

With `synchronousMode: quorum` the primary delays the `COMMIT` response until at least one standby has *flushed* (written to disk) the relevant WAL. This gives **zero RPO** for any single-standby loss — no committed transaction can be lost even if the primary dies immediately after the ack.

Mode	Behaviour	Trade-off
`none`	Async — commit before standby acks	Maximum throughput; seconds of RPO possible
`first`	Wait for first standby to flush	< 1ms extra latency; RPO = 0 for 1 loss

`quorum`	Wait for quorum of standbys	Highest durability; best for financial data

14.6.5 Automatic failover

If `automaticFailover: true` and the primary is unreachable for `promotionTimeout` seconds, the operator:

1. Queries `pg_stat_replication` on each standby to find the least-lagging one.
2. Runs `SELECT pg_promote()` on the chosen standby.
3. Re-labels it `role=primary`; the **rw-pool Service** endpoint flips within one kube-proxy sync cycle (< 1 s).
4. Configures remaining standbys to follow the new primary via `pg_rewind`.
5. Writes `status.currentPrimary` and emits a `Normal/Failover` Event.

The old primary, if it recovers, is automatically demoted and rejoins as a standby — no manual intervention needed.

14.6.6 Connection pooling

With `connectionPooler.enabled: true` the operator deploys **PgBouncer** as a 2-replica stateless Deployment in front of Postgres. In `transaction` pool mode each server connection is held only for the duration of a transaction and then returned to the pool — allowing hundreds of app threads to share a small number of actual Postgres connections:

```
400 client conns (PgBouncer) → 25 server conns (Postgres primary)
```

Without a pooler, each microservice replica opens its own persistent connections. With 30+ `orders` pods the raw connection count can push Postgres past `max_connections` and cause `FATAL: sorry, too many clients already`.

```
# Inspect pool state
kubectl -n data exec deploy/pgbouncer -- psql -p 6432 pgbouncer -c "SHOW POOLS;"
```

Chapter 14 checklist — Part III complete

- Stateful workloads use **StatefulSets** with `volumeClaimTemplates` (one PVC per replica).
- PVCs bind via `WaitForFirstConsumer` so volumes land in the pod's failure domain.
- **RWO block** for databases; `reclaimPolicy: Retain` protects the data.
- Passwords injected from **Secrets** (Chapter 13), not baked into images.
- Understood: deleting a StatefulSet **keeps** its PVCs — reclaim storage deliberately.

- Production Postgres uses **CRDs** (`PostgresReplicationCluster`, `ReplicationSlot`) to declare streaming replication, failover, pooling, and WAL archiving declaratively.

- `synchronousMode: quorum` achieves **RPO = 0** for single-standby loss.

- Logical `ReplicationSlot` CRs carry a `retentionPolicy` to prevent WAL bloat.

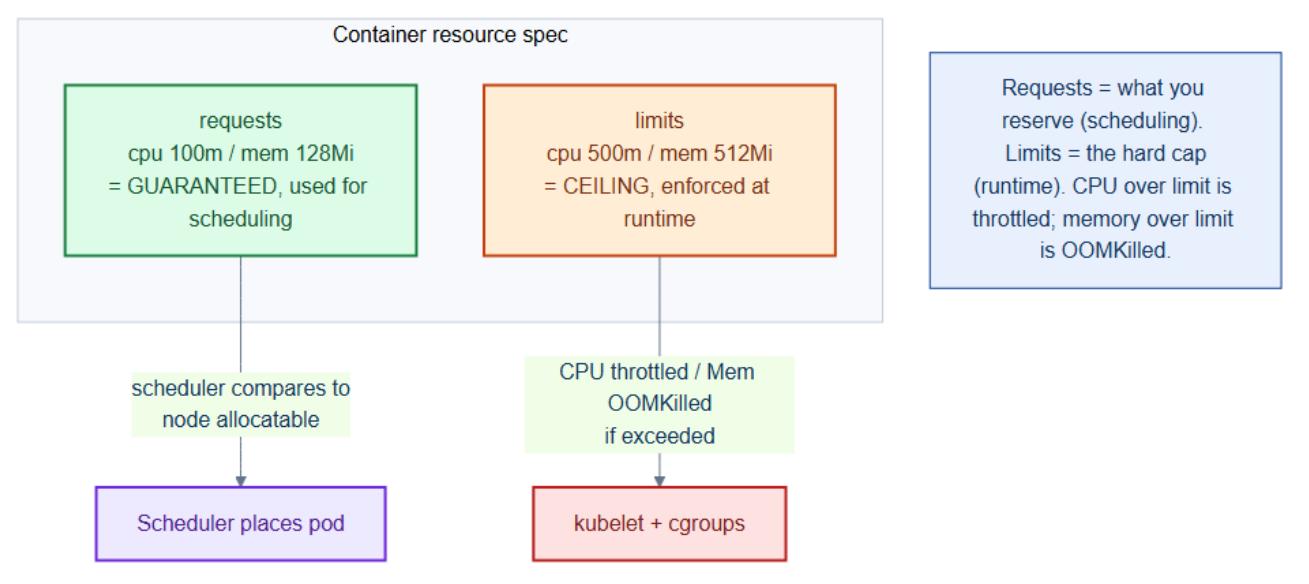
TicketHub is now fully containerized and deployed — stateless services, stateful data, config, and storage. **Part IV** makes it reliable and elastic: resource management, autoscaling, scheduling, and health.

---

15. Resource Management — Requests, Limits, QoS, LimitRange & ResourceQuota

Chapter 9 introduced the resource *model*; now we make it operational. Getting **requests and limits** right is the single biggest lever on cluster stability and cost. Set them too low and pods get throttled or OOM-killed; too high and you waste half the cluster. This chapter turns TicketHub's raw capacity into predictable, fair, protected service.

15.1 Requests vs. limits — two different jobs



	Requests	Limits
Meaning	Reserved, guaranteed amount	Hard ceiling
Used by	The **scheduler** (placement)	The **kubelet/cgroups** (runtime)
CPU over it	Impossible (it's reserved)	**Throttled** (compressible)
Memory over it	Impossible	**OOMKilled** (incompressible)

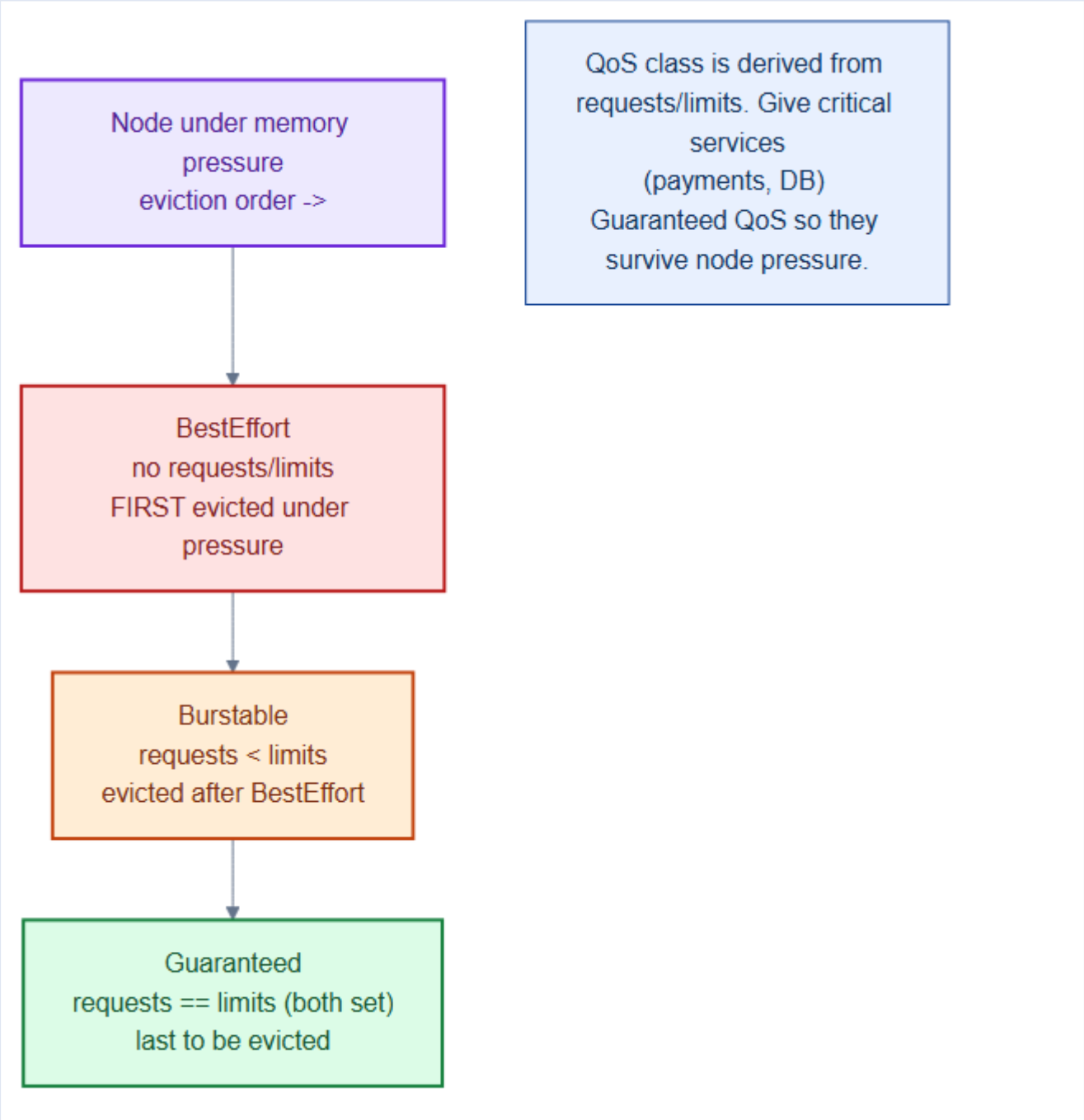
```
resources:
 requests: { cpu: "150m", memory: 192Mi }   # scheduler reserves this
 limits:   { cpu: "750m", memory: 768Mi }   # runtime cap
```

> **Mental model — a hotel reservation vs. the fire-code limit**

> The **request** is your **guaranteed reserved room** — the hotel (scheduler) won't overbook it. The **limit** is the **fire-code max occupancy** — exceed it on CPU and security throttles the party; exceed it on memory and the room is evacuated (OOMKilled). CPU is *compressible* (slow you down); memory is *incompressible* (evict).

15.2 Quality of Service classes

Kubernetes derives a **QoS class** from your requests/limits, and uses it to decide **who gets evicted first** when a node runs out of memory.



QoS class	Condition	Eviction order
Guaranteed	requests == limits (both set, every container)	Evicted **last**
Burstable	at least one request set, but ≠ limits	Middle
BestEffort	no requests/limits at all	Evicted **first**

Give revenue-critical services Guaranteed QoS

Set `requests == limits` for **payments**, **orders**, and the **databases** so they land in the **Guaranteed** class and are the *last* to be evicted under node memory pressure. Leave elastic, non-critical work (search reindex, batch reports) Burstable. Never run production services as **BestEffort** — they're first out the door.

15.3 How to actually pick the numbers

1. Start from observed usage (Prometheus, `kubectl top`, VPA in *recommend* mode — Chapter 16).
2. **Requests** ≈ steady-state P50–P75 usage (so the scheduler packs efficiently).
3. **Memory limit** ≈ P99 + headroom (OOM is fatal, be generous).
4. **CPU limit** — often set generously or omitted for latency-sensitive services (throttling adds latency); always keep the **request** accurate.

Reading percentiles (P50, P95, P99)

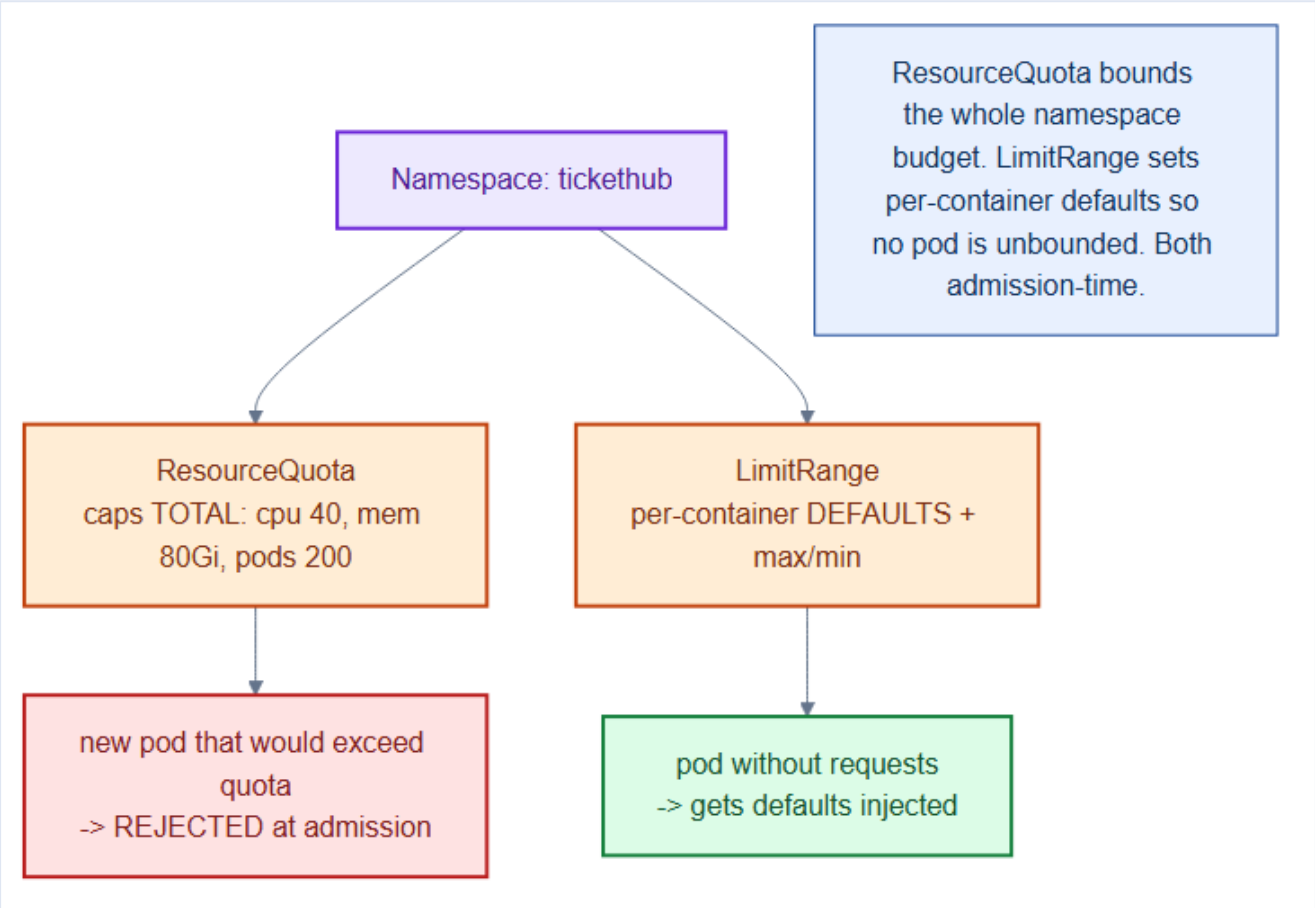
A **percentile** is the value below which that fraction of samples fall: **P50** (the median) is typical usage; **P99** is a near-worst case ("99% of samples are below this"). Size **memory limits** near **P99 + headroom** (an OOM kill is fatal), but set **requests** near **P50–P75** so the scheduler packs nodes efficiently for the common case.

A missing memory request is a scheduling landmine

A container with **no memory request** is treated as needing ~zero, so the scheduler packs nodes to the brim — then the first real memory spike triggers cascading **OOMKills** and evictions. Always set memory requests; enforce it with a LimitRange.

15.4 Guardrails: LimitRange and ResourceQuota

Individual specs can't be trusted to be complete, so the namespace enforces guardrails (introduced in Chapter 9, applied here):



- **LimitRange** injects **default** requests/limits into any container that omits them, and sets per-container **min/max**.
- **ResourceQuota** caps the **total** requests/limits/object-counts for the whole namespace, rejecting at admission anything that would exceed the budget.

```
# already in repo/manifests/00-namespaces/quota-limits.yaml
apiVersion: v1
kind: LimitRange
metadata: { name: tickethub-defaults, namespace: tickethub }
spec:
 limits:
   - type: Container
     default:        { cpu: "500m", memory: 512Mi }   # limit if omitted
     defaultRequest: { cpu: "100m", memory: 128Mi }   # request if omitted
     max:            { cpu: "4",    memory: 8Gi }      # nobody hogs a whole node
```

```
kubectl -n tickethub describe quota tickethub-quota    # see used vs hard
kubectl -n tickethub top pods                          # live usage vs requests
```

15.5 Nuances, Gotchas & Architect Considerations

> **Nuances — subtle behaviours to internalise**

- **CPU requests are used for scheduling AND CPU share allocation (cgroups)**, not for hard limits. A pod requesting `500m` CPU on a node with spare cycles can burst to multiple cores — `request` only guarantees its proportional share under contention. Only `limit` hard-caps CPU via the CFS bandwidth controller.

- **Memory limit OOM is immediate and silent**: when a container exceeds its memory limit, the kernel sends `SIGKILL` with `OOMKilled` reason — no warning, no graceful shutdown. This is different from CPU throttling (which just slows the container). Size memory limits with headroom for GC pauses (JVM) or memory spikes.

- **`BestEffort` pods are evicted first under node pressure**, but eviction order within a QoS class depends on how much over their request a pod is running. A `Burstable` pod using 10× its request is evicted before a `Burstable` pod using 1.1× — even if the second pod has a smaller absolute memory footprint.

Gotchas — traps that catch experienced engineers

- **Setting CPU limit = CPU request**: this gives the container a `Guaranteed` QoS class (good for priority), but it pins the CPU at exactly the request — the container cannot burst even when the node has idle capacity. For most application pods, set no CPU limit and let them burst freely; set only a request to guide scheduling.

- **ResourceQuota counts requests, not actual usage**: a namespace with `cpu: 10` quota and 10 pods each requesting `1` CPU has hit the quota even if all pods are idle. Submitting a new pod fails with `exceeded quota` regardless of actual node capacity.

- **Missing LimitRange defaults**: a namespace without a LimitRange allows pods with no `resources` spec — these get `BestEffort` QoS and are the first to be evicted under node pressure. Always define LimitRange defaults to ensure a minimum resource contract.

Architect Considerations

1. **Request sizing methodology**: requests should reflect the pod's p99 actual usage, not a theoretical maximum. Oversized requests cause poor bin-packing (nodes appear full while CPUs are idle). Use `kubectl top pod` / Prometheus `container_cpu_usage_seconds_total` histograms to right-size requests.

2. **CPU limit policy**: Google SRE and the Kubernetes community debate whether to set CPU limits. The consensus for latency-sensitive services: **no CPU limit** (allow bursting), set request accurately. For batch jobs: set limit = request (predictable scheduling). For third-party components: follow the vendor recommendation.

3. **Memory limit headroom factor**: set memory limit = 1.3–1.5× the p99 actual usage. This covers GC pauses (JVM), memory allocator overhead, and temporary buffers. Below 1.2× causes spurious OOMKills under load; above 2× wastes memory quota.

4. **Quota namespacing granularity**: should each microservice team have its own namespace with isolated quota, or should the `tickethub` namespace have a single aggregate quota? Per-team namespaces give charge-back visibility but multiply the management overhead.

5. **LimitRange max for VPA interaction**: VPA recommends new request/limit values. If the LimitRange `max.memory` is lower than VPA's recommendation, VPA silently skips the pod. Keep LimitRange maximums generous — they are a guardrail, not a target.

Chapter 15 checklist

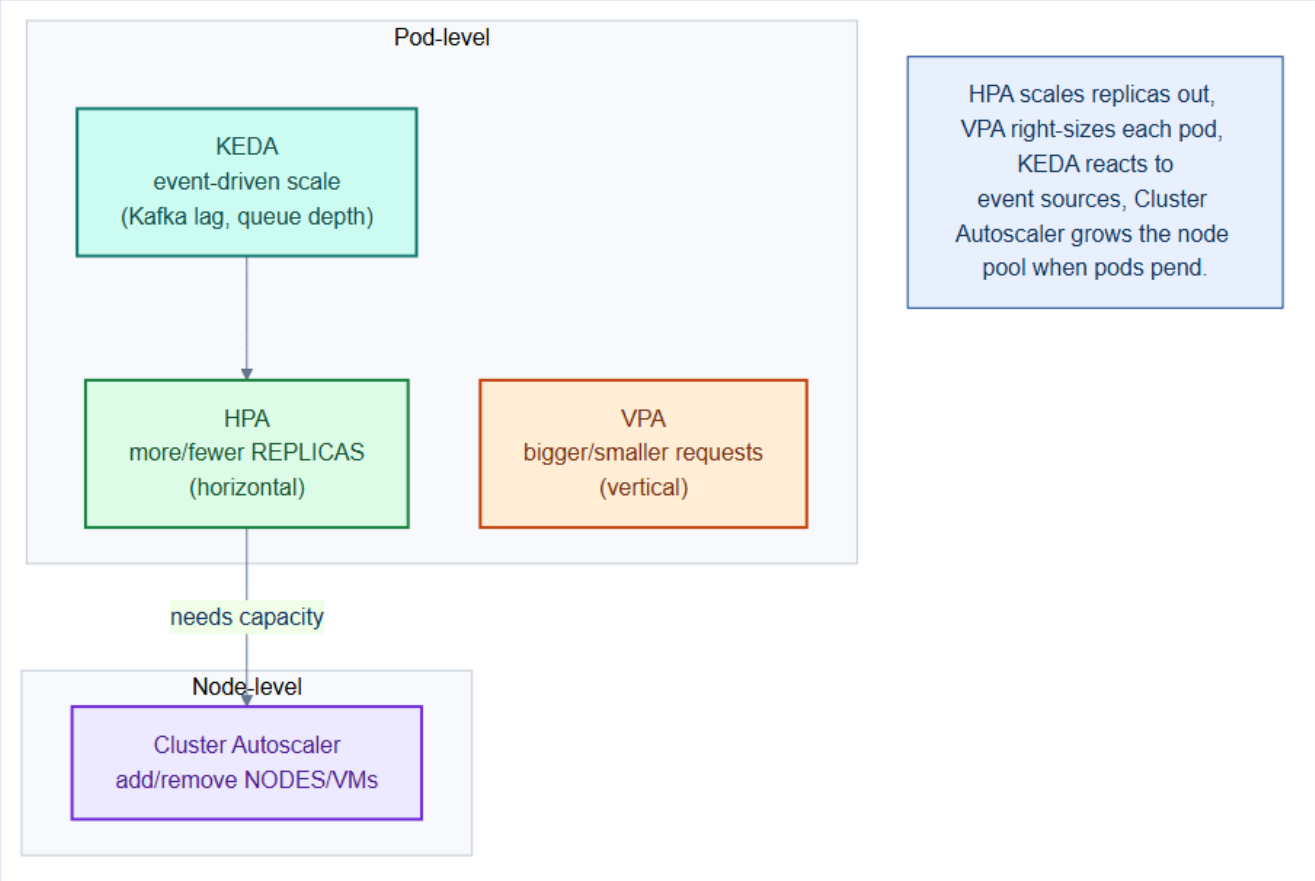
- Every container has **memory requests** (and usually CPU requests) set deliberately.
- Critical services set `requests == limits` → **Guaranteed** QoS.
- **LimitRange** guarantees no container is unbounded; **ResourceQuota** caps each namespace.
- Numbers derived from **real usage** (Prometheus/VPA-recommend), not guesses.
- No production **BestEffort** pods.

---

16. Autoscaling — Metrics Server, HPA, VPA, Cluster Autoscaler & KEDA

TicketHub's load is spiky: a hot concert on-sale can 10× traffic in seconds, then fall quiet. Static replica counts either waste money at idle or fall over at peak. **Autoscaling** makes the platform elastic across four independent dimensions — replicas, pod size, event backlog, and node count.

16.1 The four autoscalers — different axes



Autoscaler	Scales	Trigger	TicketHub example
HPA	replica **count** (horizontal)	CPU / memory / custom metric	catalog 3→20 pods on CPU
VPA	per-pod **requests** (vertical)	historical usage	right-size search pods
KEDA	replicas from **events**	Kafka lag, queue depth	notifications drain a backlog
Cluster Autoscaler	**node** count	Pending pods	add worker VMs at peak

Mental model — a restaurant at rush hour

HPA calls in **more waiters** (replicas). **VPA** decides each waiter needs **bigger trays** (requests). **KEDA** watches the **ticket rail** (event queue) and staffs to the backlog. **Cluster Autoscaler opens more tables** (nodes) when the floor is full. All four cooperate; none replaces the others.

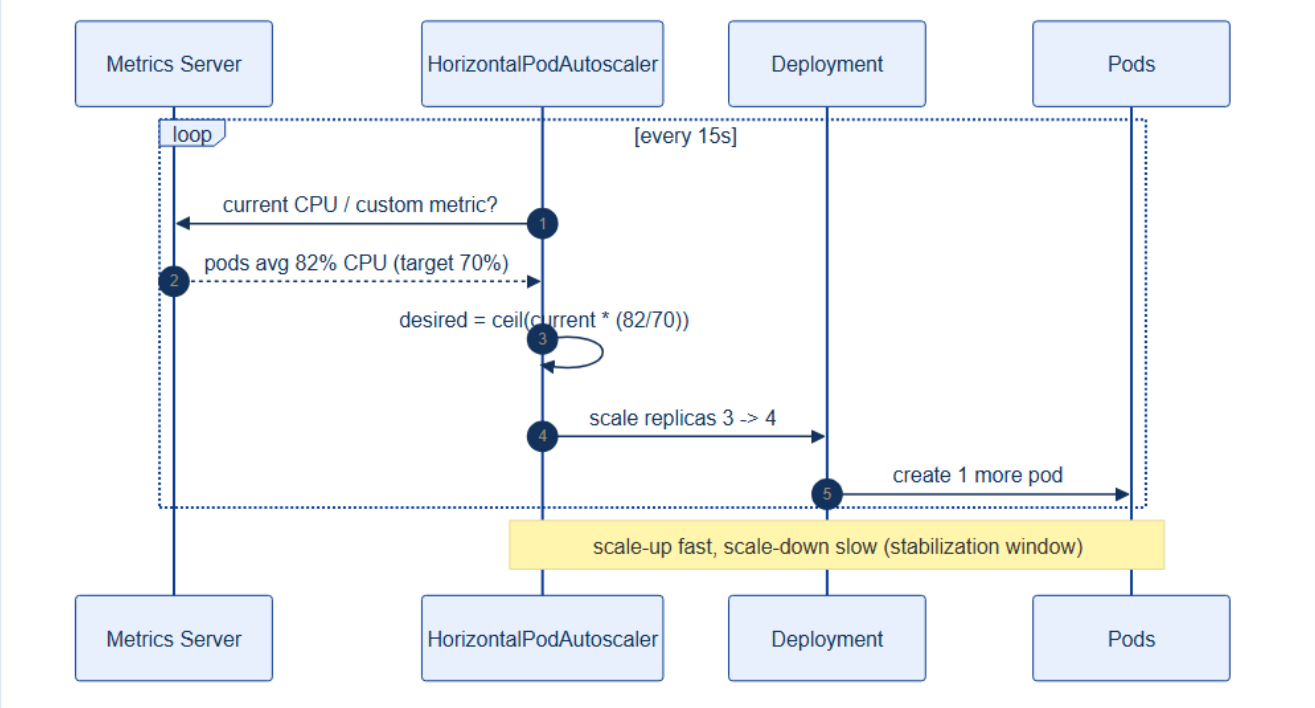
16.2 Metrics Server — the prerequisite

HPA and `kubectl top` need live CPU/memory. **Metrics Server** scrapes each kubelet and serves the metrics API. Install it first (bootstrap step 7, Chapter 9) or HPA has nothing to read.

```
kubectl top nodes        # verify Metrics Server works before configuring HPA
kubectl top pods -n tickethub
```

16.3 Horizontal Pod Autoscaler

HPA runs a control loop: read the metric, compute desired replicas, scale the Deployment.




```
desiredReplicas = ceil( currentReplicas × currentMetric / targetMetric )
# e.g. 3 replicas at 82% CPU, target 70%:  ceil(3 × 82/70) = ceil(3.51) = 4
```

```
# repo/manifests/50-scaling/catalog-hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: catalog
 namespace: tickethub
spec:
 scaleTargetRef:
```

```
    apiVersion: apps/v1
    kind: Deployment
    name: catalog
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 70 }
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300    # scale down slowly to avoid flapping
```

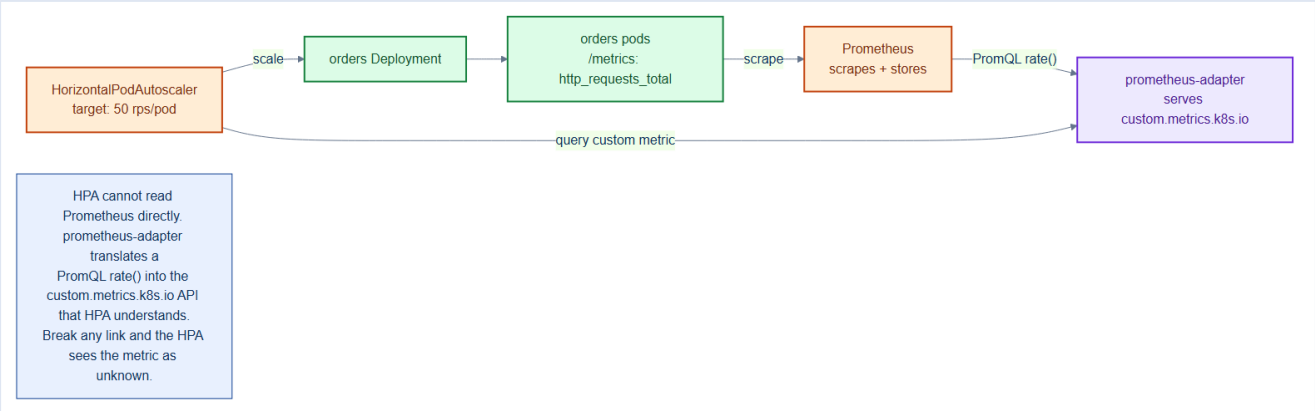
HPA needs accurate requests to work at all

HPA's CPU **utilization** is measured *against the request* (82% of the request, not of the node). If requests are wrong, HPA's math is wrong — it scales too early or too late. Chapters 15 and 16 are joined at the hip: **right-size first, then autoscale**.

16.4 Scaling on custom metrics — prometheus-adapter

The HPA above scales on CPU, which Metrics Server provides out of the box. But CPU is often the *wrong* signal — a checkout service can be CPU-idle while requests queue behind a slow database. You'd rather scale on a **business metric** like requests-per-second or p99 latency. HPA can do that — but **only if something serves those metrics through the `custom.metrics.k8s.io` API**, because HPA cannot query Prometheus directly.

That something is **prometheus-adapter**: it sits between Prometheus and the HPA, translating PromQL results into the custom-metrics API that HPA understands.



This is where Chapter 26 pays off: the `orders` service already exposes `http_requests_total`, Prometheus already scrapes it — the adapter just makes it *autoscalable*. Install the adapter and give it a rule that turns the raw counter into a per-pod rate:

```
# repo/manifests/50-scaling/prometheus-adapter-config.yaml
# Helm: prometheus-community/prometheus-adapter, this becomes its `rules.custom`.
rules:
 custom:
```

```
    - seriesQuery: 'http_requests_total{namespace!="",pod!=""}'
      resources:
        overrides:
          namespace: { resource: namespace }
          pod:       { resource: pod }
      name:
        matches: "http_requests_total"
        as: "http_requests_per_second"          # the metric HPA will ask for
      metricsQuery: 'sum(rate(<<.Series>>{<<.LabelMatchers>>}[2m])) by (<<.GroupBy>>)'
```

Now the HPA can target it. `type: Pods` averages the metric across pods and scales to hold that average:

```
# repo/manifests/50-scaling/orders-hpa-custom.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: { name: orders, namespace: tickethub }
spec:
 scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: orders }
 minReplicas: 3
 maxReplicas: 30
 metrics:
   - type: Pods
     pods:
       metric: { name: http_requests_per_second }
       target: { type: AverageValue, averageValue: "50" }   # ~50 rps per pod
```

Verify the metric is actually being served before trusting the HPA:

```
kubectl get --raw \
 /apis/custom.metrics.k8s.io/v1beta1/namespaces/tickethub/pods/*/http_requests_per_second
```

Custom metrics: HPA <- adapter <- Prometheus <- your /metrics

HPA never talks to Prometheus. The chain is: your app exposes `/metrics` (Chapter 26), Prometheus scrapes it, **prometheus-adapter** serves it on `custom.metrics.k8s.io`, and HPA reads *that* API. Break any link — no ServiceMonitor, no adapter rule — and the HPA silently reports `<unknown>` for the metric and won't scale.

Scale on a rate or ratio, never a raw counter

`http_requests_total` only ever increases — targeting it directly would scale to the moon and never come back. Always wrap counters in `rate(...)` (as the adapter rule does) so you scale on *current* load. The same applies to latency: use a `histogram_quantile` over a window, not a cumulative sum.

16.5 Vertical Pod Autoscaler

Where HPA adds *more* pods, **VPA** makes *each* pod the right size by adjusting requests from observed usage. Run it in **`recommend`** mode to inform your manifests without surprise restarts.

```
# repo/manifests/50-scaling/search-vpa.yaml
apiVersion: autoscaling.k8s.io/v1
```

```
kind: VerticalPodAutoscaler
metadata: { name: search, namespace: tickethub }
spec:
 targetRef: { apiVersion: apps/v1, kind: Deployment, name: search }
 updatePolicy: { updateMode: "Off" }   # "Off" = recommend only (safe default)
```

Don't run HPA and VPA on the same metric

HPA-on-CPU and VPA-in-Auto-mode both reacting to CPU will fight each other. Safe combos: **HPA on CPU/custom + VPA in recommend-only**, or **VPA-Auto on memory + HPA on a custom metric**. Never both auto-adjusting the same resource.

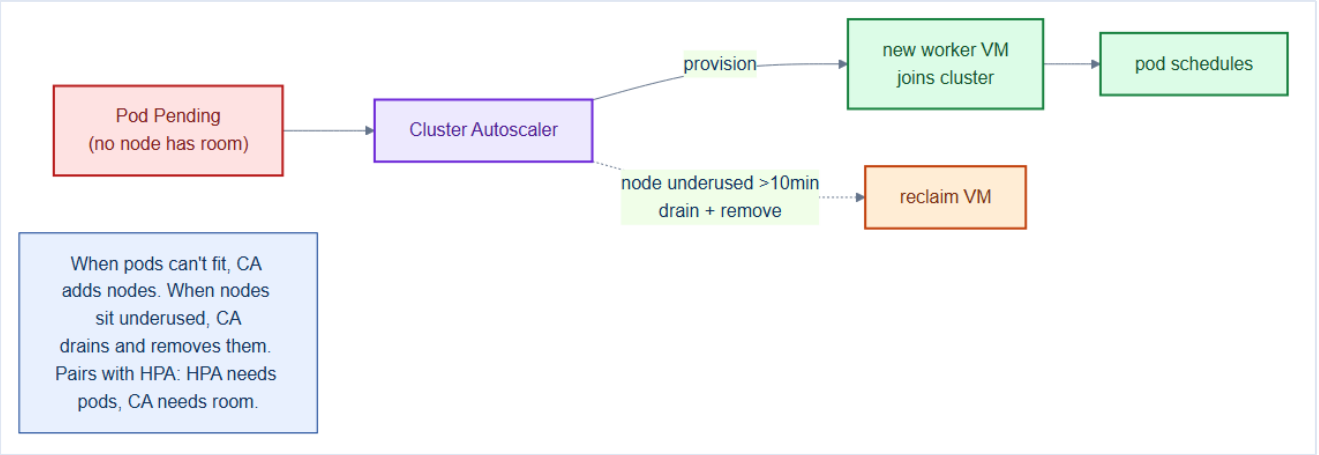
16.6 KEDA — event-driven scaling

CPU is a poor signal for a queue consumer. **KEDA** scales **notifications** directly on **Kafka consumer lag** — and can scale to **zero** when idle.

```
# repo/manifests/50-scaling/notifications-keda.yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: { name: notifications, namespace: tickethub }
spec:
 scaleTargetRef: { name: notifications }
 minReplicaCount: 0                    # scale to zero when no messages
 maxReplicaCount: 30
 triggers:
   - type: kafka
     metadata:
       bootstrapServers: kafka-0.kafka.data:9092
       consumerGroup: notifications
       topic: ticket-events
       lagThreshold: "100"             # 1 replica per 100 messages of lag
```

16.7 Cluster Autoscaler — elastic nodes

HPA/KEDA can only place pods if there's room. When pods go **Pending** for lack of capacity, the **Cluster Autoscaler** provisions new worker VMs and drains/removes them when idle.



The bare-metal caveat. On a cloud, the autoscaler calls an API and a new node appears from an effectively infinite pool. On **Proxmox bare metal there is no infinite pool** — you can only autoscale within the physical RAM and cores you actually own. Making it work needs real plumbing: the **Cluster API Proxmox provider (CAPMOX)** clones new worker VMs from the golden template (Chapter 2), and the autoscaler is wired to that node group. Expect **minute-scale** provisioning (clone + boot + join + CNI ready), not the seconds a cloud gives you.

On bare metal, prefer standing headroom over just-in-time nodes

Because a bare-metal scale-up is slow and physically bounded, don't rely on it to absorb a sudden on-sale spike — the new node arrives *after* users have already seen errors. The pragmatic pattern: keep a **warm buffer** of spare capacity (a couple of idle nodes, or headroom pods via a low-priority "balloon" PriorityClass, Chapter 17) so HPA/KEDA can place pods **instantly**, and let the Cluster Autoscaler top the pool back up in the background. Autoscaling on bare metal manages *cost over hours*, not *bursts over seconds*.

16.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **HPA has a stabilization window**: by default, scale-down is suppressed for 300 seconds after the last scale event to prevent thrashing. During a load spike that lasts 4 minutes, the HPA may keep extra pods running for 5+ minutes after load drops. This is intentional — tune `stabilizationWindowSeconds` for your traffic pattern.

- **KEDA can scale to zero; HPA cannot**: HPA's minimum is 1 replica. KEDA's `ScaledObject` with `minReplicaCount: 0` genuinely scales the Deployment to zero pods when there's no work — saving resources for batch/event-driven services. The trade-off: a cold-start delay on the first message.

- **VPA mutates pods at admission time**: VPA applies recommendations by evicting pods and recreating them with new resource specs. This means a VPA with `updateMode: Auto` is continuously evicting pods based on usage — acceptable for batch, destructive for stateful services. Always use `updateMode: Off` (recommend only) for databases.

Gotchas — traps that catch experienced engineers

- **HPA + VPA on the same Deployment**: if HPA controls replicas and VPA controls resources, they fight. HPA may scale up to 10 replicas; VPA may then lower each pod's request, causing HPA to scale back down (thinking load is lower). Only use VPA on workloads NOT controlled by HPA, or use the `VerticalPodAutoscalerCheckpoints` approach.

- **Custom metric HPA with Prometheus adapter**: if the Prometheus query returns no data (empty series), the adapter returns `0`. HPA interprets this as "zero load" and scales to `minReplicas` — potentially dropping all pods during a monitoring outage. Configure

`behavior.scaleDown.stabilizationWindowSeconds: 600` as a safety buffer.

- **Cluster Autoscaler vs pod eviction**: CA adds nodes based on `Pending` pods. If your node group has a maximum size and CA hits it, pods stay `Pending` indefinitely with no obvious error. Monitor `cluster_autoscaler_unschedulable_pods_count` and set an alert.

Architect Considerations

1. **Scale floor vs cost floor**: KEDA's scale-to-zero is great for cost — but the first Kafka message after a cold start triggers a pod create (~30s), during which messages queue. Define the acceptable message-processing latency SLO and compare it against the cold-start time before enabling scale-to-zero for `notifications`.

2. **Horizontal vs vertical scaling decision**: stateless services (catalog, orders, gateway) scale horizontally (more pods). Stateful services (Postgres, Kafka) scale vertically (larger pods) or with sharding. Mixing strategies on the same workload requires careful VPA/HPA co-ordination.

3. **Node group diversity for Cluster Autoscaler**: CA only adds nodes it knows about. With a single node group (all general workers), CA cannot add data-pool or infra-pool nodes. If Prometheus or Kafka needs more capacity, CA cannot help — you must manually expand the data pool. Design node groups to match your scaling axes.

4. **KEDA ScaledJob vs ScaledObject**: for short-lived batch tasks, `ScaledJob` creates new Job instances per queue message rather than scaling a long-running Deployment. This gives perfect work isolation (one crash doesn't affect others) but higher pod overhead. Choose based on job duration and failure isolation requirements.

5. **Autoscaler interaction with PodDisruptionBudget**: a PDB with `minAvailable: 2` on a 3-replica Deployment blocks the CA from removing a node if it would violate the PDB. Always pair PDB min with a replica count that allows graceful scale-down.

Chapter 16 checklist

- **Metrics Server** installed and verified (`kubectl top` works).
- Stateless services have **HPA** on CPU/custom metrics with sane min/max + scale-down window.
- **Custom-metric HPA** wired via **prometheus-adapter** (scale on rps/latency, not just CPU).
- **VPA in recommend mode** informs request sizing; not fighting HPA on the same metric.
- Queue consumers scaled by **KEDA** on lag (scale-to-zero where safe).

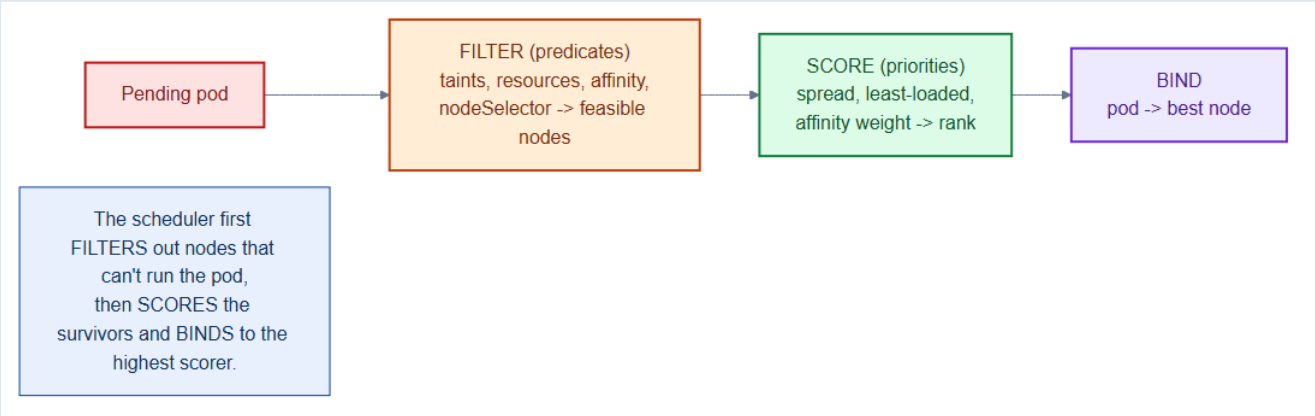
- **Cluster Autoscaler** grows/shrinks the node pool so HPA/KEDA always have room.

---

17. Scheduling & Placement — PriorityClass, Affinity, Taints, Topology Spread & PDB

Autoscaling decides *how many* pods; **scheduling** decides *where* they run. Left alone, the scheduler does a good job — but a production architect steers it deliberately: keep databases on data nodes, spread replicas across racks so one failure can't take a service down, and make sure revenue-critical pods win when the cluster is full.

17.1 How the scheduler decides



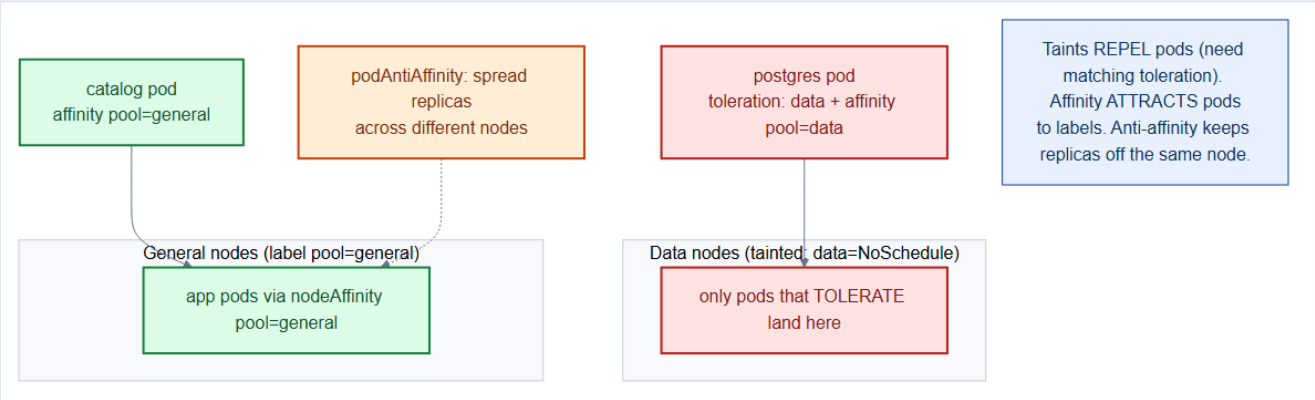
For every Pending pod the scheduler runs two phases:

1. **Filter (predicates)** — eliminate nodes that *can't* run it: not enough free requests, taint not tolerated, `nodeSelector`/affinity mismatch, volume zone conflict.
2. **Score (priorities)** — rank the survivors: spread, least-loaded, affinity weights. Highest score gets the pod (**bind**).

> **Mental model — seating guests at a wedding**
>
> **Filtering** removes impossible tables (allergy conflicts, no free seats). **Scoring** picks the *best* remaining table (near friends, away from the loud speakers). The scheduler seats one guest (pod) at a time onto the best feasible node.

17.2 Taints & tolerations — repelling pods

A **taint** on a node repels pods that don't explicitly **tolerate** it. This is how the data and infra pools (Chapter 3) stay reserved.




```
# node carries: kubectl taint nodes worker-data-1 data=true:NoSchedule
tolerations:
 - key: "data"
   operator: "Equal"
   value: "true"
   effect: "NoSchedule"      # only tolerating pods (Postgres, Kafka) may land
```

17.3 Affinity & anti-affinity — attracting and separating

Where taints *repel*, **affinity** *attracts* pods to labels, and **anti-affinity** keeps replicas apart.

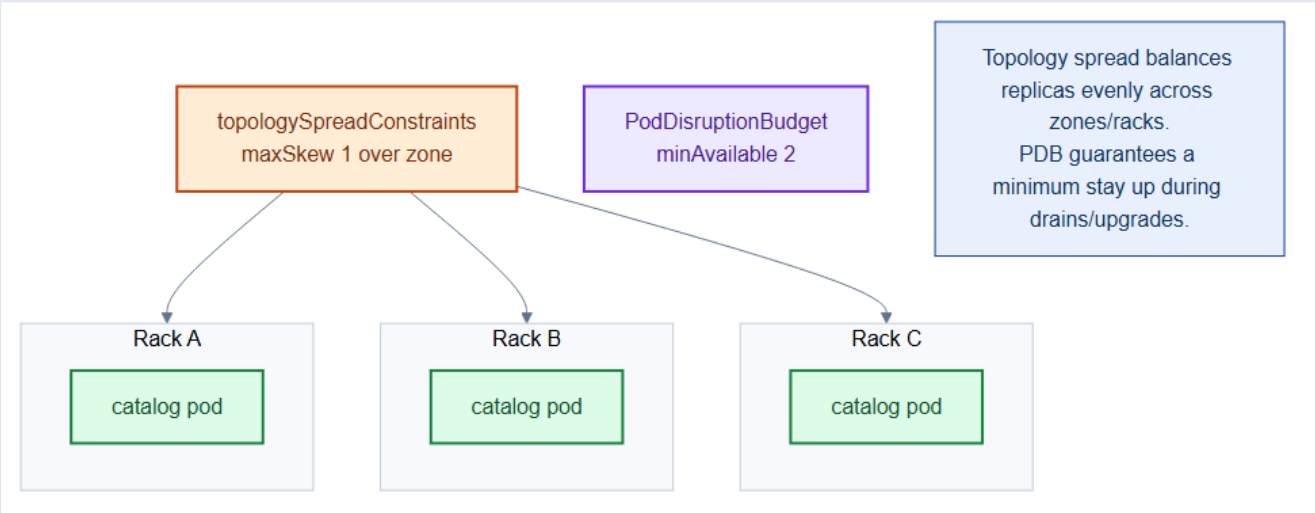
```
affinity:
 nodeAffinity:                                  # run only on data nodes
   requiredDuringSchedulingIgnoredDuringExecution:
     nodeSelectorTerms:
       - matchExpressions:
           - { key: pool, operator: In, values: [data] }
 podAntiAffinity:                               # spread replicas across hosts
   preferredDuringSchedulingIgnoredDuringExecution:
     - weight: 100
       podAffinityTerm:
         labelSelector:
           matchLabels: { app: orders }
         topologyKey: kubernetes.io/hostname
```

Anti-affinity is what makes replicas actually redundant

Three `orders` replicas on the **same node** give you zero resilience — one node failure kills all three. `podAntiAffinity` on `topologyKey: hostname` forces them onto **different nodes**, so the whole point of replication (surviving a node loss) holds.

17.4 Topology spread & Pod Disruption Budgets

Topology spread constraints balance replicas evenly across failure domains (zones/racks); a **PodDisruptionBudget (PDB)** guarantees a minimum stay running during *voluntary* disruptions (node drains, upgrades).




```
# spread constraint in the pod spec
topologySpreadConstraints:
 - maxSkew: 1
   topologyKey: topology.kubernetes.io/zone
   whenUnsatisfiable: ScheduleAnyway
   labelSelector: { matchLabels: { app: catalog } }
---
# repo/manifests/50-scaling/pdb.yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: { name: catalog, namespace: tickethub }
spec:
 minAvailable: 2                        # never let a drain drop below 2
 selector: { matchLabels: { app: catalog } }
```

On bare metal, the zone label must exist or the spread is a silent no-op

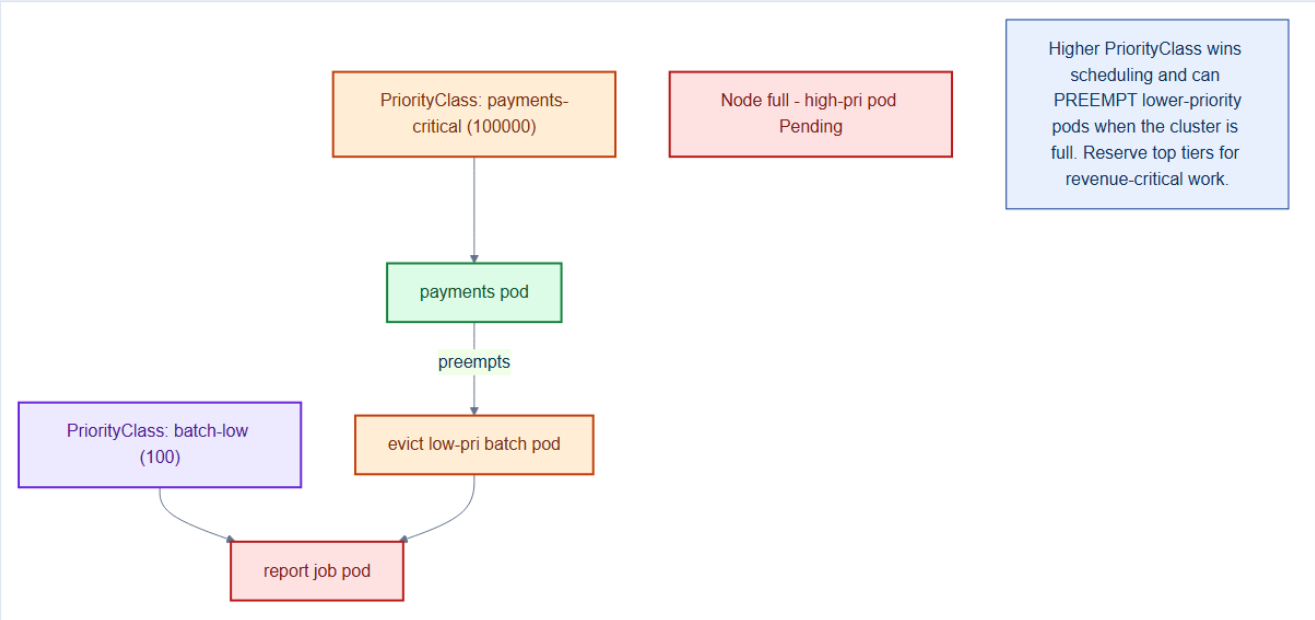
`topology.kubernetes.io/zone` is only meaningful if the nodes actually carry it. A cloud sets it automatically; **your bare-metal cluster does not** — you label nodes by rack yourself (Chapter 3). If the label is missing, every node looks like one giant zone, `maxSkew` is trivially satisfied, and all replicas can pile into a single rack while the constraint reports success. With `whenUnsatisfiable: ScheduleAnyway` it fails *open* (schedules anyway); use `DoNotSchedule` only once you're sure the labels exist.

Without a PDB, a node drain can take your whole service down

`kubectl drain` (upgrades, Chapter 27) evicts *all* pods on a node at once. If two of your three replicas happened to share that node and there's no PDB, the service can briefly hit zero healthy pods. A PDB makes the drain **wait** until replacements are Ready elsewhere. Every user-facing service needs one.

17.5 PriorityClass & preemption

When the cluster is full, **PriorityClass** decides who wins — and lets high-priority pods **preempt** (evict) lower-priority ones to make room.




```
# repo/manifests/50-scaling/priorityclasses.yaml
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: payments-critical }
value: 100000
globalDefault: false
description: "Revenue-critical services; may preempt batch work."
---
apiVersion: scheduling.k8s.io/v1
kind: PriorityClass
metadata: { name: batch-low }
value: 100
description: "Reports/reindex; first to be preempted."
```

17.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Affinity and anti-affinity are evaluated at scheduling time, not continuously**: once a pod is placed, it is never evicted just because the affinity rule is violated (e.g., if the preferred node is retainted after scheduling). Descheduler (an optional add-on) can periodically re-balance, but vanilla Kubernetes does not.

- **`topologySpreadConstraints` uses `labelSelector` to count matching pods, not running pods**: a `Pending` pod counts toward the spread calculation. If two pods are simultaneously scheduled to the same zone, the spread constraint is temporarily violated — Kubernetes tolerates this momentary imbalance.

- **PriorityClass preemption evicts the lowest-priority pod** that, when removed, gives the high-priority pod enough resources. But the evicted pod's graceful termination period still applies — Kubernetes waits for it to terminate before scheduling the high-priority pod. During `terminationGracePeriodSeconds`, both the evicted and the new pod are competing for resources.

Gotchas — traps that catch experienced engineers

- **`requiredDuringSchedulingIgnoredDuringExecution` anti-affinity with replicas > zones**: if you require each pod to be on a different zone and you have 3 pods but only 2 zones, the third pod stays `Pending` forever. Use `preferredDuringSchedulingIgnoredDuringExecution` unless you can guarantee enough nodes in each zone.

- **Forgetting DaemonSet tolerations when adding taints**: adding `NoSchedule` taints to data/infra nodes without updating DaemonSet tolerations breaks monitoring (Falco, node-exporter) on those nodes — silently.

- **PodDisruptionBudget `unhealthyPodEvictionPolicy: AlwaysAllow`** (Kubernetes 1.27+) allows voluntary disruptions to proceed even if the pod is already unhealthy. Without this, a pod in CrashLoopBackOff blocks node drains indefinitely — a common cluster upgrade blocker.

Architect Considerations

1. **Rack-awareness vs zone-awareness**: Kubernetes only knows zones, not racks. If your 3 racks are in the same zone (same data center), a zone failure still takes out all 3 racks. For true rack-level HA, add a custom `topology.tickethub.io/rack` label and use it in `topologySpreadConstraints`.

2. **Cluster-level PriorityClass policy**: who can create high-priority PriorityClasses? A team that creates `value: 1000000` can starve ALL other workloads by preemption. Restrict PriorityClass creation to cluster admins via RBAC (Chapter 19).

3. **Descheduler for re-balancing**: Kubernetes schedules but doesn't continuously re-balance. After node addition or failure recovery, pods are not automatically redistributed. The `descheduler` project adds this capability — evaluate it before assuming your topology spread constraints are being honoured over time.

4. **Topology spread with HPA**: when HPA scales a Deployment from 3 to 12 replicas under load, `topologySpreadConstraints` guides placement. But if one zone's nodes are full, the scheduler falls back to a zone with capacity — violating the spread. Ensure zone capacity is symmetric and sized for peak replica counts.

5. **Graceful termination vs PDB interaction**: a PDB prevents pod eviction if it would violate `minAvailable`. During a rolling update, a pod being terminated counts as "unavailable" until it fully stops. If `terminationGracePeriodSeconds` is long (60s+) and the PDB is tight (`minAvailable: 2` on 3 replicas), the rollout serializes to 1 pod at a time and takes minutes. Tune graceful termination and PDB together.

Chapter 17 checklist

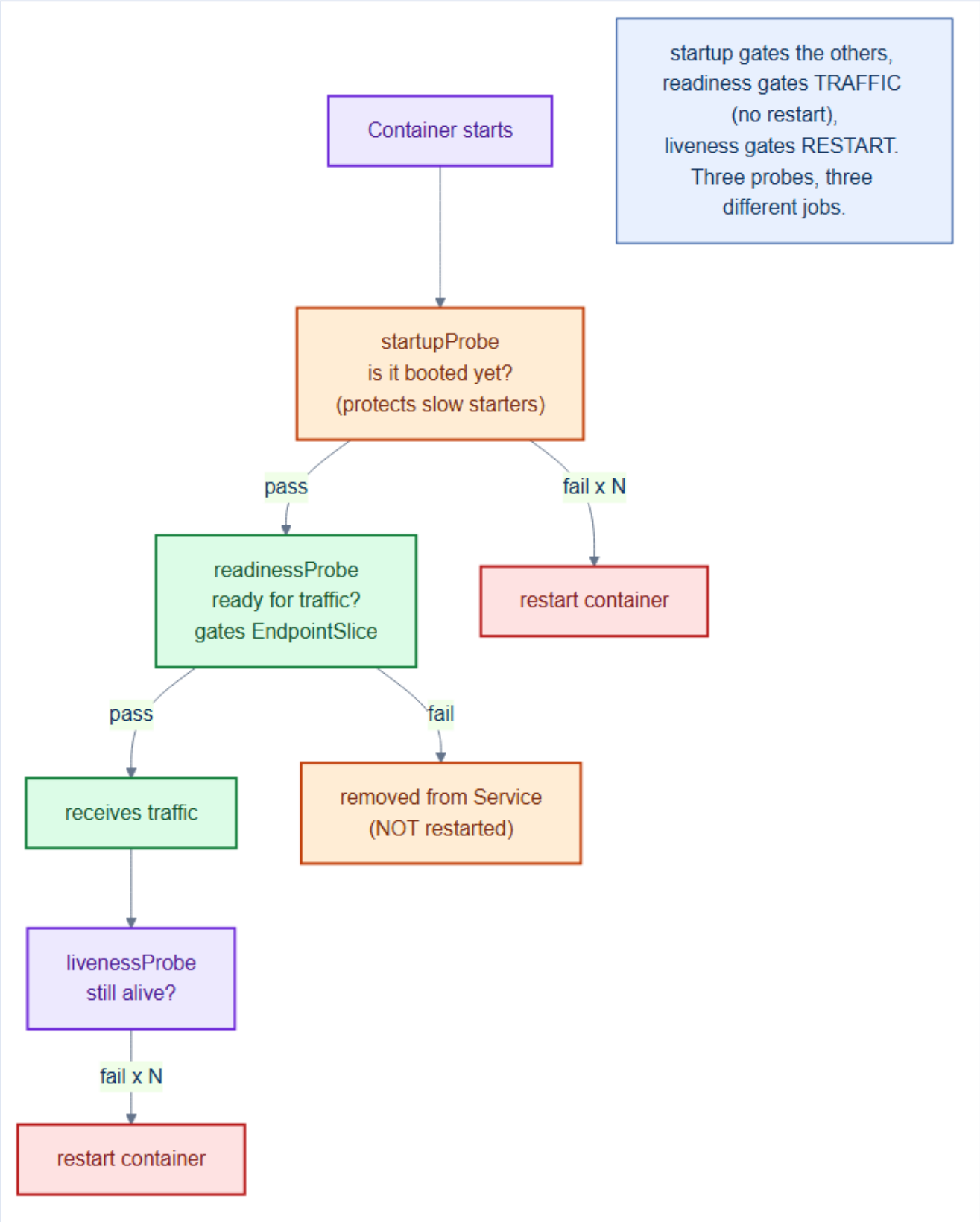
- Data/infra pools **tainted**; only matching workloads **tolerate** them.
- Replicas use **podAntiAffinity** + **topologySpreadConstraints** across nodes/zones.
- Every user-facing service has a **PodDisruptionBudget**.
- **PriorityClasses** protect revenue-critical pods and let them preempt batch work.
- Placement is deliberate, not accidental — verified with `kubectl get pods -o wide`.

---

18. Health & Lifecycle — Probes, Rollout Strategies & Graceful Shutdown

A pod that is *running* is not necessarily *working*, and a pod that is *stopping* shouldn't drop live requests. This chapter closes Part IV with the mechanisms that make TicketHub self-healing and zero-downtime: **probes** that tell Kubernetes the truth about pod health, **rollout strategies** that ship new versions without an outage, and **graceful shutdown** that finishes in-flight work.

18.1 Three probes, three different jobs



Probe	Question	On failure	Gates
startupProbe	Has it booted yet?	Restart container	Protects slow starters from liveness

readinessProbe	Ready for traffic?	**Remove from Service** (no restart)	EndpointSlice membership
livenessProbe	Still alive/healthy?	**Restart** container	Self-healing

```
startupProbe:                   # give slow apps up to 30x2s = 60s to boot
 httpGet: { path: /healthz, port: 8080 }
 failureThreshold: 30
 periodSeconds: 2
readinessProbe:                 # only Ready pods get traffic
 httpGet: { path: /readyz, port: 8080 }
 initialDelaySeconds: 3
 periodSeconds: 5
livenessProbe:                  # restart if wedged
 httpGet: { path: /healthz, port: 8080 }
 initialDelaySeconds: 10
 periodSeconds: 10
```

Mental model — a new employee's first day

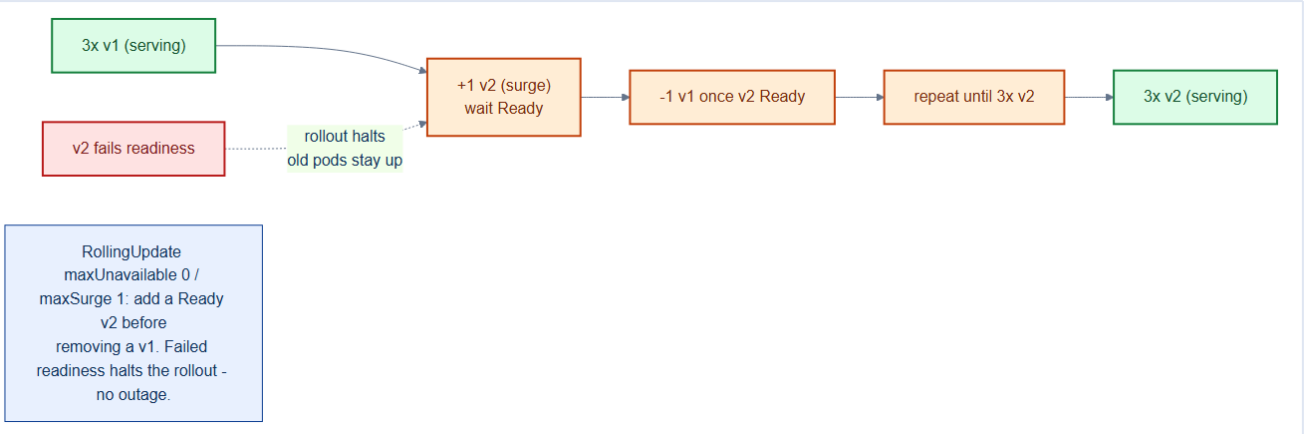
startup = "has the new hire finished orientation?" (don't judge them yet). **readiness** = "are they at their desk ready to take calls?" (route work only when yes). **liveness** = "have they passed out?" (if unresponsive, send them home and call a replacement). Three different questions — never answer them with one probe.

readiness ≠ liveness — the classic outage

Point **liveness** at a check that also depends on the **database**, and a brief DB blip makes Kubernetes **restart every pod at once** — turning a small dependency hiccup into a full outage. Liveness must test only *this process*. Dependency health belongs in **readiness** (which just pauses traffic, no restart).

18.2 Rollout strategies

The default **RollingUpdate** replaces pods gradually; with `maxUnavailable: 0` there's never a capacity dip, and a failing new version **halts the rollout** automatically.



```
strategy:
 type: RollingUpdate
 rollingUpdate:
   maxUnavailable: 0        # add a Ready new pod BEFORE removing an old one
   maxSurge: 1
```

```
kubectl -n tickethub rollout status deploy/catalog     # watch convergence
kubectl -n tickethub rollout undo deploy/catalog       # instant rollback to prior ReplicaSet
```

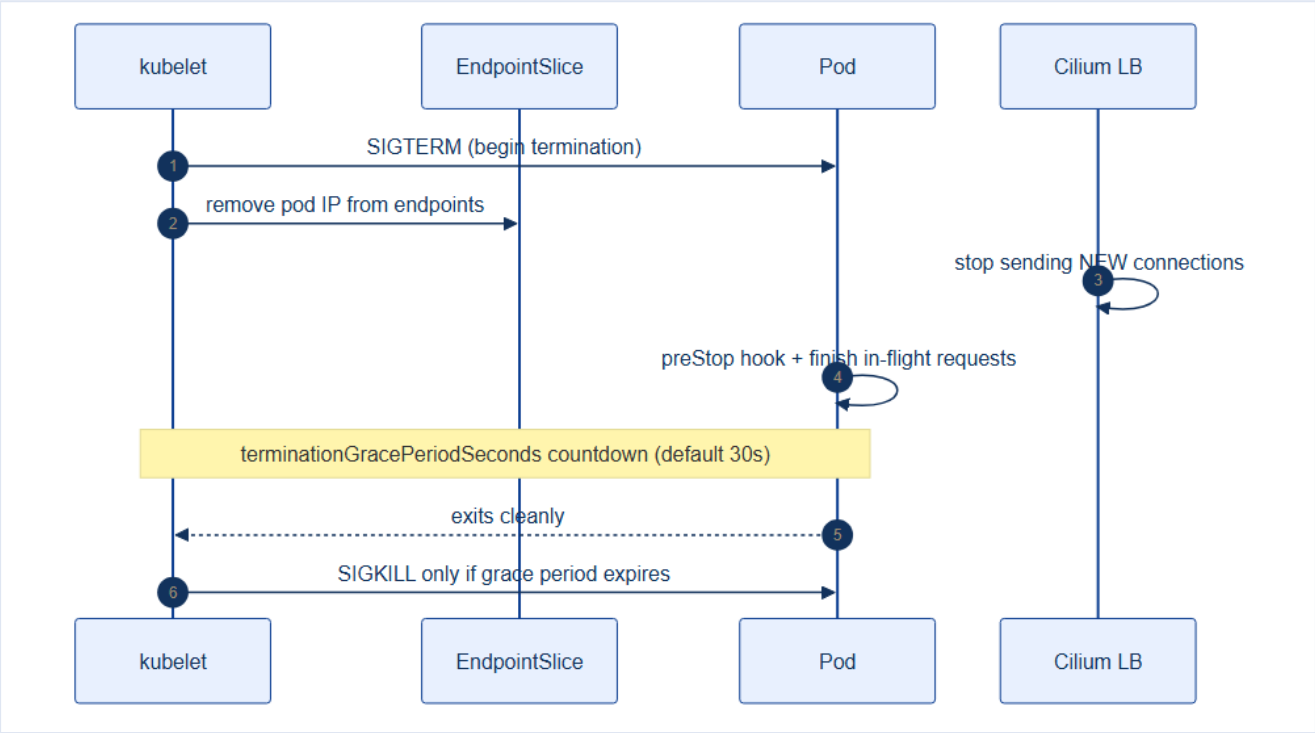
Strategy	How	When
RollingUpdate	Gradual pod-by-pod swap	Default for all services
Recreate	Kill all, then start new	Only when versions can't coexist
Blue-Green	Full parallel stack, flip traffic	High-risk releases (via Argo Rollouts)
Canary	Shift a % of traffic, watch metrics	Progressive delivery (Argo Rollouts, Ch 28)

Readiness probes are what make rolling updates safe

`maxUnavailable: 0` only helps if "available" means truly *ready*. Because the new pod joins the Service **only after** its readiness probe passes, users are never routed to a half-booted pod. Rollout safety = readiness probe + `maxUnavailable: 0`.

18.3 Graceful shutdown

When a pod is removed (scale-down, rollout, drain), Kubernetes must not sever live connections. The termination sequence is precise:



1. Pod IP is removed from the **EndpointSlice** → no *new* connections routed.
2. Container receives **SIGTERM**; an optional **preStop** hook runs.
3. App finishes in-flight requests and exits within `terminationGracePeriodSeconds` (default 30s).
4. **SIGKILL** only if the grace period expires.

```
terminationGracePeriodSeconds: 45
lifecycle:
 preStop:
   exec:
     command: ["sh", "-c", "sleep 5"]   # let endpoint removal propagate first
```

Handle SIGTERM in your app, or you drop requests

If the process ignores **SIGTERM** and just dies on SIGKILL, in-flight requests are cut mid-response every deploy. The app must catch SIGTERM, stop accepting new work, drain active requests, then exit. The `preStop sleep` covers the brief race between endpoint removal and the load balancer noticing.

18.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **All three probe types use the same failure threshold logic** (`failureThreshold × periodSeconds`) but serve different purposes: liveness kills and restarts the container; readiness removes it from Service endpoints (no restart); startup suppresses liveness during the startup window. A wrong probe type causes the wrong behavior — a liveness probe that triggers during a

traffic spike causes a restart cascade instead of graceful back-pressure.

- **`preStop` hook runs concurrently with SIGTERM in some container runtimes**: the hook is not guaranteed to complete before SIGTERM is sent in all cases. If your shutdown sequence depends on the hook finishing first (e.g., draining a connection pool before accepting SIGTERM), add a `sleep` in the hook equal to your expected drain time as a belt-and-suspenders measure.

- **Rolling update `maxSurge` and `maxUnavailable` are evaluated as a PAIR**: with `maxUnavailable: 0` and `maxSurge: 1`, the update creates one new pod and waits for it to pass readiness before killing one old pod. The deployment is always at full capacity — ideal for zero-downtime. With `maxUnavailable: 1` and `maxSurge: 0`, it kills one pod first, then creates a replacement — briefly drops below capacity.

Gotchas — traps that catch experienced engineers

- **Liveness probe too aggressive during GC pauses**: a JVM doing a full GC may pause for 5-10 seconds. If `liveness.timeoutSeconds: 1` and `failureThreshold: 3`, the container is killed after ~3 seconds of GC — causing a restart loop under load. Set `timeoutSeconds: 5` and `failureThreshold: 3` (15s total) for JVM services.

- **Readiness probe checking downstream dependencies**: a readiness probe that calls `SELECT 1` on Postgres means a Postgres outage marks ALL orders pods as unready — removing them from the Service endpoint and returning 503 to users even though the pods themselves are healthy. Check local health only in readiness probes; check downstream health in separate alerts.

- **`terminationGracePeriodSeconds: 0`** for "fast" rolling updates: this kills containers immediately on SIGTERM with no grace period. In-flight requests are dropped. Always allow enough time for connection draining: `terminationGracePeriodSeconds` ≥ the longest expected request duration + 5s buffer.

Architect Considerations

1. **Startup probe vs `initialDelaySeconds`**: `initialDelaySeconds` is a blunt instrument — it delays all probes by a fixed time regardless of actual startup progress. `startupProbe` is smarter: it polls until the app is actually ready, then hands off to liveness/readiness. Always use `startupProbe` for services with variable startup times (JVM warm-up, schema migrations).

2. **Probe granularity**: a `/healthz` endpoint that returns 200 is not meaningful if it doesn't actually test the service's ability to serve traffic. Define three layers: `/healthz` (process alive — for liveness), `/readyz` (can serve requests — for readiness, checks DB connection pool), `/startupz` (initialization complete — for startup probe).

3. **Rolling update speed vs risk**: `maxUnavailable: 0, maxSurge: 1` is safest (never below capacity) but slowest (one pod at a time). `maxUnavailable: 25%, maxSurge: 25%` is 4× faster but briefly runs at 75% capacity. Size your `minReplicas` so that `replicas × (1 - maxUnavailable)` still meets your RPS SLO during rollout.

4. **Blue/green vs rolling for schema-breaking changes**: a rolling deployment of a service with a breaking API change means old and new versions serve traffic simultaneously. If the API break is a response field rename, clients see inconsistency. Use blue/green (create a separate Deployment, switch Service selector atomically) for schema-breaking changes.

5. **Canary with traffic splitting**: Argo Rollouts or Flagger can send 5% of traffic to the new version (canary) and automatically roll back if the error rate exceeds a threshold. This is the production-safe deployment strategy for TicketHub's payments path — zero-risk progressive delivery.

Chapter 18 checklist — Part IV complete

- **startup / readiness / liveness** probes each defined for their distinct job.
- Liveness tests **only the process**; dependency health lives in **readiness**.
- Rollouts use **RollingUpdate + maxUnavailable: 0**; rollback is one command.
- Apps handle **SIGTERM** and drain within `terminationGracePeriodSeconds`.
- Result: **self-healing, zero-downtime** deploys and scale events.

TicketHub is now reliable and elastic. **Part V** locks it down: RBAC, Pod Security, NetworkPolicy, Kyverno, Falco, and supply-chain security.

---

19. RBAC & Service Accounts — Least Privilege for Every Actor

Part V hardens TicketHub. Security starts with **who can do what**. **Role-Based Access Control (RBAC)** governs every request to the API server — human or workload. The guiding rule is **least privilege**: each actor gets exactly the permissions it needs, and nothing more.

19.1 The four RBAC objects

Object	Answers	Scope
Role	*what* verbs on *which* resources	One namespace
ClusterRole	same, cluster-wide or reusable	Cluster
RoleBinding	grant a Role to a subject	One namespace
ClusterRoleBinding	grant a ClusterRole cluster-wide	Cluster

Subjects are `User`, `Group` (humans, from the auth layer), or `ServiceAccount` (workloads).

> **Mental model — keys and keyrings in a building**
>
> A **Role** is a **key** that opens specific doors (verbs on resources). A **RoleBinding** is handing that key to a **person or robot** (subject). A **Role** in one namespace is a key that only works on that floor; a **ClusterRole** works building-wide. RBAC is **additive** — there's no "anti-key". Default is deny-all; you only ever grant.

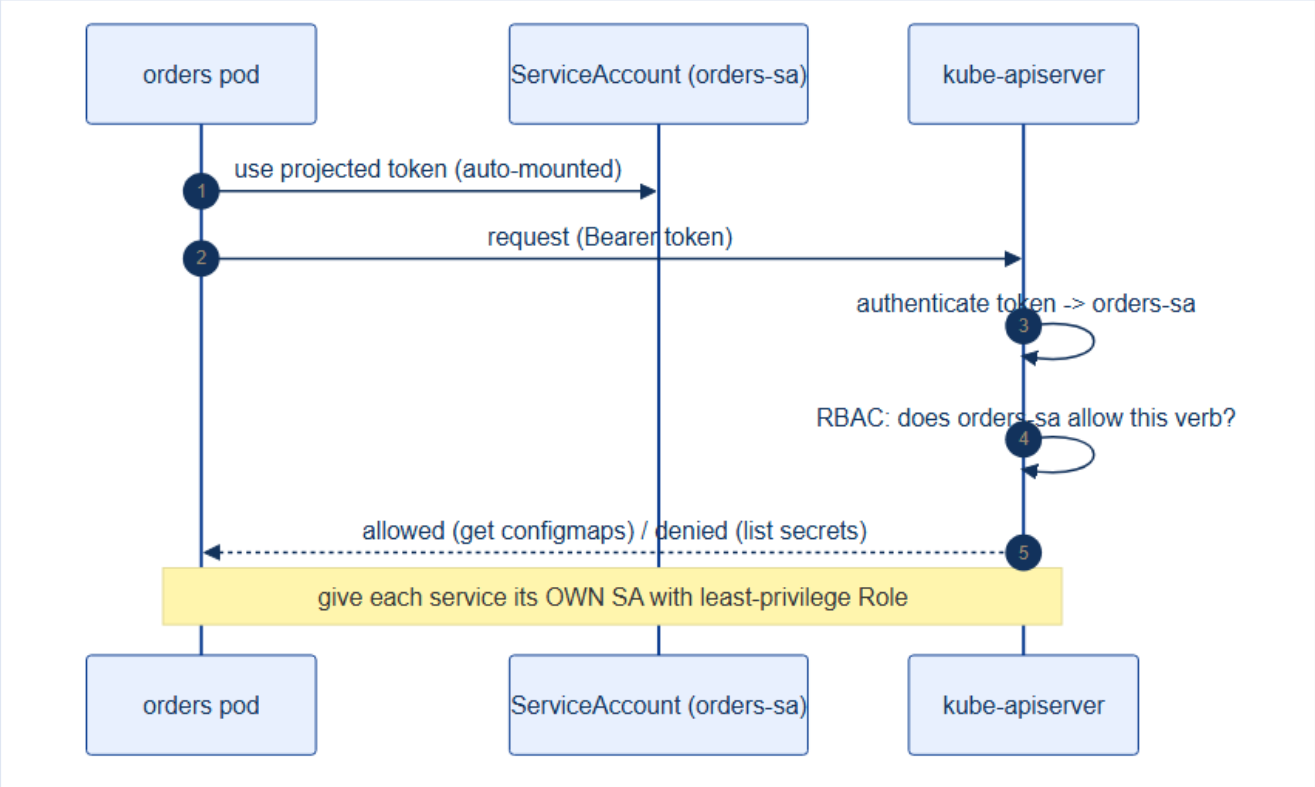
19.2 RBAC is additive and default-deny

There is **no deny rule** in RBAC. If no binding grants a permission, it's refused. This makes reasoning simple: permissions only ever accumulate from the bindings that name you.

```
# repo/manifests/60-security/orders-rbac.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: orders-reader
 namespace: tickethub
rules:
 - apiGroups: [""]
   resources: ["configmaps"]
   verbs: ["get", "list", "watch"]       # read config only — no secrets, no write
```

19.3 Give every workload its own ServiceAccount

Every pod authenticates to the API server as a **ServiceAccount (SA)**. By default it's the namespace's `default` SA — a trap, because bindings on `default` leak to *every* pod. Give each service a **dedicated SA** with a minimal Role.




```
# See: repo/manifests/ for the full manifest
apiVersion: v1
kind: ServiceAccount
metadata:
 name: orders-sa
 namespace: tickethub
automountServiceAccountToken: false   # only mount if the app truly calls the API
---
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: orders-reader-binding
 namespace: tickethub
subjects:
 - kind: ServiceAccount
   name: orders-sa
roleRef:
 kind: Role
 name: orders-reader
 apiGroup: rbac.authorization.k8s.io
```

```
# in the Deployment pod spec
spec:
 serviceAccountName: orders-sa
```

Turn off automount unless the pod calls the API

Most application pods never talk to the Kubernetes API — yet by default a token is mounted into every one, handing an attacker who lands in the container a credential. Set `automountServiceAccountToken: false` on the SA (or pod) unless the workload genuinely needs API access. Fewer tokens = smaller blast radius.

19.4 Auditing and avoiding over-permission

```
kubectl auth can-i --list --as=system:serviceaccount:tickethub:orders-sa -n tickethub
kubectl auth can-i delete secrets --as=system:serviceaccount:tickethub:orders-sa -n tickethub  # ->
```

Never bind cluster-admin to a workload or humans by default

`cluster-admin` via a ClusterRoleBinding is god mode. A compromised pod with it owns the cluster. Avoid wildcard verbs (`["*"]`) and wildcard resources; scope Roles to named resources. Humans get narrow, namespaced roles; break-glass admin is separate, audited, and rarely used.

19.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **RBAC is additive only** — you cannot explicitly deny a permission. If a user has a RoleBinding that grants `get pods` and another that grants `list pods`, they have both. The only way to "deny" access is to not grant it in the first place and remove all relevant bindings. This makes RBAC reasoning about "what CAN this principal NOT do?" difficult — enumerate permissions via `kubectl auth can-i --list --as <user>`.

- **ServiceAccount tokens are long-lived by default in older Kubernetes versions**: before Kubernetes 1.22, SA tokens were stored as Secrets with no expiry. From 1.24+, the token projection

mechanism creates short-lived tokens (1 hour) automatically. If you're running workloads on 1.21 or below, audit token expiry explicitly.

- **`ClusterRoleBinding` to a namespaced `Role`** is not possible — a ClusterRoleBinding must reference a ClusterRole. However, a `RoleBinding` CAN reference a ClusterRole: this applies the ClusterRole's permissions only within the binding's namespace. Use this pattern to define roles once as ClusterRoles and bind them per-namespace.

Gotchas — traps that catch experienced engineers

- **`system:masters` group bypasses all RBAC**: adding a user to `system:masters` (e.g., in the kubeconfig generated by kubeadm) gives permanent cluster-admin with no way to revoke — RBAC cannot deny `system:masters`. Never distribute the admin kubeconfig; use OIDC + RoleBindings for human access.

- **Operator service accounts with ClusterRole `*` verbs**: a hastily scaffolded operator that grants `verbs: ["*"]` on `resources: ["*"]` across `apiGroups: ["*"]` is a cluster takeover vector if the operator pod is compromised. Audit and scope every operator's RBAC at install time.

- **`automountServiceAccountToken: true` is the default**: every pod gets a mounted SA token that can call the Kubernetes API. A compromised pod with the default SA can `kubectl get secrets -n kube-system` if the SA has even basic RBAC. Always set `automountServiceAccountToken: false` on SAs for workloads that don't need API access.

Architect Considerations

1. **OIDC integration for human access**: kubeadm-generated certificates are fine for cluster bootstrap, but human access in production should use an OIDC provider (Dex, Keycloak, Azure AD) so that user identities are tied to corporate directory, sessions expire, and access can be revoked centrally.

2. **Least-privilege service account design**: each microservice should have a dedicated ServiceAccount with only the permissions it actually uses. The `orders` service needs to read its own ConfigMaps; it does not need to list pods or create Secrets. Audit actual API calls with `kubectl auth can-i` and audit logs, then prune.

3. **RBAC for namespace self-service**: a team that can create namespaces can also bind ClusterRoles within them — effectively elevating themselves. Restrict `namespace create` to platform admins and provide a namespace provisioning workflow (Argo CD ApplicationSet or a custom Kubernetes controller) that applies RBAC templates.

4. **Aggregated ClusterRoles**: the `view`, `edit`, and `admin` ClusterRoles are aggregate roles that automatically include permissions from any ClusterRole with the matching aggregation label. When

you install a new operator (e.g., cert-manager), its CRD views should be aggregated into the `view` role so developers can `kubectl get certificates` with their normal access.

5. **Audit log analysis**: RBAC decisions are logged in the Kubernetes audit log. Use a tool like `rbac-police` or `audit2rbac` to analyze audit logs and identify over-privileged service accounts. Run this analysis quarterly.

Chapter 19 checklist

- Every service runs under its **own ServiceAccount**, never `default`.
- Roles grant **specific verbs on named resources**; no wildcards.
- `automountServiceAccountToken: false` unless the pod calls the API.
- **No** `cluster-admin` on workloads; human access is namespaced and least-privilege.
- Permissions audited with `kubectl auth can-i`.

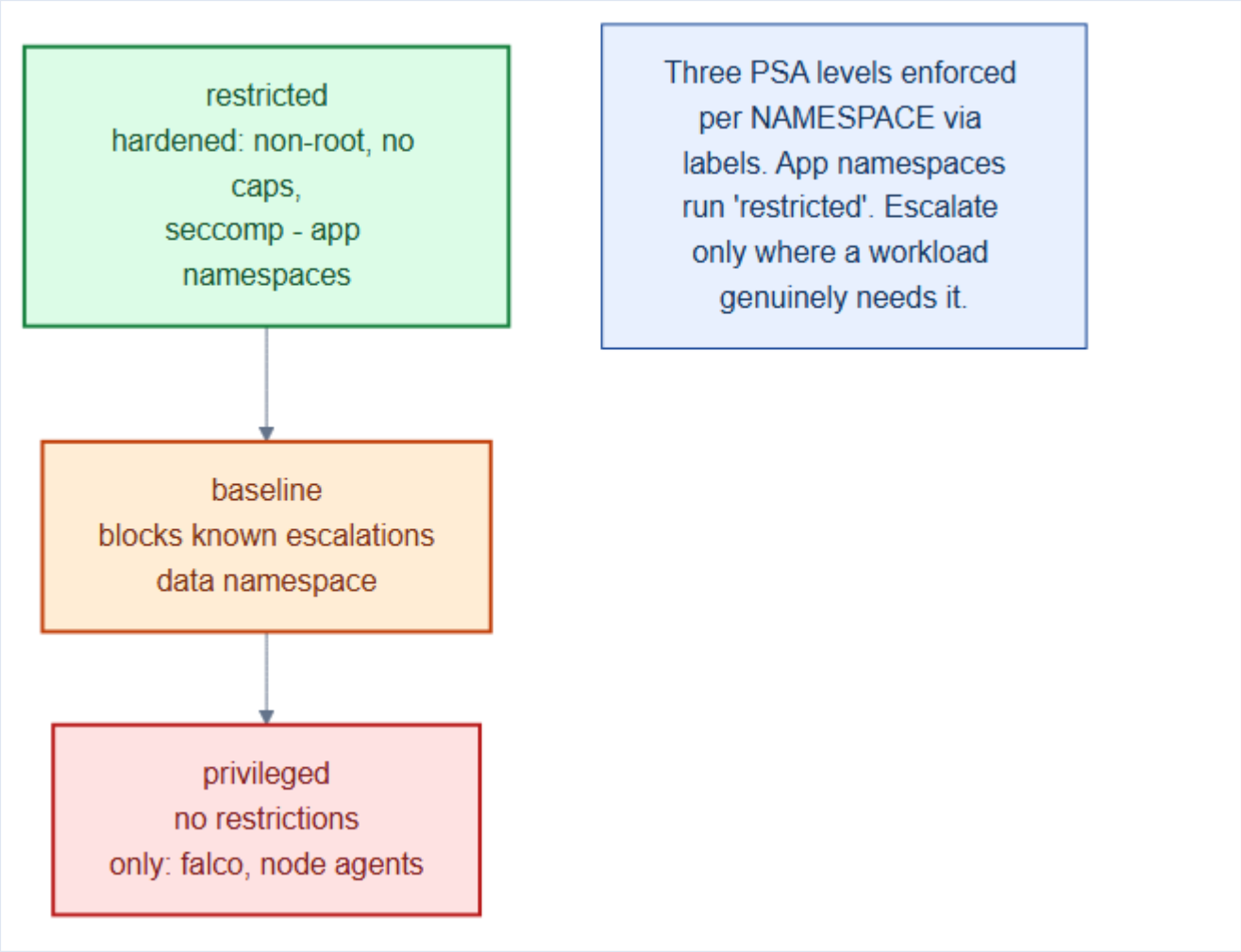
---

20. Pod Security Admission & SecurityContext Hardening

RBAC controls the API; this chapter controls **what a pod may do on the node**. A container that runs as root, mounts the host filesystem, or holds Linux capabilities is a breakout waiting to happen. **Pod Security Admission (PSA)** enforces baseline guardrails per namespace, and **SecurityContext** hardens each pod in depth.

20.1 Pod Security Admission — three levels per namespace

PSA is built into the API server. You label a namespace with one of three levels, and non-compliant pods are rejected (or warned) at admission.



Level	Allows	Use in TicketHub
privileged	Everything	Only `security` ns (Falco needs host access)
baseline	Blocks known privilege escalations	`data` ns (DBs need a few extras)

restricted	Hardened: non-root, no caps, seccomp	All **app** namespaces

```
# already baked into repo/manifests/00-namespaces/namespaces.yaml (Ch 9)
metadata:
 name: tickethub
 labels:
   pod-security.kubernetes.io/enforce: restricted    # reject violations
   pod-security.kubernetes.io/warn: restricted       # warn on kubectl
   pod-security.kubernetes.io/audit: restricted       # log to audit
```

Mental model — building codes vs. a home inspection

PSA is the **building code** for a namespace: every pod must meet the standard to be admitted. SecurityContext (next) is how each pod **proves it's up to code** — non-root wiring, no dangerous tools, sealed windows. PSA is enforced at the door; SecurityContext is what you build into the pod.

20.2 The three enforcement modes

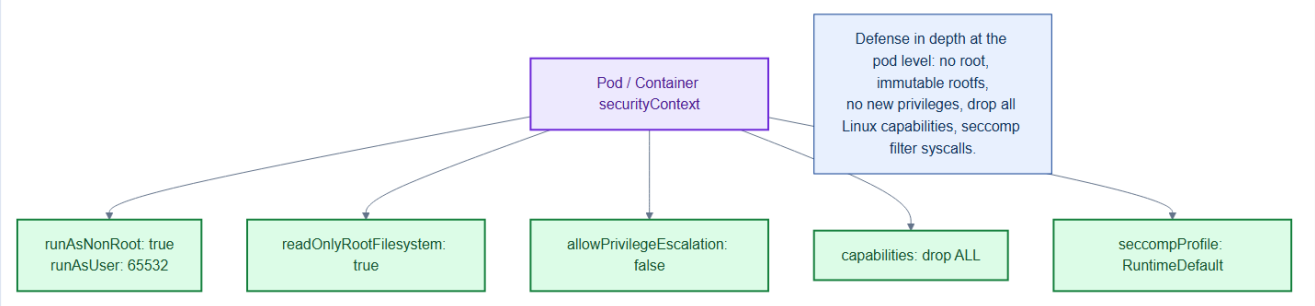
- **enforce** — reject violating pods (production).
- **warn** — allow but return a warning to `kubectl` (great for migration).
- **audit** — allow but record in the audit log.

Roll out restricted with warn/audit first

Flip a namespace to `warn: restricted` + `audit: restricted` *before* `enforce`. You see every violation without breaking running workloads, fix the manifests, then set `enforce`. Retrofitting `enforce` onto a live namespace blind is how you cause an outage.

20.3 SecurityContext — defense in depth per pod

The `restricted` level demands a hardened **securityContext**. Here is the TicketHub standard applied to every app pod:




```
# pod-level
securityContext:
 runAsNonRoot: true
```

```
 runAsUser: 65532
 fsGroup: 65532
 seccompProfile:
   type: RuntimeDefault
# container-level
containers:
 - name: orders
   securityContext:
     allowPrivilegeEscalation: false
     readOnlyRootFilesystem: true
     capabilities:
       drop: ["ALL"]
```

Setting	Stops
`runAsNonRoot` / `runAsUser`	Running as uid 0
`readOnlyRootFilesystem`	Tampering with the image at runtime
`allowPrivilegeEscalation: false`	`setuid`/`sudo`-style escalation
`capabilities: drop: [ALL]`	Raw sockets, mounting, ptrace, etc.
`seccompProfile: RuntimeDefault`	Dangerous syscalls

This pairs with the Dockerfile from Chapter 10

`readOnlyRootFilesystem: true` and `runAsNonRoot` only work if the **image** was built for it — a non-root `USER`, and any writable paths mounted as `emptyDir` volumes. Security is designed at build time (Ch 10) and *enforced* at admission (here). If the app needs to write, give it a mounted `emptyDir` at that path, not a writable rootfs.

```
# writable scratch without a writable rootfs
volumeMounts:
 - { name: tmp, mountPath: /tmp }
volumes:
 - name: tmp
   emptyDir: {}
```

Never set privileged: true on an app pod

`privileged: true` disables essentially all isolation — the container can access host devices and is one step from owning the node. Reserve it for node-level agents (Falco, CNI) in dedicated namespaces, and gate even those with Kyverno (Chapter 22).

20.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Pod Security Admission (PSA) `warn` mode writes warnings to the API server response, not to the pod's logs or events** — operators using CI pipelines must parse `kubectl apply` stderr for `Warning: would violate PodSecurity` messages. Many CI systems suppress stderr by default, making PSA warn mode silently useless.

- **`securityContext.readOnlyRootFilesystem: true`** prevents writes to the container's root filesystem, but tmpfs mounts (via `emptyDir: { medium: Memory }`) and volumeMounts are writable. An application that writes logs to `/tmp` must mount an `emptyDir` at `/tmp` explicitly or the container will crash.

- **`capabilities.drop: [ALL]` without adding back `NET_BIND_SERVICE`**: if your container binds to port 80 or 443, dropping ALL capabilities prevents binding to privileged ports (< 1024). Either run on a non-privileged port (8080) — the correct approach — or add back `NET_BIND_SERVICE` only.

Gotchas — traps that catch experienced engineers

- **`privileged: true` bypasses ALL namespace isolation**: a privileged container can mount the host filesystem, load kernel modules, and escape the namespace entirely. It is equivalent to root on the host. Never use `privileged: true` in application pods; even Falco's DaemonSet uses a minimal set of capabilities instead.

- **`runAsNonRoot: true` without a specific UID fails unexpectedly**: if the container image's `USER` directive sets UID 0 (root), the pod will fail with `container has runAsNonRoot and image has non-numeric user root`. Always pair `runAsNonRoot: true` with `runAsUser: <non-zero UID>` for deterministic behavior.

- **PSA `enforce` on `kube-system` breaks cluster components**: `kube-system` pods (kube-proxy, Cilium) require privileged capabilities. Applying `restricted` policy to `kube-system` will block the CNI agent pods and crash the cluster. Never apply PSA enforcement to `kube-system`, `kube-public`, or any platform namespace without testing first.

Architect Considerations

1. **PSA vs Kyverno for pod security**: PSA enforces three fixed policy levels (privileged, baseline, restricted) — no customization. Kyverno (Chapter 22) can express the same policies with custom carve-outs. For an enterprise platform, Kyverno gives the flexibility to say "allow this one specific privileged workload with an explicit exception" while PSA's `warn` mode acts as a safety net.

2. **Seccomp profile selection**: the default Docker seccomp profile blocks ~300 dangerous syscalls. The Kubernetes `RuntimeDefault` seccomp profile is equivalent. For payments/security-critical pods, a custom seccomp profile that allows ONLY the syscalls the binary actually uses (generated with `strace` or `seccompgen`) provides tighter isolation.

3. **Image UID/GID governance**: standardize on a non-root UID range for all team images (e.g., `1000-1999`). Add a Kyverno policy that rejects images claiming UID 0 at admission time. This prevents the "just run as root for local dev" habit from reaching production.

4. **Privileged DaemonSets namespace segregation**: Falco, Cilium, and node-exporter require elevated privileges. Run them in a dedicated `security` or `monitoring` namespace with explicit PSA `privileged` label. Never co-locate privileged DaemonSets with application workloads in the same namespace.

5. **Security context inheritance testing**: define a security context regression test suite that runs against every new container image: verify `runAsNonRoot`, `readOnlyRootFilesystem`, no `CAP_SYS_ADMIN`. Add this to your CI pipeline as a policy gate before images reach the registry.

Chapter 20 checklist

- App namespaces enforce **PSA restricted**; `security` ns privileged, `data` ns baseline.
- Rolled out via **warn/audit → enforce**, not blind.
- Every app pod: **non-root, readOnlyRootFilesystem, drop ALL caps, no priv-escalation, seccomp**.
- Writable paths provided via `emptyDir`, not a writable rootfs.
- **No** `privileged: true` outside dedicated node-agent namespaces.

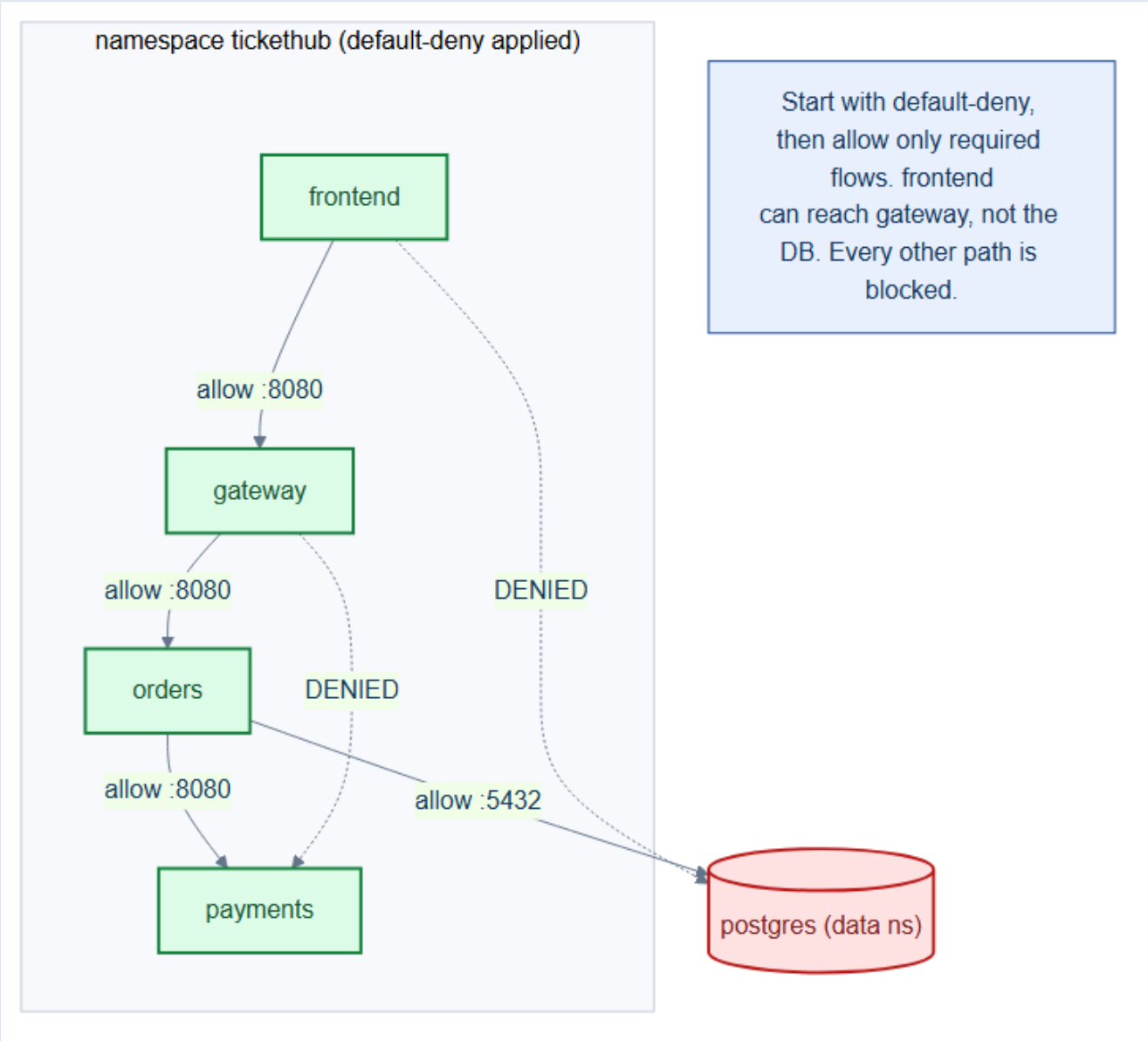
---

21. Network Policies — Zero-Trust Segmentation with Cilium

By default, **every pod in a Kubernetes cluster can talk to every other pod** — a flat, open network. For a platform handling payments that's unacceptable. **NetworkPolicies** turn the network zero-trust: deny everything, then allow only the flows TicketHub actually needs. Cilium (Chapter 6) enforces them in eBPF and extends them to L7.

21.1 The default-deny foundation

The first policy in every namespace denies all ingress and egress. From there you whitelist specific flows.



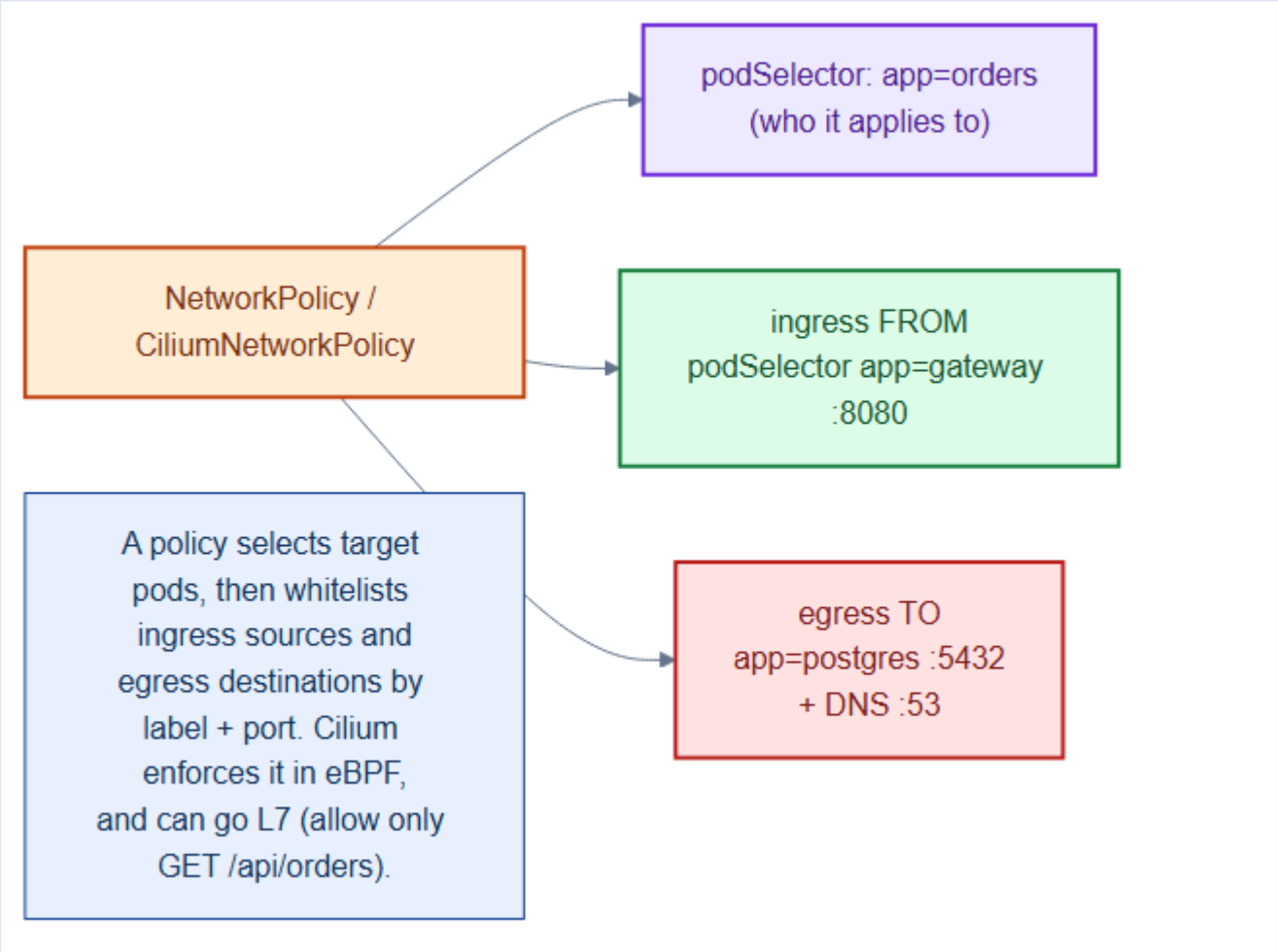

```
# repo/manifests/60-security/network-policies.yaml (default-deny section)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny-all
```

```
 namespace: tickethub
spec:
 podSelector: {}                  # selects ALL pods in the namespace
 policyTypes: [Ingress, Egress]   # deny both directions by default
```

Mental model — office doors that default to locked

Out of the box the cluster is an **open-plan office**: anyone walks into any room. Default-deny **locks every door**. Then you issue **specific keycards**: "frontend may enter gateway on port 8080", "orders may enter postgres on 5432". No keycard, no entry. An attacker who compromises `frontend` still can't reach the database.

21.2 Anatomy of an allow policy



A policy **selects target pods**, then whitelists **ingress sources** and **egress destinations** by label and port:

```
# repo/manifests/60-security/network-policies.yaml (orders-allow section)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: orders-allow
```

```
 namespace: tickethub
spec:
 podSelector:
   matchLabels: { app: orders }
 policyTypes: [Ingress, Egress]
 ingress:
   - from:
       - podSelector: { matchLabels: { app: gateway } }   # only gateway may call orders
     ports:
       - { protocol: TCP, port: 8080 }
 egress:
   - to:                                                   # orders may reach payments
       - podSelector: { matchLabels: { app: payments } }
     ports: [{ protocol: TCP, port: 8080 }]
   - to:                                                   # and Postgres in data ns
       - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: data } }
         podSelector: { matchLabels: { app: postgres } }
     ports: [{ protocol: TCP, port: 5432 }]
   - to: []                                                # and DNS
     ports:
       - { protocol: UDP, port: 53 }
```

Forgetting egress DNS breaks everything subtly

Once you apply default-deny **egress**, pods can't resolve DNS — every service call fails with a lookup error, not a connection error, which is maddening to debug. Always include an egress allow for **UDP/TCP 53 to kube-dns**. This is the #1 NetworkPolicy gotcha.

21.3 Cilium's L7 superpower

Standard NetworkPolicy stops at L3/L4 (IP + port). **CiliumNetworkPolicy** goes to **L7** — allow only specific HTTP methods/paths, Kafka topics, or DNS names:

```
# See: repo/manifests/ for the full manifest
apiVersion: cilium.io/v2
kind: CiliumNetworkPolicy
metadata: { name: orders-l7, namespace: tickethub }
spec:
 endpointSelector: { matchLabels: { app: orders } }
 ingress:
   - fromEndpoints:
       - matchLabels: { app: gateway }
     toPorts:
       - ports: [{ port: "8080", protocol: TCP }]
         rules:
           http:                              # gateway may only GET/POST these paths
             - { method: "GET",  path: "/api/orders.*" }
             - { method: "POST", path: "/api/orders" }
```

Segment by namespace, then by service

Coarse-grained first: block cross-namespace traffic except where required (apps → data on DB ports only). Then fine-grained within a namespace: service-to-service on exact ports. Cilium's L7 rules add

a third layer — even an allowed caller can only hit the specific API it's supposed to. This is defense in depth for the network.

21.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **NetworkPolicy is additive — there is no `deny` rule, only the absence of an `allow`**: the default-deny policy (`podSelector: {}`, empty ingress and egress) drops everything. Each subsequent policy adds specific allows. Two policies that both select the same pod have their ingress/egress rules UNION-ed — you cannot use a later policy to undo an earlier allow.

- **Cilium's `CiliumNetworkPolicy` supports L7 (HTTP/gRPC) rules, while the standard `NetworkPolicy` is L3/L4 only**: if you need to allow `POST /orders` but deny `DELETE /orders` from the same source, use `CiliumNetworkPolicy`. Standard `NetworkPolicy` cannot express HTTP method or path rules.

- **`namespaceSelector` matches on namespace LABELS, not names**: `matchLabels: { kubernetes.io/metadata.name: tickethub }` works because Kubernetes auto-adds this label to namespaces (from v1.21+). For older clusters, you must add the label manually to the namespace, or the selector silently matches nothing.

Gotchas — traps that catch experienced engineers

- **Forgetting egress DNS (`port 53, kube-dns`)**: a default-deny egress policy that doesn't allow port 53 to the `kube-system` namespace breaks DNS resolution for the pod — causing connection failures that look like network policy blocks but are actually DNS failures. Always add a DNS egress allow to every namespace default-deny policy.

- **Applying policy to DaemonSets before adding their egress rules**: if you apply default-deny to the `monitoring` namespace before adding egress rules for `node-exporter → Prometheus scrape`, the node-exporter pods become unreachable. Test policy in `warn` mode with Hubble flow observability before enforcing.

- **Network Policy not supported by all CNIs**: Flannel + Calico combination, Weave, and some cloud CNIs have incomplete NetworkPolicy support. If you switch CNI, re-test all NetworkPolicy semantics. Cilium is fully compliant with the spec AND extends it — one of its key advantages.

Architect Considerations

1. **Policy generation strategy**: writing NetworkPolicy by hand is error-prone. Use Hubble flow observability (Chapter 6) to observe actual traffic flows, then export them as policy drafts with `hubble observe --output policy`. Review and trim before applying — the generated policy is a starting

point, not a final answer.

2. **Namespace isolation boundary**: should the `data` namespace (Postgres, Kafka) be completely isolated from all namespaces except `tickethub`? Or should monitoring (Prometheus scrape) from `monitoring` ns also be allowed? Define the per-namespace trust model as a policy matrix before implementation.

3. **Microservice-to-microservice policy granularity**: a single `allow tickethub → data port 5432` policy is simpler but allows any tickethub pod to reach Postgres. A tighter `allow orders-app → postgres port 5432` policy (using pod label selectors) limits blast radius if a less-privileged service is compromised.

4. **Policy testing in CI**: add a `NetworkPolicy conformance test` to CI that deploys test pods and verifies that allowed connections succeed and denied connections are blocked. Tools like `cyclonus` or `netassert` automate this. Without automated tests, policy regressions are invisible.

5. **Cilium FQDN policies for external egress**: Cilium supports `toFQDNs: [{matchName: "api.stripe.com"}]` to allow egress to specific external domains by DNS name — far more robust than IP-range-based egress rules (which break when Stripe rotates IPs). Use FQDN policies for all external API calls.

Chapter 21 checklist

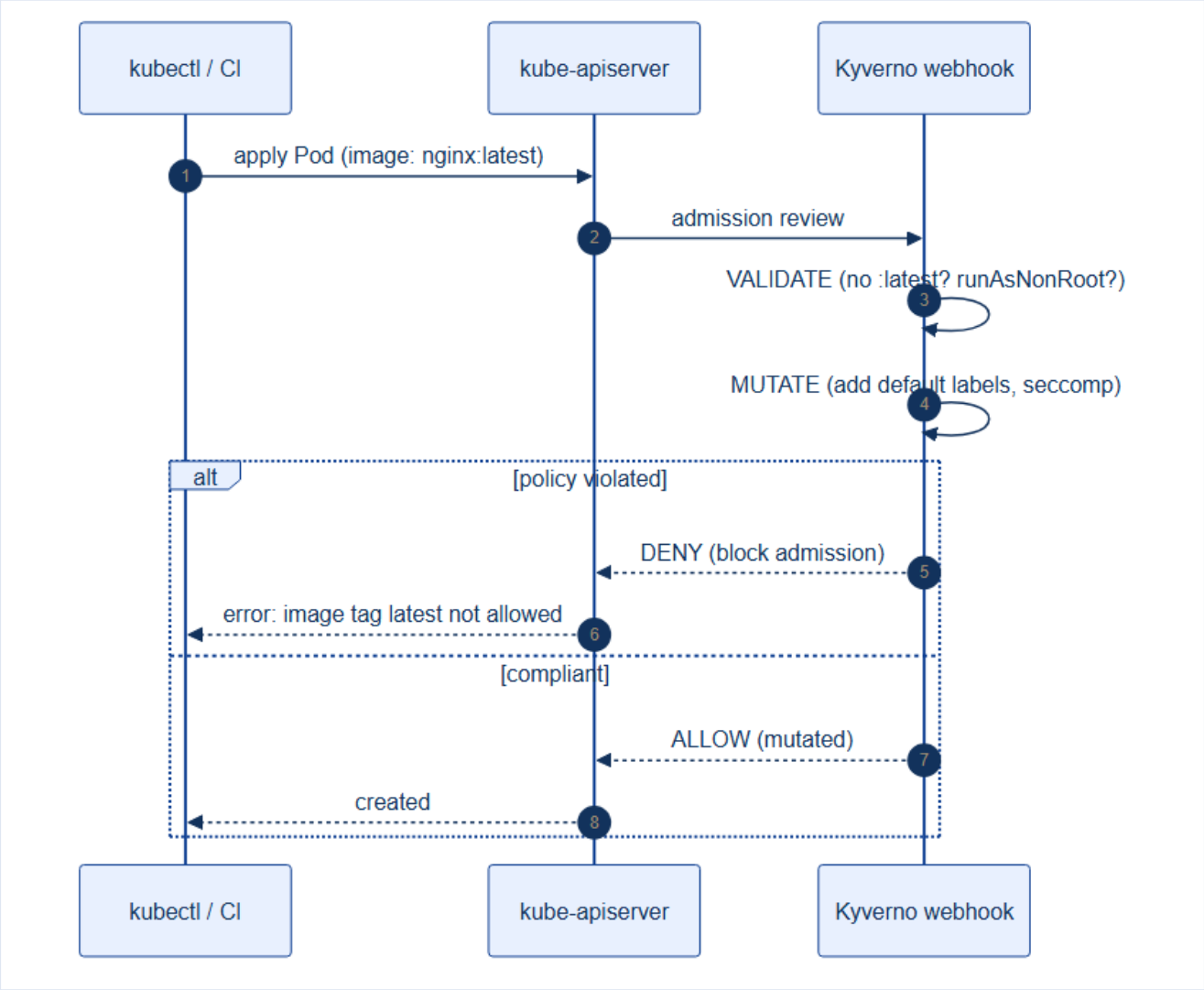
- **default-deny** ingress+egress in every app and data namespace.
- Allow policies whitelist **only required** service-to-service flows by label + port.
- **Egress DNS (53)** explicitly allowed everywhere.
- Cross-namespace traffic (apps→data) restricted to DB ports only.
- Sensitive paths tightened with **CiliumNetworkPolicy L7** rules.
- Flows verified in **Hubble** (Chapter 6): `hubble observe --verdict DROPPED`.

---

22. Policy as Code — Kyverno (Validate, Mutate, Generate)

PSA and RBAC are powerful but coarse. Real organizations have dozens of finer rules: *no `:latest` tags, every pod must have resource limits, images only from our registry, every namespace gets a default-deny NetworkPolicy*. Encoding these by hand is unenforceable. **Kyverno** makes them **policy as code** — plain YAML policies enforced at admission, cluster-wide, automatically.

22.1 Kyverno sits in the admission path

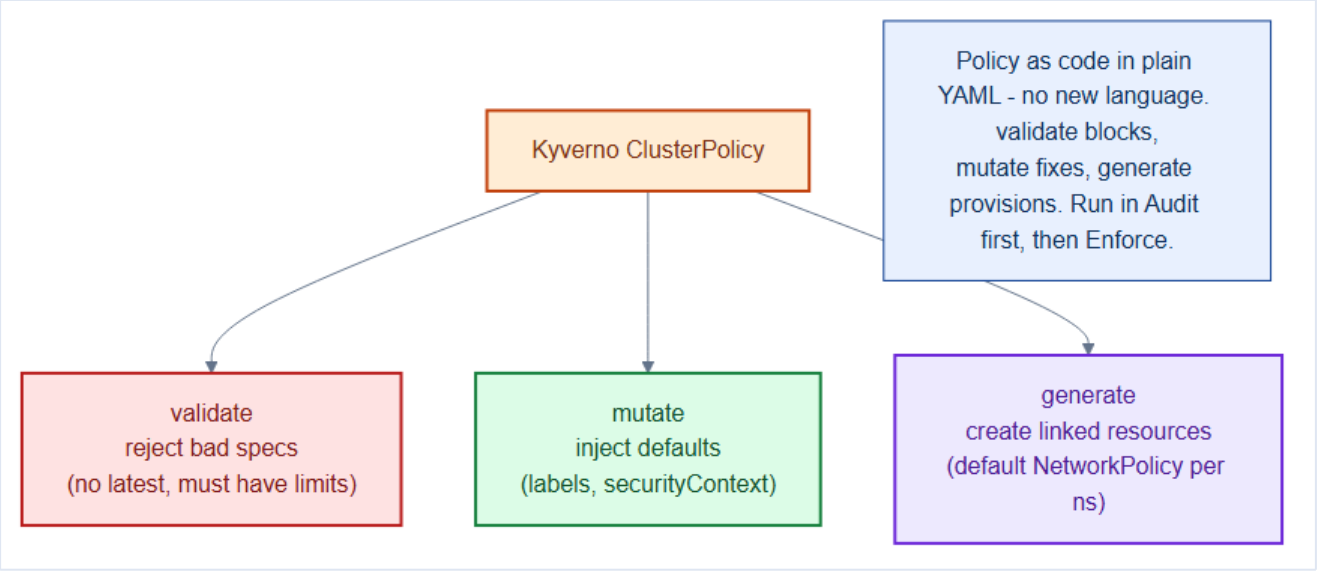


Kyverno is an **admission webhook**. Every create/update passes through it before persistence, where it can **validate** (allow/deny), **mutate** (modify), or **generate** (create companion resources).

> **Mental model — an automated plan-checker at city hall**

> Before any building permit (manifest) is approved, it goes to an **automated plan-checker**. It **rejects** plans that violate code (validate), **auto-corrects** small omissions like adding a required label (mutate), and **files companion paperwork** for you such as the fire-safety plan (generate). Nothing gets built without passing — consistently, without a human in the loop.

22.2 The three policy actions



Action	Does	TicketHub example
validate	Allow/deny based on rules	Reject `:latest`; require resource limits
mutate	Modify the resource	Inject `seccompProfile`, default labels
generate	Create linked resources	Auto-create default-deny NetworkPolicy per new namespace

22.3 Validate — block bad specs

```
# repo/manifests/60-security/kyverno-policies.yaml (disallow-latest section)
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
 name: disallow-latest-tag
spec:
 validationFailureAction: Enforce      # Audit first, then Enforce
 rules:
   - name: require-image-tag
     match:
       any: [{ resources: { kinds: [Pod] } }]
     validate:
       message: "Images must use an immutable tag or digest, not latest."
       pattern:
         spec:
           containers:
             - image: "!*:latest"
```

22.4 Mutate — inject secure defaults

```
# repo/manifests/60-security/kyverno-policies.yaml (default-securitycontext section)
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: add-default-securitycontext }
spec:
 rules:
   - name: default-seccomp
     match:
       any: [{ resources: { kinds: [Pod], namespaces: [tickethub] } }]
     mutate:
       patchStrategicMerge:
         spec:
           securityContext:
             seccompProfile: { type: RuntimeDefault }
```

22.5 Generate — provision companion resources

```
# See: repo/manifests/ for the full manifest
# every new namespace automatically gets a default-deny NetworkPolicy
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: add-default-deny }
spec:
 rules:
   - name: default-deny-netpol
     match:
       any: [{ resources: { kinds: [Namespace] } }]
     generate:
       apiVersion: networking.k8s.io/v1
       kind: NetworkPolicy
       name: default-deny-all
       namespace: "{{request.object.metadata.name}}"
       data:
         spec:
           podSelector: {}
           policyTypes: [Ingress, Egress]
```

Always start in Audit, then switch to Enforce

Set `validationFailureAction: Audit` first. Kyverno reports every violation in **PolicyReports** without blocking anything, so you see the blast radius across existing workloads. Fix them, then flip to `Enforce`. Rolling out `Enforce` blind can reject legitimate deploys and page you at 2am.

Kyverno vs. OPA/Gatekeeper

Both are admission-policy engines. **Kyverno** uses plain **YAML** (no new language) and uniquely supports **generate** and image verification natively — a gentle on-ramp for Kubernetes teams. **OPA/Gatekeeper** uses the **Rego** language, more powerful for complex logic but a steeper learning curve. TicketHub standardizes on Kyverno for its YAML ergonomics.

22.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Kyverno mutation policies run BEFORE validation policies** in the admission chain. This means a mutate policy that injects a default `securityContext` runs first, then a validate policy can check that `securityContext` is present — allowing you to enforce invariants while also providing defaults for teams that haven't set them.

- **`background: true` (default)** means Kyverno evaluates policies against existing resources periodically, not just at admission time. A new validate policy with `validationFailureAction: enforce` will log violations on pre-existing resources but will NOT delete or modify them — only new or updated objects are blocked.

- **ClusterPolicy vs Policy scope**: `ClusterPolicy` is cluster-wide; `Policy` is namespace-scoped. Use `ClusterPolicy` for platform-wide invariants (no `latest` tag, no root containers) and namespace-scoped `Policy` for team-specific rules (allowed image registries for namespace X).

Gotchas — traps that catch experienced engineers

- **`validationFailureAction: enforce` on a policy that breaks a system component**: if you apply a `ClusterPolicy` that blocks pods without a required label and the Cilium DaemonSet pods don't have that label, Cilium pods cannot be recreated after a crash — taking down all node networking. Always test policies in `audit` mode first and explicitly exclude system namespaces with `exclude.resources.namespaces`.

- **Kyverno webhook timeout**: Kyverno injects an admission webhook with a default timeout of 10 seconds. If the Kyverno pod is unavailable or slow, ALL pod admissions time out — blocking ALL deployments cluster-wide. Set `failurePolicy: Ignore` on non-critical policies and ensure Kyverno runs with PDB `minAvailable: 1` or is in HA mode.

- **Generate policies and ownership**: when Kyverno generates a NetworkPolicy in a new namespace, it becomes the owner. Manually editing that NetworkPolicy will cause Kyverno to re-sync it back to the generated version. Either don't generate resources you intend to customize, or use `synchronize: false` to generate-once-and-abandon.

Architect Considerations

1. **Policy as code in git**: Kyverno ClusterPolicies should live in the `repo/manifests/60-security/` directory and be deployed via Argo CD (Chapter 28). This makes policy changes auditable (git history), reviewable (PR process), and automatically enforced across environments.

2. **Kyverno vs OPA/Gatekeeper**: Kyverno uses native Kubernetes YAML for policies (lower learning curve); OPA Gatekeeper uses Rego (more expressive for complex rules). For a platform

team that primarily maintains Kubernetes manifests, Kyverno's YAML-native approach reduces the cognitive overhead. For complex multi-system policy (spanning cloud APIs, CI/CD, and Kubernetes), OPA is more consistent.

3. **Exception management**: Kyverno supports `PolicyException` objects (v1.9+) that grant named workloads exemptions from specific policies. This is better than disabling the policy for everyone — use PolicyExceptions for the `cilium-system` pods that legitimately need `privileged: true`.

4. **Image signature verification at scale**: Kyverno's `verifyImages` policy (with cosign) verifies every image pull against a public key. The verification adds ~100ms to pod scheduling. For clusters with hundreds of pod starts per second, verify the signing verification overhead is acceptable — cache the results in the Kyverno OCI cache.

5. **Policy drift detection**: Kyverno's background scan generates `PolicyReport` and `ClusterPolicyReport` objects with violation counts. Expose these to Grafana via the policy-reporter sidecar. A dashboard showing violation counts per namespace and per policy gives the platform team real-time visibility into compliance posture.

Chapter 22 checklist

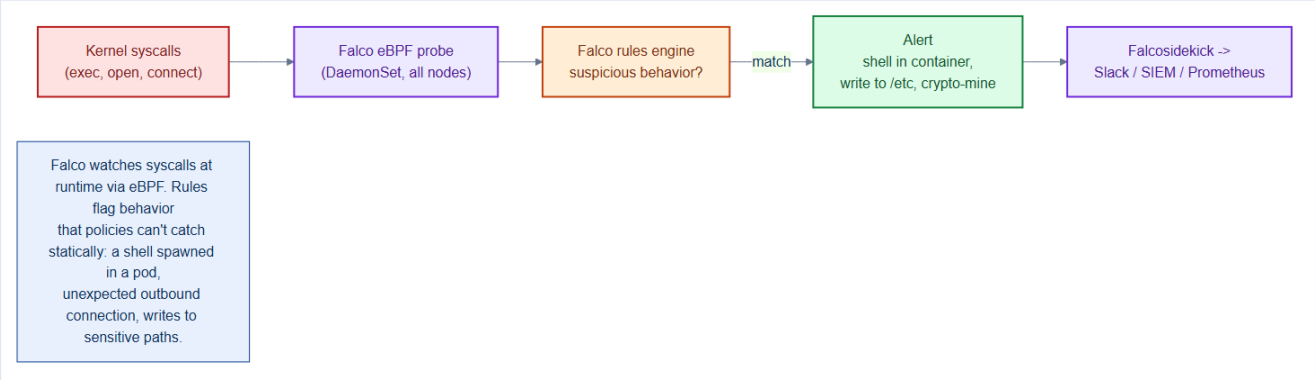
- Kyverno installed; core policies as code in Git (validate/mutate/generate).
- **Validate**: no `:latest`, images only from the internal registry, limits required.
- **Mutate**: secure defaults (seccomp, labels) injected automatically.
- **Generate**: default-deny NetworkPolicy created for every new namespace.
- Every policy rolled out **Audit** → **Enforce**; violations tracked in PolicyReports.

---

23. Runtime Threat Detection — Falco

RBAC, PSA, NetworkPolicy, and Kyverno are **preventive** — they stop bad things at admission or in the network. But some threats only appear **at runtime**: a compromised dependency spawns a shell, a container starts crypto-mining, an attacker reads `/etc/shadow`. **Falco** is the runtime security camera — it watches kernel activity live and alerts on suspicious behavior policies can't catch statically.

23.1 How Falco sees everything



Falco runs as a **DaemonSet** (one pod per node, Chapter 11) with an **eBPF probe** in the kernel. It observes every **syscall** — process execution, file opens, network connections — and runs them against a **rules engine**. Matches become alerts routed to Slack, a SIEM, or Prometheus.

> **Mental model — a security camera vs. door locks**
>
> All the earlier chapters were **locks** — they stop the wrong people entering. Falco is the **CCTV + motion sensor** *inside* the building. Even if someone slips through a lock (a zero-day, a supply-chain compromise), Falco sees them do something they shouldn't — open the vault, climb through a vent — and raises the alarm in real time.

23.2 What Falco detects that nothing else can

Behavior	Why static policy misses it
Shell spawned in a container	The image was fine; the *runtime* behavior is the attack
Write to `/etc`, `/bin`, or SSH keys	Happens after admission
Unexpected outbound connection	Data exfiltration / C2
Reading sensitive files (`/etc/shadow`)	Legit-looking process, illegit action
Container escape attempts	Kernel-level, invisible to the API server

23.3 A Falco rule

Falco ships with a strong default ruleset; you extend it for TicketHub specifics:

```
# repo/manifests/60-security/falco-rules.yaml (excerpt)
- rule: Shell spawned in TicketHub container
 desc: A shell was executed inside an application pod - likely compromise
 condition: >
   spawned_process and container
   and proc.name in (bash, sh, zsh, ash)
   and k8s.ns.name = "tickethub"
 output: >
   Shell in container (pod=%k8s.pod.name ns=%k8s.ns.name
   cmd=%proc.cmdline user=%user.name)
 priority: WARNING
 tags: [container, shell, mitre_execution]
```

```
# Falco installed via Helm as a DaemonSet in the 'security' namespace (PSA privileged)
# with Falcosidekick forwarding alerts to Slack + Prometheus Alertmanager.
```

Falco needs privileged access — isolate it

To read kernel syscalls Falco runs **privileged**, which is exactly what Chapter 20 forbids for app pods. That's why it lives in a dedicated `security` namespace labeled PSA `privileged`, and why a Kyverno policy exempts only *that* namespace. The security tooling gets the access it needs, fenced off from everything else.

23.4 From alert to response

Detection is only useful if it drives action. Route Falco → **Falcosidekick** → your incident pipeline:

- **Slack/PagerDuty** — human alert for high-priority rules.
- **Prometheus/Alertmanager** — metrics + dashboards (Chapter 26).
- **SIEM** — long-term forensic storage.
- **Automated response** — e.g., a controller that cordons the node or kills the pod on a critical match.

Tune out the noise or the signal dies

Falco's defaults are chatty — a wall of alerts trains people to ignore all of them. Baseline your workloads, silence expected behavior (a legit init container that writes config), and reserve high-priority notifications for genuine threats. An untuned Falco is worse than none, because real alerts drown.

23.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Falco rules are evaluated for every syscall event** on the node — there is a performance cost. Default Falco installs see 1-5% CPU overhead per node. Custom rules that add expensive string comparisons or regular expressions can push this higher. Profile rule performance with `falco --list-syscalls` and Falco's built-in stats output.

- `spawned_process` **events fire for EVERY new process** — including shell commands run legitimately by init systems, healthchecks, and entrypoint scripts. A "shell spawned in container" rule without a trusted-container allowlist will generate enormous noise at cluster scale, causing alert fatigue. Tune allowlists carefully before enabling.

- **Falco kernel module vs eBPF driver**: the kernel module gives full syscall coverage but requires `--privileged` and is blocked by secureboot. The eBPF driver is more portable and works with secureboot. Cilium also uses eBPF — ensure the eBPF programs don't conflict by checking kernel eBPF map limits (`ulimit -l`).

Gotchas — traps that catch experienced engineers

- **Falco rules are NOT NetworkPolicy**: Falco detects and alerts on suspicious activity; it does not block it. A Falco rule for "unexpected outbound connection" fires an alert but the connection proceeds. Combine Falco alerts with automated responses (Kubernetes admission webhook, Falco Sidekick → Kubernetes API to label/quarantine the pod) for actual blocking.

- **Custom rule precedence**: Falco evaluates rules in file order. A custom rule file that overrides a default rule must use `override: { condition: replace }` explicitly. Silently adding a rule with the same name results in both rules firing, doubling the alert volume.

- **Alert sink reliability**: Falco emits alerts to stdout by default. In a containerized deployment, this means alerts flow through the container log pipeline (Promtail → Loki). If Loki is down, alerts are lost. Always configure a Falco Sidekick integration with a durable sink (PagerDuty, Slack webhook, dedicated S3 bucket) for security-critical alerts.

Architect Considerations

1. **Falco as the last line of defense**: Falco is a detective control — it observes and alerts but doesn't prevent. The order of security layers is: supply chain (Chapter 24) → image policy (Kyverno) → network policy (Chapter 21) → pod security (Chapter 20) → runtime detection (Falco). Each layer reduces the blast radius; Falco is what catches what the others miss.

2. **Rule tuning vs alert fatigue trade-off**: too few rules = real attacks missed; too many rules = analyst fatigue and ignored alerts. Define a triage process: every new Falco alert type must have a runbook before the rule is enabled in production. Alerts without runbooks get disabled.

3. **Incident response integration**: Falco Sidekick can trigger an Argo Workflow or a Kubernetes operator that automatically: labels the offending pod `status=quarantined`, applies a NetworkPolicy blocking all egress, and creates a PVC snapshot for forensics. Design this response automation before an incident occurs.

4. **eBPF-based detection completeness**: Falco with eBPF driver captures syscall-level events. It cannot observe encrypted traffic payloads (TLS) or in-memory operations that don't make syscalls. For full observability of a compromised process, supplement Falco with memory forensics (LiME) or eBPF-based tracing (Tetragon by Cilium).

5. **Compliance mapping**: Falco rules can be mapped to CIS Kubernetes Benchmark controls, NIST 800-53, or PCI-DSS requirements. Document which Falco rules satisfy which compliance controls — this makes audit preparation significantly faster.

Chapter 23 checklist

- Falco runs as a **DaemonSet** (eBPF) on every node, in the isolated `security` ns.
- Default ruleset enabled, **extended** with TicketHub-specific rules.
- Alerts routed via **Falcosidekick** to Slack/SIEM/Prometheus.
- Rules **tuned** to suppress known-good behavior — low false-positive rate.
- A response path exists (page, and ideally auto-isolate on critical matches).

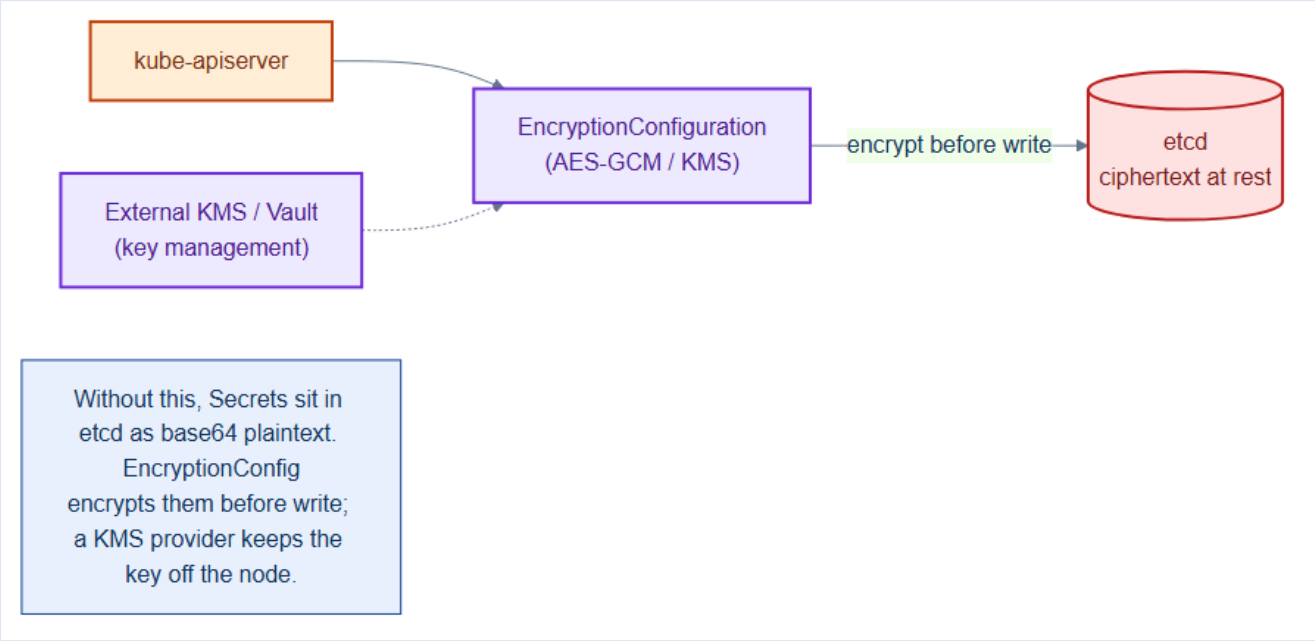
---

24. Secrets at Rest, Image Signing & Supply-Chain Security

The final preventive layer protects two things attackers love: the **secrets stored in etcd** and the **images you run**. A secret is only as safe as the disk under etcd; an image is only as trustworthy as its provenance. This chapter encrypts the former and cryptographically verifies the latter.

24.1 Encrypting Secrets at rest

By default, Kubernetes Secrets are stored in etcd as **base64 — not encrypted** (Chapter 13). Anyone with etcd disk access or a backup reads them plainly. An **EncryptionConfiguration** encrypts them before they hit disk.




```
# See: repo/manifests/ for the full manifest
# /etc/kubernetes/enc/encryption-config.yaml (referenced by kube-apiserver flag)
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
 - resources: ["secrets"]
   providers:
     - kms:                          # best: key lives in an external KMS/Vault
         name: vault-kms
         endpoint: unix:///var/run/kmsplugin/socket.sock
     - aescbc:                       # fallback local key
         keys:
           - name: key1
             secret: <base64-32-byte-key>
     - identity: {}                  # read old plaintext during migration
```

```
# apiserver flag:  --encryption-provider-config=/etc/kubernetes/enc/encryption-config.yaml
# re-encrypt existing secrets after enabling:
kubectl get secrets -A -o json | kubectl replace -f -
```

KMS provider beats a local key

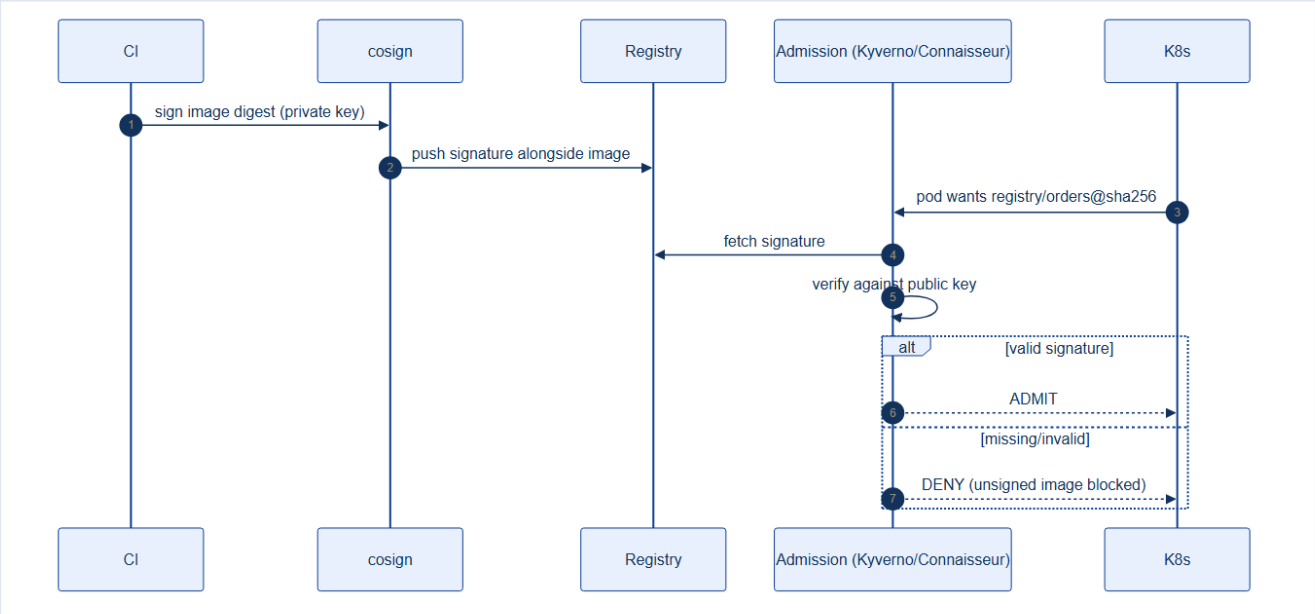
A local `aescbc` key sits on the control-plane node — steal the node, steal the key. A **KMS provider** (Vault, cloud KMS) keeps the **key-encryption-key off the cluster**; the apiserver calls out to decrypt, so an etcd backup alone is useless to an attacker. For a payments platform, KMS is the standard.

Mental model — a safe vs. a locked safe with the key elsewhere

Base64 secrets in etcd are documents in an **unlocked drawer**. `aescbc` locks the drawer but tapes the **key to the bottom**. **KMS** locks the drawer and keeps the key in a **different building** (the KMS) that logs every access. Only the last one protects you when someone walks off with the drawer (an etcd backup).

24.2 Image signing — trust what you run

Chapter 10 built and scanned images. But how does the cluster know an image is **really yours** and unmodified? **Sign** it with cosign, and **verify** the signature at admission — refuse anything unsigned.




```
# repo/manifests/60-security/kyverno-policies.yaml (verify-images section)
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: { name: verify-image-signatures }
spec:
 validationFailureAction: Enforce
 rules:
   - name: verify-tickethub-images
     match:
       any: [{ resources: { kinds: [Pod] } }]
     verifyImages:
       - imageReferences: ["registry.internal/tickethub/*"]
         attestors:
           - entries:
               - keys:
```

```
                  publicKeys: |-
                    -----BEGIN PUBLIC KEY-----
                    MFkwEwYHKoZ...          # cosign public key
                    -----END PUBLIC KEY-----
```

Now an unsigned or tampered image is **denied at admission** — the supply chain is closed loop: build → scan → sign (Ch 10) → **verify** (here).

24.3 The full supply-chain picture

Stage	Control	Chapter
Build	Multi-stage, distroless, non-root	10
Scan	Trivy blocks HIGH/CRITICAL CVEs	10
Sign	cosign signs the digest	10
Provenance	SBOM + SLSA attestation	24
Verify	Kyverno denies unsigned images	24
Run	PSA + SecurityContext	20
Watch	Falco runtime detection	23

Pin by digest, not tag, to make signatures meaningful

A signature covers an **image digest** (`@sha256:...`). If you deploy by mutable tag, the tag can be repointed to a different (unsigned) image after verification. Deploy by **digest** (Chapter 10) so what you verified is exactly what runs. Tag + signature without digest pinning is a false sense of security.

Generate an SBOM and attestation too

Beyond signing, produce a **Software Bill of Materials** (`syft`) and a **SLSA provenance attestation** describing *how* the image was built. Store them alongside the signature so that when the next Log4Shell drops, you can query which images contain the vulnerable component in seconds instead of days.

24.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **etcd encryption at rest encrypts the value stored in etcd, not the transmission**: the kube-apiserver decrypts secrets when it reads them from etcd and re-encrypts when writing. Secret values flowing over the API server's TLS connection are in plaintext inside the API server's memory — a compromised API server process can still read all secrets.

- **Cosign image signing verifies the image manifest digest**, not the image content. Two different images CAN have the same signed digest (preimage resistance of SHA-256 makes this computationally infeasible, but the trust is in the digest, not re-downloading and comparing bytes). The signature chain of trust is: registry push → Sigstore Rekor log entry → `cosign verify` at admission.

- `imagePullPolicy: IfNotPresent` (the default for tagged images) means a pod restart on the same node re-uses the cached image WITHOUT re-verifying the Kyverno image signature policy. A signed image that is later found malicious will keep running on nodes that have the cached layer until the cache is cleared. Combine signature verification with periodic digest pin updates in your manifests.

Gotchas — traps that catch experienced engineers

- **KMS encryption key rotation**: when you rotate the KMS key used for etcd encryption, all existing Secrets must be rewritten to etcd with the new key (`kubectl get secrets --all-namespaces -o json | kubectl replace -f -`). Missing this step means some Secrets are still encrypted with the old key — and if the old key is revoked, those Secrets become permanently unreadable.

- **Cosign keyless signing with Sigstore Fulcio** uses a short-lived OIDC certificate — the signing identity expires. If you rebuild and re-sign an image months later, the old signature is valid but the signing certificate is expired. `cosign verify` still passes (the transparency log records the original valid-time signature), but forensic tooling must handle this correctly.

- **Trivy/Grype CVE database lag**: vulnerability scanners use a CVE database that may be hours to days behind the NVD feed. A scanner that shows "no CRITICAL CVEs" does not mean the image is safe — it means no KNOWN CRITICAL CVEs as of the last database update. Schedule daily scans, not just build-time scans.

Architect Considerations

1. **Defense in depth for secrets**: encryption at rest (etcd KMS) + encryption in transit (TLS) + runtime access control (RBAC) + secret lifecycle management (Vault rotation) + audit logging (who accessed what secret when) are five independent layers. Implement all five — losing any one layer doesn't compromise the whole.

2. **SBOM as a supply chain artefact**: generating an SBOM (`syft`) at build time and attaching it to the image (via cosign SBOM attestation) creates a persistent record of what went into the image. This is invaluable during a supply-chain incident: instead of rebuilding to check if a vulnerable library

is present, query the SBOM.

3. **Registry admission control**: configure the container registry to only accept pushes from CI/CD pipelines (not from developer laptops). Every image in the registry was built by a reproducible, audited pipeline. This is a preventive control that complements Kyverno's image verification.

4. **Key management HSM integration**: the cosign private key should live in a Hardware Security Module (HSM) or cloud KMS, not on disk. A signing key on disk is as compromisable as any other file. Use Vault Transit Secrets Engine or AWS KMS as the cosign signing backend.

5. **Incident response for a compromised image**: if a signed production image is found to contain a backdoor, what is your response playbook? Key steps: revoke the signature (add to Sigstore revocation list), identify all running instances (`kubectl get pods -o json | jq 'select(.spec.containers[].image == "...")'`), trigger emergency rolling update with a clean image, preserve forensic evidence. Pre-write this playbook before you need it.

Chapter 24 checklist

- **EncryptionConfiguration** enabled for Secrets, ideally via a **KMS** provider.
- Existing Secrets **re-encrypted** after enabling.
- Images **signed** (cosign) and **verified at admission** (Kyverno), unsigned = denied.
- Deploys pinned by **digest** so signatures are meaningful.
- **SBOM + provenance** generated and stored for fast CVE response.

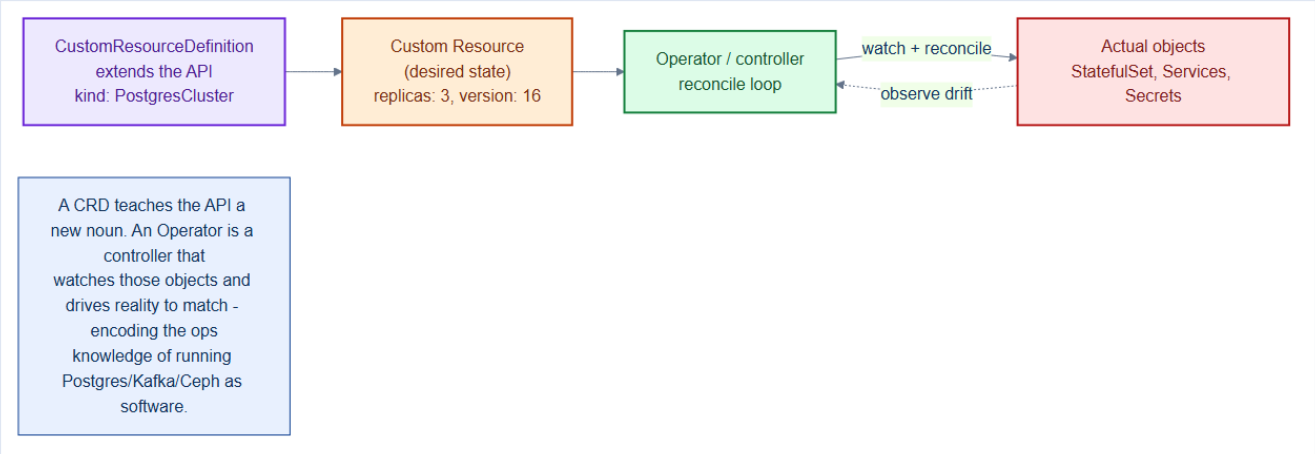
---

25. Extending Kubernetes — CRDs & Operators

Almost every platform component in this book — Cilium, Rook-Ceph, MetalLB, cert-manager, Kyverno, Prometheus — is installed by defining **custom resources** that Kubernetes doesn't ship with. That's not a coincidence: it's the **operator pattern**, the way Kubernetes is meant to be extended. This chapter closes Part V by explaining the machinery you've been using all along.

25.1 CustomResourceDefinitions — teaching the API new nouns

A **CRD** registers a new resource *kind* with the API server. After applying a CRD, `kubectl get postgresclusters` works as if it were built in — same RBAC, same `kubectl`, same declarative YAML.



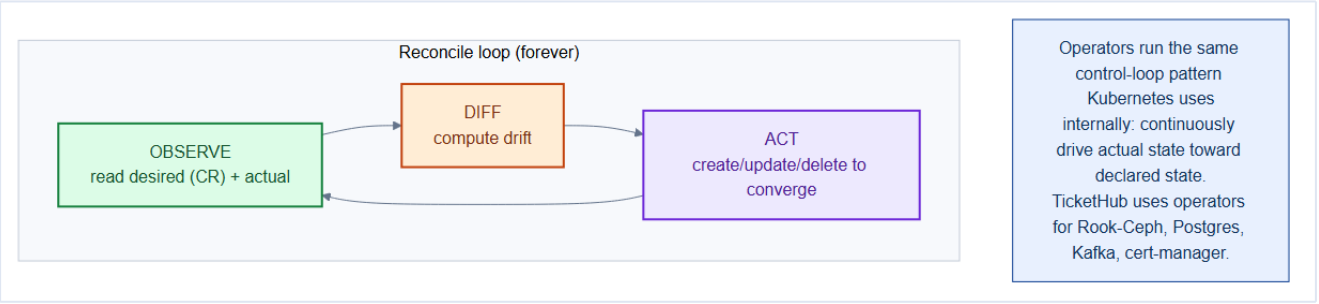
Native	Custom (via CRD)
`kind: Deployment`	`kind: CephCluster`
`kind: Service`	`kind: CiliumNetworkPolicy`
`kind: Secret`	`kind: ClusterPolicy` (Kyverno)

```
# a CRD (abbreviated) — this is what an operator ships
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata: { name: postgresclusters.tickethub.io }
spec:
 group: tickethub.io
 names: { kind: PostgresCluster, plural: postgresclusters }
 scope: Namespaced
 versions:
   - name: v1
     served: true
     storage: true
     schema:
       openAPIV3Schema:
         type: object
         properties:
```

```
          spec:
            type: object
            properties:
              replicas: { type: integer }
              version:  { type: string }
```

25.2 The operator — a controller with domain knowledge

A CRD by itself is inert — just a new noun in the database. The **operator** is the **controller** that gives it behavior: it watches those custom resources and drives the real world to match, using the same **reconcile loop** Kubernetes uses internally.




```
loop forever:
   observe  = read desired state (the CR) + actual state (real objects)
   diff     = compute the difference
   act      = create / update / delete to converge
```

Mental model — hiring an expert DBA who never sleeps

Running Postgres HA by hand needs a **DBA**: provision disks, configure replication, fail over the primary, take backups. An **operator encodes that DBA's knowledge as software**. You declare `kind: PostgresCluster, replicas: 3`; the operator does everything a human DBA would — continuously, at 3am, without a ticket. The CRD is the *request form*; the operator is the *tireless expert* fulfilling it.

25.3 Why this matters for TicketHub

You've relied on operators throughout the build:

Custom resource	Operator	What it automates
`CephCluster`	Rook	Provisions/heals Ceph (Ch 8)
`CiliumNetworkPolicy`	Cilium	eBPF network enforcement (Ch 6, 21)
`IPAddressPool`	MetalLB	Bare-metal LoadBalancer IPs (Ch 7)

`Certificate`	cert-manager	Issues/renews TLS (Ch 7)
`ClusterPolicy`	Kyverno	Admission policy (Ch 22)
`ScaledObject`	KEDA	Event-driven scaling (Ch 16)

Prefer a mature operator over hand-rolled YAML for stateful systems

You *could* run Postgres with a raw StatefulSet (Chapter 14) — but failover, backups, and version upgrades are then your problem. A battle-tested operator (CloudNativePG, Strimzi for Kafka) encodes years of operational hard-won lessons. For complex stateful software, adopt the operator; reserve hand-written manifests for your own stateless apps.

25.4 When to write your own

Most teams **consume** operators; occasionally you **build** one — to encode *your* domain, e.g. a `TicketHubTenant` CRD that provisions a namespace, quota, network policies, and per-tenant databases in one declarative object. Frameworks like **Kubebuilder** and the **Operator SDK** scaffold the controller so you write only the reconcile logic.

An operator is a privileged controller — scope it tightly

Operators typically hold broad RBAC (they create Deployments, Secrets, PVCs). A compromised or buggy operator is high-blast-radius. Install operators from trusted sources, pin versions, review their RBAC, and give each the **narrowest** ClusterRole that lets it do its job (Chapter 19).

25.5 DB Replication as a CRD domain — a worked example

This section walks through the two CRDs that model Postgres HA replication for TicketHub (`repo/manifests/20-data/postgres-replication-crd.yaml`) as a concrete, fully annotated example of the operator pattern in practice.

25.5.1 The domain model

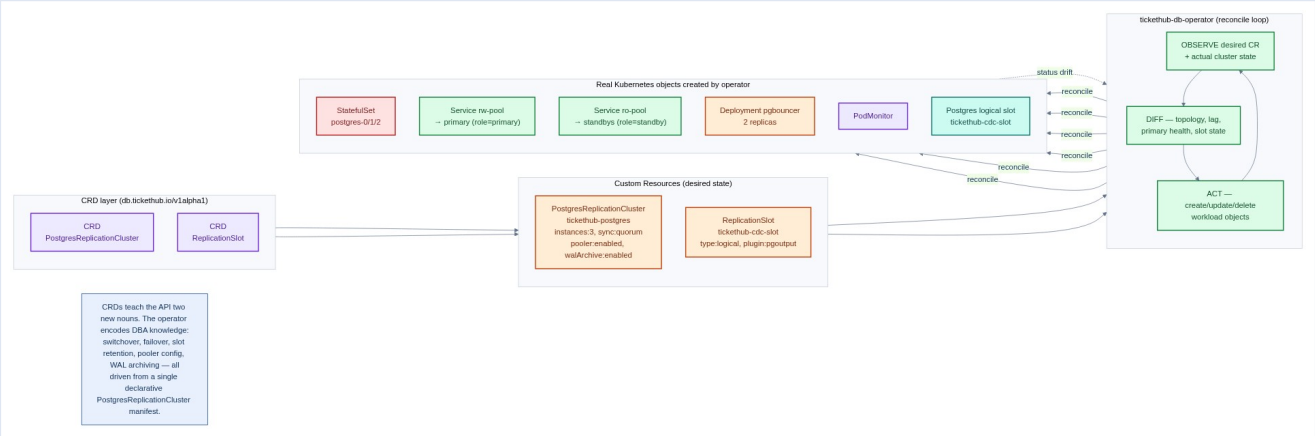
Running Postgres with streaming replication involves multiple moving parts:

Concern	Without CRDs	With CRDs + operator
Primary/standby topology	Init-containers + shell scripts	Declared as `spec.instances` + `replication.*`
Failover	Custom watchdog sidecar	`failover.automaticFailover: true`

Connection pooling	Hand-written PgBouncer manifests	`connectionPooler.enabled: true`
WAL archiving (PITR)	Custom cron + pgbackrest scripts	`walArchive.destination: s3://...`
Replication slot lifecycle	DBA monitors `pg_replication_slots`	`kind: ReplicationSlot` with `retentionPolicy`
Observability	Manually deploy postgres_exporter	`monitoring.enablePodMonitor: true`

The CRD approach doesn't hide complexity — it **relocates** it from your per-cluster scripts into a shared, versioned, tested operator.

25.5.2 CRD structure and operator reconcile flow



Two CRDs compose the domain:

`PostgresReplicationCluster` — the top-level resource declaring the desired cluster state. Key `spec` fields:

```
spec:
 instances: 3                       # 1 primary + 2 standbys
 replication:
   synchronousMode: quorum          # RPO = 0 for 1-standby loss
   walLevel: replica                # replica | logical
   hotStandby: true                 # read-only Service for standbys
 walArchive:
   enabled: true
   destination: s3://tickethub-wal/postgres
 failover:
   automaticFailover: true
   promotionTimeout: 60
 connectionPooler:
   enabled: true
   poolMode: transaction
 storage:
   size: 20Gi
   storageClass: rook-ceph-block
```

`ReplicationSlot` — a dependent resource for named logical or physical replication slots. Logical slots (for CDC pipelines like Debezium) require `walLevel: logical` on the parent cluster:

```
apiVersion: db.tickethub.io/v1alpha1
kind: ReplicationSlot
metadata:
 name: tickethub-cdc-slot
 namespace: data
spec:
 clusterRef: tickethub-postgres
 type: logical
 plugin: pgoutput
 retentionPolicy:
   maxWalSize: 2Gi
   inactiveTimeout: 24h
```

The `retentionPolicy` is critical: an abandoned logical slot with no consumer accumulates WAL indefinitely. The operator watches `pg_replication_slots` and drops slots that breach `maxWalSize` or `inactiveTimeout`, firing a Kubernetes `Warning` event so the platform team is notified.

25.5.3 What the operator reconciles

Each time the `tickethub-db-operator` sees a `PostgresReplicationCluster` event (create, update, status-drift) it drives the following objects:

Object created/owned	Purpose
`StatefulSet postgres`	Runs N Postgres instances with per-pod PVCs
`Service postgres-rw`	ClusterIP → primary pod (`role=primary`)
`Service postgres-ro`	ClusterIP → all standby pods (`role=standby`)
`Deployment pgbouncer`	PgBouncer connection pool (if `connectionPooler.enabled`)
`PodMonitor postgres`	Prometheus scrape config for `postgres_exporter` sidecar
`Secret pgbouncer-config`	PgBouncer `pgbouncer.ini` generated from CR spec
Physical `ReplicationSlot` CRs	Auto-created per standby by the operator

25.5.4 The operator RBAC

Because the operator manages StatefulSets, Services, Secrets, and its own CRDs, its ClusterRole is broad — but scoped as tightly as possible:

```
# repo/manifests/20-data/postgres-replication-crd.yaml (ClusterRole excerpt)
rules:
```

```
  - apiGroups: [db.tickethub.io]
    resources: [postgresreplicationclusters, replicationslots,
                postgresreplicationclusters/status, replicationslots/status]
    verbs: [get, list, watch, create, update, patch, delete]
  - apiGroups: [apps]
    resources: [statefulsets]
    verbs: [get, list, watch, create, update, patch, delete]
  - apiGroups: [""]
    resources: [pods, pods/exec, services, endpoints,
                persistentvolumeclaims, configmaps, secrets]
    verbs: [get, list, watch, create, update, patch, delete]
  - apiGroups: [monitoring.coreos.com]
    resources: [podmonitors]
    verbs: [get, list, watch, create, update, patch, delete]
```

`pods/exec` is needed for `pg_promote()` and `pg_rewind` — the operator shells into the standby to run the promotion. This is **high-privilege**; the operator ServiceAccount is restricted to the `data` namespace via RoleBinding if the `pods/exec` scope were namespace-only, but because StatefulSet management is cluster-scoped here a ClusterRoleBinding is used. In production, prefer a per-namespace operator deployment to limit the blast radius.

25.5.5 Observing the running cluster

```
# Short-form thanks to shortNames in the CRD
kubectl -n data get pgrc
# NAME                PRIMARY          INSTANCES  READY  PHASE    AGE
# tickethub-postgres   postgres-0       3          3      Healthy  4d

# Replication lag per standby
kubectl -n data get pgrc tickethub-postgres -o jsonpath='{.status.replicationLag}'
# {"postgres-1":"0/0 (0 bytes)","postgres-2":"128/0 (128 bytes)"}

# Inspect the logical CDC slot
kubectl -n data get rslot
# NAME                CLUSTER              TYPE     ACTIVE  WALLAG
# tickethub-cdc-slot   tickethub-postgres   logical  true    512 bytes
```

25.5.6 Failover sequence

When the primary disappears the operator fires the following reconcile actions (visible in `kubectl describe pgrc tickethub-postgres`):

1. **Detect** — health-check goroutine misses `promotionTimeout` heartbeats.
2. **Elect** — rank standbys by `pg_last_wal_receive_lsn`; least lag wins.
3. **Promote** — `exec postgres-1 -- pg_ctl promote`.
4. **Re-label** — `kubectl label pod postgres-1 role=primary`; Service endpoint shifts in < 1 s.
5. **Rewind** — when `postgres-0` recovers, run `pg_rewind` to re-sync it as a new standby.
6. **Status** — update `status.currentPrimary: postgres-1`, emit Event.

The entire sequence is typically complete within 30–90 seconds — far faster and more reliable than a human paged at 3 AM.

Chapter 25 checklist — Part V complete

- Understand CRDs (**new API nouns**) + operators (**controllers that reconcile them**).
- Stateful systems (DB, Kafka, Ceph) run via **mature operators**, not hand-rolled YAML.
- Operator RBAC reviewed and **scoped tightly**; versions pinned; sources trusted.
- Custom operators (if any) scaffolded with Kubebuilder/Operator SDK.
- **DB replication** is modelled as two CRDs: `PostgresReplicationCluster` (topology, failover, pooling, WAL archive) and `ReplicationSlot` (slot lifecycle + retention).
- `synchronousMode: quorum` + `automaticFailover: true` achieves RPO = 0 with automated promotion — no DBA required at 3 AM.

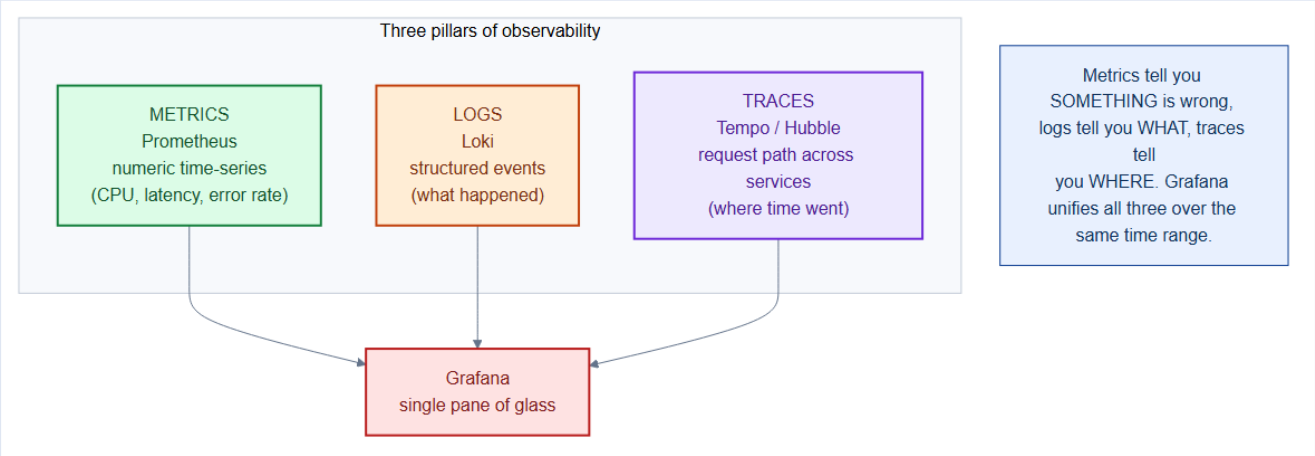
TicketHub is now **secured end to end**: identity (RBAC), the pod (PSA/SecurityContext), the network (NetworkPolicy), policy (Kyverno), runtime (Falco), and the supply chain (encryption + signing). **Part VI** operates it: observability, backup/DR, upgrades, and GitOps delivery.

---

26. Observability — Prometheus, Grafana, Loki & Hubble

TicketHub is now built, scaled, and secured — but if you can't **see** what it's doing, you're flying blind. When a concert on-sale spikes and checkout slows, you need to know *within seconds* whether it's CPU, the database, a bad deploy, or a network policy. **Observability** turns a running cluster into a system you can reason about.

26.1 The three pillars



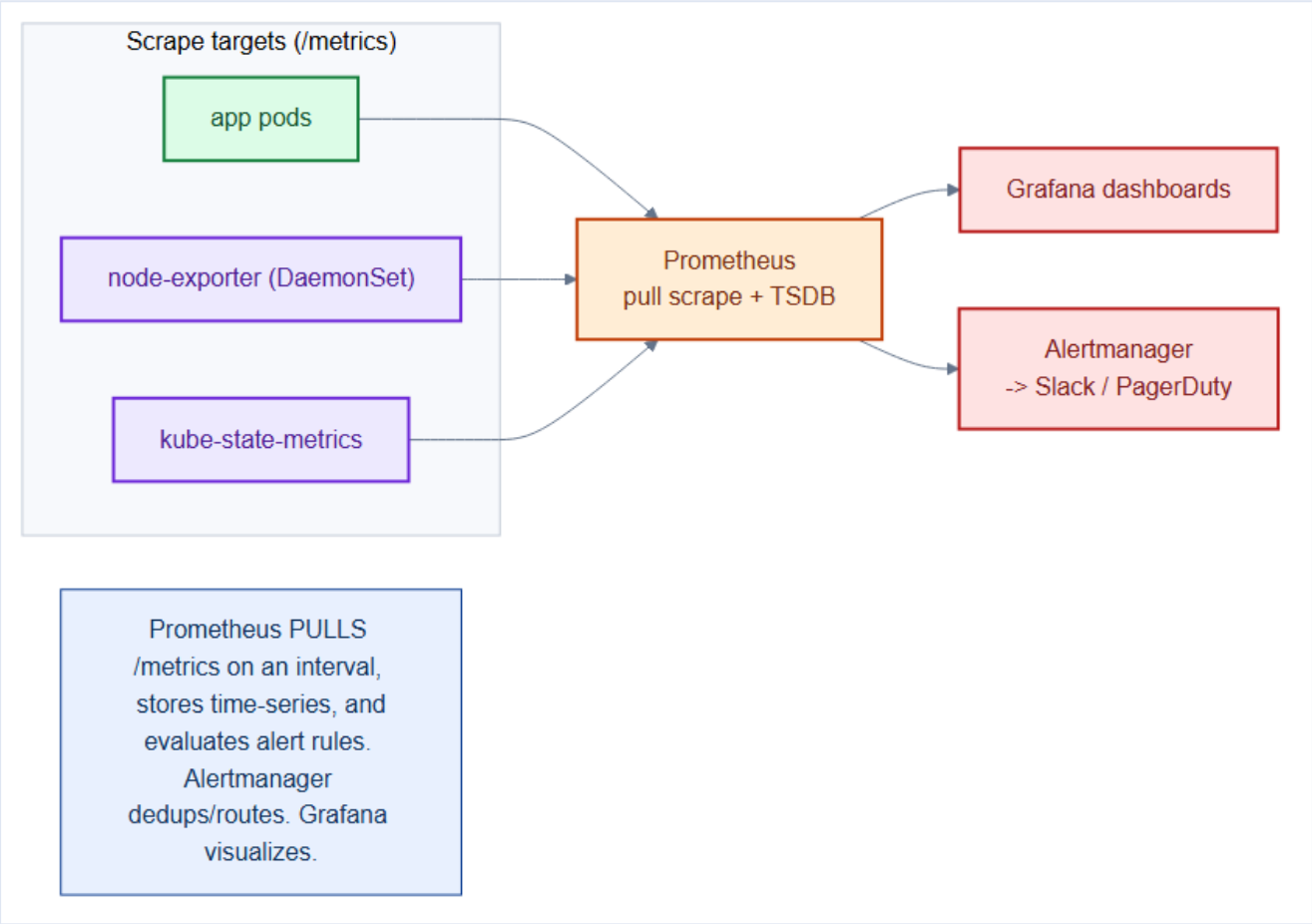
Pillar	Tool	Answers
Metrics	Prometheus	*Is* something wrong? (numeric trends)
Logs	Loki	*What* happened? (events)
Traces	Tempo / Hubble	*Where* did the time go? (request path)

Grafana unifies all three over the same time window — one pane of glass.

> **Mental model — a hospital patient monitor**
>
> **Metrics** are the **vital signs** — heart rate, blood pressure, trending on a screen; an alarm fires when they leave range. **Logs** are the **nurse's notes** — discrete events describing what occurred. **Traces** are the **X-ray** showing exactly where the blockage is. You need all three: vitals tell you there's a problem, notes and imaging tell you what and where.

26.2 Metrics with Prometheus

Prometheus **pulls** a `/metrics` endpoint from every target on an interval, stores time-series in its TSDB, evaluates alert rules, and hands data to Grafana and Alertmanager.



Deployed via the **kube-prometheus-stack** Helm chart, which brings Prometheus, Alertmanager, Grafana, `node-exporter` (Chapter 11 DaemonSet), and `kube-state-metrics`. Services expose metrics and are discovered by a **ServiceMonitor**:

```
# repo/manifests/70-observability/orders-monitoring.yaml (ServiceMonitor section)
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
 name: orders
 namespace: tickethub
 labels: { release: kube-prometheus-stack }
spec:
 selector:
   matchLabels: { app: orders }
 endpoints:
   - port: metrics          # scrape orders' /metrics
     interval: 15s
```

A **PrometheusRule** turns metrics into alerts — this one uses the RED method (Rate, Errors, Duration):

```
# repo/manifests/70-observability/orders-monitoring.yaml (PrometheusRule section)
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata: { name: tickethub-slo, namespace: tickethub }
spec:
 groups:
   - name: tickethub.rules
     rules:
```

```
      - alert: OrdersHighErrorRate
        expr: |
          sum(rate(http_requests_total{app="orders",code=~"5.."}[5m]))
            / sum(rate(http_requests_total{app="orders"}[5m])) > 0.02
        for: 5m
        labels: { severity: page }
        annotations:
          summary: "Orders 5xx error rate above 2% for 5m"
```

Alert on symptoms (SLOs), not causes

Page humans on **user-facing symptoms** — high error rate, high latency, checkout failures — not on every high-CPU blip. CPU can be 90% and users perfectly happy; that's a scaling signal (HPA, Chapter 16), not a 2am page. The **RED** (Rate/Errors/ Duration) and **USE** (Utilization/Saturation/Errors) methods keep alerts meaningful.

26.3 Instrumenting a service — where the telemetry comes from

The alert above queries `http_requests_total` and `http_request_duration_seconds` — but Prometheus doesn't invent those series. **The application produces them.** A ServiceMonitor with nothing to scrape is just a scheduled 404. This section closes the loop: how the `orders` service actually emits metrics and traces.

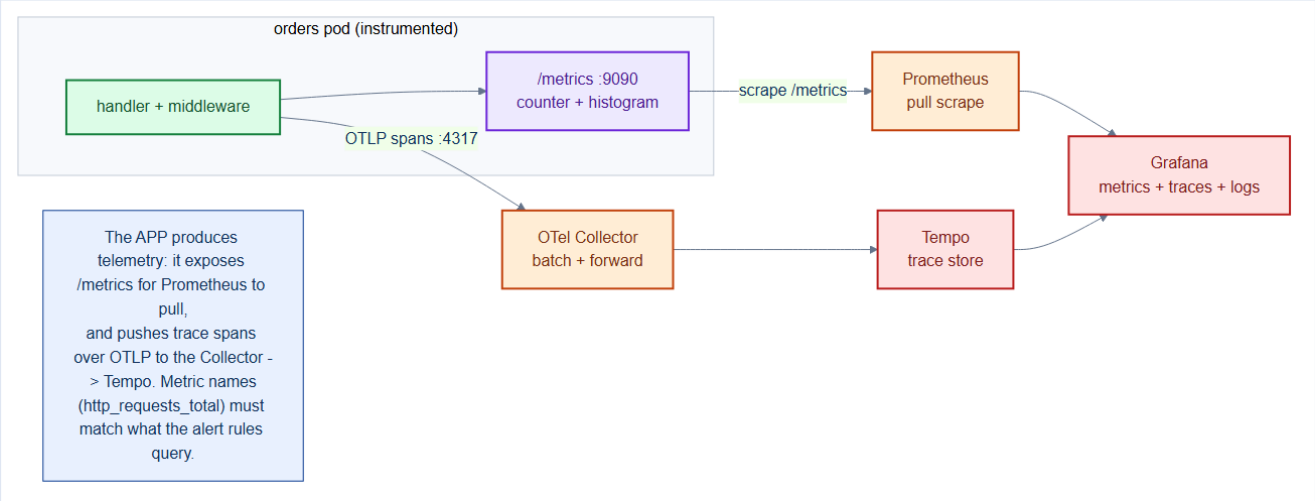
Metrics — the Prometheus client library. You register two series (a **counter** for request volume/errors, a **histogram** for latency), increment them in a middleware, and expose `/metrics`:

```
// repo/services/orders/cmd/orders/observability.go
var (
   httpRequests = promauto.NewCounterVec(
       prometheus.CounterOpts{Name: "http_requests_total"},
       []string{"method", "path", "code"},            // the labels the alert filters on
   )
   httpDuration = promauto.NewHistogramVec(
       prometheus.HistogramOpts{
           Name:    "http_request_duration_seconds",
           Buckets: prometheus.DefBuckets,             // -> _bucket series for p99
       },
       []string{"method", "path"},
   )
)

func metricsMiddleware(next http.Handler) http.Handler {
   return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
       start := time.Now()
       rec := &statusRecorder{ResponseWriter: w, code: 200}
       next.ServeHTTP(rec, r)
       httpRequests.WithLabelValues(r.Method, r.URL.Path, strconv.Itoa(rec.code)).Inc()
       httpDuration.WithLabelValues(r.Method, r.URL.Path).Observe(time.Since(start).Seconds())
   })
}
```

The counter's `code` label is exactly what `code=~"5.."` matches; the histogram is what `histogram_quantile(0.99, ...)` reads. **Name your metrics to match your alerts, or the alerts are**

fiction. The service exposes `/metrics` on a dedicated port `9090`, and the Deployment's Service names that port `metrics` — which is what the ServiceMonitor's `port: metrics` resolves to.



Traces — OpenTelemetry. Metrics tell you *orders is slow*; a **trace** tells you *the slow part was the 380 ms Postgres call, not the app*. You initialize an OTLP exporter once, then wrap the router so every request becomes a **span**:

```
// Export spans over OTLP to the collector -> Tempo.
exp, _ := otlptracegrpc.New(ctx, otlptracegrpc.WithEndpoint(endpoint))
tp := sdktrace.NewTracerProvider(
   sdktrace.WithBatcher(exp),
   sdktrace.WithResource(resource.NewWithAttributes(
       semconv.SchemaURL, semconv.ServiceName("orders"))),
)
otel.SetTracerProvider(tp)

// One line wraps every handler in a span and propagates context downstream.
instrumented := otelhttp.NewHandler(metricsMiddleware(app), "orders")
```

The magic is **context propagation**: `otelhttp` injects a `traceparent` header on outbound calls, so when `orders` calls `payments`, both spans share one **trace ID**. Grafana stitches them into a single waterfall. The spans flow `app -> OTLP -> OpenTelemetry Collector -> Tempo`, deployed in `repo/manifests/70-observability/otel-collector-tempo.yaml`.

Logs — just write structured JSON to stdout. No library ceremony: emit one JSON object per line to stdout and let Loki's agent (next section) collect it. Include the `trace_id` so a log line links straight to its trace:

```
log.Printf(`{"level":"error","msg":"payment declined","order_id":%q,"trace_id":%q}`,
   orderID, traceIDFromContext(r.Context()))
```

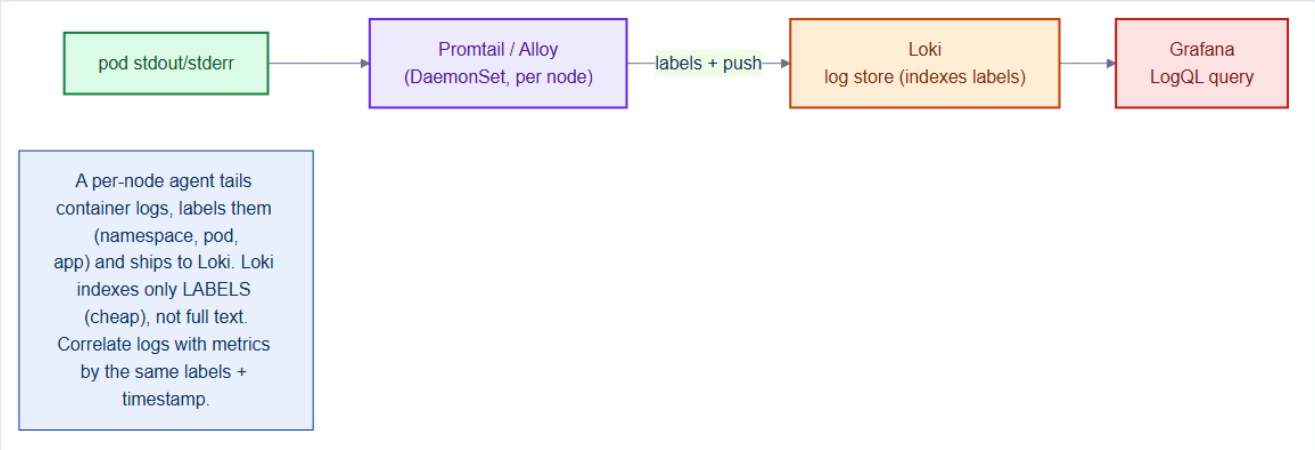
> **Instrument once, correlate everywhere**

> The payoff of shared conventions: the **same** `app` and `path` labels appear on metrics, the **same** `trace_id` appears on logs and spans, and Grafana pivots between all three over one time window. A spiking `http_requests_total{code="500"}` -> click to the Loki lines with that `trace_id` -> click to

the Tempo waterfall showing the failed Postgres span. Three pillars, one incident, ten seconds.

26.4 Logs with Loki

Loki is "Prometheus for logs" — a per-node agent (**Promtail/Alloy**, a DaemonSet) tails container stdout/stderr, **labels** each line (namespace, pod, app), and ships it. Loki indexes only the **labels** (cheap), not full text.




```
# LogQL — find orders errors during the incident window
{namespace="tickethub", app="orders"} |= "error" | json | status >= 500
```

Because Loki and Prometheus share the **same labels**, you jump from a spiking metric to the exact logs in one click in Grafana.

26.5 Network observability with Hubble

Chapter 6 installed **Hubble** on top of Cilium. It's the "trace" pillar for the network — the live service map, and which flows were **allowed or dropped** by the NetworkPolicies from Chapter 21:

```
hubble observe --namespace tickethub --verdict DROPPED   # see what policy blocked
```

Store observability data on durable, separate storage

Prometheus and Loki hold your incident-response history — put them on **Ceph-backed PVs** (Chapter 8) in the `monitoring` namespace, sized for real retention, and back them up (Chapter 27). Metrics/logs on ephemeral `emptyDir` vanish exactly when a node dies — the moment you most need to look back. Keep them off the nodes they observe.

26.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Prometheus `rate()` vs `irate()`**: `rate()` uses the full scrape interval window and is more resistant to single-sample spikes; `irate()` uses only the last two samples and reacts faster. For alerting rules (where you want to react to sustained increases), use `rate()`. For dashboards showing instantaneous throughput, `irate()` gives more responsive graphs.

- **Loki indexes only labels, not log content**: a query `{app="orders"} |= "ERROR"` first selects log streams by label (fast — B-tree lookup) and then scans the selected stream content for "ERROR" (slow — sequential scan). Design Loki label schemes with cardinality in mind: labels with 1000+ values (e.g., `pod_ip` or `request_id`) create exploding cardinality that breaks Loki's compaction and query performance.

- **Tempo trace sampling**: sending 100% of traces to Tempo at high throughput is expensive. A head-based sampling rate of 1-5% is typical for high-volume services. But you may miss rare error traces. Tail-based sampling (make the sampling decision AFTER seeing the full trace, keeping all error traces) provides better coverage — configure the OTel Collector to use tail-based sampling for error traces.

Gotchas — traps that catch experienced engineers

- **`up` metric as the only Prometheus health check**: `up == 0` means Prometheus failed to scrape the target, but `up == 1` does NOT mean the service is healthy — it means Prometheus successfully scraped `/metrics`. A service that is returning 500s but still serves metrics will have `up == 1`. Always alert on your business metrics (`http_requests_total{status="5xx"}`), not just `up`.

- **Grafana datasource secret rotation**: Grafana stores datasource credentials (Prometheus, Loki URLs with auth) in its database. Rotating Prometheus bearer tokens requires updating Grafana's datasource config AND reloading it — a step often missed when rotating credentials.

- **OTel Collector memory limiter placement**: the `memory_limiter` processor must be the FIRST processor in the pipeline, before batching. If placed after the batcher, the batcher accumulates spans until the memory limit is already exceeded — causing uncontrolled OOM instead of graceful backpressure.

Architect Considerations

1. **Metrics cardinality governance**: Prometheus performance degrades when the total number of unique time series (metric name × label combinations) exceeds ~1M per Prometheus instance. High-cardinality labels like `user_id`, `request_id`, or `url_path` in application metrics can explode series counts. Define a cardinality budget per service and enforce it via Prometheus recording rules that aggregate away high-cardinality dimensions.

2. **Log retention vs cost trade-off**: Loki stores logs in object storage (Ceph S3) which is cheap but query-slow for large windows. Define log retention tiers: 7 days hot (fast query), 30 days warm

(slower), 90 days cold (compliance archive). Loki supports compaction and deletion policies per tenant/stream.

3. **Distributed tracing sampling strategy**: for an online ticketing platform, EVERY failed transaction should be traced end-to-end (tail sampling). Successful transactions can be sampled at 1%. This requires a tail-based sampler in the OTel Collector that buffers spans and makes the sampling decision when the root span completes.

4. **On-call alert quality**: every alert that pages someone at 3 AM must have a corresponding runbook. An alert without a runbook is a noise source, not a signal. Require runbooks as a PR requirement for any new `PrometheusRule`. Track mean time to acknowledge (MTTA) and mean time to resolve (MTTR) per alert as quality metrics.

5. **SLO burn rate alerts**: instead of alerting on raw error rates (`error_rate > 1%`), alert on SLO burn rate (`error_budget_consumed_rate > 5× normal` for 1-hour window). This gives high-signal, low-noise paging — you only get paged when you are burning through your error budget faster than acceptable, not on every transient spike.

Chapter 26 checklist

- **kube-prometheus-stack** running; services expose `/metrics` via **ServiceMonitors**.
- Alerts defined on **SLO symptoms** (RED/USE), routed by Alertmanager to Slack/PagerDuty.
- **Loki + Promtail** shipping labeled logs; correlate with metrics by shared labels.
- **Hubble** for network flow visibility (allowed/dropped).
- Observability data on **durable Ceph storage** with retention, not `emptyDir`.

---

27. Backup, DR, Upgrades & Node Maintenance

A production cluster is never "done" — nodes need patching, Kubernetes needs upgrading, and someday something *will* fail catastrophically. This chapter covers keeping TicketHub alive through planned maintenance and unplanned disaster: **backup**, **disaster recovery**, **cluster upgrades**, and **node maintenance** — all without dropping a single ticket sale where avoidable.

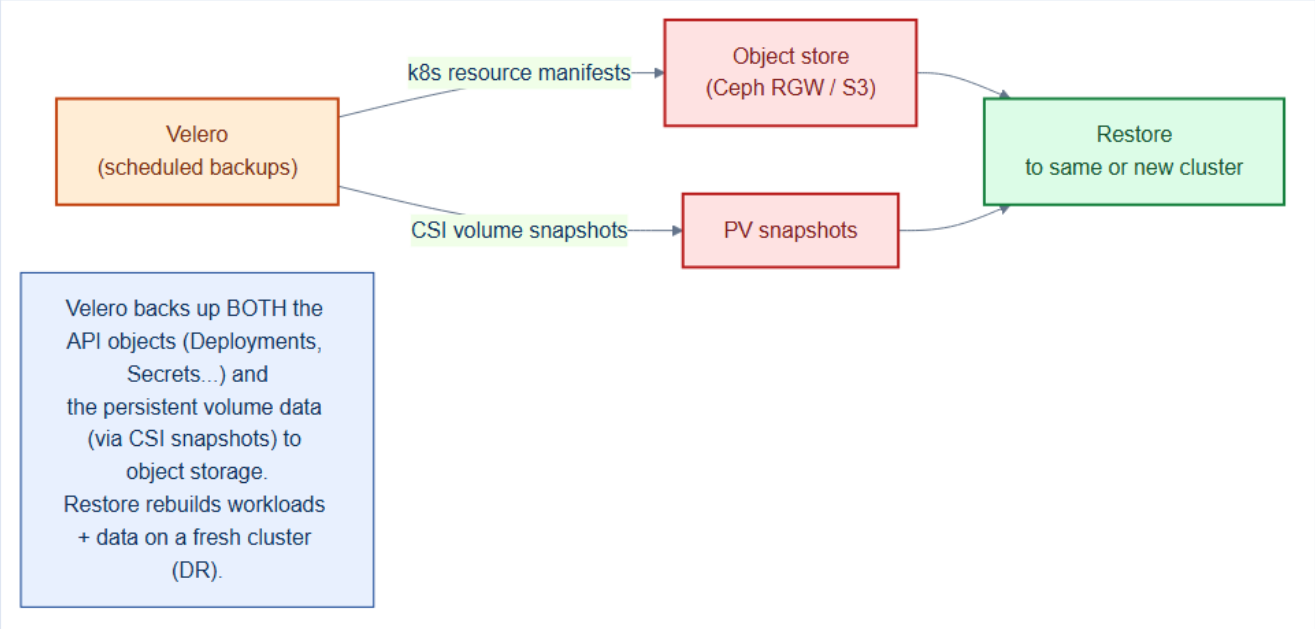
27.1 What actually needs backing up

Layer	Contains	Backup method
etcd	All cluster state (every object)	`etcdctl snapshot` (scheduled)
PV data	Database contents, Kafka logs	CSI volume snapshots (Ceph)
API objects	Deployments, ConfigMaps, RBAC	Velero
Git repo	Desired state (Chapter 28)	Git is already the backup

Because TicketHub is **GitOps-delivered** (next chapter), the *manifests* are safe in Git. What Git *doesn't* hold is **stateful data** and **secrets** — that's what backup tooling protects.

27.2 Velero — application-level backup & DR

Velero backs up both the **Kubernetes API objects** and the **persistent volume data** (via CSI snapshots) to object storage (Ceph RGW / S3), and restores them to the same or a **brand-new cluster** — the foundation of disaster recovery.



```
# repo/manifests/70-observability/velero-schedule.yaml
apiVersion: velero.io/v1
kind: Schedule
metadata:
 name: daily-tickethub
 namespace: velero
spec:
 schedule: "0 2 * * *"            # 02:00 daily
 template:
   includedNamespaces: [tickethub, data]
   snapshotVolumes: true          # CSI snapshot the PVs too
   ttl: 720h0m0s                  # keep 30 days
```

```
velero backup create adhoc-before-upgrade --include-namespaces tickethub,data
velero restore create --from-backup daily-tickethub-20260811020000   # DR
```

Mental model — backups are a time machine, DR is a spare house

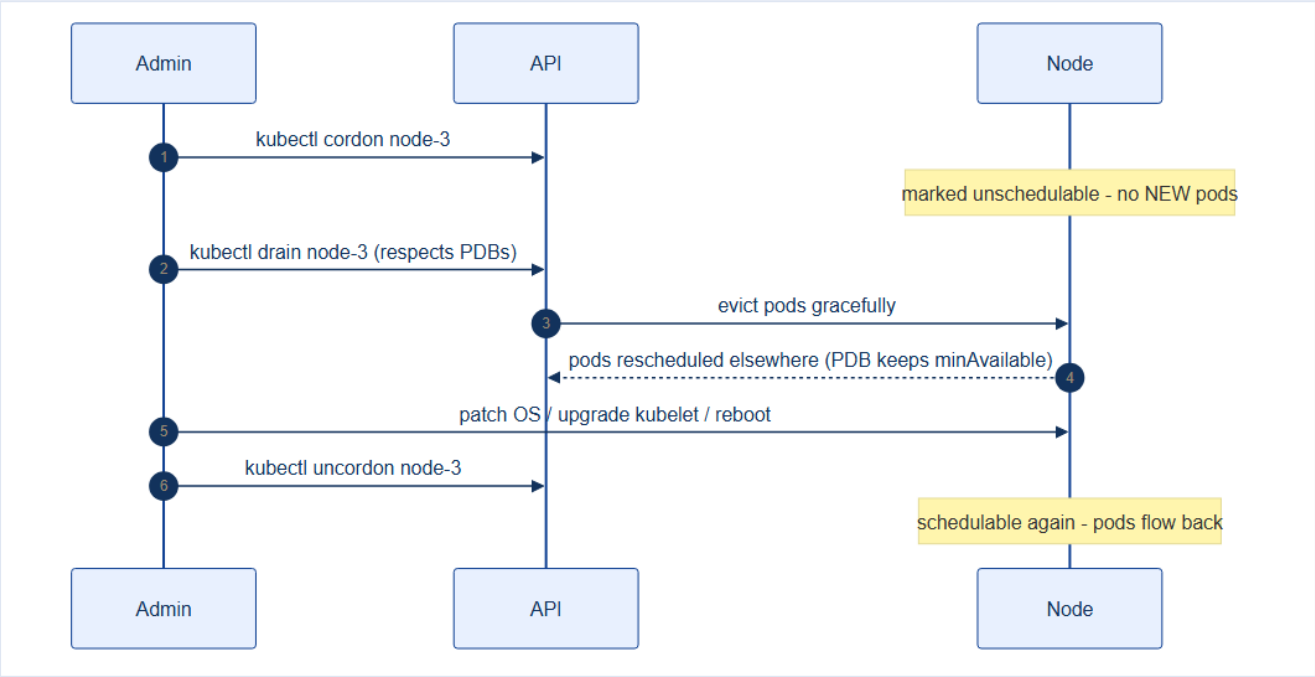
A **backup** is a **time machine**: go back to yesterday's state if today breaks. **DR** is having a **fully furnished spare house** in another town: if your house burns down, you move in and keep living. Velero is both — snapshots to rewind, and cross-cluster restore to relocate. An untested backup is a spare house you've never checked has a roof.

A backup you haven't restored is a hope, not a plan

Schedule a **periodic restore drill** into a scratch cluster/namespace. Teams discover at the worst possible moment that snapshots were app-inconsistent, a PV class didn't exist on the target, or a secret was missing. Define **RPO** (how much data you can lose) and **RTO** (how fast you must be back), and prove you meet them.

27.3 Node maintenance — cordon & drain

To patch or reboot a node, evict its pods **gracefully** first. `cordon` stops new pods landing; `drain` evicts existing ones while **respecting PodDisruptionBudgets** (Chapter 17).



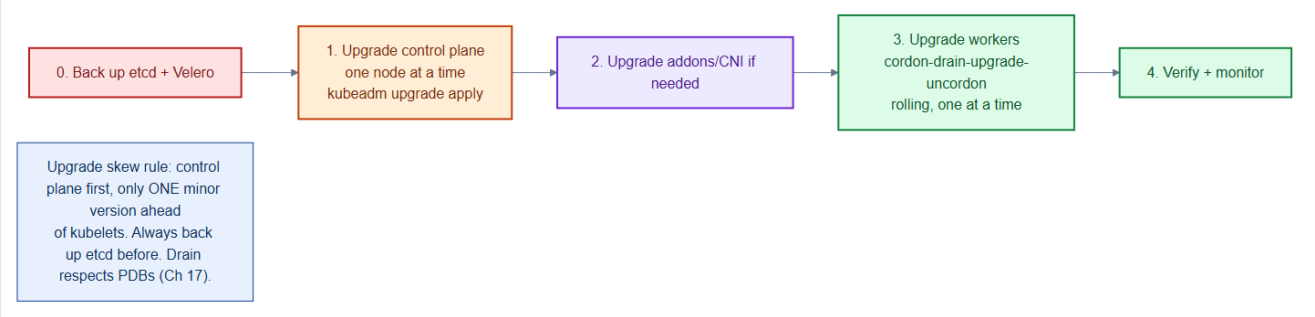

```
kubectl cordon node-3                       # unschedulable
kubectl drain node-3 --ignore-daemonsets --delete-emptydir-data   # evict, honors PDBs
# ... patch OS / reboot ...
kubectl uncordon node-3                      # back into rotation
```

> **Drain without PDBs can take a service to zero**

> `drain` evicts *all* a node's pods at once. If a PDB doesn't guarantee `minAvailable`, and replicas happened to co-locate, the service briefly hits zero. This is exactly why Chapter 17 mandated a PDB on every user-facing service — maintenance safety depends on it.

27.4 Upgrading Kubernetes

Upgrade in a strict order, honoring the **version-skew** rules, with an etcd backup first:




```
# 0. back up first
etcdctl snapshot save /backup/etcd-$(date +%F).db
velero backup create pre-upgrade --include-namespaces tickethub,data

# 1. control plane, one node at a time
kubeadm upgrade plan
kubeadm upgrade apply v1.31.0        # on cp-1, then cp-2, cp-3
```

```
# 3. workers, rolling: cordon -> drain -> upgrade kubelet -> uncordon
```

Rule	Detail
Order	Control plane **before** workers
Skew	kubelet may be **one minor version** behind the API server, never ahead
One at a time	Preserve HA quorum (etcd, Chapter 3) throughout
Never skip minors	1.30 → 1.31 → 1.32, not 1.30 → 1.32

27.5 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Velero backs up Kubernetes resource definitions (etcd objects), NOT application data volumes** by default. The `--include-volumes` flag or CSI volume snapshot integration is required to back up PVC data. A Velero backup without volume snapshots can restore a StatefulSet definition but not the database data it contained.

- **etcd backup is a different layer from Velero backup**: etcd snapshot (`etcdctl snapshot save`) backs up the raw cluster state including Secrets, RBAC, and CRDs. Velero backup backs up namespaced resources but cannot restore cluster-scoped objects (Nodes, ClusterRoles, StorageClasses) without specific `--include-cluster-resources` flags. Both are required for full DR.

- **Kubernetes version skew policy**: you can upgrade only one minor version at a time (`1.29 → 1.30`, not `1.29 → 1.31`). Control plane components can be ahead of kubelets by up to 2 minor versions during rolling upgrades, but kubelets cannot be ahead of the API server. The upgrade order is always: etcd → kube-apiserver → other CP components → kubelets.

Gotchas — traps that catch experienced engineers

- **`velero backup create` vs `velero schedule create`**: one-time backups expire and are deleted based on the TTL. Without a schedule, a manual backup from 6 months ago is your most recent backup when you need it most. Always configure a scheduled backup from day 1.

- **PVC snapshot CSI driver compatibility**: Velero CSI volume snapshots require the storage driver to support the `VolumeSnapshot` API. Rook-Ceph's CSI driver supports it, but you must install the `snapshot.storage.k8s.io` CRD and the external-snapshotter controller separately. Not having these installed means Velero silently skips volume backups.

- **In-place node upgrade vs blue-green**: `kubeadm upgrade node` upgrades the kubelet in place. If the upgrade fails mid-way, the node may be in a partially upgraded state that prevents normal operation. Always have a node replacement strategy (provision new node, drain old, decommission) as a fallback — especially for production clusters where rebuild time is critical.

Architect Considerations

1. **RTO and RPO definition**: define these BEFORE building the backup system. For TicketHub: is a 4-hour RTO acceptable (rebuild cluster + restore backup)? Is a 1-hour RPO acceptable (lose up to 1 hour of orders)? These requirements drive the backup frequency, snapshot consistency level, and restore automation investment.

2. **Backup verification — "trust but verify"**: a backup that has never been tested is a hypothesis, not a guarantee. Schedule quarterly DR drills: restore the entire `data` namespace to a separate cluster and run smoke tests. Track the actual restore time — it's almost always longer than estimated.

3. **Cluster upgrade strategy for bare metal**: you cannot "spin up a new node" on demand like in cloud. For bare metal, the upgrade strategy is: drain workers one by one, upgrade kubelet, uncordon. For control plane: use the HA topology (3 CP nodes) so you upgrade one at a time with 2/3 quorum intact.

4. **etcd compaction and defragmentation**: etcd accumulates historical revision data that is only freed by compaction (`etcdctl compact`) and defragmentation (`etcdctl defrag`). A production cluster that has been running for months without defragmentation can have etcd databases 10× larger than necessary, increasing backup size and restore time.

5. **Multi-cluster DR topology**: a single on-prem cluster with backup to the same data center Ceph storage is not a true DR — a data center fire destroys both the cluster and the backup. For genuine DR, Velero backups must be replicated to an off-site location (different data center, cloud storage bucket).

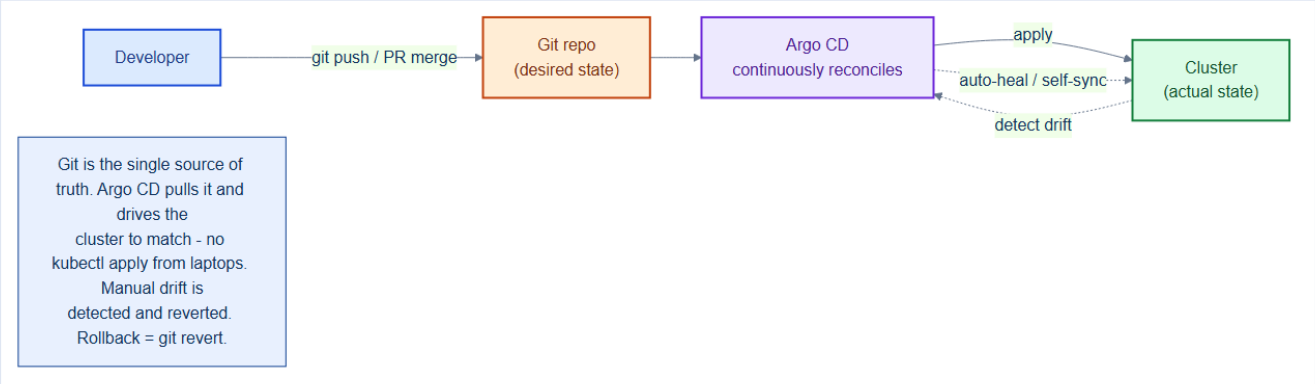
Chapter 27 checklist

- **etcd snapshots** + **Velero** backups scheduled; PV data snapshotted.
- **RPO/RTO** defined and validated by periodic **restore drills**.
- Node maintenance always via **cordon** → **drain** → **uncordon**, honoring PDBs.
- Upgrades: **back up first**, control plane before workers, one node at a time, no skipped minors.
- DR playbook documented and rehearsed on a scratch cluster.

28. GitOps Delivery with Argo CD

Throughout this book we've written YAML and imagined `kubectl apply`. In production, **nobody applies manifests from a laptop**. Instead, **Git is the single source of truth**, and **Argo CD** continuously reconciles the cluster to match it. This is **GitOps** — it makes deployments auditable, reproducible, self-healing, and trivially reversible.

28.1 The GitOps model



1. Desired state (all the manifests from this book) lives in **Git**.
2. A change is a **pull request** — reviewed, approved, merged.
3. **Argo CD** detects the commit and **applies** it to the cluster.
4. If anything **drifts** from Git (a manual `kubectl edit`), Argo CD **detects and reverts** it.

> **Mental model — Git as the thermostat setting**
>
> You don't walk around the house adjusting each radiator (`kubectl apply` per resource). You set the **thermostat** (Git), and the system continuously works to match it. Open a window and the temperature drifts — the thermostat notices and compensates (Argo CD self-heals). Want it warmer? Change the *setting* (a commit), not the radiators. The desired state is declared in one reviewed place; the system enforces it.

28.2 Push vs. pull delivery

	Traditional (push)	GitOps (pull)
Trigger	CI runs `kubectl apply`	Argo CD pulls Git
Credentials	CI holds cluster admin	Cluster pulls; CI never touches it
Drift	Undetected	Detected + reverted

Audit	Scattered CI logs	Git history = full audit trail
Rollback	Re-run old pipeline	`git revert`

> **GitOps means the cluster pulls — CI never holds cluster credentials**

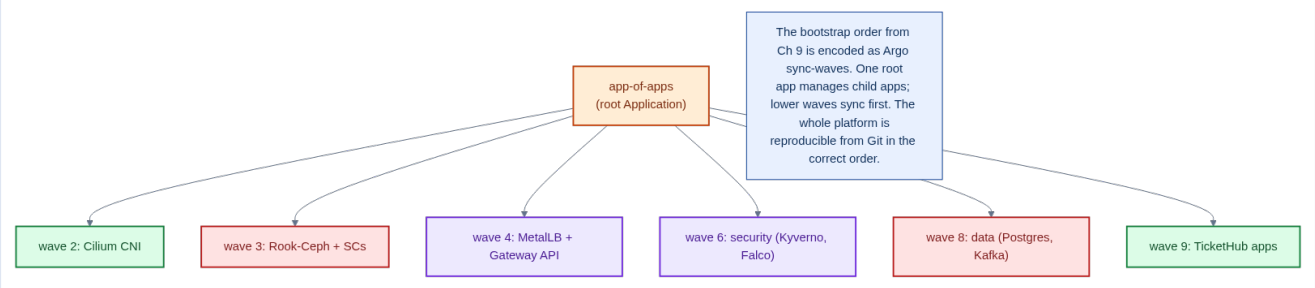
> In push delivery, your CI system holds cluster-admin and becomes a juicy target. In GitOps, **Argo CD (inside the cluster) pulls from Git** — CI only builds images and opens PRs, and never has kubectl access. This shrinks the attack surface and makes every change traceable to a signed commit.

28.3 The Application and app-of-apps

An Argo CD **Application** maps a Git path to a cluster destination:

```
# repo/argocd/tickethub-app.yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
 name: tickethub
 namespace: argocd
spec:
 project: default
 source:
   repoURL: https://git.internal/tickethub/platform.git
   targetRevision: main
   path: repo/manifests/30-workloads
 destination:
   server: https://kubernetes.default.svc
   namespace: tickethub
 syncPolicy:
   automated:
     prune: true                # delete resources removed from Git
     selfHeal: true             # revert manual drift
   syncOptions: [CreateNamespace=true]
```

The **app-of-apps** pattern uses one **root** Application to manage all the others — and encodes the **bootstrap order from Chapter 9** as **sync waves**:




```
metadata:
 annotations:
   argocd.argoproj.io/sync-wave: "3"    # Rook-Ceph in wave 3, apps in wave 9
```

This is the payoff of Chapter 9's ordering discipline: the **entire platform reconstructs itself from Git, in the correct order**, on a fresh cluster — the ultimate DR complement to Chapter 27.

28.4 Progressive delivery

Argo CD deploys the declared state; **Argo Rollouts** adds **canary** and **blue-green** strategies (Chapter 18) on top — shift 10% of traffic to the new version, watch the Prometheus error rate (Chapter 26), and **auto-rollback** if the SLO degrades.

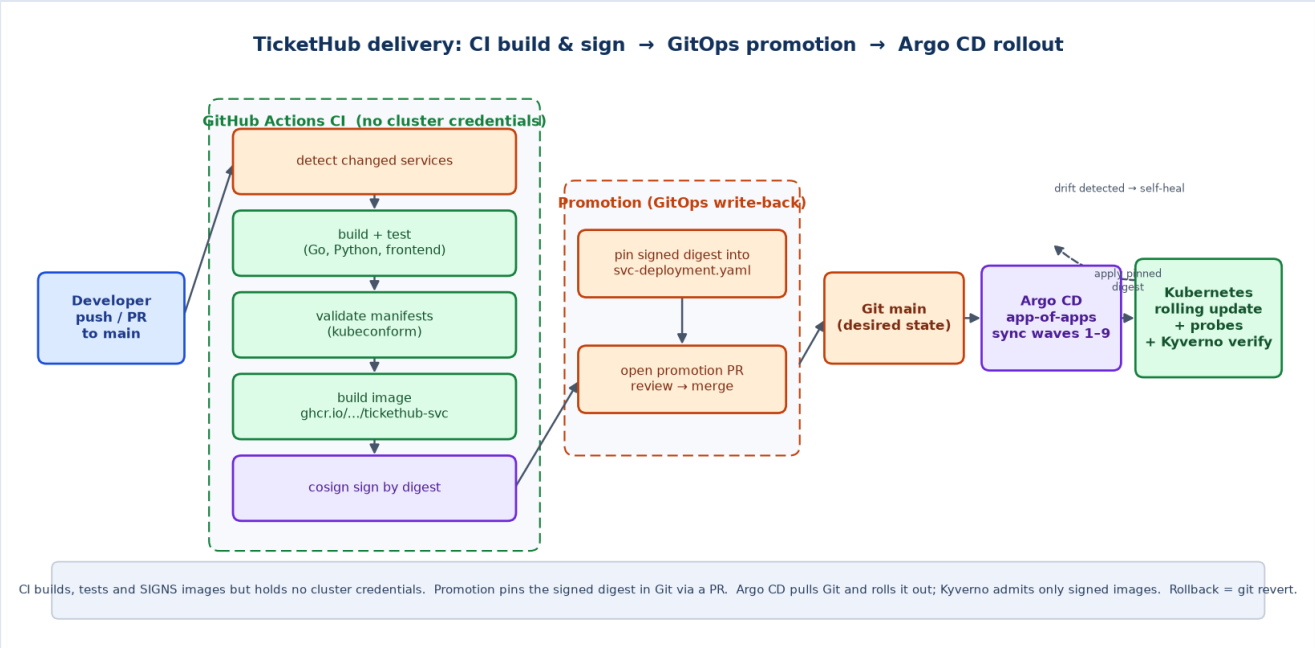
```
strategy:
 canary:
   steps:
     - setWeight: 10
     - pause: { duration: 5m }        # bake, watch metrics
     - setWeight: 50
     - pause: { duration: 5m }
     - setWeight: 100
```

> **Secrets still don't go in Git — even with GitOps**

> GitOps wants *everything* in Git, but plaintext secrets must **not** be. Use the **External Secrets** pattern from Chapter 13 (Git holds the `ExternalSecret` reference; the vault holds the value) or Sealed Secrets. Argo CD syncs the reference; the real credential never lands in a commit.

28.5 Automated image promotion — from signed image to running Pod

GitOps says *Git is the desired state*, but something has to **write the new image version into Git**. That is **image promotion**, and it is the bridge between CI (which builds artifacts) and Argo CD (which deploys them).



The flow, end to end:

1. **CI builds and signs** — on a push to `main`, GitHub Actions detects which services changed, builds and tests them, builds a container image tagged by commit SHA, pushes it to GHCR, and **signs it by digest** with Cosign. CI holds **no cluster credentials**.

Promotion writes back to Git — a `promote` job rewrites the `image:` field of each changed service's `repo/manifests/30-workloads/<svc>-deployment.yaml` to the **signed digest** (`ghcr.io/<org>/tickethub-<svc>@sha256:…`), then opens a **pull request**:

```yaml
# before
image: registry.internal/tickethub/orders:v1
# after promotion
image: ghcr.io/acme/tickethub-orders@sha256:828ad8c7... ```

**A reviewer merges the PR** — this is the deployment approval. Because `main` is branch-protected, promotion opens a PR rather than pushing directly; the merge is the audited "go" decision.

4. **Argo CD rolls it out** — Argo CD sees the changed digest in Git and syncs it. The Deployment's `RollingUpdate` strategy brings up a new Pod, waits for `/readyz`, then retires an old one. Kyverno's `verify-image-signatures` policy (Chapter 22/24) admits **only** signed images, so an unsigned or tampered digest is rejected at admission.

> **Pin the digest, not the tag**
>
> Promotion writes the **digest** (`@sha256:…`), not a moving tag like `:v1` or `:latest`. A digest is the image's immutable content identity: it guarantees the exact bytes CI built and signed are the exact bytes the cluster runs, and it makes Kyverno signature verification meaningful. Tags can be re-pointed; digests cannot.

> **GITHUB_TOKEN cannot bypass branch protection**
>
> If `main` requires pull requests, the promote job must **open a PR** — a direct `git push origin main` with the default `GITHUB_TOKEN` is rejected by the ruleset. Enable *Settings → Actions → General → Allow GitHub Actions to create and approve pull requests* so the job can open the promotion PR automatically.

### Packaging the workloads as a Helm chart

The 9 services are near-identical (same probes, security context, ports, rollout strategy), so instead of 9 hand-written manifest sets they are delivered by **one Helm chart** at `repo/charts/tickethub/`. A single `values.yaml` lists each service (image, ports, config, scaling, network policy); the chart templates render
```

the `Deployment`, `Service`, `HorizontalPodAutoscaler`, `PodDisruptionBudget`, `NetworkPolicy`, and migration `Job` for all of them.

Argo CD renders the chart directly — the workloads `Application` points at the chart path and Argo runs `helm template` internally, then reconciles the output:

```
source:
 repoURL: https://github.com/karthikb35/kubernetes_full_microservices.git
 path: repo/charts/tickethub
 helm:
   releaseName: tickethub
```

Promotion changes accordingly: the CI job pins each signed digest into the service's `image:` field in `values.yaml` (instead of a raw Deployment), and Argo CD re-renders the chart. This is the clean split — **Helm** decides *how manifests are generated*, **Argo CD** decides *how they reach the cluster*.

Helm and Argo CD are complementary, not alternatives

Helm is a templating/packaging tool; Argo CD is a GitOps reconciler. Using them together gives DRY, parameterised manifests *and* pull-based, self-healing delivery — the same signed-digest promotion flow, with one `values.yaml` instead of nine files.

28.6 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **Argo CD `sync` applies manifests in dependency order via sync waves**, but the wave mechanism is opt-in (annotation `argocd.argoproj.io/sync-wave: "N"`). Without explicit wave annotations, Argo CD applies all resources simultaneously — which can create ordering failures (e.g., a Deployment being created before its ConfigMap or Secret exists).

- **Argo CD App of Apps does not automatically prune child Applications when removed from the parent**: if you remove a child Application from the App of Apps, Argo CD marks it `OutOfSync` but doesn't delete it unless `prune: true` is set. Dangling Applications keep running and consuming resources indefinitely.

- **`kubectl apply` vs Argo CD sync**: Argo CD uses server-side apply with a `argocd` field manager. If you also run `kubectl apply` manually on the same resource, field manager conflicts can cause Argo CD to revert your changes or generate `FieldValueConflict` errors. All changes to Argo CD-managed resources MUST go through git.

Gotchas — traps that catch experienced engineers

- **Argo CD sync with `--force` flag deletes and recreates resources**: unlike `kubectl apply --force`, this is destructive — Argo CD will DELETE a running StatefulSet and recreate it, causing a full restart. Only use force sync when absolutely necessary (stuck CRD migration), never as a routine

operation.

- **Repo server access to private registries**: Argo CD's repo server must have git credentials to pull from private repos AND registry credentials if using Helm OCI charts from a private registry. Missing credentials cause silent sync failures with opaque "repository not found" errors.

- **Automated sync + Kyverno mutating webhooks**: Argo CD's drift detection compares the DESIRED manifest (git) against the LIVE manifest (cluster). Kyverno's mutation adds fields to the live manifest that aren't in git — causing Argo CD to always show the app as `OutOfSync`. Configure Argo CD's `ignoreDifferences` for Kyverno-injected fields to prevent false-positive sync loops.

Architect Considerations

1. **Mono-repo vs multi-repo**: a single `repo/manifests/` tree (as in this project) is easy to navigate but creates a single failure domain for gitops — a broken PR that blocks merge prevents ALL service updates. A multi-repo layout (one repo per service team) gives team autonomy but multiplies Argo CD Application count and makes cross-service dependencies harder to express.

2. **Secrets in git with Argo CD**: application secrets cannot be stored in plaintext in git. Options: Sealed Secrets (encrypted in git, decrypted in cluster), External Secrets (fetched from Vault at sync time), Argo CD Vault Plugin (template substitution at sync time). External Secrets is the cleanest architecture — git contains only the ExternalSecret declaration, Vault holds the actual value.

3. **Sync windows for compliance**: some environments require that no changes are applied between Friday 5pm and Monday 9am (change freeze). Argo CD `SyncWindow` supports this — define `denyWindows` for change freeze periods. Without this, an auto-synced Argo CD will apply a Friday 11pm merge immediately.

4. **Rollback strategy**: Argo CD "rollback" is `git revert` + sync. There is no in-cluster rollback button that is independent of git state. Ensure your team understands this: to roll back a bad deploy, you must create a git commit that reverts the change and merge it to main. Design your branch protection and merge strategy around this constraint.

5. **ApplicationSet for multi-environment**: use Argo CD ApplicationSet with a directory generator to automatically create Applications for every environment directory (`envs/staging/`, `envs/prod/`). This avoids manually copying Application objects between environments and ensures all environments have the same Application structure.

Chapter 28 checklist

- **Git is the single source of truth**; changes flow through reviewed PRs.

- **Argo CD** syncs with `selfHeal` + `prune`; manual drift is reverted.

- **app-of-apps + sync waves** encode the Chapter 9 bootstrap order — full cluster reproducible from Git.

- CI **builds/signs images and opens PRs only** — never holds cluster credentials.

- Risky releases use **Argo Rollouts** canary with metric-based auto-rollback.

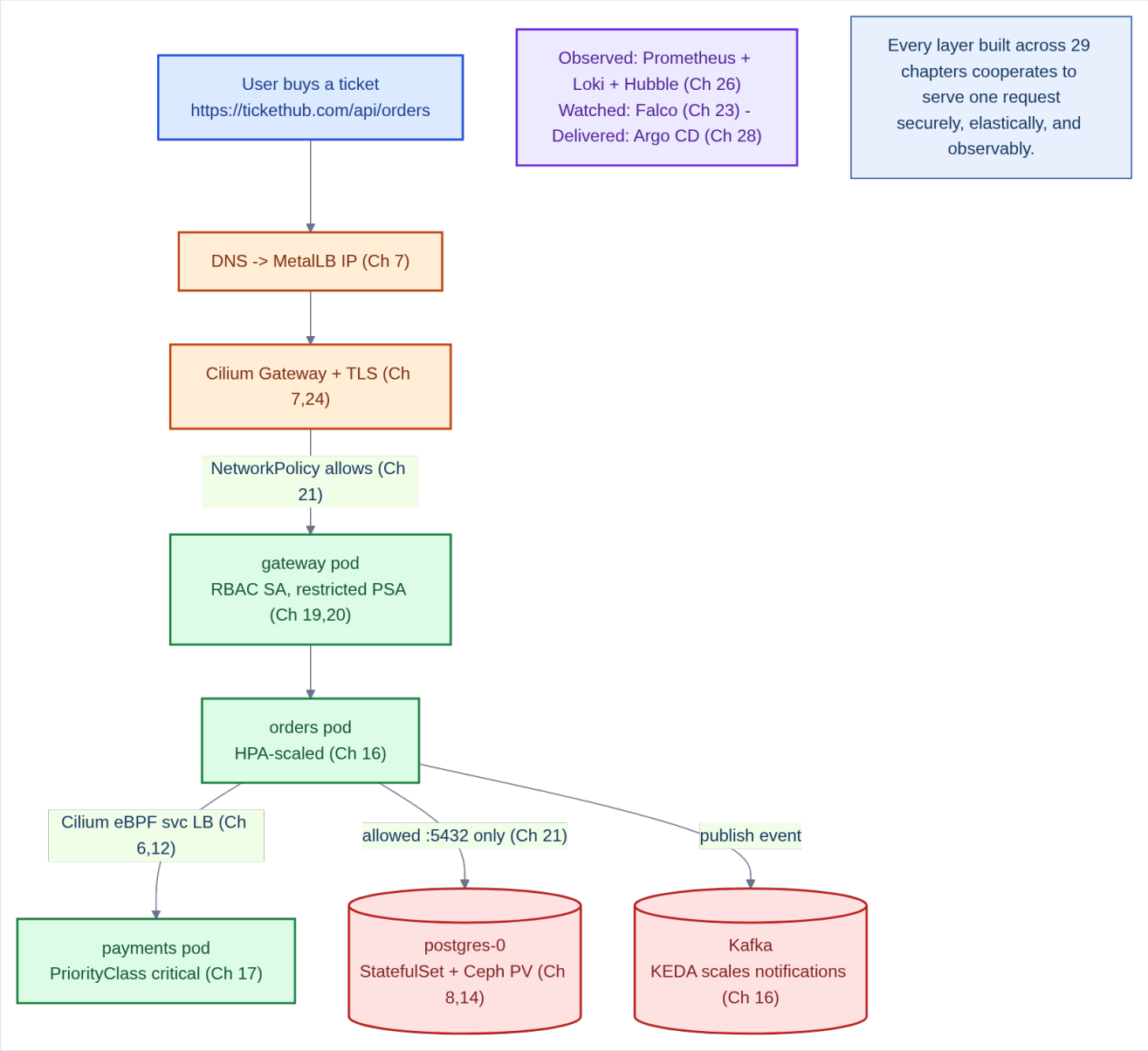
- Secrets via **External/Sealed Secrets**, never plaintext in Git.

---

29. End-to-End Recap — A Request's Full Journey

We began with bare-metal servers in a data center and ended with a secured, observable, GitOps-delivered microservices platform. This final chapter ties all 28 preceding chapters together by following **one user request** — buying a concert ticket — through every layer we built, and then stepping back to see the whole stack at once.

29.1 One ticket purchase, every layer



Trace a single `POST /api/orders` from tap to confirmation:

1. **DNS → MetalLB** (Ch 7) — the domain resolves to a `LoadBalancer` IP that MetalLB advertises from bare metal, with no cloud provider.

2. **Gateway API + TLS** (Ch 7) — the Cilium Gateway terminates HTTPS with a cert-manager certificate and routes `/api` by an HTTPRoute host/path rule.

3. **NetworkPolicy check** (Ch 21) — Cilium's eBPF datapath confirms the traffic is *allowed* to reach the gateway; every other path is denied by default.

4. **gateway pod** (Ch 11, 12) — a Deployment behind a ClusterIP Service, running as a **non-root, restricted** pod (Ch 20) under a **least-privilege ServiceAccount** (Ch 19).

5. **orders pod** (Ch 12, 16) — reached by DNS service discovery; **HPA** has scaled it out for the on-sale surge, load-balanced by Cilium eBPF (Ch 6).

6. **payments pod** (Ch 17) — a **PriorityClass**-critical service that survives node pressure and preempts batch work.

7. **postgres-0** (Ch 8, 14) — a **StatefulSet** pod with a stable identity and a **Ceph-backed PV**, reachable only on port 5432 per NetworkPolicy.

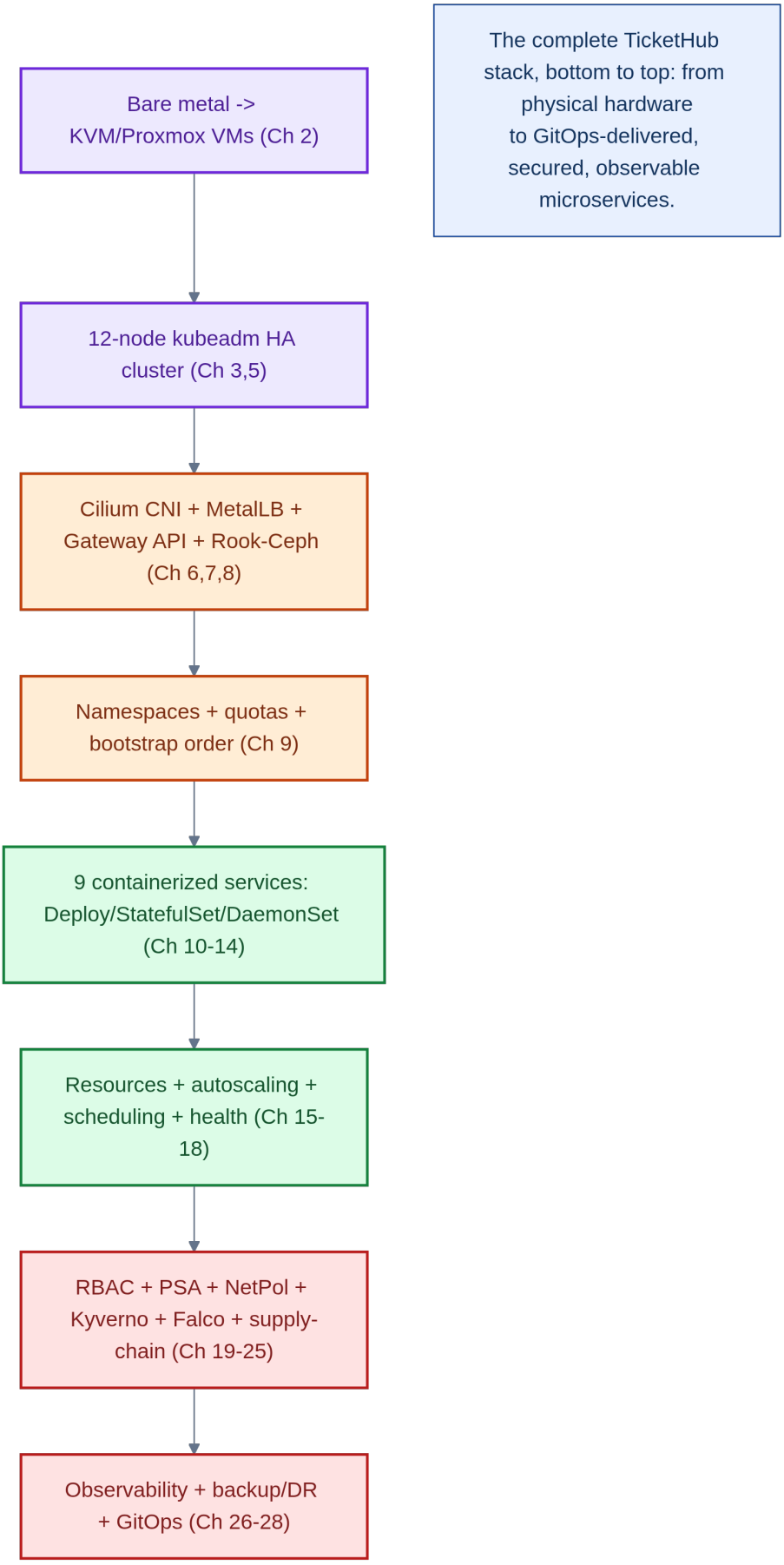
8. **Kafka event** (Ch 16) — orders publishes `ticket-purchased`; **KEDA** scales the notifications consumer on queue lag to email the ticket.

9. **Observed, watched, delivered** (Ch 23, 26, 28) — the whole flow emits **metrics/logs/traces** (Prometheus/Loki/Hubble), is **watched at runtime** by Falco, and every component was **delivered by Argo CD** from Git.

> **No single feature ships a ticket — the system does**

> The purchase succeeds because *every* layer cooperates: the network lets exactly the right pods talk, autoscaling gives them capacity, scheduling keeps the critical ones alive, storage persists the order, security fences the whole thing, and observability proves it worked. Architecture is the discipline of making these layers **compose**.

29.2 The complete stack, bottom to top


The complete TicketHub stack, bottom to top: from physical hardware to GitOps-delivered, secured, observable microservices.
Bare metal -> KVM/Proxmox VMs (Ch 2)
12-node kubeadm HA cluster (Ch 3,5)
Cilium CNI + MetalLB + Gateway API + Rook-Ceph (Ch 6,7,8)
Namespaces + quotas + bootstrap order (Ch 9)
9 containerized services: Deploy/StatefulSet/DaemonSet (Ch 10-14)
Resources + autoscaling + scheduling + health (Ch 15-18)
RBAC + PSA + NetPol + Kyverno + Falco + supply-chain (Ch 19-25)
Observability + backup/DR + GitOps (Ch 26-28)

Part	Layer	Chapters
I	Physical & architecture design (bare metal → VMs → topology → network)	1–4
II	Cluster install & core platform (kubeadm, Cilium, MetalLB, Ceph, namespaces)	5–9
III	Building & deploying services (Dockerfiles, controllers, Services, config, storage)	10–14
IV	Reliability, scaling & scheduling (resources, autoscaling, placement, health)	15–18
V	Security & governance (RBAC, PSA, NetPol, Kyverno, Falco, supply chain, CRDs)	19–25
VI	Observability & operations (monitoring, backup/DR, GitOps)	26–28

29.3 The architect's enduring principles

29.3 Nuances, Gotchas & Architect Considerations

Nuances — subtle behaviours to internalise

- **The request journey is not a straight line**: in the full sequence (User → CDN → Gateway → API Gateway svc → Orders → Inventory → Payments → Kafka → Notifications), each hop involves DNS resolution, TLS handshake (potentially), TCP connection establishment, and application processing. Instruments show the TOTAL latency but each component in the chain can introduce jitter independently — distributed tracing (Tempo) is required to identify where p99 latency spikes originate.

- **Kubernetes' asynchronous reconciliation means eventual consistency everywhere**: when you `kubectl apply` a Deployment change, the API server accepts it immediately, but the scheduler, kubelet, and container runtime each have their own reconcile cycle. The time from `apply` to all pods serving the new version can be 10-120 seconds depending on image pull time and readiness probe duration.

- **The security model is defense in depth, not a single perimeter**: each layer (TLS, NetworkPolicy, RBAC, PSA, Kyverno, Falco) independently limits blast radius. An attacker who bypasses one layer still faces the others. The weakest link in the TicketHub security model is the shared `tickethub` namespace — services share namespace scope even though they have individual RBAC and NetworkPolicy.

Gotchas — traps that catch experienced engineers

- **The mental model of "microservices are independent" breaks at the data layer**: Postgres, Kafka, and Redis are shared infrastructure. A Postgres volume fill, a Kafka partition leadership election storm, or a Redis BGSAVE blocking event affects ALL services that depend on them — regardless of pod isolation. Monitor data-layer SLIs as aggressively as application-layer SLIs.

- **GitOps and human-made kubectl changes create invisible drift**: an operator who directly edits a running ConfigMap bypasses git, creating state that Argo CD will revert on next sync. Establish a cultural and tooling norm: `kubectl edit` is a debugging tool, not a change management tool. All persistent changes go through git.

- **Observability requires active maintenance**: Prometheus alerts go stale (metric names change after service refactors), Grafana dashboards drift from reality, and Loki label schemes accumulate technical debt. Schedule a quarterly observability review: which alerts fired in the last quarter? Which were false positives? Which incidents had no alert?

Architect Considerations

1. **Total Cost of Ownership (TCO) review**: after building the full cluster, calculate the operational overhead: how many engineer-hours per week does cluster maintenance consume? How does this compare to managed Kubernetes (EKS, GKE)? On-prem gives control and lower cloud cost at the expense of operational burden — re-validate this trade-off annually.

2. **Runbook completeness audit**: for every chapter in this book, there is a corresponding operational scenario. Does a runbook exist for: Postgres primary failover? Kafka broker crash? etcd member failure? Node OOM eviction cascade? Certificate near-expiry alert response? Runbook completeness is a direct measure of operational readiness.

3. **Chaos engineering readiness**: before declaring the cluster production-ready, run controlled chaos experiments: kill a control-plane node (does etcd recover?), kill the primary Postgres pod (does the operator promote a standby?), saturate a data node's disk (does the alert fire before Ceph goes critical?). Use Chaos Mesh or LitmusChaos to automate these experiments.

4. **Graduation path**: this textbook builds a single on-prem cluster. Real production platforms grow: second cluster for DR, multi-region, multi-tenancy. Review the design choices that would need to change at each scale step: CNI (Cluster Mesh), storage (multi-site Ceph), gitops (multi-cluster

ApplicationSet), RBAC (federated identity).

5. **Documentation as living infrastructure**: the `docs/` in this repository are the authoritative reference for how the cluster was built and why each decision was made. Treat architecture decision records (ADRs) with the same rigor as code: every major design decision has a corresponding ADR document committed to the repository.

The ten principles this book was built on

1. **Design failure domains first** — VMs, node pools, and quorum before workloads (Ch 2–3).
2. **Everything declarative, in Git** — the cluster is reproducible, not hand-crafted (Ch 28).
3. **Bootstrap order is sacred** — CNI → storage → policy → data → apps (Ch 9).
4. **Right-size, then autoscale** — accurate requests make QoS and HPA honest (Ch 15–16).
5. **Redundancy only counts if it's spread** — anti-affinity + PDB + topology spread (Ch 17).
6. **Zero-trust by default** — deny-all network and least-privilege RBAC, then allow (Ch 19, 21).
7. **Harden the pod and the supply chain** — non-root, signed images, encrypted secrets (Ch 20, 24).
8. **Detect what you can't prevent** — Falco watches runtime; alerts fire on SLOs (Ch 23, 26).
9. **Practice failure** — backups you restore, upgrades you rehearse (Ch 27).
10. **Automate the operational knowledge** — operators and GitOps, not tribal memory (Ch 25, 28).

29.4 Where to go next

TicketHub is a complete foundation. Natural extensions: a **service mesh** (Istio/Cilium Service Mesh) for mTLS and fine traffic control; **multi-cluster** federation for regional HA; **cost governance** (Kubecost); **FinOps-aware autoscaling**; and **progressive delivery** maturity with automated canary analysis. Each builds on — and reuses — the primitives in this book.

You are now the architect

Every concept — pods to operators, CIDRs to sync waves — now connects to a real decision you can defend: *why this node pool, why this StorageClass, why this NetworkPolicy, why this sync wave.* That web of justified decisions, from the bare metal up, **is** cluster architecture. TicketHub was the vehicle; the judgment is yours to reuse.

End of book — "Designing, Installing & Operating a Production Kubernetes Cluster." The companion `repo/` contains every Dockerfile and manifest referenced across these 29 chapters, organized by bootstrap order for `kubectl apply` or Argo CD sync.

Appendix A — Glossary

A quick-reference for the vocabulary used throughout this book. Terms are alphabetical; the **Ch** column points to where the concept is developed in depth. For the foundational five (object model, kubectl, Helm, CIDR, the stateful backends) see the **Chapter 0 primer**.

Term	Definition	Ch
ACID	Atomicity, Consistency, Isolation, Durability — the guarantees a single database transaction gives. Across *separate* microservice databases these don't hold, which is why we use the **Saga** pattern instead.	1
Admission	The third gate in the API server (after authN/authZ): validates or mutates an object before it is stored. Home of **PSA** and **Kyverno**.	0, 20, 22
Affinity / anti-affinity	Scheduling rules that *attract* pods to nodes/labels (affinity) or keep replicas *apart* (anti-affinity) for resilience.	17
ARP	Address Resolution Protocol — how a machine on a LAN maps an IP to a hardware (MAC) address. MetalLB **L2 mode** answers ARP to claim an external IP.	4, 7
AuthN / AuthZ	Authentication (*who are you?* — cert/OIDC) vs Authorization (*are you allowed?* — RBAC). Two distinct API-server gates.	0, 19
BGP	Border Gateway Protocol — how routers exchange routes. MetalLB **BGP mode** peers with your router to load-share an external IP across nodes (vs single-node L2).	4, 7

CA (Certificate Authority)	The trusted signer of certificates. `kubeadm` creates a **cluster CA** that signs an identity for every component and node (the cluster's **PKI**). The full inventory, HA SANs, and renewal are in Chapter 7A.	5, 7A
cgroup / cgroup driver	Linux control groups limit/account a process's CPU & memory (how requests/limits are enforced). The *driver* (`systemd` vs `cgroupfs`) must match between containerd and kubelet.	0, 5, 15
CIDR	`IP/prefixlen` notation (e.g. `10.244.0.0/16`). The `/N` fixes the leading N bits as the network; smaller N = larger range. Cluster CIDRs must never overlap.	0, 4
ClusterIP	The default Service type: a stable virtual IP + DNS name reachable **only inside** the cluster.	12
CNI	Container Network Interface — the plugin that gives pods IPs and wires their networking. We use **Cilium**.	6
Compensating transaction	An action that *undoes* a completed step when a later step in a **Saga** fails (e.g. `ReleaseSeats()` after a failed payment).	1
CRD / Operator	A CustomResourceDefinition teaches the API a new *kind*; an **operator** is the controller that reconciles it — the standard way to extend Kubernetes.	25
CRI	Container Runtime Interface — the API kubelet uses to run containers via a runtime like **containerd**.	5

CSI	Container Storage Interface — the standard plugin API that lets Kubernetes provision storage from any backend (Ceph RBD, cloud disks). The Rook StorageClass `provisioner` is a CSI driver.	8, 14
DaemonSet	A controller that runs exactly one pod **per node** (CNI agent, node-exporter, Falco).	11
Deployment / ReplicaSet	A Deployment manages ReplicaSets, which manage N interchangeable stateless pod replicas with rolling updates + rollback.	11
DSR (Direct Server Return)	A load-balancing mode where the reply skips the load balancer and returns straight to the client — lower latency. Cilium option.	6
eBPF	Small sandboxed programs loaded into the Linux kernel. Cilium uses eBPF for routing, service load-balancing, and policy — far faster than iptables at scale.	4, 6
EndpointSlice	The auto-maintained list of **Ready** pod IPs behind a Service; readiness probes gate membership.	12
etcd	The cluster's key/value database holding all state. Consensus via **Raft**; needs an odd quorum (3 or 5).	3, 5
front-proxy CA	A separate CA that signs the aggregation-layer client cert, isolating extension/aggregated API servers from the main cluster CA.	7A

Gateway API	The role-oriented successor to Ingress: `GatewayClass` (infra provider), `Gateway` (platform team owns listeners/TLS/IP), `HTTPRoute` (app team owns paths/backends). Delivered as CRDs; TicketHub uses Cilium's implementation.	7
gRPC	A fast, strongly-typed, binary RPC protocol (HTTP/2) for **service-to-service** calls. TicketHub uses gRPC internally and REST/JSON at the edge.	1, 12
Headless Service	A Service with `clusterIP: None` that returns individual pod IPs and gives StatefulSet pods stable per-pod DNS (`postgres-0.postgres…`).	12, 14
Helm	The Kubernetes package manager: a **chart** (templated YAML) + your **values** → an installed **release**.	0
hostNetwork	A pod setting that puts the pod on the *node's* network namespace (bypassing the Pod CIDR). Used sparingly (e.g. edge/Gateway data plane).	4
HPA / VPA / KEDA / Cluster Autoscaler	Autoscalers for, respectively: replica **count**, per-pod **size**, **event-driven** replicas, and **node** count.	16
HTTPRoute	A Gateway API object owned by an app team that attaches to a `Gateway` and declares path/header/method matches and weighted backends. Replaces the routing rules of an Ingress.	7
Ingress	The original (now **feature-frozen**) Kubernetes object for HTTP edge routing + TLS. Superseded by the **Gateway API**; TicketHub migrated off it.	7

IPAM	IP Address Management — how the CNI hands out pod IPs from the Pod CIDR.	6
JWT	JSON Web Token — a signed, self-contained token proving a user's identity/claims. The Users/Auth service issues them; the gateway verifies them.	1
keepalived / VRRP	keepalived uses the VRRP protocol to float a **virtual IP** between nodes, so the API endpoint stays reachable if one load-balancer node dies.	3
kube-proxy	The classic component that programs `iptables`/IPVS rules to route Service traffic to pods. Cilium **replaces** it with eBPF for speed and simplicity.	4, 6
LimitRange / ResourceQuota	Per-namespace guardrails: LimitRange sets default/min/max per container; ResourceQuota caps the namespace total.	9, 15
mTLS (mutual TLS)	Both sides of a connection present and validate certificates, so client *and* server prove identity. Every control-plane link is mTLS; east-west service mTLS is added via an internal CA or a mesh.	7A
NetworkPolicy	Firewall rules for pod traffic by label/port. Default-deny then allow; Cilium extends to L7.	21
NDP	Neighbor Discovery Protocol — the IPv6 equivalent of ARP; MetalLB L2 uses it for IPv6.	4
OIDC	OpenID Connect — the token-based protocol external identity providers (Okta, Entra ID) use to authenticate *humans* to the API server.	0, 19

OSD / mon / mgr	Ceph daemons: OSD stores data (one per disk), mon keeps the cluster map + quorum, mgr does metrics/orchestration.	8
PDB (PodDisruptionBudget)	Guarantees a minimum number of a service's pods stay up during *voluntary* disruptions (node drains, upgrades).	17, 27
Percentile (P50 / P95 / P99)	The value below which that % of samples fall. P99 latency = "99% of requests are faster than this." Size memory limits near P99; requests near P50–P75.	15
PKI	Public Key Infrastructure — the system of a CA plus the certificates it signs. The cluster runs its own; see Chapter 7A for the full hierarchy and lifecycle.	5, 7A
PV / PVC / StorageClass	A PVC *requests* storage, a StorageClass describes *how* to provision it, and a PV is the *actual* volume created.	8, 14
QoS class	Guaranteed / Burstable / BestEffort — derived from requests vs limits; decides eviction order under memory pressure.	15
Quorum	The majority of members a consensus system needs to agree (2 of 3, 3 of 5). Below quorum, etcd stops accepting writes.	3
Raft	The consensus protocol etcd uses to keep replicas in agreement via an elected leader + majority quorum.	3
RBAC	Role-Based Access Control — grants *verbs on resources* to subjects (users/groups/ServiceAccounts). Additive, default-deny.	19

RED / USE	Alerting methods: RED = Rate, Errors, Duration (for services); USE = Utilization, Saturation, Errors (for resources).	26
Reconcile loop	The observe → diff → act cycle every controller/operator runs to drive actual state toward desired state.	0, 25
REST	A resource-oriented HTTP/JSON API style, browser-friendly; used at TicketHub's edge (vs gRPC internally).	1
Saga	A pattern for multi-step transactions across microservices: a sequence of local commits, each with a **compensating** undo if a later step fails.	1
SAN / certSANs	Subject Alternative Names — the list of DNS names/IPs a certificate is valid for. `apiServer.certSANs` must include the HA **VIP** or calls through the load balancer fail TLS.	7A
SBOM / SLSA	A Software Bill of Materials lists an image's components; SLSA provenance attests *how* it was built — both speed up CVE response.	24
seccomp	A Linux kernel filter restricting which syscalls a process may make; `RuntimeDefault` blocks dangerous ones.	20
ServiceAccount	The identity a **pod** uses to call the API server. Give each workload its own, least-privilege one.	19
SPIFFE	A standard for cryptographic **workload identity** (a SPIFFE ID per workload). Service meshes issue and auto-rotate SPIFFE identities to do transparent mTLS.	7A

StatefulSet	A controller for stateful pods: stable ordinal identity (`postgres-0`) + one persistent volume per replica, ordered operations.	11, 14
Static pod	A pod the kubelet runs directly from a node file (not the API). The control-plane components run this way.	0, 5
Taint / Toleration	A taint *repels* pods from a node; only pods that *tolerate* it may land. Pairs with labels to build dedicated node pools.	3, 17
trust-manager	A cert-manager companion that publishes a CA bundle as a ConfigMap into every namespace, so workloads can trust an internal CA without manual copying.	7A
veth pair	A virtual cable with one end in a pod's network namespace and one on the node; the CNI creates it to connect the pod.	0, 6
VIP (Virtual IP)	A single floating IP fronting the three API servers (via keepalived + HAProxy) so clients use one stable endpoint.	3, 5
VLAN	A logically separated LAN segment; we split management traffic from application/LB traffic onto different VLANs.	4
Network namespace	A private, isolated network stack (own interfaces/routes) — the boundary that gives each pod its own network.	0, 6

End of Appendix A. The companion `repo/` holds every manifest and Dockerfile referenced in the chapters above, organized by bootstrap order.